Networking

K19G

2026-10-18

Table of contents

Networking **33**
Curriculum map . . . 33
Journey map . . . 34
What "done" looks like . . . 34
Lab ground rules . . . 35
Related books . . . 35
Formats . . . 35

Lab Platform **36**

Lab Host — macOS **37**

Lab Host — macOS **38**
Goals . . . 38
Prerequisites . . . 38
Virtualization and container runtime . . . 38
Options . . . 38
Checklist . . . 39
Install Containerlab . . . 39
Networking notes on macOS . . . 39
First smoke test . . . 39
Next . . . 39

Lab Host — Linux **40**

Lab Host — Linux **41**
Goals . . . 41
Prerequisites . . . 41
Container runtime . . . 41
Typical path (Debian/Ubuntu-style) . . . 41
Kernel / modules . . . 42
Install Containerlab . . . 42
Host networking and firewall . . . 42
First smoke test . . . 42
Next . . . 42

Containerlab Fundamentals **43**

Containerlab Fundamentals **44**
Learning goals . . . 44

Prerequisites . . . 44
How Containerlab fits together . . . 45
Minimal topology: two Linux nodes . . . 46
Triangle of FRR routers (book default pattern) . . . 46
Startup config sketch (`config/r1/daemons` + `frr.conf`) . . . 47
Lifecycle . . . 48
Topology YAML anatomy . . . 48
Kinds you will actually use . . . 48
Links and endpoints . . . 48
Multipoint / LAN segments . . . 49
Management network . . . 49
Inspect, exec, save . . . 49
Lab directory layout (recommended) . . . 50
Graph and documentation . . . 50
Partial failure recovery . . . 50
Predict → observe → fix lab . . . 51
Setup . . . 51
Predict . . . 51
Observe . . . 51
Fix inject . . . 52
Harden . . . 52
Common mistakes . . . 52
Version awareness (mid-2026) . . . 52
Security and shared hosts . . . 53
Summary . . . 53

Netlab Fundamentals **54**

Netlab Fundamentals **55**
Learning goals . . . 55
The Dual-Engine Model: Why Both Tools Matter . . . 56
Installation & Setup . . . 57
Prerequisites . . . 57
Automated Platform Provisioning . . . 57
The Anatomy of a Netlab Topology . . . 57
Minimal Triangle Topology with OSPF . . . 57
What Happens Behind the Scenes? . . . 58
Operating Netlab Labs . . . 59
1. Starting and Stopping Labs . . . 59
2. Connecting to Devices . . . 59
3. Graphing Topologies . . . 59
Automated Verification with `netlab validate` . . . 60
Simulating Real-World Links with `netlab tc` . . . 61
Multi-NOS Portability: The Ultimate Learning Superpower . . . 61
Summary & What Comes Next . . . 62

Free Image Matrix & Resources **63**

Free Image Matrix & Resources **64**
Learning goals . . . 64
Principles . . . 64
Role matrix . . . 64
What we deliberately de-emphasize . . . 65
FRR as the workhorse . . . 65
Linux endpoints . . . 66
SR Linux community (optional track) . . . 66
SONiC and VyOS (optional) . . . 67
Sizing the lab host . . . 67
RAM savers . . . 67
Disk . . . 68
Version pinning strategy . . . 68
Resource planning worksheet . . . 68
Network resource limits on the host . . . 69
Documentation & community map . . . 69
RFCs worth bookmarking (foundations) . . . 69
Predict → observe: image sanity lab . . . 70
Multi-implementation mindset . . . 70
Ethics and redistribution . . . 70
Summary . . . 71

Linux Networking for Lab Hosts **72**

Linux Networking for Lab Hosts **73**
Learning goals . . . 73
Learning goals in practice . . . 73
Core objects . . . 73
Network namespaces . . . 74
veth pairs: virtual cables . . . 74
Bridges: software switches . . . 75
Routing between namespaces . . . 76
Failure drill: disable forwarding . . . 77
Failure drill: missing return route . . . 77
VLAN subinterfaces (preview of trunks) . . . 77
Policy leftovers that break labs . . . 77
rp_filter (reverse path filtering) . . . 77
Forwarding and firewall . . . 78
Disable offloads when captures look weird (optional) . . . 78
Mapping to Containerlab . . . 78
Clean teardown habits . . . 79
Example `down.sh` . . . 79
Predict → observe → fix drill (checklist) . . . 79
Useful one-liners cheatsheet . . . 80
Security note for shared hosts . . . 80
Summary . . . 80

Lab Hygiene **81**

Lab Hygiene **82**
Learning goals . . . 82
Why hygiene is a skill . . . 82
Addressing plans . . . 82
Template . . . 83
Allocation strategy (personal lab supernets) . . . 83
/30 vs /31 vs /24 on links . . . 83
Naming conventions . . . 84
Lab name . . . 84
Node names . . . 84
Interface names . . . 84
Git branches / folders . . . 84
Directory layout (canonical) . . . 85
README skeleton . . . 85
Git habits . . . 86
Never commit . . . 86
Diagram habit . . . 86
Configuration hygiene . . . 87
FRR example: comment intent . . . 87
verify.sh hygiene . . . 87
Cross-lab contamination . . . 88
Time hygiene . . . 88
Definition of done (lab) . . . 88
Predict → observe → fix: hygiene drill . . . 89
Collaboration norms (optional) . . . 89
Summary . . . 90

Networking Foundations **91**

What Is Networking? **92**

What Is Networking? **93**
Learning goals . . . 93
Building blocks . . . 93
What problem networking solves . . . 93
Switching vs routing (first cut) . . . 94
Protocols as contracts . . . 94
Why labs beat slideware . . . 94
Observation toolkit (preview) . . . 95
Checkpoint . . . 95
Next . . . 95

TCP/IP and the Layered Stack **96**

TCP/IP and the Layered Stack **97**
Learning goals . . . 97
TCP/IP model (working map) . . . 97
OSI as a legend (not a religion) . . . 97

What each layer "owns" . . . 98
Link . . . 98
Internet (IP) . . . 98
Transport . . . 98
Application . . . 98
Demultiplexing (up the stack) . . . 99
Layered troubleshooting . . . 99
Encapsulation (preview) . . . 99
Checkpoint . . . 99
Next . . . 99

Packets, Frames, Encapsulation **100**

Packets, Frames, Encapsulation **101**
Learning goals . . . 101
Vocabulary . . . 101
Encapsulation path (send) . . . 101
Decapsulation path (receive) . . . 102
Maximum sizes and MTU . . . 102
Captures: reading the onion . . . 102
Tunnels and overlays (preview) . . . 102
Lab sketch (two nodes) . . . 103
Checkpoint . . . 103
Next . . . 103

Link Layer and Ethernet **104**

Link Layer and Ethernet **105**
Learning goals . . . 105
Why link layer matters . . . 105
Ethernet frame (simplified) . . . 105
MAC addresses . . . 106
Broadcast domain vs collision domain . . . 106
Switch vs router (again) . . . 106
MAC learning (preview) . . . 106
Virtual Ethernet in labs . . . 107
Common L2 failures (foundations list) . . . 107
Checkpoint . . . 107
Next . . . 108

IPv4 Addressing and Subnetting **109**

IPv4 Addressing and Subnetting **110**
Learning goals . . . 110
IPv4 structure . . . 110
CIDR and masks . . . 111
On-link decision . . . 111
Private and documentation ranges . . . 111

Subnetting for multi-tier labs . . . 111
Point-to-point links . . . 112
Summarization (intro) . . . 112
VLSM . . . 112
Host configuration checklist . . . 114
Worked subnet: `/26` from a `/24` . . . 114
Private vs public (RFC 1918) . . . 114
Paper drills . . . 115
Checkpoint . . . 115
Next . . . 115

IPv6 Essentials **116**

IPv6 Essentials **117**
Learning goals . . . 117
Why IPv6 in this book . . . 117
Address format . . . 117
Address kinds (must know) . . . 118
Interface identifiers and /64 . . . 118
Linux static example . . . 118
Neighbor Discovery (preview) . . . 118
Dual-stack thinking . . . 119
Checkpoint . . . 119
Next . . . 119

ARP, ND, and Local Resolution **120**

ARP, ND, and Local Resolution **121**
Learning goals . . . 121
When local resolution happens . . . 121
ARP (IPv4) . . . 121
Incomplete / failed ARP . . . 122
Neighbor Discovery (IPv6) . . . 122
Gratuitous ARP and updates . . . 122
Proxy ARP (awareness) . . . 122
Lab sketch . . . 122
Checkpoint . . . 123
Next . . . 123

ICMP and Path Diagnostics **124**

ICMP and Path Diagnostics **125**
Learning goals . . . 125
ICMPv4 types (operator set) . . . 125
ICMPv6 . . . 125
Interpreting ping . . . 126
Traceroute mechanics (conceptual) . . . 126
Path MTU Discovery (preview) . . . 126
Filters and policy . . . 126

Lab sketch . . . 127
Checkpoint . . . 127
Next . . . 127

Transport: TCP and UDP **128**

Transport: TCP and UDP **129**
Learning goals . . . 129
Ports and sockets . . . 129
UDP . . . 129
TCP . . . 130
Three-way handshake (must know) . . . 130
States (operator view) . . . 130
TCP vs UDP selection . . . 130
NAT and transports (preview) . . . 131
Security basics . . . 131
TCP state machine (operator view) . . . 131
Three-way handshake and teardown (why captures look busy) . . . 132
Lab sketch . . . 132
Checkpoint . . . 132
Next . . . 132

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall **133**

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall **134**
Learning goals . . . 134
Mental model . . . 134
Concepts — the knobs . . . 136
Lab — reproduce the stall and measure the fixes . . . 137
Lab — catch the stall in `ss` . . . 140
Lab — what Nagle is for . . . 141
Case — choosing the fix . . . 142
The trap . . . 142
The boring rule . . . 143
Try this . . . 143
Next . . . 143
Verification . . . 144
Sources . . . 144

Common Application Protocols **145**

Common Application Protocols **146**
Learning goals . . . 146
Map (operator cheat sheet) . . . 146
DNS . . . 146
Roles . . . 146
Query types (common) . . . 147
Failure patterns . . . 147
DHCP (IPv4) . . . 147

HTTP and HTTPS . . . 148
SSH . . . 148
TLS in one paragraph . . . 148
NTP and time . . . 148
Lab sketch . . . 149
Checkpoint . . . 149
Next . . . 149

Host Forwarding and Routing Intro **150**

Host Forwarding and Routing Intro **151**
Learning goals . . . 151
Connected vs remote . . . 151
Linux route table essentials . . . 151
Longest prefix match . . . 152
Default gateway . . . 152
Static routes (host or simple router) . . . 152
IP forwarding (Linux as router) . . . 152
Policy routing (awareness) . . . 153
Firewall vs routing . . . 153
Lab sketch . . . 153
Checkpoint . . . 153
Next . . . 153

Control, Data, and Management Planes **154**

Control, Data, and Management Planes **155**
Learning goals . . . 155
Data plane . . . 155
Control plane . . . 155
Management plane . . . 156
Interaction . . . 156
Troubleshooting order (habit) . . . 157
Lab habit . . . 157
Checkpoint . . . 157
Next . . . 157

Layer 2 **158**

Ethernet & MAC Learning **159**

Ethernet & MAC Learning **160**
Learning goals . . . 160
Concepts . . . 161
Frame essentials . . . 161
Learning and flooding . . . 161
MAC addresses . . . 161
ARP: glue for IPv4 on Ethernet . . . 162

Linux bridge FDB . . . 163
Lab A: two hosts, one bridge (hand or Containerlab) . . . 163
Containerlab topology . . . 163
Predict . . . 164
Observe . . . 164
Fix inject . . . 164
Harden . . . 165
Lab B: three hosts — flooding visibility . . . 165
MAC moves and flaps . . . 165
Common L2 failures (pre-VLAN) . . . 165
Promiscuous mode and labs . . . 165
Security digression (short) . . . 166
Predict worksheet (paper) . . . 166
FRR / NOS note . . . 166
Summary . . . 166

VLANs & Trunks **168**

VLANs & Trunks **169**
Learning goals . . . 169
Concepts . . . 170
Broadcast domains . . . 170
802.1Q tag . . . 170
Router roles . . . 170
Addressing plan example . . . 170
Lab topology (conceptual) . . . 171
Containerlab sketch using Linux VLAN-aware bridge . . . 171
`config/sw1.sh` (VLAN filtering bridge) . . . 172
Predict → observe → fix . . . 173
Predict . . . 173
Observe . . . 173
Failure drill: native / PVID mismatch . . . 173
Failure drill: missing allowed VLAN . . . 174
Router-on-a-stick preview . . . 174
Verification checklist . . . 174
verify.sh sketch . . . 174
Design tips . . . 175
Common mistakes . . . 175
Summary . . . 175

Loop Prevention **176**

Loop Prevention **177**
Learning goals . . . 177
Why loops kill L2 . . . 177
Spanning tree behavior . . . 177
Priority and ties . . . 178
Port states (classic story) . . . 178

Design alternatives (model maturity) . . . 178
Lab: three switches, intentional loop . . . 179
Topology YAML (Linux bridges) . . . 179
Dangerous mode: dumb bridges without STP . . . 180
Observe storm (careful) . . . 180
Safer mode: enable STP on the bridge . . . 180
Root placement drill . . . 181
Failure drill: link down on active tree edge . . . 181
BPDUs and protection concepts (awareness) . . . 182
When not to extend L2 . . . 182
Predict worksheet . . . 182
Verification checklist for STP labs . . . 182
Summary . . . 183

Link Aggregation **184**

Link Aggregation **185**
Learning goals . . . 185
Concepts . . . 185
One logical link . . . 185
Static vs LACP . . . 185
Hashing . . . 186
Split brain / misconfig . . . 186
Linux bonding quickstart . . . 186
Containerlab topology (two paths, bond on routers) . . . 187
`config/r1-bond.sh` . . . 188
Predict → observe → fix . . . 188
Predict . . . 188
Observe . . . 189
Failover drill . . . 189
Mismatch drill . . . 189
Hashing awareness drill . . . 189
STP interaction . . . 189
Operational checks (NOS-neutral) . . . 190
Common mistakes . . . 190
Summary . . . 190

L2 Failure Drills **191**

L2 Failure Drills **192**
Learning goals . . . 192
Baseline lab . . . 192
Evidence kit . . . 193
Drill catalog . . . 193
Drill 1 — Access port wrong VLAN . . . 193
Drill 2 — Trunk missing VLAN . . . 193
Drill 3 — Native VLAN mismatch . . . 194
Drill 4 — Cable / interface down . . . 194

Drill 5 — Broadcast storm (controlled) . . . 194
Drill 6 — STP blocked port confusion . . . 195
Drill 7 — MAC move . . . 195
Drill 8 — Duplicate IP . . . 195
Drill 9 — Duplicate MAC (advanced) . . . 196
Drill 10 — LAG member down . . . 196
Drill 11 — Inter-VLAN path mistaken for L2 . . . 196
Drill 12 — Host firewall . . . 197
Incident note template . . . 197
Scorecard . . . 197
Combined verify.sh ideas . . . 198
Checkpoint: L2 competence . . . 198
Summary . . . 198

Layer 3 **200**

IP Addressing **201**

IP Addressing **202**
Learning goals . . . 202
IPv4 structure . . . 202
On-link decision . . . 203
Subnetting for labs . . . 203
Summarization . . . 203
Paper drill . . . 204
Special-use ranges (know them) . . . 204
IPv6 essentials for dual-stack labs . . . 204
Dual-stack habit . . . 205
Linux configuration patterns . . . 205
Persistent lab style . . . 205
Multiple addresses . . . 205
Lab: addressing plan + dual-stack hosts . . . 205
Predict . . . 206
Observe . . . 206
Failure: overlapping subnets . . . 207
Failure: bad summary (static preview) . . . 207
Calculation practice (quick) . . . 207
Host vs router addressing habits . . . 207
Verification checklist . . . 207
Summary . . . 208

Routing Fundamentals **209**

Routing Fundamentals **210**
Learning goals . . . 210
Concepts . . . 210
Forwarding equation . . . 210
RIB vs FIB . . . 211

Longest prefix match . . . 211
Metrics and distance (ideas) . . . 211
Connected routes . . . 211
Host vs router . . . 212
Gateway ARP . . . 212
Next-hop resolution failures . . . 212
Asymmetric routing . . . 212
Lab: competing prefixes . . . 213
Predict / observe . . . 214
Lab: router with two interfaces . . . 214
ARP/ND deep enough for routing . . . 215
TTL and traceroute . . . 215
Policy preview . . . 215
Predict → observe → fix drills . . . 216
Drill: missing return route . . . 216
Drill: wrong gateway MAC path . . . 216
Drill: more-specific shadow . . . 216
Cheatsheet . . . 216
Summary . . . 216

Static Routing Labs **218**

Static Routing Labs **219**
Learning goals . . . 219
Topology: triangle with hosts . . . 219
Containerlab topology . . . 221
FRR config sketches . . . 222
Linux-only alternative . . . 222
Deploy and baseline . . . 223
Predict . . . 223
Observe . . . 223
Floating statics . . . 223
Failover drill . . . 223
Return path drill (mandatory) . . . 224
Blackhole and reject . . . 224
verify.sh . . . 224
Addressing.md excerpt . . . 224
Default routes at edge . . . 225
IPv6 statics (optional parallel) . . . 225
Common mistakes . . . 225
Summary . . . 225

ICMP & Path MTU **227**

ICMP & Path MTU **228**
Learning goals . . . 228
ICMP essentials (IPv4) . . . 228
Path MTU discovery (idea) . . . 229

Lab: MTU mismatch . . . 229
Topology . . . 229
Predict . . . 230
Observe . . . 230
Inject: drop ICMP unreachable on r1 . . . 231
Restore . . . 231
Harden . . . 231
Traceroute and ICMP filters . . . 231
Capture workflow . . . 231
iperf3 large transfers . . . 232
IPv6 notes . . . 232
Operational checklist . . . 232
Predict worksheet . . . 232
verify.sh additions . . . 233
Summary . . . 233

Interior Routing **234**

OSPF Basics **235**

OSPF Basics **236**
Learning goals . . . 236
Concepts . . . 236
Distance-vector vs link-state . . . 236
OSPF adjacency state machine (short) . . . 236
Areas . . . 237
Cost . . . 237
Network types (awareness) . . . 237
Addressing plan . . . 237
Method A: The Fast Netlab Workflow (Recommended) . . . 237
Deploy and Validate with Netlab . . . 238
Method B: Raw Containerlab & Manual FRR Configurations . . . 239
Topology YAML (`ospf-tri.clab.yml`) . . . 239
FRR Daemons File (`config/r1/daemons`) . . . 240
FRR OSPF Config (`config/r1/frr.conf`) . . . 240
Deploy and Verify Manually . . . 241
Predict . . . 241
Neighbor not Full? . . . 241
Cost change drill . . . 241
Link failure drill . . . 242
Passive interfaces / LAN notes . . . 242
Verification checklist . . . 242
verify.sh . . . 242
Common mistakes . . . 243
Beyond this chapter . . . 243
Summary . . . 243

BGP Basics 244

BGP Basics 245
Learning goals . . . 245
Concepts . . . 245
Autonomous systems . . . 245
eBGP vs iBGP . . . 246
Session state . . . 246
Path selection overview . . . 246
NLRI and attributes . . . 246
Lab A: two AS eBGP . . . 246
Topology . . . 247
daemons . . . 247
r1 frr.conf . . . 248
r2 . . . 248
Verify session . . . 248
Predict . . . 249
Lab B: iBGP sketch (optional same day) . . . 249
Session failure drill . . . 249
Next-hop reachability drill . . . 249
Dual-home teaser . . . 250
verify.sh . . . 250
Safety habits . . . 250
Common mistakes . . . 250
Summary . . . 251

Policy & Filters 252

Policy & Filters 253
Learning goals . . . 253
Why policy exists . . . 253
Prefix lists (FRR) . . . 254
Route-maps as idea . . . 254
Lab: filter unauthorized prefixes . . . 254
Customer r1 advertises extra junk . . . 255
ISP r2 policy . . . 255
Predict . . . 255
Observe . . . 255
Harden . . . 255
Local-pref steer drill . . . 256
Outbound preference (customer) . . . 256
Redistribution loop awareness . . . 256
Example static→OSPF with filter (sketch) . . . 257
ACL-style packet filters vs routing policy . . . 257
Distribute-list / filter-list ideas . . . 257
Failure drills . . . 257
Drill 1 — over-broad permit any . . . 257
Drill 2 — accidental deny all . . . 258

Drill 3 — next-hop-self missing (iBGP) . . . 258
verify.sh policy checks . . . 258
Policy design worksheet . . . 258
Summary . . . 259

BFD — Fast Failure Detection **260**

BFD — Fast Failure Detection **261**
Learning goals . . . 261
Why carrier is not enough . . . 261
Concepts — BFD in one page . . . 262
Addressing plan (BFD lab) . . . 263
Lab build (Linux namespaces + FRR) . . . 263
Drill 1 — the silent failure (no BFD) . . . 266
Predict . . . 266
Observe . . . 266
Explain . . . 267
Drill 2 — add BFD and repeat . . . 267
Drill 3 — the slower side wins . . . 269
Predict . . . 269
Observe . . . 269
Drill 4 — BFD for a static route . . . 270
Drill 5 — one-sided BFD (predict, then try) . . . 270
Choosing timers . . . 271
Verification checklist . . . 272
Common mistakes . . . 272
Summary . . . 272
Sources . . . 273

RPKI — Route Origin Validation **274**

RPKI — Route Origin Validation **275**
Learning goals . . . 275
Concepts — RPKI in one page . . . 275
Addressing plan (RPKI lab) . . . 278
Lab build (Linux namespaces + FRR + StayRTR) . . . 279
Drill 1 — validation is a label, not a filter . . . 282
Predict . . . 282
Observe . . . 282
Explain . . . 283
Drill 2 — drop Invalid, de-prefer NotFound . . . 283
Drill 3 — the maxLength trap . . . 284
Predict . . . 284
Explain . . . 285
Drill 4 — state changes under accepted routes . . . 285
Predict . . . 285
Observe . . . 285
Explain . . . 286

Drill 5 — the cache goes away (predict, then try) . . . 286
Checking real-world origins . . . 287
Deploying it for real . . . 287
Verification checklist . . . 288
Common mistakes . . . 288
Summary . . . 289
Sources . . . 289

Segment Routing, SRv6, and TI-LFA **291**

Segment Routing, SRv6, and TI-LFA **292**
Learning goals . . . 292
Thesis . . . 292
Mental model . . . 292
SR data planes . . . 293
Worked examples . . . 294
Case 1: Architecture in one page (RFC 8402) . . . 294
Case 2: TI-LFA repair flavours (RFC 9855 §5) . . . 294
Case 3: What the PLR does to the packet (RFC 9855 §6–7) . . . 294
Case 4: Numbers from RFC 9855 Appendix B (SR-MPLS measurements) . . . 295
The trap . . . 295
The boring rule . . . 295
Try this . . . 295
Sources . . . 296

Ops and Automation **297**

Observability for Networks **298**

Observability for Networks **299**
Learning goals . . . 299
Observability layers . . . 299
Baseline: what “healthy” looks like . . . 299
Interface and error signals . . . 300
Control-plane signals . . . 300
OSPF . . . 300
BGP . . . 301
Path quality . . . 301
Packet capture practice . . . 301
Capture points . . . 301
Logging . . . 302
FRR . . . 302
Kernel . . . 302
Metrics sketch (Prometheus-shaped thinking) . . . 302
Minimal blackbox idea . . . 302
Telemetry awareness (2026) . . . 303
Incident timeline template . . . 303
Lab drill: observe a BGP flap . . . 303

Lab drill: silent MTU blackhole . . . 304
Runbook snippet library . . . 304
Privacy and safety . . . 304
Summary . . . 304

Automation & Lab-as-Code 305

Automation & Lab-as-Code 306
Learning goals . . . 306
Principles . . . 306
Directory layout (automation-ready) . . . 306
Makefile pattern . . . 307
up.sh / down.sh . . . 307
Config strategies . . . 308
Safe interactive explore → promote . . . 308
Light Ansible (optional) . . . 308
Templating addressing: DIY vs. Netlab . . . 309
The DIY Approach (Jinja2 + custom scripts) . . . 309
The Netlab Declarative Approach . . . 309
Verification: verify.sh vs. netlab validate . . . 310
Custom Shell Script (`verify.sh`) . . . 310
Netlab Automated Validation . . . 311
CI smoke (GitHub Actions sketch) . . . 311
Image pins in automation . . . 311
Idempotency drills . . . 311
Secret hygiene . . . 312
Graph and inventory export . . . 312
Predict → observe → fix . . . 312
Anti-patterns . . . 312
Summary . . . 313

Capstone Outline 314

Capstone Outline 315
Learning goals . . . 315
Definition of done (all capstones) . . . 315
Capstone C1 — Campus-style dual core (open images) . . . 316
Intent . . . 316
Skills exercised . . . 316
Success metrics . . . 316
Suggested stack . . . 316
Capstone C2 — Dual-homed site to provider edge . . . 317
Intent . . . 317
Skills exercised . . . 317
Success metrics . . . 317
verify extras . . . 317
Capstone C3 — Mini fabric snack . . . 317
Intent . . . 317

Skills exercised . . . 318
Stretch (optional) . . . 318
Capstone C4 — Incident week . . . 318
Intent . . . 318
Skills exercised . . . 319
Success metrics . . . 319
How to pick . . . 319
Project template (copy) . . . 319
DESIGN.md skeleton . . . 320
RETRO.md prompts . . . 320
Integration with the book spine . . . 320
Beyond this book (next horizons) . . . 320
Final checkpoint: "expert" as defined in the syllabus . . . 321
Summary . . . 321

Edge and Services **322**

First-Hop Redundancy **323**

First-Hop Redundancy **324**
Learning goals . . . 324
Concepts . . . 324
The problem . . . 324
First-hop redundancy idea . . . 324
Protocol family (vendor-agnostic) . . . 325
Priority, preempt, timers . . . 325
What FHRP is *not* . . . 325
ARP and the VIP . . . 326
Addressing plan . . . 326
Topology YAML . . . 326
FRR daemons . . . 327
FRR VRRP sketch (r1) . . . 328
Linux alternative (keepalived) . . . 328
Deploy and baseline . . . 329
Predict (baseline) . . . 329
Observe . . . 329
Drill 1 — Master failure . . . 329
Predict → observe → fix . . . 329
Drill 2 — Split brain / LAN partition (concept) . . . 330
Predict . . . 330
Observe . . . 330
Fix / harden . . . 330
Drill 3 — Priority change live . . . 331
Predict . . . 331
Observe . . . 331
Return path and asymmetric edges . . . 331
Verification checklist . . . 331

verify.sh sketch . . . 332
Common mistakes . . . 332
Optional SR Linux note (community) . . . 332
Summary . . . 332

DHCP Relay and NAT **334**

DHCP Relay and NAT **335**
Learning goals . . . 335
Concepts — DHCP . . . 335
Why not static everything? . . . 335
Four-way dance (v4) . . . 335
Critical fields . . . 336
Relay failure modes . . . 336
Concepts — NAT family . . . 336
When NAT is appropriate in this book . . . 336
Addressing plan (combined edge lab) . . . 337
Topology YAML . . . 337
Edge addressing and forward . . . 338
DHCP server (dnsmasq) on dhcpsrv . . . 339
Minimal same-subnet lab (sanity first) . . . 339
Relay path (concept + command) . . . 339
Predict (DHCP) . . . 339
Observe . . . 340
Fix . . . 340
NAT — MASQUERADE on edge . . . 340
Predict . . . 340
Observe . . . 341
Fix . . . 341
Drill — break DHCP then NAT . . . 341
FRR on the edge (optional co-location) . . . 341
Verification checklist . . . 342
Common mistakes . . . 342
Summary . . . 342

Name, Time, and Log Hygiene **343**

Name, Time, and Log Hygiene **344**
Learning goals . . . 344
Why this chapter exists . . . 344
Concepts — naming . . . 345
Layers of DNS use . . . 345
Records you actually need . . . 345
Split horizon awareness . . . 345
Concepts — time . . . 345
Concepts — logs . . . 346
Addressing plan (hygiene lab) . . . 346
Topology YAML . . . 346

Bring-up addressing (example) . . . 347
DNS lab — dnsmasq authoritative snack . . . 348
Predict . . . 348
Observe / fix . . . 348
Drill — DNS outage looks like network outage . . . 348
Predict → observe → fix . . . 348
Time sync — chrony sketch . . . 349
Skew drill . . . 349
Predict . . . 349
Observe . . . 349
Logging hygiene . . . 350
Local journal pattern . . . 350
syslog-style forward (concept) . . . 350
What to log for network incidents . . . 350
FRR logging (when routers are FRR) . . . 350
Hygiene checklist (use every serious lab) . . . 351
Predict → observe → fix mini-scenarios . . . 351
Scenario A — "Git clone fails" in a lab VM . . . 351
Scenario B — "Certificate expired" on brand-new lab CA . . . 351
Scenario C — "It failed an hour ago" with no trace . . . 351
Optional: name the lab graph . . . 352
Common mistakes . . . 352
Summary . . . 352

Edge Dual-Home Lab **353**

Edge Dual-Home Lab **354**
Learning goals . . . 354
Design choices (pick one primary) . . . 354
Pattern A — Single CE, two eBGP uplinks (simpler) . . . 354
Pattern B — Dual CE + VRRP (more realistic campus edge) . . . 354
Addressing plan . . . 355
Topology YAML . . . 355
FRR daemons . . . 356
CE1 config sketch . . . 357
CE2 config differences . . . 358
PE1 / PE2 sketch . . . 358
Deploy and baseline verify . . . 359
Predict (baseline) . . . 359
Drill 1 — Primary PE session loss . . . 360
Predict → observe → fix . . . 360
Drill 2 — Force VRRP failover . . . 360
Predict . . . 360
Drill 3 — Policy rejection . . . 361
Predict . . . 361
Drill 4 — Name/time smoke (hygiene) . . . 361
verify.sh . . . 361
Definition of done (this lab) . . . 361

Optional SR Linux PE . . . 362
Common mistakes . . . 362
Summary . . . 362

DNS Authoritative and Recursive 363

DNS Authoritative and Recursive 364
Learning goals . . . 364
Why roles matter . . . 364
Concepts — recursive vs authoritative . . . 365
Authoritative . . . 365
Recursive . . . 365
Stub . . . 365
Concepts — SOA, NS, glue . . . 366
Concepts — caching . . . 366
Concepts — DNSSEC at operator level . . . 367
Addressing plan (DNS roles lab) . . . 367
Topology YAML . . . 367
Authoritative zone (CoreDNS sketch) . . . 368
Direct auth checks (no recursion) . . . 369
Recursive (Unbound) sketch . . . 369
dig verification workflows . . . 370
1. Happy path via recursive . . . 370
2. Prove auth is the source of truth . . . 370
3. Trace mindset (public names) . . . 370
4. Glue / NS consistency . . . 370
5. DNSSEC operator glance (public) . . . 371
Predict → observe → fix . . . 371
Scenario A — client NXDOMAIN for `www.lab.example` . . . 371
Scenario B — auth works, recursive SERVFAIL . . . 371
Scenario C — "update not visible" . . . 371
Scenario D — AA vs AD confusion . . . 371
Verification checklist . . . 372
Common mistakes . . . 372
Summary . . . 372

QUIC Version 2 and Compatible Version Negotiation 373

QUIC Version 2 and Compatible Version Negotiation 374
Learning goals . . . 374
Concepts — why a "version 2" that does nothing new? . . . 374
Concepts — Compatible VN vs Incompatible VN . . . 375
Version Information (transport parameter 0x11) . . . 376
Packet-version rules after a Compatible switch (v1 v2) . . . 376
When v2 matters vs when it does not . . . 377
Worked observation — Firefox and Compatible VN . . . 377
Try this (browser) . . . 378
Mental model — one diagram . . . 378

The trap . . . 378
The boring rule . . . 379
Sources . . . 379

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft) 380

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft) 381
Learning goals . . . 381
Concepts in one page . . . 381
Build the lab . . . 382
Drill 1: three kinds of AAAA, one client that races . . . 382
Drill 2: the same name, a client that does not race . . . 383
Drill 3: read the race on the wire . . . 384
What version 3 changes . . . 384
The trap . . . 385
The boring rule . . . 385
Try this . . . 386
Sources . . . 386

QUIC Load Balancing with nftables: Routing on Connection IDs 387

QUIC Load Balancing with nftables: Routing on Connection IDs 388
Learning goals . . . 388
Mental model . . . 388
The CID layout . . . 389
Build the lab . . . 390
Why not paste the published ruleset? . . . 396
Drill 1: handshakes stick to a client address . . . 398
Drill 2: established packets follow the server ID . . . 398
Drill 3: two connections, one 4-tuple, and why zones exist . . . 399
Drill 4: real QUIC, including migration . . . 400
Drill 5: drain be3, add be4, deregister be3, reset its clients . . . 401
The trap . . . 402
Notes and pitfalls . . . 404
The boring rule . . . 404
Try this . . . 404
Sources . . . 404

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499 406

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499 407
Learning goals . . . 407
Mental model . . . 407
Concepts — two halves of a QUIC stream . . . 409
Concepts — which HTTP/3 error code . . . 410
Case — what a proxy must do . . . 411
Case — cancel-safe origins . . . 412
Lab — cancel mid-response on localhost . . . 412
The trap . . . 421

The boring rule . . . 421
Try this . . . 421
Sources . . . 422

Policy, QoS, and Hardening **423**

ACL Thinking **424**

ACL Thinking **425**
Learning goals . . . 425
Concepts . . . 425
Three different "filters" people mix up . . . 425
Policy statement template . . . 425
First-match vs best-match . . . 426
Stateful vs stateless . . . 426
Direction and attachment . . . 426
Written policy for the lab . . . 426
Topology YAML . . . 427
Addressing . . . 428
Implement with iptables (readable teaching form) . . . 428
nftables sketch (same policy) . . . 429
Predict → observe → fix drills . . . 430
Drill 1 — Allowed path . . . 430
Drill 2 — Denied path . . . 430
Drill 3 — SSH source restriction . . . 430
Drill 4 — Asymmetric paths . . . 430
Drill 5 — Wrong plane . . . 431
FRR: route policy is still "ACL thinking" . . . 431
Object-group idea (vendor-agnostic) . . . 431
Change control for ACLs . . . 431
Verification checklist . . . 432
Common mistakes . . . 432
Summary . . . 432

Control-Plane Protection **433**

Control-Plane Protection **434**
Learning goals . . . 434
Concepts . . . 434
Two paths through a router . . . 434
Why unprotected CPUs melt . . . 435
Policy matrix (start simple) . . . 435
Hardening adjacent controls . . . 435
Lab topology . . . 436
Addressing . . . 437
Baseline routing (static snack) . . . 437
Linux receive-path protection (teaching CoPP) . . . 437

Predict → observe → fix . . . 438
Drill 1 — Transit still works under "attack" to RE . . . 438
Drill 2 — SSH from non-mgmt . . . 439
Drill 3 — Over-tight BGP protection . . . 439
Drill 4 — Peer-only OSPF . . . 439
FRR-side levers (awareness) . . . 439
Building a CoPP document . . . 439
IPv6 notes . . . 440
Optional SR Linux / NOS CoPP . . . 440
Verification checklist . . . 440
Common mistakes . . . 440
Summary . . . 441

QoS Vocabulary **442**

QoS Vocabulary **443**
Learning goals . . . 443
Concepts — the pipeline . . . 443
Vocabulary table . . . 444
DSCP / PHB (operator view) . . . 444
Policing vs shaping . . . 445
Linux lab topology . . . 445
Addressing . . . 445
Experiment 1 — bottleneck without QoS . . . 446
Predict . . . 446
Observe . . . 446
Experiment 2 — two classes with HTB . . . 446
Predict . . . 447
Observe . . . 447
Fix . . . 447
Experiment 3 — police vs shape feel . . . 447
Marking sketch (DSCP) . . . 447
Predict . . . 448
Observe . . . 448
What lab QoS cannot prove . . . 448
FRR / NOS note . . . 448
Design mini-checklist . . . 448
Predict → observe → fix template . . . 449
Common mistakes . . . 449
Summary . . . 449

Hardening Checklists **450**

Hardening Checklists **451**
Learning goals . . . 451
How to use these lists . . . 451
Checklist 0 — Lab hygiene (every topology) . . . 451

Checklist 1 — Management plane . . . 452
Linux probes . . . 452
Checklist 2 — Control plane & routing . . . 452
FRR probes . . . 453
Checklist 3 — Data plane filters . . . 453
Checklist 4 — Edge services . . . 453
Checklist 5 — Secrets, users, supply chain . . . 454
Checklist 6 — Resilience & abuse . . . 454
Role profiles (quick) . . . 454
Lab host (Containerlab hypervisor) . . . 454
FRR edge CE . . . 454
FRR P / fabric leaf . . . 455
Linux bridge "switch" . . . 455
Exception register template . . . 455
Automating gates — `verify-hard.sh` . . . 455
Predict → observe → fix (meta-drill) . . . 456
Mapping to prior chapters . . . 456
Optional SR Linux . . . 456
Definition of done — hardened lab . . . 457
Common mistakes . . . 457
Summary . . . 457

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow 458

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow 459
Learning goals . . . 459
Mental model . . . 460
Concepts — what a SYN cookie is . . . 461
Lab — setup: a private network namespace with a void . . . 461
Lab — accept queue: a server that never calls `accept()` . . . 463
Lab — `somaxconn` silently caps the backlog . . . 465
Lab — a spoofed SYN flood, three ways . . . 465
Lab — what a cookie connection keeps . . . 468
Case — reading a production listener . . . 469
The trap . . . 470
The boring rule . . . 470
Try this . . . 471
Verification . . . 471
Sources . . . 471

Overlays and Tunnels 473

Why Encapsulate 474

Why Encapsulate 475
Learning goals . . . 475
The core picture . . . 475
Why people encapsulate . . . 476

Why *not* to encapsulate . . . 476
Encapsulation as Russian dolls . . . 477
Identifiers you must name . . . 477
Failure domains . . . 478
MTU tax . . . 478
ECMP and entropy . . . 479
Control plane vs data plane (again) . . . 479
Mini lab — see encapsulation with tcpdump . . . 479
GRE between a and b across under . . . 480
Predict → observe → fix . . . 480
Design questions checklist . . . 480
Mapping to the journey map . . . 481
Common mistakes . . . 481
Summary . . . 481

Site Tunnels **482**

Site Tunnels **483**
Learning goals . . . 483
When site tunnels are the right tool . . . 483
Topology — two sites + "Internet" . . . 484
Addressing plan . . . 484
Lab 1 — GRE + static routes . . . 485
Predict . . . 486
Observe . . . 486
Fix . . . 486
Lab 2 — MTU blackhole drill . . . 486
Predict . . . 486
Lab 3 — Underlay failure . . . 487
Predict . . . 487
Route-based vs policy-based VPN . . . 487
Lab 4 — WireGuard route-based (if packages available) . . . 487
Predict → observe . . . 488
FRR over GRE (dynamic optional) . . . 488
Policy-based IPsec awareness . . . 488
Security notes (vendor-agnostic) . . . 489
verify.sh sketch . . . 489
Common mistakes . . . 489
Summary . . . 489

WireGuard: Cryptokey Routing and the Pre-Shared Key **490**

WireGuard: Cryptokey Routing and the Pre-Shared Key **491**
Learning goals . . . 491
Concepts in one page . . . 491
Lab build . . . 492
Drill 1: cryptokey routing drops a perfectly encrypted packet . . . 494
Drill 2: a PSK mismatch, seen from the wire . . . 495

Drill 3: fix and rotate the PSK live . . . 496
Deploying it for real . . . 496
Verification checklist . . . 497
Common mistakes . . . 497
Summary . . . 497
Sources . . . 498

OpenVPN Data Channel Offload (DCO) 499

OpenVPN Data Channel Offload (DCO) 500
Thesis . . . 500
Mental model . . . 500
Worked examples . . . 501
Case 1: Confirm the kernel side before you touch configs . . . 501
Case 2: Let OpenVPN auto-enable DCO (and force it off for A/B) . . . 502
Case 3: One-sided DCO still helps . . . 503
Case 4: Routing mental shift under DCO . . . 503
The trap . . . 503
The boring rule . . . 504
Try this . . . 504
Sources . . . 504

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch 505

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch 506
Learning goals . . . 506
Concepts in one page . . . 506
Worked examples . . . 507
Case 1: Two tunnels, one default — the collision . . . 507
Case 2: `fwmark` + policy routing (single full tunnel) . . . 507
Case 3: Multi-tunnel with distinct marks . . . 508
Case 4: Kill switch = firewall, not WireGuard . . . 509
Case 5: Namespace alternative for “all traffic through one tunnel” . . . 509
The trap . . . 509
The boring rule . . . 510
Try this . . . 510
Sources . . . 510

VXLAN Data Plane 511

VXLAN Data Plane 512
Learning goals . . . 512
Concepts . . . 512
Roles . . . 512
Header (mental) . . . 513
Data-plane only learning . . . 513
Multicast vs head-end replication . . . 513
When VXLAN is appropriate . . . 513
Topology — two VTEPs + underlay + hosts . . . 513

Addressing . . . 514
VXLAN + bridge on leaf1 . . . 515
Deploy checks . . . 516
Predict . . . 516
Observe . . . 516
Fix . . . 516
Drill — MTU . . . 517
Predict . . . 517
Drill — wrong VNI isolation . . . 517
Predict . . . 517
Drill — MAC move / learning . . . 517
Predict . . . 517
Flood scope risk . . . 518
FRR / SR Linux awareness . . . 518
Verification checklist . . . 518
Common mistakes . . . 518
Summary . . . 518

EVPN Control-Plane Intro **520**

EVPN Control-Plane Intro **521**
Learning goals . . . 521
Why EVPN exists . . . 521
Objects (vendor-agnostic) . . . 522
Control vs data verification split . . . 522
Minimal design story — 2 leaf . . . 522
FRR capability note (mid-2026) . . . 523
Topology sketch (EVPN lab) . . . 523
RR config idea . . . 524
Predict → observe → fix drills . . . 525
Drill 1 — Session before service . . . 525
Drill 2 — Host MAC appears as EVPN route . . . 525
Drill 3 — Kill EVPN, keep underlay . . . 525
Drill 4 — RT mismatch (concept) . . . 525
L2 VNI vs L3 VNI (awareness) . . . 525
Optional SR Linux community path . . . 526
Security / isolation checklist . . . 526
What this intro deliberately skips . . . 526
Common mistakes . . . 527
Summary . . . 527

Overlay Lab **528**

Overlay Lab **529**
Learning goals . . . 529
Pick a track . . . 529
Definition of done (all tracks) . . . 529

Track A — Dual-site GRE (reference build) . . . 530
Success metrics . . . 530
verify.sh . . . 530
Failure matrix A . . . 531
Track B — VXLAN two-leaf (reference build) . . . 531
Success metrics . . . 531
bringup excerpt . . . 531
verify.sh B . . . 531
Failure matrix B . . . 532
Capture helper . . . 532
Track C — EVPN intro (stretch) . . . 532
Option 1: Netlab EVPN-VXLAN Topology (Recommended) . . . 532
Option 2: Manual FRR Configuration . . . 533
Success metrics . . . 533
Failure matrix C . . . 534
Integrated “mini fabric snack” (optional merge) . . . 534
Observability expectations . . . 534
Hardening lite (overlay) . . . 534
Lab report template . . . 535
Predict → observe → fix (capstone habit) . . . 535
Time-box guidance . . . 535
Common mistakes . . . 535
Summary . . . 536

Fabrics and Multi-Area **537**

Clos Leaf-Spine **538**

Clos Leaf-Spine **539**
Learning goals . . . 539
Why Clos-style fabrics . . . 539
Roles . . . 540
Oversubscription (awareness) . . . 540
Underlay protocol choices . . . 541
Addressing plan (2×2) . . . 541
Topology YAML . . . 541
FRR daemons . . . 543
OSPF underlay sketch (l1) . . . 543
eBGP underlay sketch (optional alternate) . . . 544
Deploy and ECMP verify . . . 544
Predict . . . 544
Observe multipath . . . 544
Drill 1 — Spine loss . . . 545
Predict → observe → fix . . . 545
Drill 2 — One leaf uplink down . . . 545
Predict . . . 545
Drill 3 — Polarization awareness . . . 545

East-west vs north-south . . . 546
Overlay placement (preview) . . . 546
Verification checklist . . . 546
Common mistakes . . . 546
Summary . . . 547

Multi-Area OSPF **548**

Multi-Area OSPF **549**
Learning goals . . . 549
Concepts . . . 549
Why areas . . . 549
Backbone rule (practical) . . . 549
Roles . . . 550
LSA awareness (not flash cards) . . . 550
Summarization . . . 550
Topology . . . 550
Topology YAML . . . 551
FRR configs . . . 552
r1 (area 1) . . . 552
r3 (area 2) . . . 552
r2 (ABR) . . . 552
Deploy and baseline . . . 553
Predict . . . 553
Observe . . . 553
Drill 1 — Summarization blackhole . . . 554
Predict . . . 554
Fix . . . 554
Drill 2 — ABR failure . . . 554
Predict . . . 554
Drill 3 — Area mismatch . . . 555
Predict . . . 555
Stub area ideas (awareness) . . . 555
Multi-area vs fabric BGP . . . 555
Verification checklist . . . 556
Common mistakes . . . 556
Summary . . . 556

Fabric with FRR **557**

Fabric with FRR **558**
Learning goals . . . 558
Lab repo layout . . . 558
Design defaults for this book . . . 559
eBGP unnumbered (modern fabric style) . . . 559
Numbered eBGP fabric (reliable teaching) . . . 559
Spine s1 excerpt . . . 560
Leaf l1 excerpt . . . 560

OSPF alternate (same wiring) . . . 561
Declarative Fabric with Netlab (Fast & Scalable) . . . 561
Deploy and Verify with Netlab . . . 562
Manual Containerlab & FRR Implementation . . . 562
Containerlab topology (`clos22.clab.yml`) . . . 563
Operational show kit . . . 563
verify.sh . . . 563
Failure scripts . . . 564
Predict → observe → fix . . . 564
Policy on fabric (light) . . . 564
Adding overlay later (hook points) . . . 564
Scale checklist 2×2 → 4×4 . . . 565
Observability . . . 565
Optional SONiC / SR Linux leaf . . . 565
Common mistakes . . . 565
Summary . . . 566

Multi-Implementation Lab **567**

Multi-Implementation Lab **568**
Learning goals . . . 568
Design principle . . . 568
Recommended topology . . . 569
Addressing / ASN (fixed) . . . 569
Topology YAML (FRR + optional srl) . . . 569
FRR side . . . 570
SR Linux mapping (conceptual) . . . 571
Verification questions (dialect-free) . . . 571
FRR answers . . . 571
SR Linux answers (illustrative) . . . 572
predict → observe → fix matrix . . . 572
verify.sh (heterogeneous) . . . 572
Overlay optional add-on . . . 572
Hardening gates (reuse part 08) . . . 573
Lab report template . . . 573
When interop hurts . . . 573
Common mistakes . . . 574
Summary . . . 574

Networking

Personal notes on learning **computer networking from first principles to expert NetOps**, with labs built as **code** in Containerlab using **open-source and free network images** only (FRR, Linux, optional SR Linux / SONiC / VyOS).

> **Note**
>
> This book is **vendor-agnostic**. CLIs differ; protocols do not. Start with the **lab platform** (macOS/Linux + Containerlab), then a deep **networking foundations** pass (TCP/IP, addressing, core protocols), then L2 → L3 → routing → edge → policy → overlays → fabrics → ops. Topology figures are illustrated SVGs under `images/`.

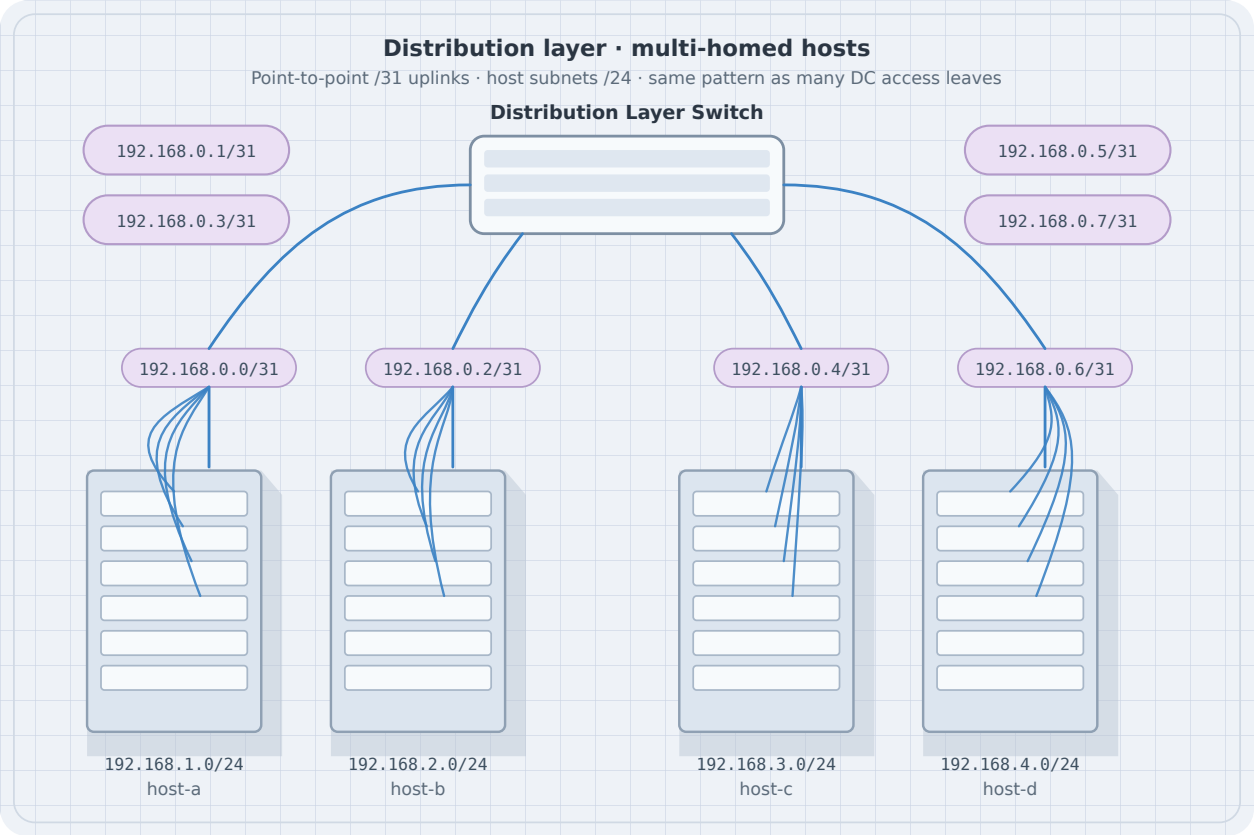


Figure 1: Distribution switch with multi-homed servers

Curriculum map

Part	Focus
01 Lab platform	macOS/Linux host setup, Containerlab, free images, host networking, hygiene
02 Networking foundations	TCP/IP stack, packets, Ethernet intro, IPv4/IPv6, ARP/ND, ICMP, TCP/UDP, common apps, host routing, planes
03 Layer 2	Ethernet/MAC deep, VLANs, loops, LAG, failure drills
04 Layer 3	Advanced addressing/routing labs, statics, ICMP/PMTU in multi-hop designs
05 Interior routing	OSPF, BGP, policy & filters
06 Ops & automation	Observability, lab-as-code, capstone outline
07 Edge & services	First-hop HA, DHCP/NAT in edge designs, hygiene, dual-home lab
08 Policy, QoS, hardening	ACLs, CoPP, QoS vocabulary, hardening
09 Overlays & tunnels	Encap, site tunnels, VXLAN, EVPN intro
10 Fabrics & multi-area	Clos, multi-area OSPF, FRR fabric, multi-NOS

Journey map

```
Lab platform (macOS / Linux + Containerlab)
  → Networking foundations (TCP/IP, addressing, protocols)
  → Layer 2 depth
  → Layer 3 / routing depth
  → Interior routing (OSPF, BGP)
  → Edge & services
  → Policy / QoS / hardening
  → Overlays & tunnels
  → Fabrics
  → Ops & automation
```

What “done” looks like

- Design multi-site L2/L3 networks with clear underlay vs overlay roles
- Implement them in Containerlab with free images
- Prove behavior with tables, neighbors, and packet captures
- Break links and policies on purpose; restore and document
- Automate lab lifecycle

Lab ground rules

- Prefer **free/open** node kinds (Containerlab + FRR + Alpine/Debian)
- Every serious lab: topology file + addressing plan + failure scenario
- Capture evidence when it teaches something tables do not

Related books

Book	Overlap
Linux	Host networking commands (`ip`, `ss`, `tcpdump`, firewalls)
NixOS	Declarative network config on NixOS hosts
Go	`net/http`, probes, and load-test projects
VCS	Lab-as-code repos and CI for topology files

Formats

HTML, PDF, and EPUB via the monorepo portal when built.

Lab Platform

Lab Host — macOS

Lab Host — macOS

Prepare a **macOS** machine as the Containerlab control host: virtualization, container runtime, resources, and networking gotchas before you deploy topologies.

> **Note**
>
> **Stub chapter** — outline for the lab-platform on-ramp. Expand with exact install commands and verified versions as you freeze a lab baseline.

Goals

- Run Docker-compatible containers with enough CPU/RAM for FRR + Linux nodes
- Install Containerlab and pull free lab images
- Understand macOS-specific limits (VM layer, file sharing, port publishing)

Prerequisites

- Recent macOS with virtualization support (Apple Silicon or Intel)
- Admin rights for Docker/Colima and Homebrew
- Disk space for images (plan multi-GB for NOS images)

Virtualization and container runtime

Options

Approach	Notes
Docker Desktop	Simple GUI; license/terms apply for some orgs
Colima + Docker CLI	Lightweight VM; common for CLI-first labs
Lima / other	Possible; keep one path documented for this book

Checklist

1. Install runtime and confirm `docker version` / `docker info`
2. Allocate CPU and memory for multi-node labs (start modest; scale up)
3. Confirm architecture (`linux/arm64` vs `linux/amd64`) matches images you pull

Install Containerlab

- Prefer the Containerlab install docs for the current macOS method
- Verify: `containerlab version`
- Confirm the binary can talk to your Docker socket

Networking notes on macOS

- Lab nodes run **inside a Linux VM** — host `localhost` port maps differ from native Linux
- Bridge and macvlan behaviors are not identical to bare-metal Linux
- Prefer topologies that do not assume bare-metal SR-IOV or host netns tricks on day one

First smoke test

1. Deploy a **two-node** FRR or Linux topology from Containerlab examples
2. `containerlab inspect` / `docker ps` — nodes up
3. `docker exec` into a node; `ping` the peer
4. Destroy the lab; confirm no orphan containers

Next

- Parallel path: Lab Host — Linux
- Then Containerlab Fundamentals

Lab Host — Linux

Lab Host — Linux

Prepare a **Linux** machine (bare metal or VM) as the Containerlab control host: kernel/container runtime, permissions, and host networking.

> **Note**
>
> **Stub chapter** — outline for the lab-platform on-ramp. Expand with distro-specific package names as you freeze a baseline (Debian/Ubuntu recommended for docs).

Goals

- Install Docker or a Containerlab-supported runtime
- Install Containerlab and free images
- Avoid permission and firewall traps that break lab deploy

Prerequisites

- 64-bit Linux with virtualization if nested (for some images)
- Root or passwordless sudo for install steps
- Enough RAM/disk for your target topology scale

Container runtime

Typical path (Debian/Ubuntu-style)

1. Install Docker Engine (or Podman only if you know Containerlab support for your version)
2. Add your user to the `docker` group (or use root consistently in labs)
3. Enable and start the service; `docker info` succeeds

Kernel / modules

- Bridge, veth, and overlay-related features must work
- Nested virt: confirm if you need it for chosen NOS images

Install Containerlab

- Follow Containerlab install for Linux
- Verify: `containerlab version`
- Optional: bash completion

Host networking and firewall

- Local firewalls (ufw, firewalld, nftables) can block management or published ports
- Don't disable security blindly — open what the lab needs
- IP forwarding may be required for some designs later

First smoke test

1. Deploy a minimal two-node topology
2. Confirm links and management connectivity
3. Capture with `tcpdump` on a veth if useful
4. `containerlab destroy -t <file>` — clean teardown

Next

- macOS path: Lab Host — macOS
- Then Containerlab Fundamentals

Containerlab Fundamentals

Containerlab Fundamentals

Containerlab turns a YAML topology into runnable network labs: create nodes, wire virtual links, attach a management network, and destroy everything cleanly. This chapter is the operator's manual for the rest of the book's labs.

Learning goals

By the end of this chapter you can:

- Read and write a minimal `*.clab.yml` topology
- Deploy, inspect, connect, and destroy labs
- Choose free node **kinds** (FRR, Linux, etc.)
- Understand management network vs data-plane links
- Export a graph and use consistent naming
- Recover from partial deploys and leftover state

Prerequisites

- Linux host or VM with nested virt as needed (Docker or podman per Containerlab docs)
- `containerlab` installed (install guide)
- Permission to run privileged containers / sudo as required
- Images pullable from public registries for free kinds

Check:

```
containerlab version
docker version   # or podman
```

How Containerlab fits together

```
topology.clab.yml


 containerlab deploy

         create containers (nodes)
         create veth/links between interfaces
         attach management network (SSH/mgmt IPs)
         optional startup configs / binds
```

Destroy reverses the graph. **Source of truth is the YAML** (plus mounted configs), not hand edits inside nodes.

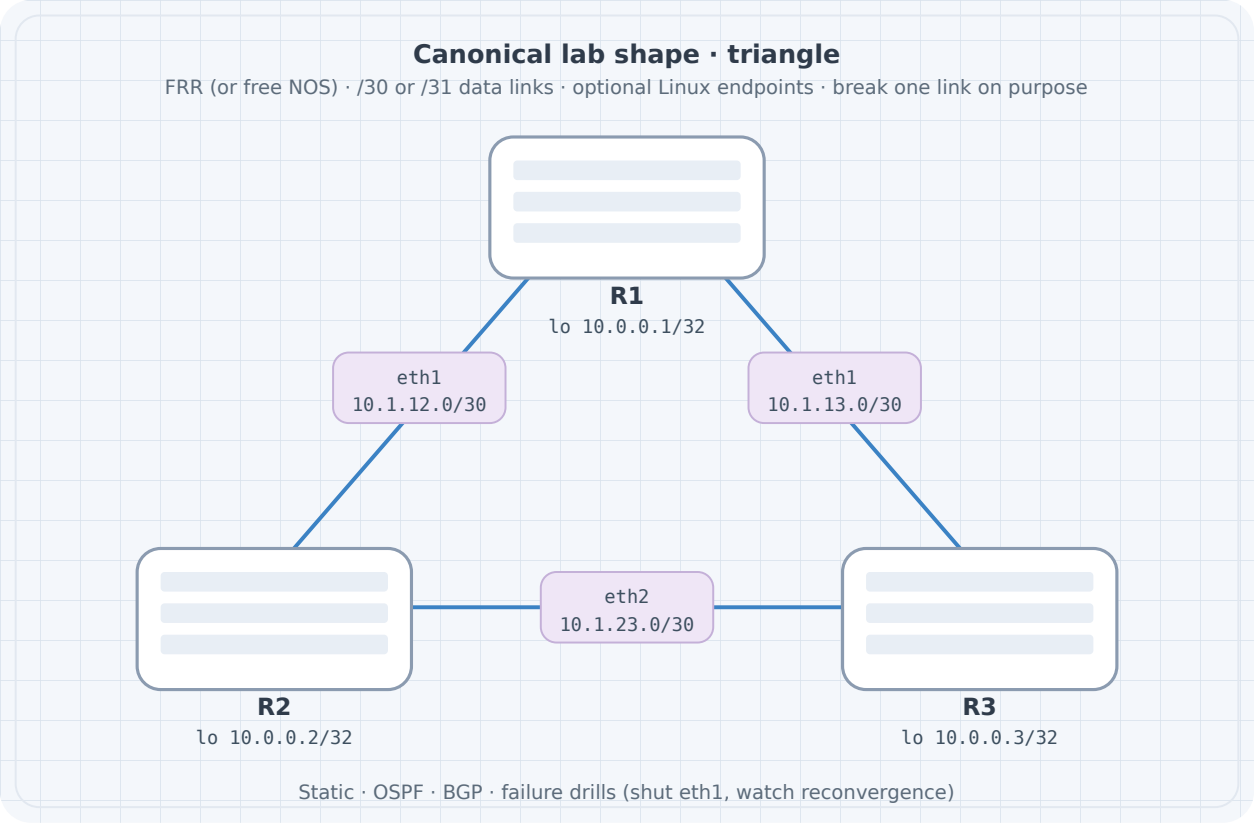


Figure 1: Triangle lab

Minimal topology: two Linux nodes

```
name: twoping

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
    h2:
      kind: linux
      image: alpine:3.20

  links:
    - endpoints: ["h1:eth1", "h2:eth1"]
```

Save as `twoping.clab.yml`.

```
sudo containerlab deploy -t twoping.clab.yml
sudo containerlab inspect -t twoping.clab.yml
```

Address the link and test (interfaces may need addressing—Alpine is raw):

```
# Names often: clab-twoping-h1
docker exec -it clab-twoping-h1 sh
# inside:
ip addr add 10.0.0.1/24 dev eth1
ip link set eth1 up
# on h2:
# ip addr add 10.0.0.2/24 dev eth1 && ip link set eth1 up
# ping 10.0.0.1
```

For real labs you bake addressing into startup scripts or use NOS images that apply configs. Pattern: **deploy → configure via files → verify**.

Triangle of FRR routers (book default pattern)

FRR containers give you a real control plane without licenses.

```
name: triangle

topology:
  nodes:
    r1:
      kind: linux
```

```
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
```

Note: Pin image tags deliberately (example tag may differ—check current FRR container tags). Each node mounts its own `./config/rN` to `/etc/frr`.

Startup config sketch (`config/r1/daemons` + `frr.conf`)

FRR images expect daemons enabled. Example `frr.conf` ideas (interface names must match):

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.0.12.1/24
!
interface eth2
 ip address 10.0.13.1/24
!
ip route 0.0.0.0/0 10.0.12.2
!
line vty
```

Exact FRR container entrypoint behavior evolves—follow the image README. The **topology pattern** (binds + links) is the durable skill.

Lifecycle

```
sudo containerlab deploy -t triangle.clab.yml
sudo containerlab graph -t triangle.clab.yml   # if supported in your version
sudo containerlab inspect -t triangle.clab.yml
docker exec -it clab-triangle-r1 vtysh
sudo containerlab destroy -t triangle.clab.yml --cleanup
```

Topology YAML anatomy

Key	Purpose
`name`	Lab name prefix for containers/namespaces
`topology.nodes`	Node inventory
`topology.links`	Point-to-point (or multipoint via extras)
`kind`	How Containerlab treats the node
`image`	Container image
`binds`	Host path mounts
`exec`	Commands after start (use sparingly; prefer files)
`env`	Environment variables
`ports`	Publish container ports to host (mgmt tools)
`mgmt-ipv4`	Static management address (optional)

Kinds you will actually use

Kind / image pattern	Role
`linux` + Alpine/Debian	Hosts, simple routers with FRR image
FRR container on `linux` kind	OSPF/BGP labs
`srl` (SR Linux community)	Modern NOS practice (free image path)
VyOS / SONiC images	Optional diversity

Always confirm **license-free** use for the image you pull. This book never requires paid virtual NOS licenses.

Links and endpoints

```
  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
```

Rules of thumb:

- Interface names must match what the NOS expects or what you configure
- Point-to-point links are veth pairs under the hood
- Do not reuse the same interface name twice on one node
- Leave `eth0` for management if the kind uses it for mgmt

Multipoint / LAN segments

Patterns include attaching multiple nodes via a bridge node or kind-specific multipoint links. Early labs stick to **point-to-point** for clarity; L2 chapters introduce switch nodes.

Management network

Containerlab creates a management network so you can SSH/API to nodes without using data-plane links.

```
sudo containerlab inspect -t triangle.clab.yml
# Note mgmt IPv4 addresses
ssh admin@<mgmt-ip>   # credentials depend on image
```

Predict: Breaking a data-plane link does not remove SSH via mgmt (unless you broke the host docker network). That separation mirrors production OOB vs in-band—use it.

Inspect, exec, save

```
# Table of nodes and mgmt IPs
sudo containerlab inspect -t triangle.clab.yml

# Shell
docker exec -it clab-triangle-r1 bash

# FRR
docker exec -it clab-triangle-r1 vtysh -c 'show interface brief'

# Destroy
sudo containerlab destroy -t triangle.clab.yml --cleanup
```

`--cleanup` removes lab containers and lab-specific wiring; use it to avoid ghost interfaces.

Lab directory layout (recommended)

```
labs/triangle/
  triangle.clab.yml
  README.md
  addressing.md
  verify.sh
  config/
    r1/
    r2/
    r3/
  captures/
```

```
cd labs/triangle
sudo containerlab deploy -t triangle.clab.yml
./verify.sh
```

Graph and documentation

```
sudo containerlab graph -t triangle.clab.yml
```

Import diagrams into your journal. Combine with a hand-drawn addressing plan—graph alone lacks subnets.

Partial failure recovery

If deploy dies halfway:

```
sudo containerlab destroy -t triangle.clab.yml --cleanup
docker ps -a | grep clab
# remove stragglers if needed
sudo containerlab deploy -t triangle.clab.yml
```

Also check:

```
docker images | head
df -h          # disk full breaks pulls
ip link | wc -l  # too many leftover veths
```

Predict → observe → fix lab

Setup

Deploy `twoping` with addresses applied via `exec` for speed (acceptable for learning; later prefer binds):

```
name: twoping

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - ip addr add 10.0.0.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - ip addr add 10.0.0.2/24 dev eth1
        - ip link set eth1 up
  links:
    - endpoints: ["h1:eth1", "h2:eth1"]
```

Note: `exec` timing can race interface creation on some versions—if eth1 missing, retry commands or use a small sleep in a script bind.

Predict

- `ping 10.0.0.2` from h1 succeeds
- `ip neigh` shows REACHABLE for peer

Observe

```
docker exec clab-twoping-h1 ping -c 3 10.0.0.2
docker exec clab-twoping-h1 ip neigh
docker exec clab-twoping-h1 ip -br a
```

Fix inject

```
docker exec clab-twoping-h2 ip link set eth1 down
docker exec clab-twoping-h1 ping -c 2 10.0.0.2  # fail
docker exec clab-twoping-h2 ip link set eth1 up
```

Harden

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-twoping-h1 ping -c 2 -W 1 10.0.0.2
echo OK
```

Re-deploy from clean destroy and ensure `verify.sh` still passes.

Common mistakes

Mistake	Symptom	Fix
Wrong interface name in links	Deploy error or silent no-path	Match kind docs
Editing live node only	Lost on destroy	Binds + git
Forgetting `--cleanup`	Ghost links/names	Always cleanup when done
Huge images, tiny laptop	OOM kills	Smaller kinds, fewer nodes
Mixing lab names	Container name collisions	Unique `name:` per active lab

Version awareness (mid-2026)

Containerlab releases steadily. Skills that stay:

- Topology as code
- kinds + images + links + binds
- deploy / inspect / destroy lifecycle

Syntax extras (stages, components, interfaces block style) may appear in newer docs—read containerlab.dev for your installed version when YAML fails validation.

```
containerlab version
containerlab help deploy
```

Security and shared hosts

- Labs often run privileged networking—isolate lab hosts
- Do not expose mgmt SSH to the internet
- Pull images from trusted registries; pin digests for CI later

Summary

- Containerlab materializes YAML into namespaces, containers, and links
- Master deploy → inspect → exec → destroy –cleanup
- Prefer binds/startup configs over irreproducible exec-only hacks
- Keep per-lab directories with verify scripts
- FRR + Linux free images cover most of this book's spine

Next: **Netlab Fundamentals**—how to layer a declarative intent compiler on top of Containerlab to automate IP addressing, protocol configurations, and validation suites.

Netlab Fundamentals

Netlab Fundamentals

While Containerlab excels at spinning up containers and wiring virtual Ethernet pairs, running real multi-node routing topologies requires hundreds of lines of boilerplate device configurations, IP address spreadsheets, and manual routing daemon setup.

Netlab (developed by Ivan Pepelnjak) solves this problem by providing a **declarative network intent and configuration compiler**. Rather than replacing Containerlab, Netlab uses Containerlab as its primary virtualization provider (`provider: clab`). You describe *what* you want in a high-level YAML topology; Netlab automatically plans IP subnets, assigns loopback and transit addresses, generates vendor-specific configurations (FRR, Arista cEOS, Nokia SR Linux, Cisco XRd), and instructs Containerlab to deploy the lab.

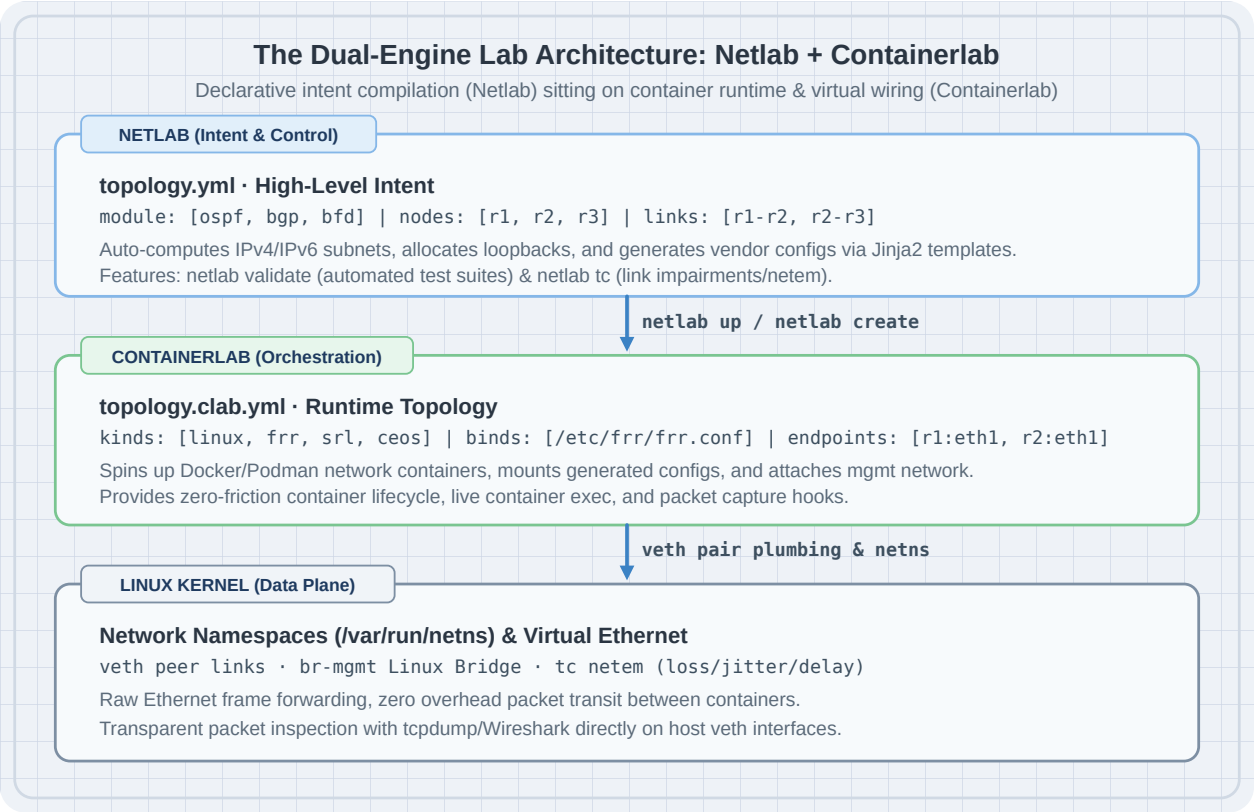


Figure 1: Dual-Engine Lab Architecture: Netlab + Containerlab

Learning goals

By the end of this chapter, you will be able to:

- Articulate the distinction between the orchestration layer (Containerlab) and the intent/configuration layer (Netlab).
- Install and configure Netlab with Containerlab on a Linux system.
- Write concise declarative topology files (`topology.yml`) with automated module activation (OSPF, BGP, BFD, VLANs).
- Inspect compiled artifacts (`netlab create`) to understand the underlying Containerlab topology and generated NOS configuration files.
- Operate the full lifecycle: `netlab up`, `netlab connect`, `netlab inspect`, and `netlab down`.
- Run automated verification test suites using `netlab validate`.
- Inject realistic network impairments (latency, jitter, packet loss) using `netlab tc`.
- Swap network operating systems (e.g., FRR to Arista cEOS or Nokia SR Linux) with a single configuration parameter.

The Dual-Engine Model: Why Both Tools Matter

When learning networking from scratch, choosing between Containerlab and Netlab is not an either/or dilemma. They serve distinct, complementary purposes across your learning journey:

Capability	Containerlab Direct (`*.clab.yml`)	Netlab with Containerlab (`topology.yml`)
Mental Model	Physical / Virtual link plumbing (`veth` pairs & namespaces)	Network architecture & protocol intent
IP Addressing	Completely manual; you must assign every IP on every interface	Fully automated IPAM; computes /30, /31, or /24 transit and /32 loopbacks
Protocol Config	Manual; you author and mount `/etc/frr/frr.conf` or vendor configs	Automated; Jinja2 templates render native NOS configs from modules
Ideal For	L1/L2 mechanics, raw ARP, custom bridges, packet crafts	OSPF, BGP, EVPN-VXLAN, multi-area fabrics, failure drills
Validation	Manual CLI checks and `docker exec` commands	Automated testing assertions with `netlab validate`
NOS Portability	Low; changing NOS requires rewriting all config files	High; change `device: frr` to `device: eos` or `device: srlinux`

Curriculum Rule of Thumb:
Use **raw Containerlab** for visceral wire-level understanding in early foundation chapters (inspecting ARP frames, Linux bridges, MTU fragmentation). Transition to **Netlab** when studying dynamic routing protocols, transit policies, and data center fabrics to eliminate syntax fatigue and focus on architectural principles.

Installation & Setup

Netlab is distributed as a Python package (`network-lab`) that interacts directly with Docker and Containerlab.

Prerequisites

Netlab requires Python 3.9+, Docker (or Podman), and Containerlab. On Ubuntu/Debian:

```
# Ensure Python virtual environment tools and Ansible core are installed
sudo apt update
sudo apt install -y python3-pip python3-venv git

# Create a dedicated virtual environment or install via pipx / pip
python3 -m pip install --upgrade network-lab
```

Automated Platform Provisioning

Netlab includes a self-bootstrapping command that verifies system requirements, installs Containerlab, configures Docker, and sets the required user groups (`docker` and `clab_admins`):

```
# Verify installation prerequisites and dependencies
netlab check

# Automatically install or update Containerlab and dependencies
sudo netlab install containerlab
```

Verify your active environment:

```
netlab test clab
```

This command deploys a two-node lightweight container lab, validates bidirectional connectivity over a virtual link, and tears down the lab cleanly.

The Anatomy of a Netlab Topology

A Netlab topology is saved by default as `topology.yml` in your working directory.

Minimal Triangle Topology with OSPF

Here is the exact equivalent of the three-router OSPF triangle that required three separate configuration files and complex volume binds in raw Containerlab:

```yaml
# topology.yml
---
provider: clab
defaults.device: frr

module: [ ospf ]

nodes:
  r1:
  r2:
  r3:

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:
```

What Happens Behind the Scenes?

When you run `netlab up`, the Netlab compiler executes a multi-step pipeline:

```
topology.yml


1. Topology Transformation (Assign IP subnets, loopbacks, router IDs, interface names)


2. Jinja2 Config Generation (Render /etc/frr/frr.conf and daemons for r1, r2, r3)


3. Containerlab File Generation (Emit clab.yml with nodes, kinds, mounts, and links)


4. Containerlab Orchestration (sudo containerlab deploy -t clab.yml)


5. Initial Configuration Push (Mount configs or apply via daemon sockets)
```

To see the compiled Containerlab YAML without starting containers, run:

```
netlab create
```

Inspect the generated artifacts in the current directory: - `clab.yml`: The generated Containerlab topology. - `hosts.yml` / `ansible.cfg`: Automation inventory for Ansible and Python scripts. -

`config/`: The auto-generated initial configurations for each device (e.g., `config/r1/frr.conf`).

Operating Netlab Labs

Netlab abstracts away container naming conventions (`clab-<labname>-<nodename>`) and provides uniform commands across any operating system.

1. Starting and Stopping Labs

```
# Create configuration, deploy containers, and configure devices in one step
netlab up

# Inspect running nodes, interfaces, and IPv4/IPv6 address assignments
netlab inspect

# Cleanly stop and destroy containers, veth pairs, and management links
netlab down
```

2. Connecting to Devices

You do not need to run `docker exec -it clab-... vtysh`. Netlab connects directly to the device CLI:

```
# Open an interactive CLI session on router r1 (starts vtysh on FRR)
netlab connect r1

# Run a non-interactive show command across a node
netlab connect r1 vtysh -c "show ip ospf neighbor"
```

3. Graphing Topologies

Netlab can automatically render visual diagrams of your topology in SVG or ASCII format:

```
# Generate a graphical visualization of the lab topology
netlab graph -o topology.png
```

Automated Verification with `netlab validate`

In enterprise NetOps, verifying that a network functions according to specification is done through automated tests rather than manual pings. Netlab provides an automated validation engine built directly into the topology file.

You can declare expected states under the `validate` key:

```
# topology.yml with validation rules
---
provider: clab
defaults.device: frr
module: [ ospf ]

nodes:
  r1:
  r2:
  r3:

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:

validate:
  ospf_neighbors:
    description: "Verify all routers reach Full OSPF adjacency"
    nodes: [ r1, r2, r3 ]
    plugin: ospf_neighbor(count=2, state='Full')

  loopback_reachability:
    description: "Verify end-to-end routing between loopbacks"
    nodes: [ r1 ]
    exec:
      r1: ping -c 2 10.0.0.2
      r1_to_r3: ping -c 2 10.0.0.3
```

Execute the test suite with a single command:

```
netlab validate
```

If an adjacency is stuck in `ExStart` due to an MTU mismatch or a broken link, `netlab validate` reports a red failure with actionable diagnostic outputs.

Simulating Real-World Links with `netlab tc`

Production networks suffer from latency, jitter, packet reordering, and drop rates. Netlab integrates Linux Traffic Control (`tc` / `netem`) directly into links.

You can define impairments declaratively in `topology.yml`:

```
links:
  - r1:
    r2:
    # Emulate an intercontinental WAN link
    bandwidth: 100mbps
    latency: 45ms
    jitter: 5ms
    loss: 0.5%
```

Or inject impairments dynamically during a live lab drill using the `netlab tc` CLI tool:

```
# Add 50ms latency to the link between r1 and r2
netlab tc set r1 eth1 --delay 50ms

# Inject 10% packet drop to observe BFD or OSPF dead timer behavior
netlab tc set r2 eth1 --loss 10%

# Clear all impairments
netlab tc clear r1 eth1
```

Multi-NOS Portability: The Ultimate Learning Superpower

One of the greatest advantages of Netlab is device abstraction. Dynamic routing protocols are vendor-agnostic standards, but vendor CLIs differ dramatically.

In Netlab, changing your entire lab from FRR to Arista EOS, Cisco IOS-XR, or Nokia SR Linux requires modifying one line:

```
# Change default device from FRR to Arista cEOS
defaults.device: eos
```

Or mixing different operating systems in the same topology:

```
nodes:
  r1:
    device: frr       # Linux FRR routing daemon
  r2:
    device: eos       # Arista cEOS
```

```
  r3:
    device: srlinux   # Nokia SR Linux
```

When you execute `netlab up`, Netlab applies standard OSPF configurations across all three distinct operating systems automatically. This enables you to focus on protocol interoperability, RFC compliance, and packet behavior without getting derailed by syntax differences.

Summary & What Comes Next

- **Containerlab** is your runtime foundation: it wires containers, creates Linux bridges, and provisions network namespaces.
- **Netlab** is your network compiler: it automates IP addressing, configures routing protocols, conducts automated testing, and simulates link conditions.
- Throughout this book:
  - Early chapters (Ethernet, ARP, VLANs, static routes) use **raw Containerlab and Linux containers** so you understand the raw kernel plumbing.
  - Advanced routing chapters (OSPF, BGP, EVPN-VXLAN, Clos fabrics) provide **Netlab topologies** as the primary recipe, alongside the compiled Containerlab configurations for complete architectural clarity.

Free Image Matrix & Resources

Free Image Matrix & Resources

The main path of this book uses **open-source and freely available** lab images only. No paid virtual NOS license is required to finish the curriculum. This chapter catalogs practical choices (as of mid-2026 practice) and how to size a lab host.

Learning goals

By the end of this chapter you can:

- Pick images for endpoints, routers, and optional modern NOS practice
- Estimate RAM/CPU for small vs medium topologies
- Avoid accidental license walls
- Pin versions for reproducibility
- Know where official docs and communities live

Principles

1. **Free to pull and run for learning** on the main path
2. **Documented interfaces** (Linux, FRR vtysh, open NOS models)
3. **Containerlab-friendly** kinds where possible
4. **Pin tags**—`latest` is for tourists

Always re-check the image's license and Containerlab kind docs before classroom or corporate use; policies change.

Role matrix

Role	Recommended default	Alternatives (still free/open path)
Endpoint host	`alpine:3.20` or Debian slim	Ubuntu minimal
Software router / FRR	`quay.io/frrouting/frr:<tag>`	FRR on Debian + manual setup
Modern NOS feel	Nokia **SR Linux** community image via Containerlab kind	SONiC community images
Firewall / edge Swiss army	VyOS rolling (confirm current license terms)	Linux nftables node
Switch-like L2	Linux bridge / OVS container; FRR + bridge	SR Linux L2 features
Traffic tools	Alpine/Debian + `iperf3`, `tcpdump`, `mtr`	Dedicated trafficgen images
Capture analysis	Host `tshark` / Wireshark	Container with tshark
Automation control	Host Python/Ansible	CI runner container

What we deliberately de-emphasize

Avoid on main path	Why
Paid IOS/NX-OS/JUNOS virts	License friction; concepts unchanged
Exam dump topologies	Wrong optimization target
Huge multi-vendor museums	RAM death; low learning density

Paid images can be a **later footnote** if your employer provides them—not a gate.

FRR as the workhorse

FRRouting implements OSPF, BGP, IS-IS, statics, and more with a Cisco-like vtysh. Container images are the fastest path:

```
# Example - check quay for current stable tags
docker pull quay.io/frrouting/frr:10.2.1
```

Strengths: control-plane labs, policy, dual-stack
Limits: not a full merchant silicon simulator; L2 features depend on how you combine Linux bridging

Lab pattern:

```
# snippet
r1:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/r1:/etc/frr
```

Enable only needed daemons (`bgpd=yes`, `ospfd=yes`) in `daemons` file to save RAM.

Linux endpoints

Alpine is small and fast:

```
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils busybox-extras
```

Debian if you need a richer package set:

```
h1:
  kind: linux
  image: debian:bookworm-slim
  exec:
    - apt-get update && apt-get install -y iproute2 iputils-ping tcpdump iperf3 mtr-tiny
```

Tip: bake a personal lab image with tools preinstalled to save time:

```
FROM alpine:3.20
RUN apk add --no-cache iproute2 iputils tcpdump iperf3 mtr bind-tools
```

```
docker build -t local/lab-endpoint:1.0 .
```

Use `local/lab-endpoint:1.0` in topologies.

SR Linux community (optional track)

Nokia SR Linux community images are commonly used with Containerlab for modern streaming telemetry and CLI/model exposure—without classic paid chassis IOS requirements. Follow current Containerlab **kind: srl** documentation for image names and credentials.

Use SR Linux when you want:

- YANG-ish / modern NOS workflows
- Leaf-spine demos closer to DC ops
- Comparison against FRR ("same OSPF idea, different dialect")

Do not block L2/L3 fundamentals waiting for SR Linux. FRR+Linux is enough for competence.

SONiC and VyOS (optional)

Image family	Good for	Watch for
SONiC	DC NOS ideas, BGP underlay practice	Image size, platform assumptions
VyOS	Edge router feature bundle	Release/license wording over time

Treat both as **breadth**, not blockers.

Sizing the lab host

Rough starting points (conservative; measure yours):

Lab scale	Nodes (example)	RAM ballpark	CPU
Tiny	2–3 Alpine	2–4 GB free	2
Small	3× FRR + 2 hosts	8 GB system	4
Medium	6–8 FRR/SRL + hosts	16 GB	6–8
Fabric snack	2 spine + 4 leaf + hosts	32 GB happier	8+

```
free -h
nproc
docker stats --no-stream
```

RAM savers

- Prefer Alpine endpoints over full Ubuntu desktops
- Disable unused FRR daemons
- Destroy labs when idle

- Avoid multiple concurrent large labs
- Do not run heavy desktop + large lab + browser zoo on 8 GB

Disk

Images accumulate:

```
docker system df
docker image prune
# careful:
# docker system prune
```

Pin and keep only tags you need.

Version pinning strategy

```
# Good
image: quay.io/frrouting/frr:10.2.1
image: alpine:3.20

# Risky for books/CI
image: quay.io/frrouting/frr:latest
image: alpine:latest
```

In CI (later chapter), pin digests:

```
image: quay.io/frrouting/frr:10.2.1@sha256:....
```

Record pins in each lab's README.

Resource planning worksheet

Before a new topology, write:

```
## Sizing
Nodes:
- 3 x FRR (~X MB each estimated)
- 2 x Alpine
Total RAM budget: ...
Host free RAM: ...
Fallback if OOM: remove r3, collapse to chain
```

If deploy dies with OOM kills:

```
dmesg -T | tail -50
docker ps -a
```

Network resource limits on the host

Resource	Symptom when exhausted
RAM	Random node death
CPU	Protocol timers flaky, loss
Inotify/watches	Rare editor/CI issues
Interface count	Fail to create veth
Docker IP pool	Mgmt network allocate fail

```
ip link | wc -l
docker network ls
```

Destroy cleaned labs free veths.

Documentation & community map

Resource	Use
containerlab.dev	Topology reference, kinds
docs.frrouting.org	OSPF/BGP/commands
RFCs (IETF)	Authoritative protocol behavior
Linux `man ip`, `man bridge`	Host dataplane
Lab repos using Containerlab	Inspiration—verify licenses

RFCs worth bookmarking (foundations)

Topic	Starting points
IP	RFC 791 (historic), RFC 8200 (IPv6)
ICMP	RFC 792, RFC 4443
ARP	RFC 826
OSPF	RFC 2328 (v2)
BGP	RFC 4271
Ethernet/VLAN	IEEE world + RFC 5517 context notes

You do not memorize RFCs; you **use them when models conflict**.

Predict → observe: image sanity lab

```yaml
name: imgcheck

topology:
  nodes:
    frr1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
    a1:
      kind: linux
      image: alpine:3.20
  links:
    - endpoints: ["frr1:eth1", "a1:eth1"]
```

Predict: Both containers running; `vtysh` answers on frr1; Alpine has `ip` after apk or use busybox ip if present.

```
sudo containerlab deploy -t imgcheck.clab.yml
docker exec clab-imgcheck-frr1 vtysh -c 'show version'
docker exec clab-imgcheck-a1 uname -a
sudo containerlab destroy -t imgcheck.clab.yml --cleanup
```

Harden: Document the exact tags that work on your machine in a personal `IMAGES.md`.

Multi-implementation mindset

Later design labs may mix FRR + Linux + optional SR Linux. Same OSPF adjacency model; different show commands. Keep a translation scrap:

Intent	FRR	Linux
Addresses	`show interface`	`ip -br a`
Routes	`show ip route`	`ip route`
BGP summary	`show bgp summary`	n/a or bird/etc

Ethics and redistribution

- Do not redistribute proprietary images
- Respect registry rate limits and ToS
- Prefer building lab content others can run **without** secrets or licenses

Summary

- Default stack: Containerlab + FRR + Alpine/Debian
- Optional free NOS: SR Linux community, SONiC, VyOS—with eyes open
- Size hosts honestly; destroy idle labs
- Pin image tags; record what works
- Official docs + RFCs beat random CLI folklore

Next: **lab hygiene**—addressing plans, naming, git layout, and diagram habits so multi-lab life stays sane.

Linux Networking for Lab Hosts

Linux Networking for Lab Hosts

Containerlab sits on Linux networking primitives: namespaces, veth pairs, bridges, routes, and basic filtering. If you only drive YAML without understanding the host, debugging feels like magic. This chapter builds the host-side intuition you need before (and during) Containerlab work.

Learning goals

By the end of this chapter you can:

- Explain how network namespaces isolate stacks
- Create veth pairs and bridges by hand
- Add addresses and routes; verify with `ip route get`
- Use `tcpdump` inside a namespace
- Map Containerlab links to veth/bridge concepts
- Apply minimal filter awareness (`nft`/`iptables`, `rp_filter`)

Learning goals in practice

You will build a **two-namespace router lab** without Containerlab, then tear it down. That is the same physics Containerlab orchestrates for you.

Core objects

Object	Role
Interface	L2 endpoint (physical, veth, bridge, vlan subint)
Network namespace	Isolated network stack (interfaces, routes, sockets)
veth pair	Virtual cable: two ends, often in different namespaces
Bridge	Software switch in the kernel

Object	Role
Route	FIB entry: destination → device / next hop
Rule	Policy routing (advanced; optional here)

```
ip -br link
ip -br addr
ip netns list
ip route
```

Network namespaces

A namespace process sees the **root** namespace. Lab nodes (containers) each get a namespace.

```
# Create two namespaces
sudo ip netns add ns1
sudo ip netns add ns2
ip netns list

# Run a command inside
sudo ip netns exec ns1 ip link
sudo ip netns exec ns1 ping -c1 127.0.0.1
```

Loopback exists per namespace; bring it up if you care about local sockets:

```
sudo ip netns exec ns1 ip link set lo up
```

veth pairs: virtual cables

```
# Create pair veth-a <-> veth-b
sudo ip link add veth-a type veth peer name veth-b

# Move ends into namespaces
sudo ip link set veth-a netns ns1
sudo ip link set veth-b netns ns2

# Name + address inside each ns
sudo ip netns exec ns1 ip link set veth-a name eth0
sudo ip netns exec ns2 ip link set veth-b name eth0

sudo ip netns exec ns1 ip addr add 10.0.0.1/24 dev eth0
sudo ip netns exec ns2 ip addr add 10.0.0.2/24 dev eth0
```

```
sudo ip netns exec ns1 ip link set eth0 up
sudo ip netns exec ns2 ip link set eth0 up

# Test L3 on-link
sudo ip netns exec ns1 ping -c 2 10.0.0.2
```

Predict: ARP resolves; ICMP works; no router required because same subnet.

Observe:

```
sudo ip netns exec ns1 ip neigh
sudo ip netns exec ns1 tcpdump -ni eth0 -c 10 arp or icmp &
sudo ip netns exec ns1 ping -c 2 10.0.0.2
```

Bridges: software switches

Connect multiple endpoints as if on a switch:

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip netns add h3

sudo ip link add br0 type bridge
sudo ip link set br0 up

# veth for each host: host side in ns, peer on host attached to bridge
for n in 1 2 3; do
  sudo ip link add veth-h$n type veth peer name veth-b$n
  sudo ip link set veth-h$n netns h$n
  sudo ip netns exec h$n ip link set veth-h$n name eth0
  sudo ip netns exec h$n ip addr add 10.10.0.$n/24 dev eth0
  sudo ip netns exec h$n ip link set eth0 up
  sudo ip netns exec h$n ip link set lo up
  sudo ip link set veth-b$n master br0
  sudo ip link set veth-b$n up
done

sudo ip netns exec h1 ping -c 2 10.10.0.2
sudo ip netns exec h1 ping -c 2 10.10.0.3
```

Bridge FDB (MAC table):

```
bridge fdb show br br0
# or
ip neigh  # not the same; FDB is L2 forward database
```

```
bridge link show
```

After pings, you should see learned MACs on the bridge ports.

Routing between namespaces

Classic: two LANs and a router namespace.

```
h1 (10.1.0.10/24) -- veth -- r (10.1.0.1 + 10.2.0.1) -- veth -- h2 (10.2.0.10/24)
```

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip netns add r

# h1 <-> r
sudo ip link add v1 type veth peer name r1
sudo ip link set v1 netns h1
sudo ip link set r1 netns r
sudo ip netns exec h1 ip link set v1 name eth0
sudo ip netns exec r  ip link set r1 name eth1
sudo ip netns exec h1 ip addr add 10.1.0.10/24 dev eth0
sudo ip netns exec r  ip addr add 10.1.0.1/24  dev eth1
sudo ip netns exec h1 ip link set eth0 up
sudo ip netns exec r  ip link set eth1 up
sudo ip netns exec h1 ip link set lo up
sudo ip netns exec r  ip link set lo up

# h2 <-> r
sudo ip link add v2 type veth peer name r2
sudo ip link set v2 netns h2
sudo ip link set r2 netns r
sudo ip netns exec h2 ip link set v2 name eth0
sudo ip netns exec r  ip link set r2 name eth2
sudo ip netns exec h2 ip addr add 10.2.0.10/24 dev eth0
sudo ip netns exec r  ip addr add 10.2.0.1/24  dev eth2
sudo ip netns exec h2 ip link set eth0 up
sudo ip netns exec r  ip link set eth2 up
sudo ip netns exec h2 ip link set lo up

# Enable forwarding on router ns
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1

# Default routes on hosts
sudo ip netns exec h1 ip route add default via 10.1.0.1
sudo ip netns exec h2 ip route add default via 10.2.0.1
```

```
# Test
sudo ip netns exec h1 ping -c 3 10.2.0.10
sudo ip netns exec h1 traceroute -n 10.2.0.10
```

Predict → observe:

```
sudo ip netns exec h1 ip route get 10.2.0.10
sudo ip netns exec r  ip route get 10.2.0.10 from 10.1.0.10 iif eth1
sudo ip netns exec r  ip -s link show eth1
```

Failure drill: disable forwarding

```
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=0
sudo ip netns exec h1 ping -c 2 10.2.0.10   # expect fail
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1
```

Harden note: container routers must allow forwarding; images often set this, but verify.

Failure drill: missing return route

If the "router" only had a connected route on one side (misconfigured static elsewhere), one-way traffic appears. With connected routes on both interfaces above, both directions work. Later static chapters amplify return-path bugs.

VLAN subinterfaces (preview of trunks)

```
# On a namespace with eth0
sudo ip netns exec h1 ip link add link eth0 name eth0.10 type vlan id 10
sudo ip netns exec h1 ip addr add 10.10.10.1/24 dev eth0.10
sudo ip netns exec h1 ip link set eth0.10 up
```

Both ends need matching VLAN IDs. Bridges can be VLAN-aware (`vlan_filtering`); Containerlab L2 labs often use OVS or NOS switches—concepts transfer.

Policy leftovers that break labs

rp_filter (reverse path filtering)

Strict reverse-path checks drop asymmetric legitimate lab traffic.

```
sysctl net.ipv4.conf.all.rp_filter
sysctl net.ipv4.conf.default.rp_filter
# Per-iface:
# sysctl net.ipv4.conf.eth0.rp_filter
```

If mysterious one-way drops appear on Linux routers, check `rp_filter` and counters.

Forwarding and firewall

```
# nftables quick view
sudo nft list ruleset
# legacy
sudo iptables -L -n -v
sudo iptables -t nat -L -n -v
```

Host firewalls (especially Docker's chains) can interact with lab bridges. When a lab "should work" and does not, inspect host filter tables—not only node configs.

Disable offloads when captures look weird (optional)

Checksum offload can make `tcpdump` show wrong checksums on local veth. Prefer understanding this over panicking:

```
sudo ethtool -k veth0 | grep sum || true
```

Mapping to Containerlab

You did by hand	Containerlab roughly does
`ip netns add`	Creates container network namespaces
veth pairs	Links between nodes
bridges / OVS	Multipoint wiring, mgmt network
addressing	From topology + image startup
`ip_forward`	Inside router nodes

When Containerlab deploy fails or links are dark, drop to:

```
ip netns list
# Containerlab often names netsns like clab-<lab>-<node>
sudo ip netns exec clab-mylab-r1 ip -br a
```

Exact naming depends on lab name and version; `containerlab inspect` helps.

```
sudo containerlab inspect -t topology.clab.yml
```

Clean teardown habits

Namespaces and veths litter a host if you exit mid-lab.

```
# Example cleanup for the objects you created
sudo ip netns del h1 2>/dev/null || true
sudo ip netns del h2 2>/dev/null || true
sudo ip netns del r  2>/dev/null || true
sudo ip netns del ns1 2>/dev/null || true
sudo ip netns del ns2 2>/dev/null || true
sudo ip link del br0 2>/dev/null || true
# dangling veths usually die with netns; if not:
ip -br link | grep veth || true
```

Prefer scripted `up.sh` / `down.sh` even for hand labs.

Example `down.sh`

```
#!/usr/bin/env bash
set -euo pipefail
for ns in h1 h2 r ns1 ns2; do
  sudo ip netns del "$ns" 2>/dev/null || true
done
sudo ip link del br0 2>/dev/null || true
echo "cleaned"
```

Predict → observe → fix drill (checklist)

Lab: three hosts on `br0` as above.

1. **Predict:** h1→h3 uses bridge flood then learn; FDB populates.
2. **Observe:** `bridge fdb show`, tcpdump on `veth-b1`.
3. **Fix inject:** `sudo ip link set veth-b3 down` — h3 unreachable; FDB ages/fails.
4. **Harden:** document “access port down vs cable unplug” equivalence in labs.

Useful one-liners cheatsheet

```
# Watch neigh events
ip monitor neigh

# Route decision
ip route get 1.1.1.1 from 10.1.0.10 iif eth0

# Stats
ip -s -s link show eth0

# All addresses every ns (rough)
for ns in $(ip netns list | awk '{print $1}'); do
  echo "===== $ns ====="
  sudo ip netns exec "$ns" ip -br a
done
```

Security note for shared hosts

Namespace labs are powerful. On a shared machine:

- Do not bridge lab interfaces onto trusted production networks without filters
- Clean up after sessions
- Avoid enabling ip_forward on the **root** namespace unless you intend the host to route

Containerlab management networks still need isolation discipline on corporate laptops.

Summary

- Namespaces isolate full network stacks—foundation of container networking
- veth pairs are virtual cables; bridges are virtual switches
- Router namespaces need addressing, up state, forwarding, and routes both ways
- Host firewall and `rp_filter` are common "invisible" lab breakers
- Hand-building once makes Containerlab YAML readable and debuggable

Next: **Containerlab fundamentals**—topology YAML, kinds, links, lifecycle—on top of these primitives.

Lab Hygiene

Lab Hygiene

Brilliant topologies rot without hygiene: colliding subnets, random hostnames, uncommitted click-ops, and mystery cables. This chapter is the professional layer on top of Containerlab—how to keep labs reproducible, reviewable, and kind to future you.

Learning goals

By the end of this chapter you can:

- Write an addressing plan before deploy
- Apply consistent node and interface naming
- Structure a lab repo/directory for git
- Keep diagrams and inventories in sync
- Run a personal definition-of-done for every lab
- Avoid cross-lab contamination on one host

Why hygiene is a skill

Without hygiene	With hygiene
"Which 10.0.0.0/24 is this?"	Documented per-lab blocks
Works once after 40 hand edits	Destroy/deploy/verify loop
Cannot explain a failure	Journal + captures
Peer cannot run your lab	README + pins + scripts

Hygiene is not bureaucracy; it is how lab craft scales.

Addressing plans

Never invent IPs mid-SSH.

Template

```
# Addressing - lab triangle-ospf

| Node | Interface | Address | Notes |
|------|-----------|---------|-------|
| r1 | eth1 | 10.0.12.1/24 | to r2 |
| r2 | eth1 | 10.0.12.2/24 | to r1 |
| r1 | eth2 | 10.0.13.1/24 | to r3 |
| r3 | eth2 | 10.0.13.2/24 | to r1 |
| r2 | eth2 | 10.0.23.1/24 | to r3 |
| r3 | eth1 | 10.0.23.2/24 | to r2 |
| h1 | eth1 | 192.168.1.10/24 | gw 192.168.1.1 (r1 lo/lan) |

Loopbacks:
| Node | lo |
|------|-----|
| r1 | 1.1.1.1/32 |
| r2 | 2.2.2.2/32 |
| r3 | 3.3.3.3/32 |

VLANs: n/a
ASNs: n/a
Mgmt: Containerlab default pool
```

Allocation strategy (personal lab supernets)

Pick a **lab supernet** you will not confuse with home LAN:

Use	Example block
Home LAN (real)	192.168.1.0/24 (do not reuse in labs if VPN splits break)
Lab underlay P2P	10.0.0.0/16 carved into /24 or /30
Lab customer VRFs	10.10.0.0/16
Loopbacks	10.255.0.0/24 as /32s
“Internet” sim	203.0.113.0/24 (documentation range) / 192.0.2.0/24

Documentation ranges (RFC 5737) are great for examples in prose; inside isolated labs private space is fine—just **write it down**.

/30 vs /31 vs /24 on links

Choice	When
/24	Beginners, lots of headroom, wasteful

Choice	When
/30	Classic P2P
/31	Efficient P2P (ensure both NOS support)

Be consistent within a lab.

Naming conventions

Lab name

```
name: l2-vlan-trunk   # short, hyphenated, unique among active labs
```

Container names become `clab-<name>-<node>`.

Node names

Pattern	Examples
Role + number	`r1`, `r2`, `sw1`, `h1`
Role + site	`edge-a`, `core-1`
Avoid	`test`, `node`, `foo`, `router`

Interface names

Stick to what the image uses (`eth1`, `eth2`… for many Linux kinds). Document any breakout or rename.

Git branches / folders

```
labs/
  03-l2-vlan-trunk/
  04-static-triangle/
  05-ospf-single-area/
```

Numeric prefixes match book order when useful.

Directory layout (canonical)

```
labs/04-static-triangle/
  README.md           # intent, pins, how to run
  topology.clab.yml
  addressing.md
  journal.md
  verify.sh
  up.sh               # optional deploy wrapper
  down.sh             # destroy --cleanup
  config/
    r1/
    r2/
    h1/
  captures/
  diagrams/
    logical.png       # or .svg / .drawio
```

README skeleton

```
# Static triangle

## Intent
Three FRR routers, static routes, dual hosts ping.

## Images
- quay.io/frrouting/frr:10.2.1
- alpine:3.20

## Run
sudo containerlab deploy -t topology.clab.yml
./verify.sh
sudo containerlab destroy -t topology.clab.yml --cleanup

## Failure drills
- shut transit link r1-r2
- remove return route on r3
```

Git habits

```
git status
git add labs/04-static-triangle
git commit -m "lab: static triangle with verify.sh"
```

Commit:

- Topology YAML
- Configs
- addressing.md
- verify.sh
- journal highlights (not necessarily huge pcaps)

Usually ignore:

- Large pcaps (store sparse samples or regenerate)
- Ephemeral logs

```
# example snippet
labs/**/captures/*.pcap
!labs/**/captures/.gitkeep
```

Never commit

- Production credentials
- Private keys
- Proprietary images

Diagram habit

Minimum viable diagram set:

1. **Logical** — nodes and links
2. **Addressing** — subnets/VLAN IDs on links
3. **Failure** — which link you shut for the drill

ASCII is valid:

```
h1--r1------r2--h2
     \      /
      \    /
        r3
```

Illustrated topology SVGs (self-contained light canvas) match this book's lab figures under `images/`; conceptual models may stay monochrome. See `images/README.md`.

Configuration hygiene

Do	Don't
Edit files under `config/`	Treasure unique vtysh-only state
Reload/restart from files	Assume memory is durable
Diff configs in git	Copy-paste from chat without review
Comment tricky policy lines	Leave "magic" prefix-lists

FRR example: comment intent

```
! block lab RFC1918 from leaking to "isp" peer
ip prefix-list LAB-PRIV seq 5 permit 10.0.0.0/8 le 32
```

verify.sh hygiene

- `set -euo pipefail`
- Explicit container names
- Timeouts on ping
- Exit non-zero on failure
- Print which check failed

```
#!/usr/bin/env bash
set -euo pipefail
PASS=0
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-static-h1 ping -c 2 -W 1 10.2.0.10 \
  || fail "h1 cannot reach h2"
```

```
echo "OK: all checks passed"
```

Cross-lab contamination

Symptoms:

- Wrong SSH target (same hostname different lab)
- Stale docker networks
- IP confusion in your brain

Mitigations:

```
sudo containerlab inspect --all 2>/dev/null || true
docker ps --format 'table {{.Names}}\t{{.Status}}' | grep clab || true
# Only one "heavy" lab running when learning
sudo containerlab destroy -t old.clab.yml --cleanup
```

Use unique lab `name:` fields always.

Time hygiene

Put dates in journals; note Containerlab and image versions:

```
Date: 2026-08-04
containerlab: 0.xx.x
frr image: 10.2.1
kernel: $(uname -r)
```

Future breakage is often a version drift story.

Definition of done (lab)

- ☐ Intent one-liner in README
- ☐ addressing.md complete
- ☐ topology deploys clean on fresh destroy
- ☐ verify.sh green

☐ One failure drill documented

☐ Diagrams match YAML

☐ Secrets absent

☐ Image tags pinned

☐ Committed to git

Predict → observe → fix: hygiene drill

1. **Predict:** A peer can clone your lab folder and pass `verify.sh` without asking you questions.
2. **Observe:** Remove your personal aliases; follow only README.
3. **Fix:** Gaps become README bullets.
4. **Harden:** Add `up.sh`/`down.sh`.

```
#!/usr/bin/env bash
# up.sh
set -euo pipefail
sudo containerlab deploy -t topology.clab.yml
./verify.sh
```

```
#!/usr/bin/env bash
# down.sh
set -euo pipefail
sudo containerlab destroy -t topology.clab.yml --cleanup
```

Collaboration norms (optional)

If sharing labs:

- Prefer open images only
- Include expected verify output sample
- Note host requirements (RAM)
- License your configs clearly (e.g. MIT notes)

Summary

- Address first, deploy second
- Name labs and nodes for humans and `docker ps`
- Canonical directory: topology, configs, verify, journal, diagrams
- Git is part of networking practice
- One clean destroy/deploy is worth ten fragile demos
- Definition of done prevents "almost labs"

Part checkpoint: topology-as-code should feel like muscle memory. Next part enters **layer 2**—Ethernet, VLANs, loops, aggregation, and failure drills.

Networking Foundations

What Is Networking?

What Is Networking?

A network moves **data** between **endpoints** over **links**, using **protocols**—agreed rules for format, addressing, and behavior. Everything later in this book (VLANs, OSPF, VXLAN) is a specialization of that idea.

Learning goals

- Name the main building blocks of a network and what problem each solves
- Distinguish local delivery (same L2 domain) from routed delivery (across networks)
- Explain why we use layered protocols instead of one monolithic format
- Tie lab tools (Containerlab, `ip`, captures) to those ideas

Building blocks

Piece	Role	Lab example
Endpoint / host	Source or sink of traffic	Alpine container, your laptop
Interface	Attachment to a link	`eth0`, `eth1`
Link	Path between interfaces	Cable, veth pair, Wi-Fi
Switch / bridge	Forward within an L2 domain by MAC	Linux bridge, virtual switch
Router	Forward between L3 networks by IP	FRR node, Linux with forwarding
Protocol	Rules and message formats	Ethernet, IPv4, TCP, DNS

What problem networking solves

Without a network, processes on different machines cannot cooperate. Networking provides:

1. **Reachability** — a path exists (or is reported as unreachable)

2. **Naming / addressing** — who is who (MAC, IP, names via DNS)
3. **Multiplexing** — many conversations share links (ports, VLANs, queues)
4. **Reliability options** — best-effort (UDP/IP) vs recovered streams (TCP)
5. **Policy** — who may talk to whom, with what priority

Switching vs routing (first cut)

```
Same subnet / VLAN          Different subnets

  Host A  switch  Host B    Host A → gateway → … → Host B
  Frame uses MAC             Packet uses IP; each hop rewrites L2
```

- **Switching (L2):** deliver a frame inside one broadcast domain; learn MACs; flood unknown unicasts.
- **Routing (L3):** deliver a packet across networks; each router decrements TTL and chooses a next hop.

You will refine both in the Layer 2 and Layer 3 parts. Foundations give vocabulary and host-level truth.

Protocols as contracts

A protocol answers:

- What bits mean (headers, fields)
- Who may send what next (state machines)
- How errors are signaled (ICMP, TCP RST, DHCP NAK)

Vendors invent CLIs; **RFCs and open implementations** define the contracts this book cares about.

Why labs beat slideware

Networks fail in partial, asymmetric, and time-dependent ways. A Containerlab topology lets you:

- Change **one** variable (shut a link, wrong mask, bad gateway)
- Observe **tables** (ARP, routes, MAC) and **packets** (tcpdump)
- Reset and try again

Observation toolkit (preview)

Tool	Use
`ip addr` / `ip route` / `ip neigh`	Host addressing and neighbors
`ping` / `traceroute` / `mtr`	Reachability and path hints
`ss` / `netstat`	Sockets and listeners
`tcpdump` / `tshark`	On-the-wire proof
Containerlab	Multi-node topologies as code

Checkpoint

Before deeper chapters, you should be able to point at a simple two-host lab and say: *these are endpoints, this is a link, delivery is L2 or L3 for this destination, and here is how I would prove it.*

Next

- TCP/IP and the Layered Stack

TCP/IP and the Layered Stack

TCP/IP and the Layered Stack

Networks are built as a **stack of layers**. Each layer solves one class of problem and offers a service to the layer above. This book uses the **TCP/IP** (Internet) model in day-to-day work; the **OSI** model remains a useful legend for vendor docs and troubleshooting language.

Learning goals

- Map problems and tools to the correct TCP/IP layer
- Translate common OSI layer numbers into TCP/IP practice
- Explain demultiplexing (how a host hands data up the stack)
- Avoid "wrong layer" diagnoses

TCP/IP model (working map)

Layer	Examples	Responsibility
Application	HTTP, DNS, SSH, DHCP, BGP (as app of TCP/UDP)	Meaning for programs; often client/server
Transport	TCP, UDP	Process-to-process; ports; reliability options
Internet	IPv4, IPv6, ICMP	Host-to-host logical addressing; routing; error signaling
Link / network access	Ethernet, Wi-Fi, PPP	Adjacent-node framing; media access

Some texts split "host-to-network" further; for labs, **Ethernet + ARP/ND** is the daily link reality.

OSI as a legend (not a religion)

OSI #	Name	Rough TCP/IP home
7	Application	Application
6	Presentation	Often folded into app (TLS, encodings)

OSI #	Name	Rough TCP/IP home
5	Session	Often folded into app/TCP
4	Transport	TCP/UDP
3	Network	IP / ICMP
2	Data link	Ethernet frames, switching
1	Physical	Cables, optics, signaling (usually out of scope here)

Practical rule: know *where the address lives* and *which header you are looking at in a capture*, not seven acronyms for a quiz.

What each layer "owns"

Link

- Local delivery between neighbors
- MAC addresses, Ethertype, FCS
- Switches forward here without caring about IP

Internet (IP)

- Global-ish logical addressing
- Best-effort packet forwarding
- TTL/hop limit; fragmentation (IPv4) or reliance on PMTU (IPv6)

Transport

- Which **process** on the host
- Ports; TCP connection state vs UDP datagrams

Application

- Protocol semantics (DNS query/response, HTTP methods)
- Often implements its own retries on top of UDP

Demultiplexing (up the stack)

On receive:

1. NIC / driver accepts a frame (MAC filter)
2. Ethertype → IP vs ARP vs other
3. IP protocol field → TCP / UDP / ICMP / …
4. Port numbers → socket / application

```
Frame [Eth | IP | TCP | HTTP ]
          ^    ^    ^     ^
          |    |    |     application
          |    |    transport demux (ports)
          |    internet demux (protocol #)
          link demux (Ethertype)
```

Layered troubleshooting

Symptom	Look first at
No link light / veth missing	Link / lab cabling
ARP incomplete	Link + IP on-link config
Ping fails, ARP OK	Routing, firewall, ICMP policy
TCP SYN no reply	ACL, listening socket, path, RST
TLS fails after TCP	Application / certificates

Encapsulation (preview)

Each layer **adds a header** when sending and **strips it** when receiving. Details: Packets, Frames, Encapsulation.

Checkpoint

Given a failure description, name the **layer** you will inspect first and **one command or table** that proves or disproves that theory.

Next

- Packets, Frames, Encapsulation

Packets, Frames, Encapsulation

Packets, Frames, Encapsulation

Data is **wrapped** in headers as it descends the stack and **unwrapped** on the way up. Clear names prevent confusion in captures and design talks.

Learning goals

- Use correct terms: data, segment/datagram, packet, frame
- Sketch encapsulation for a simple TCP and UDP flow
- Read a tcpdump line and identify outer headers
- Explain how tunnels add outer encapsulations

Vocabulary

Term	Layer (typical)	Meaning
Data / payload	Application	What the app cares about
Segment	TCP	TCP header + payload
Datagram	UDP (and sometimes IP)	UDP header + payload
Packet	IP	IP header + payload
Frame	Link	Ethernet (or other) header + payload + FCS

People say "packet" loosely; in precise talk, **frame** is L2 and **packet** is L3.

Encapsulation path (send)

1. Application writes bytes
2. Transport adds **src/dst ports** (TCP adds seq/ack/flags)
3. IP adds **src/dst IP**, TTL, protocol
4. Ethernet adds **src/dst MAC**, Ethertype

5. Bits go on the wire (or veth)

```
 Application data
+------------------+
| TCP/UDP header   |
+------------------+
| IP header        |
+------------------+
| Ethernet header  |
+------------------+
```

Decapsulation path (receive)

Reverse order: FCS check → MAC → Ethertype → IP → protocol → ports → app.

Maximum sizes and MTU

- **MTU**: max IP packet size a link will accept (commonly 1500 on Ethernet)
- Larger packets must fragment (IPv4) or fail with Packet Too Big (IPv6 + PMTUD)
- Extra encapsulation (VLAN tag, VXLAN, IPsec) **reduces** effective payload size

Details and labs: Layer 3 ICMP/PMTU chapter later; foundations need the idea.

Captures: reading the onion

```
# On a Linux lab node
tcpdump -ni eth1 -vv
tcpdump -ni eth1 icmp
tcpdump -ni eth1 'tcp port 22'
```

What you see first is the **outer** header. Inner headers appear after decap or when capturing on the right interface.

Tunnels and overlays (preview)

Overlays wrap an **inner** packet in an **outer** IP/UDP (or GRE) packet. Underlay routers only need to forward the outer packet. Full treatment: Overlays part.

```
[ Outer Eth | Outer IP | Outer UDP | VXLAN | Inner Eth | Inner IP | … ]
```

Lab sketch (two nodes)

1. Deploy two Linux nodes on one L2 segment
2. Capture while `ping` runs — ICMP inside IP inside Ethernet
3. Capture `nc` or `curl` — TCP three-way handshake fields

Checkpoint

Draw encapsulation for **DNS over UDP** and **SSH over TCP** from host A to B on the same Ethernet. Label every header.

Next

- Link Layer and Ethernet

Link Layer and Ethernet

Link Layer and Ethernet

The **link layer** moves frames between neighbors on a shared medium or point-to-point link. In almost every lab in this book, that means **Ethernet** (physical or virtual).

Learning goals

- Describe Ethernet frame fields used in daily troubleshooting
- Explain unicast, multicast, broadcast at L2
- Contrast hubs (obsolete), switches, and bridges
- Preview MAC learning (full drills in Layer 2 part)

Why link layer matters

IP cannot deliver alone: on multi-access Ethernet, the next hop is identified by **MAC**. Wrong VLAN, wrong MAC, or broken L2 path makes "routing" look broken when the problem is local.

Ethernet frame (simplified)

Field	Role
Dest MAC	Who should accept the frame
Src MAC	Who sent it (learning key on switches)
Ethertype / length	Payload type (e.g. `0x0800` IPv4, `0x86DD` IPv6, `0x0806` ARP)
Payload	IP packet, ARP, etc.
FCS	Error detection

802.1Q VLAN tags insert **TPID + TCI** (including VLAN ID) between SA and Ethertype on trunks—details in VLANs chapter.

MAC addresses

- 48-bit; written `aa:bb:cc:dd:ee:ff`
- Universally administered vs locally administered
- **Broadcast** `ff:ff:ff:ff:ff:ff`
- Multicast: group bit set

```
ip -br link
ip link show eth1
```

Broadcast domain vs collision domain

- **Collision domain:** historically shared half-duplex media; switched full-duplex Ethernet largely eliminates classic collisions
- **Broadcast domain:** extent of L2 broadcast/unknown flood—usually one VLAN

Routers **stop** L2 broadcasts; switches do not.

Switch vs router (again)

Device	Forwarding key	Changes MAC?	Changes IP?
L2 switch	Dest MAC + FDB	No (transparent)	No
Router	Dest IP + FIB	**Yes** (new Ethernet hop)	No (until NAT)

MAC learning (preview)

1. Learn **src MAC → port** on ingress
2. If dest MAC known, forward to that port
3. If unknown unicast, **flood** in the VLAN
4. Broadcast/multicast: flood per rules

Deep labs: Ethernet & MAC Learning.

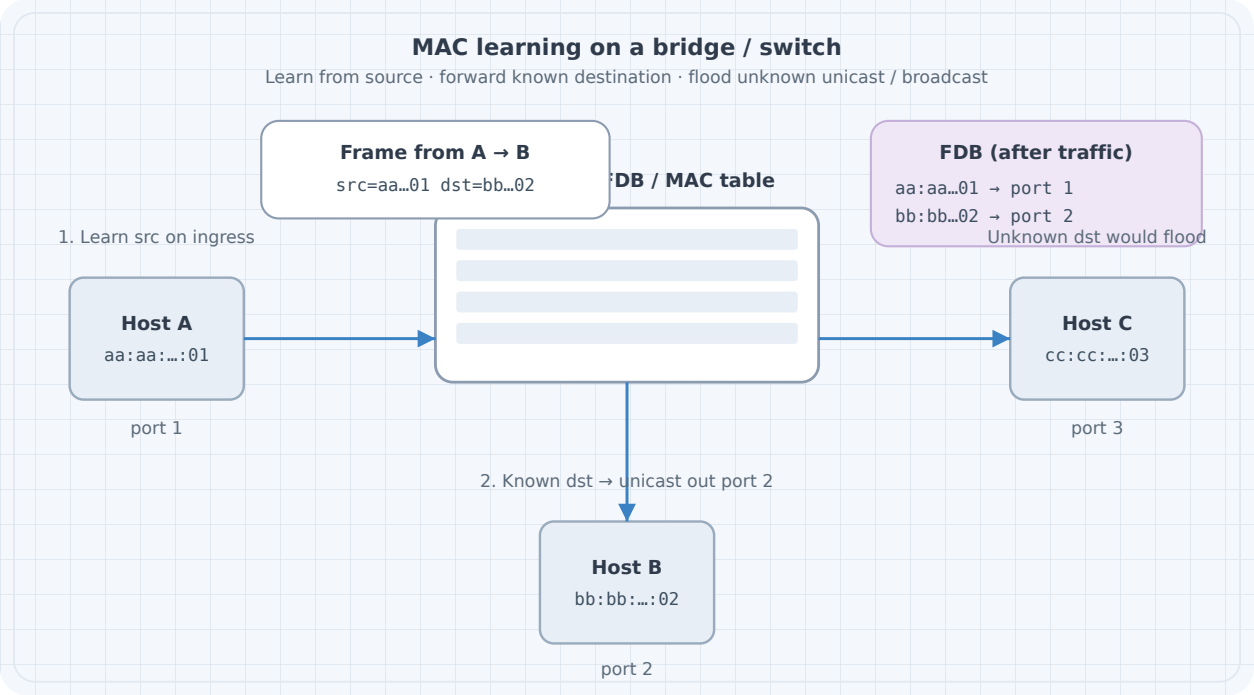


Figure 1: MAC learning

Virtual Ethernet in labs

Containerlab connects nodes with **veth pairs** and bridges—same Ethernet model, software implementation:

```
ip link
bridge link   # if bridge-utils / iproute2 bridge
```

Common L2 failures (foundations list)

Symptom	Typical cause
Incomplete ARP	Wrong subnet, wrong VLAN, silent host
Flapping MAC	Loops or dual-homing without design
No frame at all	Down interface, wrong cable/veth, admin down

Checkpoint

Explain what a switch does when it receives a frame with an **unknown** destination MAC, and what it does with the **source** MAC.

Next

- IPv4 Addressing and Subnetting

IPv4 Addressing and Subnetting

IPv4 Addressing and Subnetting

IPv4 addressing is the daily language of lab design: **prefixes**, **masks**, **subnets**, and **who is on-link**. Master this before multi-router labs.

Learning goals

- Read and write CIDR prefixes (`a.b.c.d/nn`) fluently
- Convert between prefix length and dotted masks
- Design non-overlapping lab subnets and point-to-point links
- Identify special-use and documentation ranges
- Configure addresses on Linux and verify with `ip`

IPv4 structure

- **32 bits**, usually dotted decimal (four 0–255 octets)
- **Prefix length** `/N` = number of network bits; host bits = `32 - N`
- **Network address**: host bits zero
- **Broadcast address** (classical Ethernet/IPv4): host bits one (except special cases like /31)

Prefix	Mask	Classical usable hosts	Common use
/32	255.255.255.255	1	Host route, loopback
/31	255.255.255.254	2	Point-to-point (RFC 3021)
/30	255.255.255.252	2	Classic P2P
/29	255.255.255.248	6	Small segment
/24	255.255.255.0	254	Common LAN
/16	255.255.0.0	many	Summaries / big labs
/8	255.0.0.0	huge	Rare in labs

CIDR and masks

```
192.168.10.0/24
network: 192.168.10.0
mask:    255.255.255.0
hosts:   192.168.10.1 - 192.168.10.254
bcast:   192.168.10.255
```

```
# Linux: add address
ip addr add 192.168.10.10/24 dev eth1
ip -br addr
```

On-link decision

If the destination is inside a **connected** prefix, the host resolves L2 (ARP) on that interface. Otherwise it looks up a **route** (often default via gateway).

```
ip route get 192.168.10.50
ip route get 8.8.8.8
```

Private and documentation ranges

Range	Role
`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`	Private (RFC 1918)—default lab space
`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`	Documentation (TEST-NET)—safe in examples
`127.0.0.0/8`	Loopback
`169.254.0.0/16`	Link-local (APIPA)
`224.0.0.0/4`	Multicast

Lab hygiene: pick a documented plan (`10.20.0.0/16` etc.) and never invent overlapping subnets "for now."

Subnetting for multi-tier labs

Example allocation from `10.20.0.0/16`:

Use	Block
Access / users A	`10.20.1.0/24`
Access / users B	`10.20.2.0/24`

Use	Block
Servers	`10.20.10.0/24`
P2P core	`10.20.255.0/24` carved as `/31` or `/30`
Loopbacks	`10.20.254.0/24` as `/32`s

Rules:

1. No accidental overlap
2. Leave growth room
3. Write the plan **before** deploy
4. Align future summaries with real topology

Point-to-point links

Style	Example	Notes
/31	`10.20.255.0/31` → `.0` and `.1`	Preferred in modern designs
/30	`10.20.255.0/30` → `.1` and `.2` usable	Classic textbooks

Summarization (intro)

Advertising `10.20.0.0/22` for four /24s only works if you **own** the whole /22. Bad summaries **blackhole** traffic for space you do not have.

Action	Risk
Summary too broad	Blackholes
No aggregation ever	Table growth later (IGP/BGP scale)

Layer 3 part continues summarization drills; foundations need the danger signal.

VLSM

Variable Length Subnet Masking: different prefix lengths in one design (`/24` LAN + `/31` core). Required for efficient labs—do not force everything to /24.

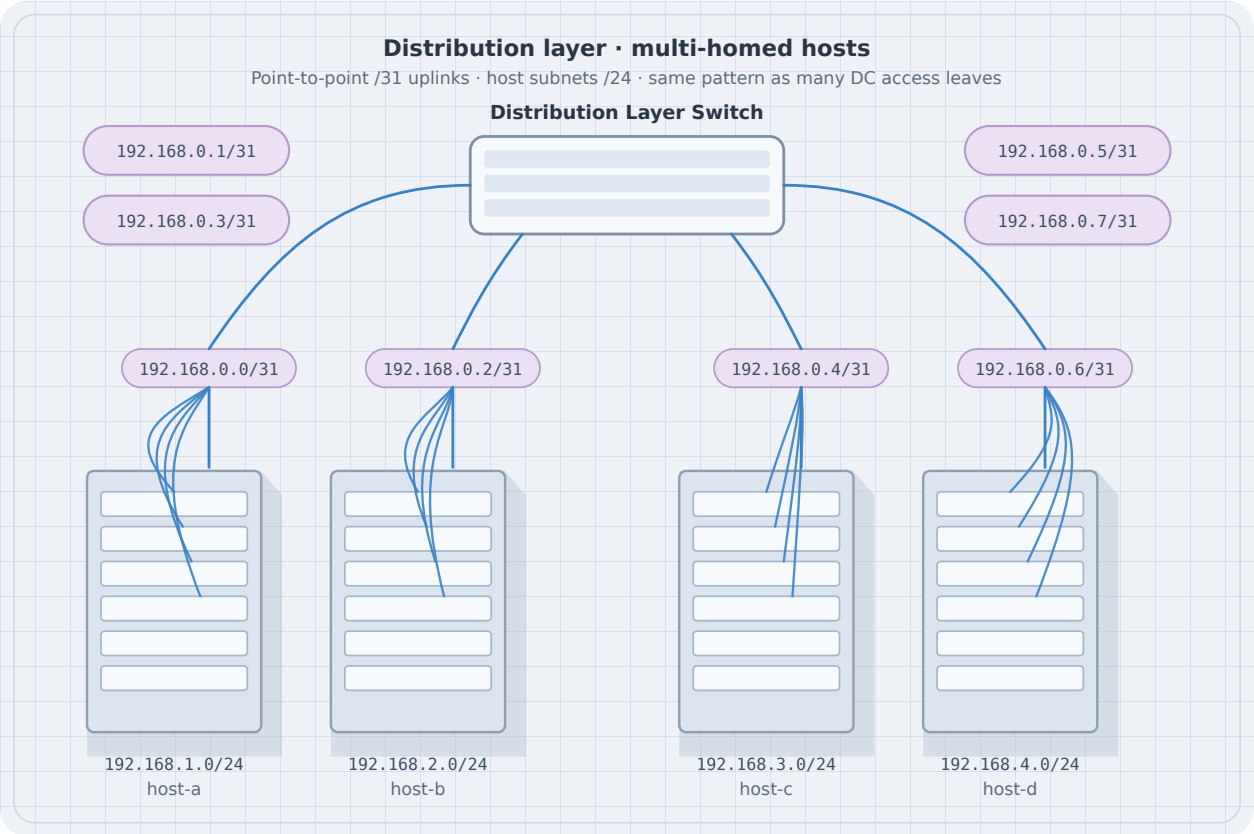


Figure 1: Dist switch multi-home

Host configuration checklist

1. Address + prefix on the correct interface
2. Up state (`ip link set eth1 up`)
3. Gateway if leaving the subnet
4. DNS only if apps need names (separate from pure L3 reachability)

```
ip addr add 10.20.1.10/24 dev eth1
ip link set eth1 up
ip route add default via 10.20.1.1
ip -br a
ip route
```

Worked subnet: /26 from a /24

Parent: `192.168.10.0/24` (256 addresses). Split into four `/26`s (64 addresses each):

Block	Network	Usable hosts	Broadcast
0	`192.168.10.0/26`	`.1`–`.62`	`.63`
1	`192.168.10.64/26`	`.65`–`.126`	`.127`
2	`192.168.10.128/26`	`.129`–`.190`	`.191`
3	`192.168.10.192/26`	`.193`–`.254`	`.255`

Host bits: `/26` → 6 host bits → $2^6 = 64$ addresses → 62 usable (reserve network + broadcast) on Ethernet-style LANs. Point-to-point often uses `/31` (RFC 3021) or `/30`.

Longest-prefix match: if a router has both `10.0.0.0/16` and `10.1.2.0/24`, a packet to `10.1.2.5` uses the `/24`. More specific wins—plan summarization so you do not punch accidental holes.

Private vs public (RFC 1918)

Range	Typical use
`10.0.0.0/8`	Large private / DC
`172.16.0.0/12`	Mid private
`192.168.0.0/16`	Home/lab LANs
`100.64.0.0/10`	CGNAT (ISP shared) — not “your” private LAN plan

Lab rule: pick one private block for the whole topology and **document** the plan before cabling Containerlab.

Paper drills

1. Network, first host, last host, broadcast for `172.16.5.0/26`
2. How many /24s fit in `10.0.0.0/16`?
3. Can `10.0.1.0/24` and `10.0.1.128/25` coexist without overlap?
4. From `10.50.0.0/16`, carve three `/24`s and one `/28` for a mgmt VLAN without overlap

Checkpoint

Design addresses for: two LANs (/24 each), one router-router link (/31), two router loopbacks (/32). No overlaps; list every interface IP.

Next

- IPv6 Essentials

IPv6 Essentials

IPv6 Essentials

IPv6 is not "IPv4 with longer addresses only." Foundations need **address types**, **prefix notation**, **link-local**, and enough config to run dual-stack labs.

Learning goals

- Read compressed IPv6 addresses and `/prefix` lengths
- Distinguish link-local, unique local, global unicast
- Configure a static IPv6 address on Linux
- Know what Neighbor Discovery replaces (ARP, parts of ICMP discovery)

Why IPv6 in this book

- Dual-stack labs mirror real networks
- ND and SLAAC appear in operations
- Many overlays and DC fabrics are IPv6-capable or IPv6-underlay

You do not need to memorize every transition technology here.

Address format

- **128 bits**, eight hextets, e.g. `2001:db8:1:2:3:4:5:6`
- Compress longest zero run with `::` (once)
- Prefix: `2001:db8:1::/64` (common LAN length is **/64**)

Documentation prefix: `2001:db8::/32` (use in examples).

Address kinds (must know)

Kind	Pattern / notes	Role
Link-local	`fe80::/10`	On-link only; required on every interface
Unique local (ULA)	`fc00::/7` (commonly `fd00::/8`)	Private-like lab space
Global unicast	`2000::/3` (in practice)	Internet-routable space
Multicast	`ff00::/8`	One-to-many
Loopback	`::1`	Local host
Unspecified	`::`	"Any" / not set

```
ip -6 addr
ip -6 route
```

Interface identifiers and /64

LAN segments almost always use **/64**. Host bits can come from:

- Manual config
- SLAAC (router advertisements)
- DHCPv6 (where used)

EUI-64 and privacy extensions exist; labs often set **static** addresses for clarity.

Linux static example

```
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip -6 route add default via 2001:db8:1::1 dev eth1
ping -6 2001:db8:1::1
```

Neighbor Discovery (preview)

IPv6 uses **ND** (ICMPv6) instead of ARP:

- Neighbor Solicitation / Advertisement
- Router Solicitation / Advertisement

Details: ARP, ND, and Local Resolution.

Dual-stack thinking

Stack	Failure isolation
IPv4 works, IPv6 fails	v6 route, RA, ND, firewall ip6tables/nft
Prefer AAAA vs A	Happy Eyeballs / app order

Test **both** when dual-stack is in scope:

```
ping -4 <host>
ping -6 <host>
```

Checkpoint

Write a dual-stack plan for one LAN: IPv4 `/24`, IPv6 `/64` from `2001:db8:10::/64`, gateway `::1` / `.1`, two hosts.

Next

- ARP, ND, and Local Resolution

ARP, ND, and Local Resolution

ARP, ND, and Local Resolution

On multi-access links, IP needs a **local** next-hop mapping: IPv4 uses **ARP**; IPv6 uses **Neighbor Discovery (ND)**. If resolution fails, routing never gets a chance.

Learning goals

- Explain when ARP/ND runs (on-link destinations and gateways)
- Read and clear neighbor tables on Linux
- Spot incomplete entries and wrong-subnet mistakes
- Capture ARP or NS/NA in a lab

When local resolution happens

1. Host decides destination is **on-link** (connected route), or
2. Host needs the **gateway's MAC** for an off-link destination

Then: resolve next-hop IP → MAC; build Ethernet header; send.

ARP (IPv4)

Message	Role
Who-has (request)	Broadcast: "Who has IP X? Tell Y"
Is-at (reply)	Unicast: "X is at MAC M"

```
ip neigh show
ip neigh show dev eth1
ping -c1 192.168.10.1
ip neigh show
# force re-resolve
ip neigh flush dev eth1
```

Incomplete / failed ARP

Cause	What you see
Target down	Incomplete, then failed
Wrong VLAN / segment	No reply
Wrong mask (thinking on-link when not)	ARP for remote IPs on LAN
Firewall drop	Silence

Neighbor Discovery (IPv6)

Message (ICMPv6)	Role
Neighbor Solicitation	Resolve or probe neighbor
Neighbor Advertisement	Provide or confirm MAC
Router Solicitation / Advertisement	Find routers; SLAAC info

```
ip -6 neigh show
ping -6 2001:db8:1::1
```

Gratuitous ARP and updates

Hosts may announce their own IP/MAC (gratuitous ARP) on address change. Useful for failover designs later (first-hop redundancy).

Proxy ARP (awareness)

A router answering ARP for non-local addresses can hide topology mistakes—or create blackholes. Prefer **correct masks and routes** over proxy ARP as a habit.

Lab sketch

1. Two hosts + gateway on `/24`
2. `tcpdump -ni eth1 arp` while first ping runs
3. Wrong mask on host: watch ARP for remote destinations

Checkpoint

Host `10.0.0.10/24`, gateway `10.0.0.1`. Destination `10.0.1.10`. Which IP is ARP'd for? Why?

Next

- ICMP and Path Diagnostics

ICMP and Path Diagnostics

ICMP and Path Diagnostics

ICMP (and ICMPv6) signal errors and support diagnostics. `ping` and `traceroute` are applications of those messages—not separate magic protocols.

Learning goals

- Map common ICMP types to meaning
- Use ping/traceroute with clear interpretation limits
- Understand TTL expiry and "why traceroute hops appear"
- Preview Path MTU Discovery

ICMPv4 types (operator set)

Type	Name	Typical meaning
0 / 8	Echo reply / request	Ping
3	Destination unreachable	No route, port unreachable, admin prohibited, …
11	Time exceeded	TTL expired (traceroute hops)

Codes refine type 3 (network unreachable, host unreachable, etc.).

ICMPv6

Similar roles: echo, destination unreachable, **packet too big**, time exceeded, ND messages (see previous chapter).

```
ping -c3 192.0.2.1
ping -6 -c3 2001:db8::1
```

Interpreting ping

Result	Possible meaning
Reply	Forward path + return path + ICMP allowed
Timeout	Drop, filter, wrong route, silent host
Unreachable	Explicit ICMP error from some node

Ping success application success. TCP port filters can still block SSH/HTTP.

Traceroute mechanics (conceptual)

1. Send probes with TTL=1, 2, 3, …
2. Each hop that decrements TTL to 0 should send **time exceeded**
3. Destination eventually responds (port unreachable / echo)

```
traceroute 192.0.2.1
# or
mtr -r -c 5 192.0.2.1
```

Limits: load balancers, ICMP rate limits, firewalls that hide hops, asymmetric return paths.

Path MTU Discovery (preview)

- Senders prefer **not** to fragment (IPv6: no in-path fragmentation)
- Hop returns **Packet Too Big** / frag-needed if DF set and MTU smaller
- Broken PMTUD → hangs for large transfers while small pings work

Deep labs: Layer 3 ICMP & Path MTU chapter.

Filters and policy

Many networks rate-limit or block ICMP. That improves some attacks and **hurts** diagnostics and PMTUD. Lab hygiene: know your nftables/iptables ICMP rules.

```
# Example observability only-do not paste blindly into production
nft list ruleset
iptables -L -n
```

Lab sketch

1. Three-node line (A—B—C)
2. Ping A→C; traceroute A→C
3. ACL drop ICMP on B: compare symptoms

Checkpoint

Why can `ping` work while `curl http://…` fails? Why can `traceroute` show `* * *` on middle hops while the destination still answers?

Next

- Transport: TCP and UDP

Transport: TCP and UDP

Transport: TCP and UDP

The **transport layer** delivers data to the correct **process** on a host using **ports**, with different reliability and connection models.

Learning goals

- Explain 5-tuples and port roles
- Contrast TCP and UDP services
- Read basic TCP handshake and teardown signals
- Use `ss` to inspect listeners and connections

Ports and sockets

- Port: 16-bit number (0–65535)
- **Well-known** ports (e.g. 22 SSH, 53 DNS, 80 HTTP) vs **ephemeral** client ports
- Socket: local address + port (+ protocol), often with peer

5-tuple: protocol, src IP, src port, dst IP, dst port — identifies a conversation.

```
ss -tulpn
ss -tan
ss -uan
```

UDP

Property	UDP
Connection	None
Reliability	None (app may add)
Ordering	Not guaranteed
Header	Lightweight
Use cases	DNS, DHCP, many tunnels, telemetry, real-time

```
# one-shot listener / client ideas
nc -u -l -p 9999
nc -u <peer> 9999
```

TCP

Property	TCP
Connection	Yes (stateful)
Reliability	Ack, retransmit
Ordering	Byte stream reassembly
Flow / congestion control	Yes
Use cases	SSH, HTTP, BGP, config APIs

Three-way handshake (must know)

1. Client → Server: **SYN**
2. Server → Client: **SYN-ACK**
3. Client → Server: **ACK**

Then data can flow. Teardown uses **FIN/ACK** (or **RST** for abort).

```
tcpdump -ni eth1 'tcp port 22'
```

States (operator view)

State	Meaning
LISTEN	Waiting for connections
SYN-SENT / SYN-RECV	Handshake in progress
ESTABLISHED	Data transfer
TIME-WAIT	Waiting to be sure old segments die
CLOSE-WAIT / FIN-WAIT	Teardown in progress

TCP vs UDP selection

Need	Prefer
Reliable byte stream	TCP

Need	Prefer
Simple request/response, app handles loss	UDP
Multicast / lightweight tunnel outer	Often UDP
Head-of-line sensitivity	Design carefully (HTTP/2 vs QUIC later topics)

NAT and transports (preview)

NATS track **transport** state (especially TCP). UDP mappings time out differently. Edge/NAT chapter expands this.

Security basics

- **Do not** expose admin TCP ports to untrusted networks without policy
- Spoofing and scan noise are normal on open nets
- TLS usually rides **on TCP** (or QUIC on UDP)—app layer crypto

TCP state machine (operator view)

```
CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT_* → TIME_WAIT → CLOSED
                (listen) LISTEN → SYN_RCVD → ESTABLISHED → CLOSE_WAIT → LAST_ACK → CLOSED
```

State you see (`ss` / `netstat`)	Meaning
`LISTEN`	Server bound, waiting for SYN
`ESTAB` / `ESTABLISHED`	Data can flow
`TIME-WAIT`	Local side closed; holding tuple so delayed segments die (normal after busy services)
`CLOSE-WAIT`	Peer closed; **your app** must close its side (often a bug if stuck)
`SYN-SENT`	Client waiting for SYN-ACK (firewall/blackhole if stuck)

Flow control vs congestion control: window advertisements limit *receiver* buffer pressure; congestion control (slow start, Reno/CUBIC, …) limits *network* load. Both shrink effective throughput; do not "tune sysctls" before measuring.

UDP: no handshake, no stream—each datagram is independent. Good for DNS, DHCP, QUIC's outer path, and lab pings of custom apps. Reliability, if needed, is **your** application's job.

Three-way handshake and teardown (why captures look busy)

1. Client → Server: `SYN` (seq=c)
2. Server → Client: `SYN-ACK` (seq=s, ack=c+1)
3. Client → Server: `ACK` (ack=s+1)

Teardown is often four segments (`FIN/ACK` each way) or `RST` on abort. In labs, filter `tcp.flags.syn==1 or tcp.flags.fin==1 or tcp.flags.reset==1` in Wireshark/tshark to see only control.

Lab sketch

1. `ss -tulpn` on a node before/after starting SSH
2. Capture handshake for SSH (`tcpdump -ni any port 22`)
3. `nc` UDP echo between two lab hosts
4. Force a stuck `CLOSE-WAIT` by killing only one side of a netcat TCP session; observe with `ss -tan`
5. Note `TIME-WAIT` count after many short connections (`ss -tan state time-wait | wc -l`)

Checkpoint

A client opens HTTPS to a server. Name protocol numbers/fields for: Ethernet type, IP protocol, TCP ports (client ephemeral + 443), and where TLS sits.

Next

- Deep dive: Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall
- Common Application Protocols

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall

Two TCP features that each save packets can together add a fixed **40 ms** to every request. **Nagle's algorithm** (sender side) holds a small segment while an earlier small segment is still unacknowledged. **Delayed ACK** (receiver side) waits, up to a timer, before acknowledging, hoping to piggyback the ACK on a reply. When a client sends one request as *two* small writes and then waits for the answer, the server cannot reply (it has only half the request), so it delays its ACK; the client cannot send the second half until that ACK arrives. Both sides wait for the timer. Nothing is lost, nothing is retransmitted, CPU is idle, and p50 latency is 40 ms on a link with a 20 µs RTT. This chapter reproduces the stall on loopback, catches it in `ss`, and measures every fix, including the one people reach for first (`TCP_NODELAY`) and what it costs.

Learning goals

- State Nagle's rule (RFC 896, RFC 9293 §3.7.4) and Linux's Minshall variant, and the delayed-ACK rule (RFC 9293 §3.8.6.3)
- Recognise the write-write-read pattern and predict the stall from it
- Read `ato`, `unacked` and `notsent` in `ss -ti` while the stall is happening
- Compare the fixes: one write (`writev`/`sendmsg`), `TCP_CORK`, `MSG_MORE`, `TCP_NODELAY` and `TCP_QUICKACK`, and say which side each one acts on
- Explain what Nagle protects against, and measure what `TCP_NODELAY` costs in segments

Lab platform: Linux 6.12, iproute2 6.15 (`ss`, `nstat`), Python 3.13.5, run as an unprivileged user over `127.0.0.1`. No namespaces, no `tc`, no root. Every listing output below is pasted from those runs. Kernel behaviour is quoted from `include/net/tcp.h` and `net/ipv4/tcp_output.c` in Linux 6.12; the same constants and `tcp_nagle_check()` are unchanged in Linux 7.2, the current stable series.

Mental model

```
client                                          server
   write(hdr, 10 B)   sent: nothing unacked     has half a request
   write(body, 10 B)                               nothing to send yet
     Nagle: a small segment is unacked → HOLD      delayed ACK: wait (ato 40 ms)
                       ... up to ~40 ms ...
          bare ACK (delack timer)
   body released                    full request → reply
```

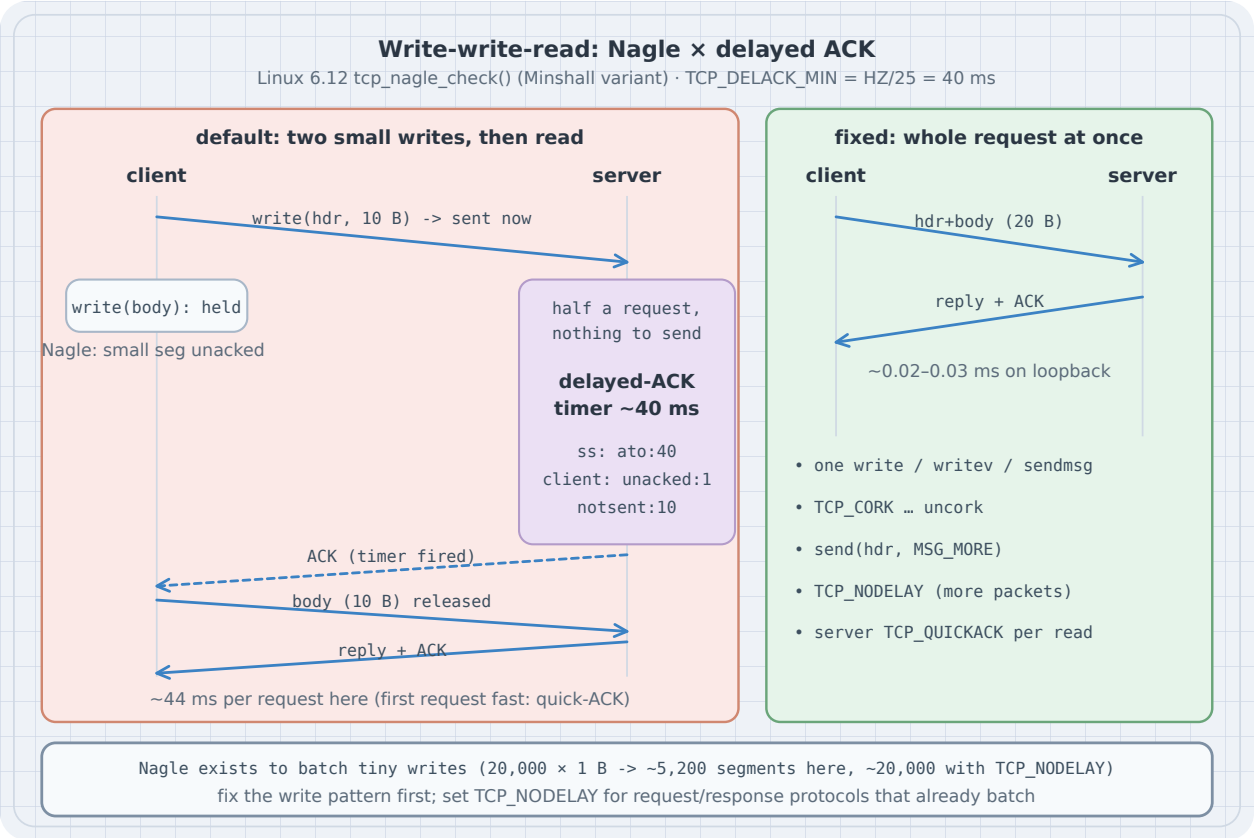


Figure 1: Write-write-read: Nagle × delayed ACK

```
                reply (+ ACK)
```

The two rules:

Rule	Where	What it says	Linux
Nagle (RFC 896; RFC 9293 §3.7.4, SHLD-7, with a per-connection off switch, MUST-17)	Sender	While data is unacknowledged, buffer small writes until the ACK arrives or a full MSS is ready	`tcp_nagle_check()` with **Minshall's variant** (RFC 9293 App. A.3): hold a sub-MSS segment only while a previous *sub-MSS* segment is unacknowledged
Delayed ACK (RFC 1122 §4.2.3.2; RFC 9293 §3.8.6.3, SHLD-18/19, MUST-40)	Receiver	ACK at least every second full-sized segment; otherwise wait (less than 0.5 s) to piggyback the ACK on data	`TCP_DELACK_MIN` = `TCP_ATO_MIN` = `HZ/25` = **40 ms**, `TCP_DELACK_MAX` = `HZ/5` = 200 ms; a new connection starts in **quick-ACK** mode (up to `TCP_MAX_QUICKACKS` = 16 immediate ACKs)

Neither rule is a bug. Together they deadlock until the timer fires, whenever all of these hold:

1. The sender writes one logical message as **two or more sub-MSS writes**.
2. The receiver **cannot respond** until it has the whole message.
3. The sender **waits for the response** before writing more.

HTTP/1.x clients that write headers and body separately, RPC frameworks that write a length prefix and then the payload, database drivers, Redis/memcached clients and SSH-like interactive protocols all fit this pattern unless they batch their writes or set `TCP_NODELAY`.

Concepts — the knobs

Knob	Side	Scope	Effect
One write: build the message in user space, or `writev()`/`sendmsg()` with an iovec	Sender	Per message	The message leaves as one segment; Nagle has nothing to hold. Portable.

Knob	Side	Scope	Effect
`TCP_CORK` (Linux)	Sender	Until cleared	Hold partial segments; clearing it sends them now. 200 ms ceiling. Overrides `TCP_NODELAY`.
`MSG_MORE` flag on `send()` (Linux)	Sender	One call	Same as `TCP_CORK` for that call: "more is coming"; the next send without the flag pushes.
`TCP_NODELAY`	Sender	Socket	Disable Nagle: every write is sent as soon as cwnd/rwnd allow. Portable (RFC 9293 MUST-17).
`TCP_QUICKACK` (Linux)	Receiver	**Not sticky**	Enter quick-ACK mode now. The stack drops back to delayed ACKs on its own (`tcp(7)`: "This flag is not permanent").
`net.ipv4.tcp_autocorking` (default 1)	Sender	Host	Coalesce small writes while an earlier packet is still queued in the host. Counter `TcpExtTCPAutoCorking`.
`ip route ... quickack 1`	Receiver	Route	Quick-ACK for every connection on that route (needs root; not run here).

Lab — reproduce the stall and measure the fixes

```
# nagle_lab.py - write-write-read over loopback: the Nagle x delayed-ACK stall, and the ways
# usage: python3 nagle_lab.py [REQUESTS]
import socket, statistics, struct, sys, threading, time

HDR, BODY = b"H" * 10, b"B" * 10          # one request, sent as two small writes
REQ, RESP = len(HDR) + len(BODY), b"R" * 20
N = int(sys.argv[1]) if len(sys.argv) > 1 else 50

def data_segs_out(s):                     # struct tcp_info -> tcpi_data_segs_out
    f = struct.unpack("8B24I4Q6I", s.getsockopt(socket.IPPROTO_TCP, socket.TCP_INFO, 160))
```

```
    return f[41]

def recv_exact(s, n):
    buf = b""
    while len(buf) < n:
        chunk = s.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def server(lsock, quickack):
    c, _ = lsock.accept()
    if quickack == "once":
        c.setsockopt(socket.IPPROTO_TCP, socket.TCP_QUICKACK, 1)
    for _ in range(N):
        if quickack == "every read":      # not sticky: re-arm before every read
            c.setsockopt(socket.IPPROTO_TCP, socket.TCP_QUICKACK, 1)
        recv_exact(c, REQ)
        c.sendall(RESP)
    c.close()

def run(mode):
    lsock = socket.create_server(("127.0.0.1", 0))
    quickack = {"QUICKACK once": "once", "QUICKACK every read": "every read"}.get(mode)
    t = threading.Thread(target=server, args=(lsock, quickack))
    t.start()
    s = socket.create_connection(lsock.getsockname())
    if mode == "TCP_NODELAY":
        s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    lat = []
    for _ in range(N):
        t0 = time.perf_counter()
        if mode == "one write":
            s.sendall(HDR + BODY)
        elif mode == "TCP_CORK":
            s.setsockopt(socket.IPPROTO_TCP, socket.TCP_CORK, 1)
            s.sendall(HDR); s.sendall(BODY)
            s.setsockopt(socket.IPPROTO_TCP, socket.TCP_CORK, 0)   # uncork = send now
        elif mode == "MSG_MORE":
            s.send(HDR, socket.MSG_MORE); s.sendall(BODY)
        else:                             # default, TCP_NODELAY, server-side QUICKACK
            s.sendall(HDR); s.sendall(BODY)
        recv_exact(s, len(RESP))
        lat.append((time.perf_counter() - t0) * 1e3)
    segs = data_segs_out(s)
```

```
    s.close(); t.join(); lsock.close()
    first = " ".join(f"{x:.2f}" for x in lat[:3])
    lat.sort()
    print(f"{mode:19} first3 [{first}]  median {statistics.median(lat):6.3f} ms"
          f"  p90 {lat[int(0.9 * N)]:6.3f} ms  total {sum(lat):6.1f} ms  data segs {segs:3}"

print(f"{N} requests of {REQ} bytes (two writes of 10), {len(RESP)}-byte reply, 127.0.0.1")
for mode in ("Nagle (default)", "TCP_NODELAY", "one write", "TCP_CORK", "MSG_MORE",
             "QUICKACK once", "QUICKACK every read"):
    run(mode)
```

```
nstat -n                  # reset the nstat history for this network namespace
python3 nagle_lab.py
nstat TcpExtDelayedACKs   # delayed ACKs sent since the reset
```

```
50 requests of 20 bytes (two writes of 10), 20-byte reply, 127.0.0.1
Nagle (default)     first3 [0.26 42.26 44.12]  median 44.014 ms  p90 44.163 ms  total 2154.4
TCP_NODELAY         first3 [0.43 0.38 0.49]  median  0.031 ms  p90  0.070 ms  total    2.9 m
one write           first3 [0.25 0.04 0.06]  median  0.029 ms  p90  0.032 ms  total    1.7 m
TCP_CORK            first3 [0.10 0.04 0.03]  median  0.031 ms  p90  0.034 ms  total    1.6 m
MSG_MORE            first3 [0.08 0.04 0.03]  median  0.030 ms  p90  0.033 ms  total    1.6 m
QUICKACK once       first3 [0.10 40.85 43.98]  median 43.990 ms  p90 44.201 ms  total 2152.9
QUICKACK every read first3 [0.10 0.06 0.04]  median  0.036 ms  p90  0.039 ms  total    1.9 m
#kernel
TcpExtDelayedACKs               134                0.0
```

Read it line by line:

- **Nagle (default):** the first request takes 0.26 ms: the connection is fresh and in quick-ACK mode, so the server ACKs the header at once. From the second request on, every request takes about **44 ms**: the 40 ms minimum delayed-ACK timeout plus a few milliseconds of timer granularity. Fifty requests take 2.15 s instead of 2 ms. The client sent 100 data segments: header and body separately.
- **TCP_NODELAY:** same two writes, same 100 segments, but the body is not held: median 31 µs.
- **one write / TCP_CORK / MSG_MORE:** the request leaves as **one** segment (50 data segments for 50 requests) and the stall is gone. This is the best result: fast *and* half the packets.
- **QUICKACK once:** setting `TCP_QUICKACK` on the server right after `accept()` does nothing useful. The connection is already in quick-ACK mode then, and it leaves that mode by itself after a few exchanges: the stall starts at request two, exactly like the default.
- **QUICKACK every read:** re-arming it before every `recv()` fixes the stall from the receiver side, with 100 segments. That works when you own the server but not the client; it is also Linux-only and costs a system call per read.
- `TcpExtDelayedACKs` counted 134: about 98 from the two stalling runs (49 stalled requests each), and the rest from other traffic in this shared network namespace. `nstat` counters are per network namespace, not per process.

Lab — catch the stall in ss

```
# catch_stall.py - freeze the moment of the stall and look at both sockets with ss -ti
import socket, subprocess, threading, time

lsock = socket.create_server(("127.0.0.1", 0))
port = lsock.getsockname()[1]
ready = threading.Event()

def server():
    c, _ = lsock.accept()
    for _ in range(3):                      # warm up: leave the initial quick-ACK phase
        c.recv(20); c.sendall(b"R" * 20)
    ready.set()
    c.recv(10)                              # got the header, still waiting for the body
    time.sleep(0.2); c.close()

t = threading.Thread(target=server); t.start()
s = socket.create_connection(("127.0.0.1", port))
for _ in range(3):
    s.sendall(b"Q" * 20); s.recv(20)
ready.wait()
s.sendall(b"H" * 10)                        # goes out at once: nothing unacknowledged
s.sendall(b"B" * 10)                        # Nagle holds this: a small segment is unacked
time.sleep(0.005)                           # 5 ms into the up-to-40 ms wait
out = subprocess.run(["ss", "-tniH", "state", "established", f"( sport = :{port} or dport =
                      capture_output=True, text=True).stdout
keep = ("ato", "mss", "unacked", "notsent", "lastsnd", "lastack", "bytes_sent", "bytes_recei
lines = out.splitlines()
for addrs, info in zip(lines[0::2], lines[1::2]):
    local, peer = addrs.split()[2:4]
    who = "server" if local.endswith(f":{port}") else "client"
    print(f"{who}  {local} -> {peer}")
    print("        " + " ".join(w for w in info.split() if w.split(":")[0] in keep))
s.close(); t.join()
```

```
python3 catch_stall.py
```

```
server  127.0.0.1:45499 -> 127.0.0.1:45560
        ato:40 mss:32768 bytes_sent:60 bytes_received:70 lastsnd:8 lastack:8
client  127.0.0.1:45560 -> 127.0.0.1:45499
        ato:40 mss:32768 bytes_sent:70 bytes_received:60 lastsnd:8 lastack:8 unacked:1 notse
```

Eight milliseconds into the stall the evidence is in two fields. On the **client**: `unacked:1` (the header segment is in flight, unacknowledged) and **`notsent:10`** (the 10-byte body is in the send queue and Nagle has not sent it). On the **server**: `ato:40`, the delayed-ACK timeout, and nothing to

send. `bytes_sent:70` on the client is three 20-byte warm-up requests plus the header; the body is not counted until it leaves. In production, `ss -tni` showing a small, persistent `notsent` on a request/response socket whose peer shows `ato:40` is the stall. A packet capture shows it as a 40 ms gap between a bare ACK from the server and the client's next small segment.

Lab — what Nagle is for

```
# tiny_writes.py - what Nagle is FOR: 20,000 one-byte writes, counted in segments on the wir
import socket, struct, threading, time

WRITES = 20_000

def data_segs_out(s):
    f = struct.unpack("8B24I4Q6I", s.getsockopt(socket.IPPROTO_TCP, socket.TCP_INFO, 160))
    return f[41]

def drain(lsock, got):
    c, _ = lsock.accept()
    n = 0
    while chunk := c.recv(65536):
        n += len(chunk)
    got.append(n); c.close()

for nodelay in (0, 1, 0, 1):
    lsock = socket.create_server(("127.0.0.1", 0)); got = []
    t = threading.Thread(target=drain, args=(lsock, got)); t.start()
    s = socket.create_connection(lsock.getsockname())
    s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, nodelay)
    t0 = time.perf_counter()
    for _ in range(WRITES):
        s.send(b"x")
    dt = time.perf_counter() - t0
    segs = data_segs_out(s)
    s.close(); t.join(); lsock.close()
    print(f"TCP_NODELAY={nodelay}  {WRITES} writes -> {segs:6} data segments"
          f"  ({WRITES / segs:5.1f} bytes/segment)  {dt * 1e3:6.1f} ms  received {got[0]}")
```

```
python3 tiny_writes.py
```

```
TCP_NODELAY=0  20000 writes ->   5418 data segments  (  3.7 bytes/segment)    36.4 ms  recei
TCP_NODELAY=1  20000 writes ->  19135 data segments  (  1.0 bytes/segment)    79.1 ms  recei
TCP_NODELAY=0  20000 writes ->   5175 data segments  (  3.9 bytes/segment)    39.8 ms  recei
TCP_NODELAY=1  20000 writes ->  20000 data segments  (  1.0 bytes/segment)    81.4 ms  recei
```

An application that writes one byte at a time (unbuffered logging, keystrokes, a naive serializer)

is what Nagle was designed for in 1984. With Nagle on, 20,000 writes left as about 5,200–5,400 segments; with `TCP_NODELAY` as 19,135–20,000 segments (the few merges come from autocorking). Each segment carries 40+ bytes of IP and TCP headers for 1 byte of data. On loopback the ACK comes back in microseconds, so Nagle can only batch about 4 bytes per segment. On a real path with a 20 ms RTT Nagle allows at most one small segment in flight per round trip, so the same loop (about 40 ms of writes) would leave as a handful of segments, each carrying everything written during one round trip (not run here: adding delay needs `tc netem`, see Try this). That is the cost of setting `TCP_NODELAY` everywhere without also batching writes: more packets, more interrupts, more per-packet CPU on every hop, and on lossy or metered links (cellular, satellite, IoT) a visible bill.

Case — choosing the fix

Situation	Fix
You own the sender code	Build the whole message and write it once (`writev`/`sendmsg`, a buffered writer you flush per message). Then decide on `TCP_NODELAY` separately.
Request/response protocol whose library already writes each message once (gRPC, HTTP/2, most modern RPC and DB drivers)	`TCP_NODELAY` on. Most such libraries already set it: Go's `net` package sets it on every TCP connection by default; Node.js `http.createServer` defaults to `noDelay: true` since v18, while a raw `net.Socket` still starts with Nagle on. Check yours.
Header + `sendfile()` body (static file servers)	`TCP_CORK` around the header write and the `sendfile()`, then uncork. Or `MSG_MORE` on the header.
Bulk transfer (backups, replication streams)	Nothing: writes are MSS, Nagle never holds them.
You own only the receiver and cannot change clients	`TCP_QUICKACK` before every read, or a `quickack 1` route for those clients' prefix. Treat it as a workaround.
Interactive byte streams (terminals)	Depends on what matters more: latency (SSH sets `TCP_NODELAY` for interactive sessions) or packet count.

The trap

Trap: "it's the network." The stall is entirely host-side: no loss, no retransmission, no queueing. Pings are fast, `ss` shows a tiny RTT (tens of microseconds on loopback), yet requests take 40 ms. The tell is latency that clusters at ~**40 ms** (or ~200 ms on stacks whose delayed-ACK timer is 200 ms) regardless of path RTT or load.

Trap: "`TCP_NODELAY` everywhere." It removes the stall but not the cause, two writes per message. You now send twice the packets (100 vs 50 data segments in the first lab), and a chatty writer can send one packet per byte (third lab). Batch first; then `TCP_NODELAY` is cheap.

Trap: "set `TCP_QUICKACK` once." It is not a mode you turn on; it is a nudge. The first lab shows "QUICKACK once" stalls exactly like the default.

Trap: "`TCP_CORK` and forget." A corked socket holds partial data up to 200 ms. Forgetting to uncork turns a 40 ms stall into a 200 ms one. `MSG_MORE` cannot be forgotten: it applies to one call.

Trap: "the first request was fast, so we're fine." Quick-ACK mode at connection start hides the stall for the first exchange. Benchmarks that open a new connection per request never see it; connection pools see it on every request.

The boring rule

Write each message with one system call. Then turn on `TCP_NODELAY` for request/response protocols, and leave Nagle on for streams of tiny writes you cannot batch. When latency clusters at 40 ms, look for `notsent` on the sender and `ato:40` on the receiver before you look at the network.

Try this

1. Change `HDR` to 1,000 bytes and `BODY` to 70,000 bytes in `nagle_lab.py`. Does the default still stall? Why does a body larger than one MSS change the answer?
2. Make the server write its reply as two 10-byte writes too. Which side stalls now, and does client-side `TCP_NODELAY` still fix it?
3. Run `nagle_lab.py` with `N=200` and plot the latency of each request. How many fast requests do you get before the stall starts on this kernel?
4. With a capture tool (`tcpdump -ni lo port PORT`) where one is available, find the bare ACK and the body segment around each 40 ms gap.
5. On a machine you control, add a 20 ms delay to a test interface (`tc qdisc add dev IF root netem delay 20ms`, as root or in a network namespace) and re-run `tiny_writes.py`. How many bytes per segment does Nagle batch now?

Next

- Common Application Protocols

Verification

- Run unprivileged on this box: Debian 13, Linux **6.12.94+**, iproute2 **6.15.0** (`ss`, `nstat`), Python **3.13.5**, `tcp_congestion_control=cubic`, `tcp_autocorking=1`, loopback MTU 65536. All outputs above are pasted from those runs; timings vary by a few percent between runs.
- Kernel constants and logic checked in Linux 6.12 source: `TCP_DELACK_MIN`/`TCP_ATO_MIN` = `HZ/25`, `TCP_DELACK_MAX` = `HZ/5`, `TCP_MAX_QUICKACKS` = 16 (`include/net/tcp.h`); `tcp_nagle_check()` and `tcp_minshall_check()` (`net/ipv4/tcp_output.c`), and re-checked unchanged in the `linux-7.2.y` tree. Linux 7.2.11 is current stable (kernel.org, 2026-10-18).
- **Not run:** `tcpdump` (not installed; nothing was installed for this chapter), the `ip route ... quickack 1` route attribute and `tc netem` delay (both need root or a separate network namespace, which this run did not use). These are described from `ip-route(8)`, `tc-netem(8)` and the kernel source.
- `nstat` counters are per network namespace; the delta above includes unrelated traffic in a shared namespace, as noted.
- No packages installed, no `sudo`, no sysctl changed.

Sources

- RFC 896 — Congestion Control in IP/TCP Internetworks (Nagle, 1984)
- RFC 9293 — Transmission Control Protocol: §3.7.4 Nagle Algorithm (SHLD-7, MUST-17), §3.8.6.3 Delayed Acknowledgments (SHLD-18, SHLD-19, MUST-40), Appendix A.3 Nagle Modification
- RFC 1122 — Requirements for Internet Hosts §4.2.3.2 (delayed ACK), §4.2.3.4 (Nagle)
- RFC 5681 — TCP Congestion Control §4.2 (generating acknowledgments)
- G. Minshall, *A Suggested Modification to Nagle's Algorithm* (Internet-Draft, 1998), reference [68] in RFC 9293
- Linux 6.12 `include/net/tcp.h` and `net/ipv4/tcp_output.c` (`tcp_nagle_check`, `tcp_minshall_check`): https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/ipv4/tcp_output.c?h=linux-6.12.y
- Kernel docs — IP sysctl: `tcp_autocorking`
- man7 — `tcp(7)` (`TCP_NODELAY`, `TCP_CORK`, `TCP_QUICKACK`, `TCP_INFO`), `send(2)` (`MSG_MORE`), `ss(8)`, `ip-route(8)` (`quickack`)
- Go — `net.TCPConn.SetNoDelay` ("The default is true (no delay)"); Node.js — `http.createServer noDelay`, `net.Socket noDelay`

Common Application Protocols

Common Application Protocols

Application protocols give **meaning** to payloads. Operators still need a working vocabulary: DNS, DHCP, HTTP, SSH, and TLS appear in almost every network design.

Learning goals

- Place common apps on TCP/UDP and well-known ports
- Describe DNS resolution flow at a high level
- Describe DHCP DORA and where relays fit
- Separate HTTP from TLS and from TCP failures
- Know what SSH needs from the network

Map (operator cheat sheet)

Protocol	Transport	Port(s)	Role
DNS	UDP/TCP	53	Name → address (and more)
DHCP	UDP	67/68	IPv4 host config
DHCPv6	UDP	546/547	IPv6 host config
HTTP	TCP	80	Web (cleartext)
HTTPS	TCP	443	HTTP over TLS
SSH	TCP	22	Secure shell / SFTP
NTP	UDP	123	Time
BGP	TCP	179	Routing (control plane app)
SNMP	UDP	161/162	Legacy management

DNS

Roles

Role	Function
Stub resolver	On the host; asks a recursive resolver
Recursive resolver	Walks the tree for the answer
Authoritative server	Holds zone data

Query types (common)

Type	Meaning
A	IPv4 address
AAAA	IPv6 address
CNAME	Alias
MX / TXT / SRV	Mail, text, services

```
# Linux lab host examples
getent hosts example.com
# if dig/nslookup installed:
# dig +short A example.com
# dig +short AAAA example.com
```

Failure patterns

Symptom	Check
Names fail, IPs work	DNS config, UDP/53 filters, resolver reachability
Slow apps	Timeouts to dead resolvers
Wrong address	Cache, split DNS, hosts file

DHCP (IPv4)

DORA: Discover → Offer → Request → Ack

- Client uses broadcasts initially (needs L2 reachability to server or **relay**)
- Server assigns IP, mask, gateway, DNS, lease time

```
Client --L2 domain-- DHCP server
Client --L2-- Relay --L3-- DHCP server
```

Edge designs with helpers/relays: later Edge & services part.

HTTP and HTTPS

```
HTTP:  App  TCP:80  Server
HTTPS: App  TCP:443  TLS   HTTP   Server
```

Failure	Layer to inspect
TCP SYN timeout	Path, ACL, listen socket
TCP up, TLS error	Certificates, SNI, protocol mismatch
TLS up, HTTP 5xx	Application

```
curl -v http://192.0.2.10/
curl -vk https://192.0.2.10/
```

SSH

- TCP/22 (by default)
- Needs bidirectional path and allowed security policy
- Used heavily for lab and production management

```
ssh user@192.0.2.10
ss -tlnp | grep 22
```

TLS in one paragraph

TLS provides encryption and authentication **above** TCP (classically). Network engineers care when: MTU/PMTUD breaks large records, middleboxes interfere, or certificates expire. Crypto suite deep-dives are optional here.

NTP and time

Broken time breaks auth (Kerberos, cert validation) and log correlation. Lab VMs often need chrony/ntp or host sync.

Lab sketch

1. DNS: break `/etc/resolv.conf`; compare `ping` by name vs by IP
2. HTTP: capture TCP/80 three-way + first GET
3. SSH: confirm LISTEN on 22; ACL drop and observe client timeout

Checkpoint

User says “the network is down” but `ping` by IP works and `curl` by name fails. List the top three protocol checks in order.

Next

- Host Forwarding and Routing Intro

Host Forwarding and Routing Intro

Host Forwarding and Routing Intro

Every host is a mini-router for its **own** traffic: connected routes, default gateway, and policy policy later. This chapter is host-centric; multi-router IGP/BGP comes in later parts.

Learning goals

- Read a Linux routing table
- Explain longest-prefix match (LPM)
- Configure a default route and a static host route
- Distinguish RIB vs what `ip route get` uses
- Enable IP forwarding on a Linux router node (preview)

Connected vs remote

Destination	Host behavior
In a connected prefix	ARP/ND on that interface; no gateway
Not connected	Lookup route → next hop → ARP/ND for next hop

Linux route table essentials

```
ip route
ip -6 route
ip route get 1.1.1.1
ip route get 192.168.10.50
```

Common entry types:

Entry	Meaning
`default via G dev IF`	Default gateway
`P.P.P.P/nn dev IF`	Connected or direct
`H via G`	Host route (/32)

Entry	Meaning
metric / proto	Preference and source (dhcp, static, …)

Longest prefix match

More specific prefixes win:

```
0.0.0.0/0        via 10.0.0.1
10.0.0.0/8       via 10.0.0.2
10.1.1.0/24      dev eth2
```

Destination `10.1.1.5` → `eth2` connected.
Destination `10.2.0.1` → via `10.0.0.2`.
Destination `8.8.8.8` → default via `10.0.0.1`.

Default gateway

Most endpoints need one:

```
ip route add default via 10.20.1.1 dev eth1
```

Without it, on-link works and off-link fails—classic "I can ping the gateway but not the internet."

Static routes (host or simple router)

```
ip route add 10.30.0.0/16 via 10.20.1.2
ip route add 192.0.2.10/32 via 10.20.1.3
```

Multi-router static designs and floating statics: Layer 3 static routing labs.

IP forwarding (Linux as router)

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
sysctl -w net.ipv6.conf.all.forwarding=1
```

With forwarding on, Linux can move packets between interfaces per the FIB—foundation of FRR data plane coexistence in labs.

Policy routing (awareness)

Multiple tables and `ip rule` exist for VRFs and advanced designs. Foundations: know they exist; do not start there.

Firewall vs routing

A **correct route** with a **drop rule** still fails. Always separate:

1. Is there a route? (`ip route get`)
2. Does L2 resolve? (`ip neigh`)
3. Does policy allow? (nft/iptables, security groups)

Lab sketch

1. Two subnets, one Linux router
2. Hosts with defaults pointing at the router
3. `ip route get` on host and router for east-west traffic
4. Break default on host; compare symptoms

Checkpoint

Host `10.1.1.10/24`, gateway `10.1.1.1`. Route table has only the connected /24. What works? What fails? What single command fixes off-subnet reachability?

Next

- Control, Data, and Management Planes

Control, Data, and Management Planes

Control, Data, and Management Planes

Separate **how the network decides**, **how it forwards**, and **how you operate it**. This split keeps troubleshooting honest from first lab to fabric design.

Learning goals

- Define data, control, and management planes with examples
- Classify failures into the right plane
- Pick evidence (tables vs protocols vs access path) per plane
- Connect planes to tools used in this book

Data plane

Forwards user and transit packets using programmed tables.

Examples	Evidence
Ethernet FDB / MAC table	`bridge fdb`, switch MAC table
IP FIB / routes used for forward	`ip route get`, CEF/FIB show
ACL counters / nft hits	Counters increment on matches
Queues / drops	Interface drop stats

Failures: blackhole route, wrong next hop, MTU drop, ACL drop, congestion.

Control plane

Builds and maintains the information data plane uses.

Examples	Evidence
ARP/ND	`ip neigh`
OSPF/BGP adjacencies	protocol show commands
MAC learning	dynamic FDB entries
Route computation	RIB entries, SPF/BGP path choice

Failures: adjacency down, authentication mismatch, policy filter, CPU overload, CoPP drops.

Management plane

How humans and automation configure and observe devices.

Examples	Evidence
SSH, HTTPS API, NETCONF	Reachability to mgmt VRF/IP
Telemetry, syslog, SNMP	Streams arrive at collectors
Console / OOB	Still works when in-band dies

Failures: ACL on vty, expired credentials, mgmt routing broken, "network down" that is only mgmt path down.

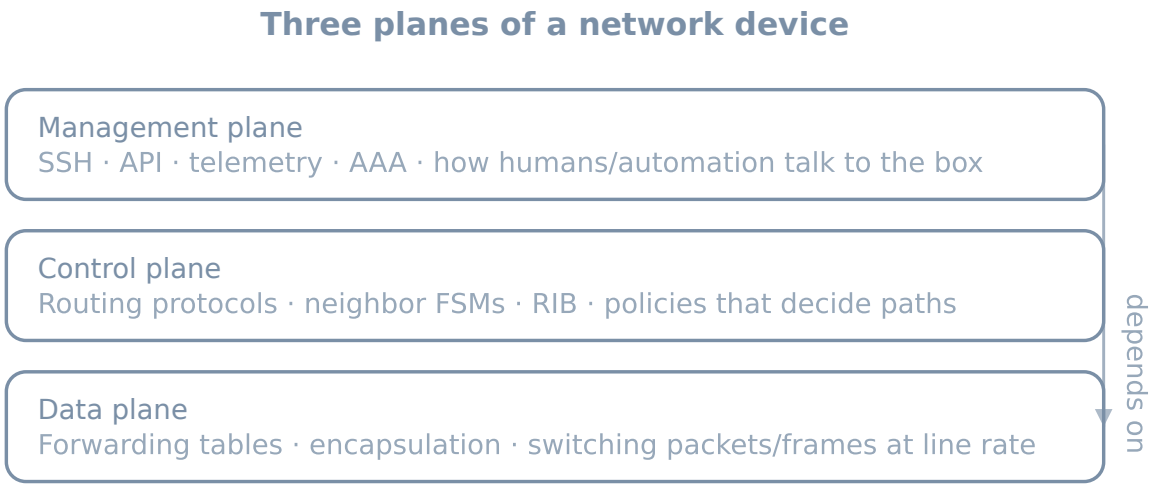


Figure 1: Planes

Interaction

```
Management  →  configures  →  Control
Control     →  programs    →  Data
Data        →  forwards       user packets
```

A BGP session (control) programs the FIB (data). You establish BGP over TCP (data plane for the control session—meta but real).

Troubleshooting order (habit)

1. **Management:** Can I access the device?
2. **Data:** Does `ping`/`curl` of interest work? What does `ip route get` say?
3. **Control:** Are neighbors up? Are routes present in the RIB?
4. Reconcile: RIB has route but FIB/policy drops?

Lab habit

When something fails, write one sentence: *"I think this is a **data / control / management** plane fault because …; I will prove it with …."*

Checkpoint

SSH to a router fails, but the router still forwards traffic between two lab hosts. Which plane is most likely broken for you, and which is still working?

Next

Depth continues in Ethernet & MAC Learning. Host and Containerlab skills remain in Lab platform.

Layer 2

Ethernet & MAC Learning

Ethernet & MAC Learning

Layer 2 is where frames move inside a broadcast domain. Before VLANs and spanning tree, master **Ethernet addressing, flooding, and MAC learning**—the behaviors every switch and bridge still implements.

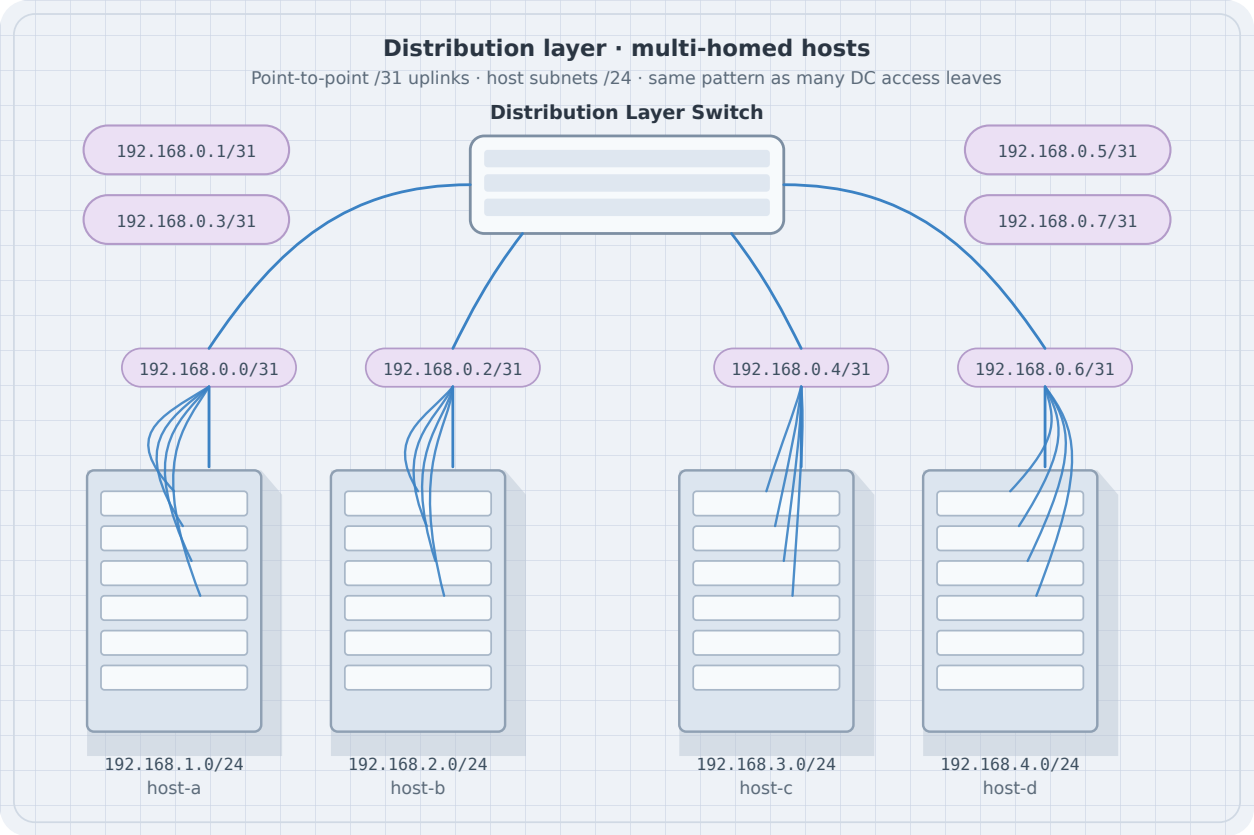


Figure 1: Distribution switch multi-home topology

Learning goals

By the end of this chapter you can:

- Explain source learning and unknown unicast flooding
- Read and interpret a MAC/FDB table

- Predict ARP's role binding IP to MAC on-link
- Build a small bridged lab with Linux or Containerlab
- Diagnose "same subnet but no ping" with neigh + FDB + capture

Concepts

Frame essentials

An Ethernet frame carries:

- Destination MAC
- Source MAC
- Ethertype (e.g. IPv4 `0x0800`, ARP `0x0806`, IPv6 `0x86DD`, 802.1Q tag)
- Payload
- FCS (hardware)

Switches forward using **destination MAC** and the VLAN context (next chapter). They **learn** from **source MAC**.

Learning and flooding

Event	Switch action
Frame arrives with source MAC S on port P	Learn S → P (for that VLAN)
Destination MAC known	Forward out that port only (not ingress)
Destination unknown unicast	Flood within VLAN
Broadcast (ff:ff:ff:ff:ff:ff)	Flood within VLAN
Multicast	Flood or constrain (IGMP snooping later world)

Stale entries age out (aging timer). Moves update the port mapping—important when you relocate a VM/container.

MAC addresses

- 48-bit, usually written `aa:bb:cc:dd:ee:ff`
- Locally administered vs universal bits exist; labs often show container random MACs

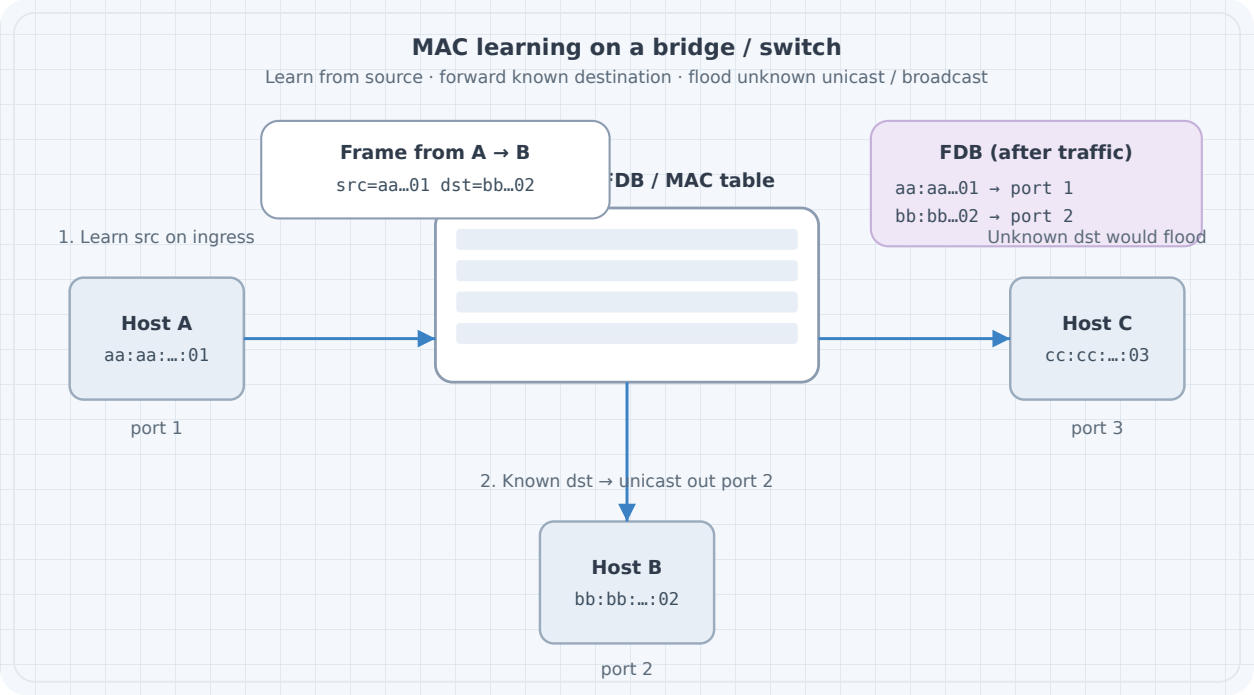


Figure 2: MAC learning: learn source, forward or flood destination

- Do not design security that only trusts MAC equality (easy to spoof)

```
ip link show eth1
# link/ether xx:xx:xx:xx:xx:xx
```

ARP: glue for IPv4 on Ethernet

On-link IPv4 needs ARP:

1. Host knows dest IP is on-link (same subnet / connected route)
2. Who has IP X? Tell Y — broadcast
3. Reply unicast with MAC
4. Neigh cache stores IP MAC

```
ip neigh show
ip neigh flush dev eth1   # lab use carefully
```

IPv6 uses Neighbor Discovery (ICMPv6) instead of ARP—same role, different messages.

Linux bridge FDB

```
# Create bridge and show FDB after traffic
bridge fdb show br br0
bridge link show
```

Dynamic entries learn from traffic; static entries can be added for experiments.

Lab A: two hosts, one bridge (hand or Containerlab)

Containerlab topology

```
name: l2-mac

topology:
  nodes:
    br1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 bridge
        - ip link add name br0 type bridge
        - ip link set br0 up
        # attach eth1/eth2 after links exist - see note below
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.2/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "br1:eth1"]
    - endpoints: ["h2:eth1", "br1:eth2"]
```

Operational note: Alpine bridge setup via `exec` may race; a mounted `entrypoint.sh` is more reliable:

```
#!/bin/sh
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add name br0 type bridge 2>/dev/null || true
ip link set br0 up
ip link set eth1 master br0
ip link set eth2 master br0
ip link set eth1 up
ip link set eth2 up
sleep infinity
```

Bind and set as command for `br1`. Hosts keep simple addressing execs.

Predict

- First ping causes ARP broadcast flooded to the other host
- FDB on `br0` learns h1 and h2 MACs on respective ports
- Subsequent unicast ICMP is switched, not broadcast

Observe

```
docker exec clab-l2-mac-h1 ping -c 3 10.10.0.2
docker exec clab-l2-mac-br1 bridge fdb show br br0
docker exec clab-l2-mac-h1 ip neigh
docker exec clab-l2-mac-h1 tcpdump -ni eth1 -e -c 20 arp or icmp
```

Fix inject

```
# Wrong mask on h2 - still "same numbers" but off-link behavior
docker exec clab-l2-mac-h2 ip addr flush dev eth1
docker exec clab-l2-mac-h2 ip addr add 10.10.0.2/32 dev eth1
docker exec clab-l2-mac-h2 ip link set eth1 up
docker exec clab-l2-mac-h1 ping -c 2 10.10.0.2
```

Restore `/24`. Journal the symptom.

Harden

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-l2-mac-h1 ping -c 2 -W 1 10.10.0.2
docker exec clab-l2-mac-br1 bridge fdb show br br0 | grep -q .
echo OK
```

Lab B: three hosts — flooding visibility

Add `h3` on `10.10.0.3/24` linked to `br1:eth3`.

Predict: Unknown unicast to a silent host floods; after traffic, FDB constrains.

Observe: tcpdump on h3 while h1 pings h2—after learning, h3 should **not** see unicast ICMP (ideal case). During ARP storms or failures, you may still see broadcasts.

```
docker exec clab-l2-mac-h3 tcpdump -ni eth1 -c 30 &
docker exec clab-l2-mac-h1 ping -c 5 10.10.0.2
```

MAC moves and flaps

```
# Simulate "move": shut one path (in multi-path labs) or change container
# Observe FDB port update and short loss
bridge fdb show br br0
```

Duplicate MACs on two ports (misconfig or bridge loops) cause flapping—spanning tree chapter addresses loops.

Common L2 failures (pre-VLAN)

Symptom	Likely cause
No ARP reply	Far host down, wrong cable/port, filter
ARP ok, no ping	Local firewall, wrong IP on host
Intermittent	MAC flap, duplex issues (physical), loop
Works after first flood only	Misread; check FDB aging and silent nodes

Promiscuous mode and labs

Bridges and captures may require promisc on interfaces:

```
ip link set eth1 promisc on
tcpdump -ni eth1
```

Containerlab links typically allow node-local captures on data interfaces.

Security digression (short)

- Port security / sticky MAC exist on enterprise gear—concept: limit learned MANs
- DHCP snooping / dynamic ARP inspection—later ops world
- For this book: understand **why** spoofing works at L2 so you do not mythologize the LAN

Predict worksheet (paper)

Domain with switch S and hosts A,B,C. A sends to B's IP first time.

1. Is the first frame after ARP unicast or might B's MAC still be unknown?
2. Does C receive the ARP request?
3. Does C receive the ICMP echo request once FDB learned?

Answers: (1) after ARP, A knows B's MAC—frame is unicast; switch floods only if FDB miss on B. (2) yes, ARP request is broadcast. (3) no, not if learning completed and no loop/mirror.

FRR / NOS note

Hardware switches expose `show mac address-table` equivalents. Linux `bridge fdb` is the same idea. When you use SR Linux or others later, translate:

Intent	Linux bridge	Typical NOS language
Show learned MACs	`bridge fdb show`	show mac table
Clear dynamic	`bridge fdb del ...`	clear mac

Summary

- Switches learn source MACs, forward known unicasts, flood unknowns/broadcasts
- ARP/ND bind IP to MAC on-link

- FDB + neigh + tcpdump diagnose most “L2 should work” failures
- Build bridge labs with Containerlab Linux nodes before complex VLAN designs

Next: **VLANs and trunks**—splitting broadcast domains without new cables.

VLANs & Trunks

VLANs & Trunks

VLANs split one physical switching fabric into multiple **broadcast domains**. Trunks carry many VLANs between switches with 802.1Q tags. Mis-matched tags and native VLANs are classic outage factories—this chapter makes them deliberate.

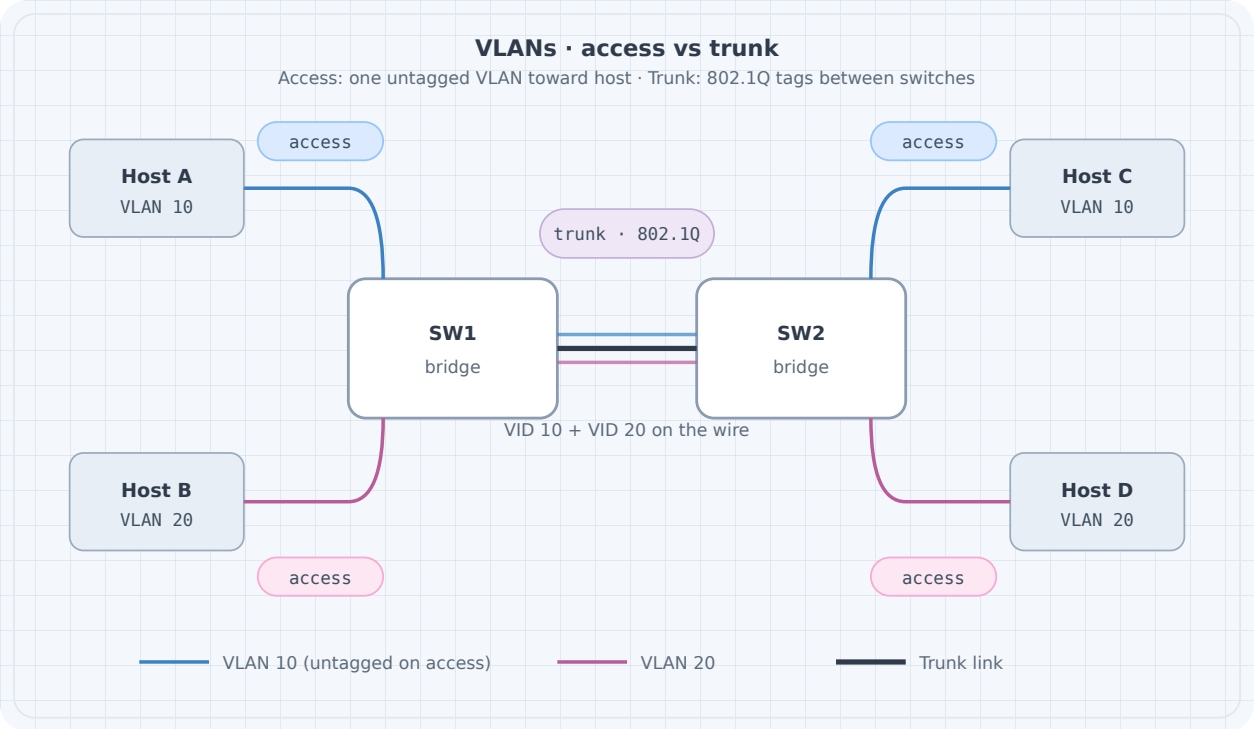


Figure 1: Access ports versus 802.1Q trunk

Learning goals

By the end of this chapter you can:

- Explain access vs trunk ports
- Design a multi-VLAN lab with consistent IDs
- Predict tag behavior on the wire
- Spot native VLAN mismatch symptoms

- Verify isolation between VLANs without a router

Concepts

Broadcast domains

Without VLANs, one bridge = one flood domain. VLAN-aware bridging tags frames with a VLAN ID (VID) and keeps separate FDBs per VLAN (implementation details vary, model holds).

Port mode	Behavior
Access	One VLAN; frames untagged on the wire toward the host
Trunk	Multiple VLANs; tags on the wire (except optional native)
Native VLAN	Untagged traffic on a trunk is mapped to this VLAN

802.1Q tag

Inserts TPID `0x8100` + PCP/DEI + VID. Capture shows tagged frames between switches:

```
tcpdump -ni eth1 -e vlan
# or
tcpdump -ni eth1 -e | head
```

Router roles

Inter-VLAN traffic needs an L3 device (router-on-a-stick, SVI, separate routed ports). Same switch + different VLANs **no IP connectivity** by design.

Addressing plan example

VLAN	Name	Subnet	Access ports
10	users	10.10.10.0/24	h1
20	iot	10.10.20.0/24	h2
99	native/mgmt lab	10.10.99.0/24	optional

Trunk between `sw1` and `sw2` allows 10,20 (and 99 if used).

Lab topology (conceptual)

```
h1 --(access VLAN 10)-- sw1 ==trunk== sw2 --(access VLAN 10)-- h3
h2 --(access VLAN 20)-- sw1 ==trunk== sw2 --(access VLAN 20)-- h4
```

Containerlab sketch using Linux VLAN-aware bridge

Full production-grade switch images are optional; Linux can teach tags.

```
name: l2-vlan

topology:
  nodes:
    sw1:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/sw1.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    sw2:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/sw2.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.20.1/24 dev eth1
        - ip link set eth1 up
    h3:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.3/24 dev eth1
```

```
        - ip link set eth1 up
    h4:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.20.4/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "sw1:eth1"]
    - endpoints: ["h2:eth1", "sw1:eth2"]
    - endpoints: ["h3:eth1", "sw2:eth1"]
    - endpoints: ["h4:eth1", "sw2:eth2"]
    - endpoints: ["sw1:eth3", "sw2:eth3"]
```

`config/sw1.sh` (VLAN filtering bridge)

```
#!/bin/sh
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add br0 type bridge vlan_filtering 1
ip link set br0 up

# access ports
for i in 1 2; do ip link set eth$i up; ip link set eth$i master br0; done
ip link set eth3 up
ip link set eth3 master br0

# clear default VLAN 1 membership as needed; set access VLANs
bridge vlan add vid 10 dev eth1 pvid untagged
bridge vlan del vid 1 dev eth1 2>/dev/null || true
bridge vlan add vid 20 dev eth2 pvid untagged
bridge vlan del vid 1 dev eth2 2>/dev/null || true

# trunk eth3: tagged 10,20
bridge vlan add vid 10 dev eth3
bridge vlan add vid 20 dev eth3
bridge vlan del vid 1 dev eth3 2>/dev/null || true

# bridge self VLANs
bridge vlan add vid 10 dev br0 self
bridge vlan add vid 20 dev br0 self

sleep infinity
```

Mirror on `sw2` with eth1→VLAN10, eth2→VLAN20, eth3 trunk.

Kernel/`bridge` command details vary slightly by version—if a flag fails, check `man bridge` on the image and adjust. The **model** (pvid untagged access, tagged trunk) is the learning target.

Predict → observe → fix

Predict

- h1 h3 (VLAN 10) ping works
- h2 h4 (VLAN 20) ping works
- h1 h2 fails (no L3 inter-VLAN)
- Capture on trunk shows VLAN tags for inter-switch frames

Observe

```
docker exec clab-l2-vlan-h1 ping -c 2 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 10.10.20.4
docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.1  # expect fail
docker exec clab-l2-vlan-sw1 bridge vlan show
docker exec clab-l2-vlan-sw1 tcpdump -ni eth3 -e -c 20
```

Failure drill: native / PVID mismatch

On sw1 trunk, map untagged to VLAN 10; on sw2 map untagged to VLAN 20. Send untagged (or mis-access) traffic.

Symptoms often seen:

- Silent discard
- Hosts “leak” into wrong segment
- Intermittent weirdness depending on tagging

Harden: Explicitly tag all lab VLANs on trunks; avoid relying on native VLAN for user data. Document native VLAN ID identically on both ends if you must use it.

Failure drill: missing allowed VLAN

Remove VLAN 20 from sw2 trunk membership.

Predict: VLAN 10 still works; VLAN 20 across switches dies.

```
# on sw2, delete vid 20 from eth3, retest h2->h4
```

Router-on-a-stick preview

```
sw trunk --- r1 eth1.10 + eth1.20
```

```
ip link add link eth1 name eth1.10 type vlan id 10
ip link add link eth1 name eth1.20 type vlan id 20
ip addr add 10.10.10.254/24 dev eth1.10
ip addr add 10.10.20.254/24 dev eth1.20
ip link set eth1.10 up
ip link set eth1.20 up
sysctl -w net.ipv4.ip_forward=1
```

Hosts use `.254` as gateway. Full L3 chapters expand routing; here you only prove **VLAN separation needs L3 to merge**.

Verification checklist

```
# Per switch
bridge vlan show
bridge fdb show

# Per host
ip -br a
ip neigh
ping -c 2 <same-vlan-far-host>
ping -c 1 -W 1 <other-vlan-host>   # should fail without router
```

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-l2-vlan-h1 ping -c 2 -W 1 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 -W 1 10.10.20.4
if docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.1; then
```

```
  echo "FAIL: inter-VLAN should be blocked" >&2
  exit 1
fi
echo OK
```

Negative checks are gold.

Design tips

Tip	Why
Plan VLAN IDs globally in the lab	Avoid "VLAN 10 means different things"
Do not reuse VLAN 1 for user data	Habit for real gear hygiene
Match trunk allow lists both ends	One-way or partial connectivity
Document native VLAN	Mismatch class of bugs
Separate voice/data IDs even in labs	Practice real patterns

Common mistakes

Mistake	Result
Access port in wrong VLAN	ARP never reaches partner
Trunk missing VID	Works locally, fails across switches
Same subnet on two VLANs	Broken design + asymmetric mess
Expecting VLANs to route	Need L3

Summary

- VLANs partition broadcast domains; trunks carry tagged sets between switches
- Access = untagged one VID; trunk = tagged many VIDs
- Align allow lists and native VLAN on both ends
- Prove isolation with intentional negative pings
- Inter-VLAN = router/SVI job

Next: **loop prevention**—what happens when redundant L2 paths exist without a control protocol.

Loop Prevention

Loop Prevention

Redundant L2 links are good for resilience and catastrophic without loop control. Broadcasts and unknown unicasts circulate forever; MAC tables flap; the LAN melts. This chapter covers the **spanning tree family** as a model (not a vendor certification deep dive) and how to see loops in the lab.

Learning goals

By the end of this chapter you can:

- Explain why Ethernet needs loop prevention
- Describe root bridge election and port roles at a conceptual level
- Predict a blocked port in a simple triangle of switches
- Build a loop, observe the blast, then enable prevention
- Prefer design alternatives when L2 redundancy is the wrong tool

Why loops kill L2

Ethernet has no TTL on frames. In a loop:

1. Broadcast / unknown unicast is flooded
2. Copies multiply around the cycle
3. CPUs and links saturate
4. Learning oscillates (MAC seen on alternating ports)

IP routers do not save you inside a pure L2 domain.

Spanning tree behavior

Classic STP/RSTP/MSTP ideas:

Idea	Meaning
Root bridge	Reference switch (lowest priority/BID wins)
Root port	Each non-root switch's best path toward root
Designated port	Best port on a segment toward downstream
Blocked / alternate	Redundant path held in reserve
BPDU	Control frames that build the tree

Modern networks often use **RSTP** or vendor rapid variants; data centers may avoid large L2 domains entirely (L3 fabric). Still, you must recognize STP language on campus gear and many NOS features.

Priority and ties

Lower bridge priority wins. Ties break by MAC. In labs, set priorities explicitly so root placement is intentional:

```
sw1 priority better (numerically lower) → root
sw2/sw3 farther
```

Port states (classic story)

Blocking → Listening → Learning → Forwarding (RSTP simplifies roles/states). During convergence, expect transient blackholes—measure them in drills.

Design alternatives (model maturity)

Approach	Loop story
STP/RSTP on bridged triangle	Protocol blocks one link
L3 access / routed triangle	No L2 loop; ECMP at L3
MC-LAG / bundling	Multi-chassis tricks (complex)
Single homing	No redundancy, no loop

For many greenfield labs after this chapter, **prefer L3 redundancy**. Learn STP so you can operate brownfield and understand failure domains.

Lab: three switches, intentional loop

```
    sw1
   /   \
 sw2---sw3
```

Hosts on sw2 and sw3.

Topology YAML (Linux bridges)

```yaml
name: l2-loop

topology:
  nodes:
    sw1:
      kind: linux
      image: alpine:3.20
      binds: ["./config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    sw2:
      kind: linux
      image: alpine:3.20
      binds: ["./config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    sw3:
      kind: linux
      image: alpine:3.20
      binds: ["./config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.2/24 dev eth1 && ip link set eth1 up
    h3:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.3/24 dev eth1 && ip link set eth1 up

  links:
    - endpoints: ["sw1:eth1", "sw2:eth1"]
    - endpoints: ["sw1:eth2", "sw3:eth1"]
```

```
    - endpoints: ["sw2:eth2", "sw3:eth2"]
    - endpoints: ["h2:eth1", "sw2:eth3"]
    - endpoints: ["h3:eth1", "sw3:eth3"]
```

Dangerous mode: dumb bridges without STP

```
#!/bin/sh
# config/sw-loop.sh - NO STP (for short controlled drill only!)
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add br0 type bridge
ip link set br0 up
for i in 1 2 3; do
  ip link set eth$i up 2>/dev/null || true
  ip link set eth$i master br0 2>/dev/null || true
done
sleep infinity
```

Warning: Run the loop experiment **briefly** on an isolated lab host. Broadcast storms can stress CPU.

Observe storm (careful)

```
# terminal 1
docker exec clab-l2-loop-h2 ping -f 10.10.0.3 &
# or send broadcasts
docker exec clab-l2-loop-sw1 ip -s link
# counters explode
```

Destroy promptly:

```
sudo containerlab destroy -t l2-loop.clab.yml --cleanup
```

Safer mode: enable STP on the bridge

Linux bridge STP:

```
ip link add br0 type bridge
ip link set br0 type bridge stp_state 1
ip link set br0 up
# attach ports...
```

```
# inspect
docker exec clab-l2-loop-sw1 cat /sys/class/net/br0/bridge/stp_state
# port states under /sys/class/net/br0/brif/*/state
```

Predict: One of the triangle links is blocked; ping between h2 and h3 still works via remaining tree; storm does not persist.

Observe:

```
docker exec clab-l2-loop-h2 ping -c 5 10.10.0.3
# check which port is blocking on each sw
for s in sw1 sw2 sw3; do
  echo "== $s =="
  docker exec clab-l2-loop-$s sh -c 'for p in /sys/class/net/br0/brif/*/state; do echo $p $(
done
```

Root placement drill

Influence root by bridge priority (when using STP implementations that expose it):

```
# example sysfs / tools depend on stack; concept: make sw1 root
ip link set br0 type bridge priority 0x1000
```

On NOS images (SR Linux, etc.), use vendor STP priority knobs—same election idea.

Predict: With sw1 root, blocked port is likely on the sw2–sw3 link (common triangle outcome with symmetric costs).

Verify: Diagram the tree; mark root ports and blocked port.

Failure drill: link down on active tree edge

1. Baseline ping
2. Shut a forwarding uplink
3. Measure time until ping recovers
4. Confirm previously blocked port becomes forwarding

```
docker exec clab-l2-loop-sw1 ip link set eth1 down
# time recovery
docker exec clab-l2-loop-h2 ping -c 20 10.10.0.3
docker exec clab-l2-loop-sw1 ip link set eth1 up
```

Journal convergence feel (RSTP vs classic differs a lot).

BPDUs and protection concepts (awareness)

Feature idea	Intent
BPDU guard	Err-disable access port if BPDU seen
Root guard	Prevent downstream from becoming root
Loop guard	Protect against unidirectional link issues

You may not implement all in Alpine labs; know the **jobs** so production configs make sense.

When not to extend L2

Smell	Prefer
L2 stretched across sites	L3 + overlay if needed
Huge broadcast domain	Segment VLANs + route
STP diameter anxiety	Routed access / leaf-spine

Spanning tree is a safety net, not an excuse for continent-sized broadcast domains.

Predict worksheet

Triangle swA–swB–swC, all costs equal, swA root.

1. Which link is most likely blocked?
2. Host on swB to host on swC: path?
3. If swA dies, what must happen?

Answers: (1) often B–C; (2) B–A–C; (3) re-election, former blocked may forward, connectivity among survivors if links remain.

Verification checklist for STP labs

- Never leave an intentional storm lab running unattended
- Record before/after port states
- Compare L2 redundancy recovery to L3 ECMP recovery later—feel the difference

Summary

- L2 loops multiply floods; no frame TTL
- Spanning tree elects a root and blocks redundant ports
- Practice triangle labs with STP on; only briefly demo storms
- Root placement is a design choice
- Mature designs often push redundancy to L3

Next: **link aggregation**—bundling links for bandwidth and failover without creating an independent loop story (when configured correctly).

Link Aggregation

Link Aggregation

Link aggregation (LAG / bonding / etherchannel-class ideas) combines multiple physical links into one logical link: more bandwidth, faster failover, and fewer STP blocked paths between the same pair of devices—when both ends agree.

Learning goals

By the end of this chapter you can:

- Explain when LAG helps vs when it hides problems
- Contrast static bundle vs LACP
- Configure a Linux bond in a lab
- Predict traffic behavior and failover
- Verify member health and hashing limitations

Concepts

One logical link

```
     +---- eth1 ----+
sw1 -+              +- sw2
     +---- eth2 ----+
           \____ bond0 / lag0 ____/
```

Upper protocols (STP, L3 adjacency) see **one** link if bundling is correct.

Static vs LACP

Mode	Pros	Cons
Static / on	Simple	Mis-wiring may blackhole; no negotiation

LACP	Detects partner; protects against some mistakes	Both ends must support/enable

Labs may start static; production almost always prefers LACP.

Hashing

Flows distribute across members by a hash (MAC/IP/L4 ports depending on config). **One TCP flow** typically stays on one member—do not expect a single iperf stream to fill N× bandwidth.

Split brain / misconfig

Classic fail: one side bundles, the other has independent ports—loops or blackholes. Always verify both ends.

Linux bonding quickstart

Kernel bonding modes (selection depends on goal):

Mode	Name	Notes
0	balance-rr	Round-robin
1	active-backup	Simple failover
4	802.3ad	LACP

```
# Example active-backup (conceptual host commands)
modprobe bonding
ip link add bond0 type bond mode active-backup miimon 100
ip link set eth1 down
ip link set eth2 down
ip link set eth1 master bond0
ip link set eth2 master bond0
ip addr add 10.0.0.1/24 dev bond0
ip link set bond0 up
ip link set eth1 up
ip link set eth2 up
cat /proc/net/bonding/bond0
```

In containers, bonding support depends on privileges and kernel modules on the **host**. Containerlab labs sometimes simulate aggregation at the NOS layer instead. If bonding is blocked in your environment, run this chapter on a Linux VM/nested host and keep Containerlab for endpoints.

Containerlab topology (two paths, bond on routers)

```yaml
name: l2-lag

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/r1-bond.sh:/startup.sh
      cmd: /bin/sh /startup.sh
      # may need privileged: true depending on clab version/kind options
    r2:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/r2-bond.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth3"]
    - endpoints: ["h2:eth1", "r2:eth3"]
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r1:eth2", "r2:eth2"]
```

config/r1-bond.sh

```
#!/bin/sh
set -e
apk add --no-cache iproute2 iputils >/dev/null
# fallback if bonding unavailable: sequential routes demo only
if ip link add bond0 type bond mode 802.3ad 2>/dev/null; then
  ip link set eth1 down
  ip link set eth2 down
  ip link set eth1 master bond0
  ip link set eth2 master bond0
  ip addr add 10.0.0.1/24 dev bond0
  ip link set bond0 up
  ip link set eth1 up
  ip link set eth2 up
else
  echo "bonding unavailable; configure two links manually for failover drill" >&2
  ip addr add 10.0.0.1/24 dev eth1
  ip link set eth1 up
  ip link set eth2 up
fi
ip addr add 192.168.1.1/24 dev eth3
ip link set eth3 up
sysctl -w net.ipv4.ip_forward=1
ip route add 192.168.2.0/24 via 10.0.0.2
sleep infinity
```

Symmetric on r2 with `10.0.0.2`, LAN `192.168.2.1/24`, route to `192.168.1.0/24 via 10.0.0.1`.

If LACP mode fails without a partner daemon, use `mode active-backup` for reliable lab failover learning.

Predict → observe → fix

Predict

- h1→h2 ping works through logical path
- Shutting **one** member keeps session alive (active-backup or LACP with remaining member)
- Single flow throughput one member speed

Observe

```
docker exec clab-l2-lag-h1 ping -c 5 192.168.2.10
docker exec clab-l2-lag-r1 cat /proc/net/bonding/bond0 2>/dev/null || true
docker exec clab-l2-lag-r1 ip -br link
```

Failover drill

```
docker exec clab-l2-lag-r1 ip link set eth1 down
docker exec clab-l2-lag-h1 ping -c 10 192.168.2.10
docker exec clab-l2-lag-r1 ip link set eth1 up
```

Measure loss count. Journal recovery time.

Mismatch drill

On r2, do **not** enslave eth2; leave as separate interface with another IP.

Predict: loops, flaps, or partial connectivity—document what **your** kernel does; fix by matching configs.

Hashing awareness drill

```
# multiple parallel streams may use more members than one stream
# on hosts with iperf3
iperf3 -s   # h2
iperf3 -c 192.168.2.10 -P 8
```

Compare `-P 1` vs `-P 8`.

STP interaction

Between two switches, a multi-link LAG is one logical link STP sees one edge. Independent unbundled links STP blocks all but one (or you loop). **Bundle before you celebrate "redundant cables."**

Check	Intent

Operational checks (NOS-neutral)

Check	Intent
Members all up	Physical/L1
LACP PDUs both ways	Negotiation
Same speed/duplex	Bundle eligibility
Counters increment on members	Actual use
Control plane adjacency single	Logical link view

Common mistakes

Mistake	Result
One end LACP, other static only	Stuck negotation / down
Different VLANs on members	Inconsistent bundle
Expect one flow = N x bandwidth	Disappointed benchmarks
Forget host firewall on bond MAC	Odd ARP

Summary

- LAG = one logical link from multiple physical members
- Prefer LACP in real life; static only with tight control
- Hashing limits single-flow throughput
- Verify both ends; failover-test by shutting members
- Unbundled parallel L2 links reintroduce STP/loop problems

Next: **L2 failure drills**—deliberate breakage across MAC, VLAN, loop, and bundle scenarios with evidence.

L2 Failure Drills

L2 Failure Drills

This chapter is a **practice arena**. You combine Ethernet, VLANs, loop prevention, and aggregation under intentional failure. Competence is measured by evidence and recovery, not by green first pings.

Learning goals

By the end of this chapter you can:

- Run a structured failure catalog against an L2 lab
- Collect the minimum evidence set for each class of break
- Recover cleanly and re-verify
- Write short incident notes a peer can follow
- Know when an "L2 issue" is actually L3 or host firewall

Baseline lab

Use a multi-switch VLAN lab (from the VLANs chapter) with:

- Two switches, trunk between them
- VLAN 10 and VLAN 20
- Hosts h1/h3 in VLAN 10, h2/h4 in VLAN 20
- Optional inter-VLAN router for a subset of drills

Document addressing in `addressing.md`. Ensure `./verify.sh` is green before any inject.

Evidence kit

```
# Host
ip -br a
ip neigh
ping -c 3 -W 1 <target>
tcpdump -ni eth1 -e -c 30 arp or icmp

# Switch / bridge
bridge vlan show
bridge fdb show
ip -s link
# STP port states if enabled
```

Save outputs under `journal.md` with timestamps.

Drill catalog

For each drill: **predict → inject → observe → restore → harden**.

Drill 1 — Access port wrong VLAN

Inject: Move h1's switch port from VLAN 10 to VLAN 20 (PVID change).

Predict: h1 cannot ARP h3; may see wrong subnet peers if any; verify.sh fails.

Observe: `bridge vlan show`, tcpdump for ARP who-has never answered by h3.

Restore: Correct PVID/untagged VLAN.

Harden: Label ports in diagrams; negative test in verify (optional).

Drill 2 — Trunk missing VLAN

Inject: Remove VLAN 10 from trunk allow list on one switch only.

Predict: Local VLAN 10 still works on that switch; cross-switch VLAN 10 dies; VLAN 20 may still work.

Observe: Asymmetric "works on same switch only."

Restore: Re-add VID on both ends.

Harden: Checklist "both ends allow list."

Drill 3 — Native VLAN mismatch

Inject: Native/PVID untagged mapping differs on trunk ends.

Predict: Untagged frames land in different broadcast domains; confusing intermittent leaks.

Observe: Captures show untagged where you expected tags; wrong FDB.

Restore: Align native VLAN; prefer tagged-only user VLANs.

Harden: Explicit lab policy: native VLAN unused for data.

Drill 4 — Cable / interface down

Inject:

```
docker exec clab-l2-vlan-sw1 ip link set eth3 down
```

Predict: Cross-switch traffic dies; FDB ages; local switching may live.

Observe: ping loss; interface counters; STP reconvergence if redundant path exists.

Restore: `ip link set eth3 up`; wait for learning/STP.

Harden: If no redundant path, document single point of failure.

Drill 5 — Broadcast storm (controlled)

Inject: Temporary bridging loop **without** STP on an isolated lab (very short).

Predict: CPU spikes, loss of management responsiveness, exploding counters.

Observe: `ip -s link`, host load.

Restore: `containerlab destroy --cleanup` immediately if unstable.

Harden: STP on; never lab this on a shared prod-like bridge.

Drill 6 — STP blocked port confusion

Inject: With STP triangle, shut the active root port path.

Predict: Alternate port activates; brief loss; new tree.

Observe: Port state sysfs or NOS show; ping `-c 50` loss burst.

Restore: Bring link up; tree may revert depending on costs.

Harden: Note expected reconvergence budget.

Drill 7 — MAC move

Inject: Move host container link from sw1 to sw2 (redeploy link or migrate veth).

Predict: FDB updates; short blackhole until relearn; ARP may refresh.

Observe: `bridge fdb show` before/after; neigh on peers.

Restore: Stable attachment.

Harden: In virtualization, understand port channels / bond moves.

Drill 8 — Duplicate IP

Inject: Configure h4 temporarily with h3's IP on VLAN 10 (careful).

Predict: Flapping ARP, intermittent delivery, wrong neigh.

Observe: `ip neigh` oscillating; tcpdump conflicting ARP replies.

Restore: Unique IPs.

Harden: Addressing plan ownership; IPAM habit.

Drill 9 — Duplicate MAC (advanced)

Inject: Spoof MAC on a second host (lab only).

```
ip link set eth1 down
ip link set eth1 address aa:bb:cc:dd:ee:ff
ip link set eth1 up
```

Predict: FDB flap; unstable delivery.

Restore: Original MAC; destroy/recreate if messy.

Harden: Understand L2 trust boundaries.

Drill 10 — LAG member down

Inject: On a bond/LAG lab, shut one member.

Predict: Traffic continues if min-links satisfied; single-flow rate unchanged.

Observe: `/proc/net/bonding/bond0`; ping continuity.

Restore: Member up.

Harden: Alert on member-down even if bond stays up.

Drill 11 — Inter-VLAN path mistaken for L2

Inject: Delete router inter-VLAN route or shut subinterface.

Predict: Same-VLAN ok; cross-VLAN fails—looks like "VLAN broken" if you misread.

Observe: traceroute never leaves gateway; ARP to gateway fails or gateway unreachable.

Restore: Router config.

Harden: Always check same-VLAN baseline before blaming trunk.

Drill 12 — Host firewall

Inject: On Linux host, drop ICMP.

```
# if nft/iptables available
iptables -A INPUT -p icmp -j DROP
```

Predict: Others cannot ping in; egress may still work.

Observe: Asymmetric ping stories.

Restore: Flush rule.

Harden: Include host endpoint in the failure picture.

Incident note template

```
## Incident: trunk missing VLAN 10 on sw2
Date:
Lab: l2-vlan

### User symptom
h1 cannot reach h3; h1 reaches local same-switch hosts.

### Prediction before fix
Trunk allow list asymmetry.

### Evidence
- bridge vlan show sw1/sw2 (paste)
- tcpdump trunk: no VLAN 10 tags toward sw2
- ping results

### Root cause
vid 10 deleted from sw2 eth3

### Fix
bridge vlan add vid 10 dev eth3

### Prevent
verify.sh checks cross-switch VLAN 10 and 20
README allow-list both ends
```

Scorecard

Run at least **six** drills across different classes. Check:

Class	Drills done
VLAN membership	
Trunk tagging	
Link down	
STP / loop	
Aggregation	
Host/IP mistakes	

Combined verify.sh ideas

```
#!/usr/bin/env bash
set -euo pipefail
# positive
docker exec clab-l2-vlan-h1 ping -c 2 -W 1 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 -W 1 10.10.20.4
# negative isolation
if docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.4 2>/dev/null; then
  echo "unexpected inter-VLAN without router" >&2
  exit 1
fi
echo OK
```

After each restore, re-run.

Checkpoint: L2 competence

You pass the part when you can:

1. Deploy multi-switch VLANs from code
2. Produce evidence for tag/allow-list bugs
3. Explain and demo loop risk
4. Fail over a link or LAG member deliberately
5. Write an incident note with root cause and prevention

Summary

- Failure drills convert L2 theory into ops judgment

- Always baseline with verify.sh
- Predict before inject; restore and re-verify after
- Separate L2, L3, and host causes with evidence
- Incident notes are part of the curriculum artifact

Next part: **layer 3**—addressing, routing fundamentals, static labs, ICMP and path MTU.

Layer 3

IP Addressing

IP Addressing

Forwarding starts with **addressing**: structure, masks, subnets, summarization, and enough IPv6 to run dual-stack labs. This chapter is calculation plus lab verification—not exam trick puzzles for their own sake.

Learning goals

By the end of this chapter you can:

- Convert between prefix length and masks fluently
- Design subnets for a multi-tier lab without overlaps
- Summarize contiguous blocks and predict blackholes from bad summaries
- Configure IPv4 and basic IPv6 on Linux
- Use documentation ranges and lab hygiene ranges deliberately

IPv4 structure

An IPv4 address is 32 bits, usually dotted decimal. A **prefix** `address/len` identifies network bits vs host bits.

Prefix	Mask	Hosts (approx usable)
/32	255.255.255.255	1 (host route)
/31	255.255.255.254	2 (P2P often)
/30	255.255.255.252	2 usable classic
/24	255.255.255.0	254 usable classic
/16	255.255.0.0	large
/8	255.0.0.0	huge

```
# Linux
ip addr add 192.0.2.10/24 dev eth1
ip -br a
ip route
```

On-link decision

If destination is inside a connected prefix, the host uses ARP/ND on that interface. Otherwise it sends to a gateway (route lookup).

```
ip route get 192.0.2.50
ip route get 8.8.8.8
```

Subnetting for labs

Example campus-ish allocation from `10.20.0.0/16`:

Use	Block
Users site A	10.20.1.0/24
Users site B	10.20.2.0/24
Servers	10.20.10.0/24
P2P core	10.20.255.0/24 as /30s
Loopbacks	10.20.254.0/24 as /32s

Rules:

1. No overlapping prefixes on different segments unless deliberate anycast
2. Leave growth room
3. Write the plan before deploy
4. Align summaries with actual aggregation topology

Summarization

If you own `10.20.1.0/24` and `10.20.2.0/24`, a summary `10.20.0.0/22` may or may not be correct depending on other contents of that /22.

Action	Risk
Summary advertises space you do not have	Blackhole for missing parts
Too-specific everywhere	Table bloat (later IGP/BGP scale)
Forgetting more-specific local routes	Traffic steers wrong

Paper drill

You have:

- 10.0.0.0/25
- 10.0.0.128/25

Correct tight summary? `10.0.0.0/24`.

You have only `10.0.0.0/25` but advertise `10.0.0.0/24` upstream without a sink. What happens to 10.0.0.200? **Blackhole** toward you.

Special-use ranges (know them)

Range	Role
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918)
127.0.0.0/8	Loopback
169.254.0.0/16	Link-local
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Documentation (RFC 5737)
224.0.0.0/4	Multicast

IPv6 essentials for dual-stack labs

IPv6 address is 128 bits; common lab prefix `/64` on LANs.

```
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip -6 route
ip -6 neigh
ping -6 -c 2 2001:db8:1::1
```

Prefix idea	Example
Documentation	`2001:db8::/32`
Unique local	`fd00::/8` (with local generation rules)
Link-local	`fe80::/10` auto on interfaces

ND replaces ARP. Router Advertisements may autoconfigure hosts—labs often use statics for control.

Dual-stack habit

Give nodes both families when learning:

```
ip addr add 10.1.0.10/24 dev eth1
ip -6 addr add 2001:db8:1::10/64 dev eth1
```

Test both paths; do not assume v4 success means v6 works.

Linux configuration patterns

Persistent lab style

In Containerlab, prefer startup scripts or image config over manual `ip addr` that vanishes.

```
#!/bin/sh
ip addr flush dev eth1
ip addr add 10.1.0.10/24 dev eth1
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip route add default via 10.1.0.1
ip -6 route add default via 2001:db8:1::1
```

Multiple addresses

```
ip addr add 10.1.0.10/24 dev eth1
ip addr add 10.1.0.11/24 dev eth1
```

Secondary addresses appear in ARP; know who answers.

Lab: addressing plan + dual-stack hosts

```
name: l3-addr

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - sysctl -w net.ipv4.ip_forward=1
```

```
        - sysctl -w net.ipv6.conf.all.forwarding=1
        - ip addr add 10.1.0.1/24 dev eth1
        - ip addr add 10.2.0.1/24 dev eth2
        - ip -6 addr add 2001:db8:1::1/64 dev eth1
        - ip -6 addr add 2001:db8:2::1/64 dev eth2
        - ip link set eth1 up
        - ip link set eth2 up
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.1.0.10/24 dev eth1
        - ip -6 addr add 2001:db8:1::10/64 dev eth1
        - ip link set eth1 up
        - ip route add default via 10.1.0.1
        - ip -6 route add default via 2001:db8:1::1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.2.0.10/24 dev eth1
        - ip -6 addr add 2001:db8:2::10/64 dev eth1
        - ip link set eth1 up
        - ip route add default via 10.2.0.1
        - ip -6 route add default via 2001:db8:2::1

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["h2:eth1", "r1:eth2"]
```

Predict

- h1 pings h2 v4 and v6 through r1
- Wrong mask on h1 (e.g. /32 only) breaks on-link perception of gateway

Observe

```
docker exec clab-l3-addr-h1 ping -c 2 10.2.0.10
docker exec clab-l3-addr-h1 ping -6 -c 2 2001:db8:2::10
docker exec clab-l3-addr-h1 ip route get 10.2.0.10
docker exec clab-l3-addr-r1 ip -br a
```

Failure: overlapping subnets

Put h2 also in `10.1.0.0/24` while connected to r1 eth2 differently—create intentional overlap and journal chaos (ARP confusion, asymmetric paths). Restore clean plan.

Failure: bad summary (static preview)

On a third router path (optional), advertise/install `10.0.0.0/8` via a dead next hop while more specifics missing—observe blackhole for unused space.

Calculation practice (quick)

1. How many /24s in a /16? → 256
2. First/last usable in 10.0.0.0/30 classic? → 10.0.0.1 and 10.0.0.2
3. Is 10.0.0.64/26 inside 10.0.0.0/24? → yes
4. Can you summarize 10.1.0.0/24 and 10.2.0.0/24 as a single /23? → no (not contiguous bit boundary as one /23 covering both cleanly without others)

Work these on paper until automatic.

Host vs router addressing habits

Role	Typical
Host	One address + default route
Router	Addresses on each L3 interface + routing table
Loopback	Stable /32 or /128 for protocols later

```
ip addr add 1.1.1.1/32 dev lo
```

Verification checklist

```
ip -br a
ip route
ip -6 route
ip route get <dst>
ping -c 2 <dst>
ping -6 -c 2 <dst6>
```

Summary

- Prefixes define on-link vs routed behavior
- Plan non-overlapping lab allocations; document them
- Summaries must match reality or they blackhole
- Dual-stack labs need explicit v6 config and tests
- `ip route get` is your friend

Next: **routing fundamentals**—RIB/FIB, longest match, next hops, and host vs router behavior including ARP/ND.

Routing Fundamentals

Routing Fundamentals

Routing is **longest-prefix match** to a next hop or interface, plus the glue that resolves the next hop to L2. This chapter ties RIB/FIB, metrics, and host vs router behavior into one model you will reuse for statics and dynamic protocols.

Learning goals

By the end of this chapter you can:

- Explain longest prefix match with competing routes
- Distinguish connected, static, and protocol routes
- Resolve next hops (ARP/ND) and see failures when resolution dies
- Use `ip route get` as a decision debugger
- Describe host default-route behavior vs multi-interface routers

Concepts

Forwarding equation

For each packet:

1. Look up destination in FIB
2. Select winning route (longest prefix, then tie-breakers)
3. Find egress interface and L2 rewrite target
4. Decrement TTL / hop limit; drop if zero

Store	Role

RIB vs FIB

Store	Role
RIB	Routing information: all candidates, protocol owners
FIB	What data plane uses for forward

On Linux, `ip route` shows kernel FIB. FRR holds protocol RIB and installs selected routes into the kernel via zebra.

```
ip route
vtysh -c 'show ip route'   # FRR RIB view
```

Longest prefix match

Routes:

- `0.0.0.0/0` via gwA
- `10.0.0.0/8` via gwB
- `10.1.2.0/24` via gwC

Destination `10.1.2.5` → **gwC**. Destination `10.9.0.1` → **gwB**. Destination `1.2.3.4` → **gwA**.

```
ip route add 10.0.0.0/8 via 192.0.2.1
ip route add 10.1.2.0/24 via 192.0.2.2
ip route get 10.1.2.5
```

Metrics and distance (ideas)

Multiple protocols may offer the same prefix. Selection uses administrative preference (distance) then metric. Linux has metrics on routes; FRR has AD per protocol. Exact numbers differ—**model** is ranked sources.

Connected routes

Configuring `10.1.0.1/24` on eth1 installs connected route `10.1.0.0/24 dev eth1`. Connected usually beats static/default for that space.

Host vs router

Host	Router
Often one default route	Many specific routes
Forwards? usually no	`ip_forward=1`
ARP for on-link and gateway	ARP for each next hop per interface
Care about DNS, apps	Care about RIB scale, policy

Gateway ARP

When host sends off-link:

- Destination IP stays the remote
- Ethernet dest is **gateway MAC**

```
# host
ip route get 8.8.8.8
ip neigh show
tcpdump -ni eth1 -e host 8.8.8.8
```

Next-hop resolution failures

Symptom: route exists, ping fails.

```
ip route get 10.2.0.10
# shows via 10.0.0.2 dev eth1
ip neigh show dev eth1
# if FAILED/INCOMPLETE for 10.0.0.2 → L2 problem or peer down
```

Control vs data: Static route "up" in config does not guarantee next hop resolves.

Asymmetric routing

Forward path R1→R2→R3, return R3→R4→R1. Stateless ping may work; stateful firewalls and some reverse-path filters break.

Always verify both directions.

```
# from each side
ping -c 2 <other>
traceroute -n <other>
```

Lab: competing prefixes

```
name: l3-lpm

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.0.1
        - ip route add 10.1.2.0/24 via 192.168.0.2
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.1/24 dev eth1
        - ip link set eth1 up
    r2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.2/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    # NOTE: for a real multi-router demo, use a bridge multipoint or
    # connect h1-r1 and h1-r2 carefully; simplest teaching variant below.
```

Cleaner multipoint teaching lab: one LAN bridge with h1, gw1, gw2:

```
name: l3-lpm

topology:
  nodes:
    br0:
      kind: linux
      image: alpine:3.20
      binds: ["./config/br.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
```

```
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.0.1
        - ip route add 10.1.2.0/24 via 192.168.0.2
    gw1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.1/24 dev eth1 && ip link set eth1 up
    gw2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.2/24 dev eth1 && ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "br0:eth1"]
    - endpoints: ["gw1:eth1", "br0:eth2"]
    - endpoints: ["gw2:eth1", "br0:eth3"]
```

Predict / observe

```
docker exec clab-l3-lpm-h1 ip route get 10.1.2.5
# expect via 192.168.0.2
docker exec clab-l3-lpm-h1 ip route get 10.9.0.1
# expect via 192.168.0.1 (default) if no better route
```

Lab: router with two interfaces

```
# r1
sysctl -w net.ipv4.ip_forward=1
ip addr add 10.1.0.1/24 dev eth1
ip addr add 10.2.0.1/24 dev eth2
# hosts with defaults via r1
```

Predict: Connected routes cover both LANs; hosts need default only; r1 needs no static for inter-LAN.

Observe:

```
ip route
ip route get 10.2.0.10 from 10.1.0.10 iif eth1
```

ARP/ND deep enough for routing

IPv4	IPv6
ARP request/reply	NS/NA
`ip neigh`	`ip -6 neigh`
Broadcast	Multicast solicited-node

```
ip monitor neigh
```

Flush for drills:

```
ip neigh flush dev eth1
```

TTL and traceroute

Each hop decrements TTL. Traceroute infers path from ICMP time exceeded.

```
traceroute -n 10.2.0.10
# or
mtr -rwzc 20 10.2.0.10
```

Filters that drop ICMP break traceroute but not always data—note for ops.

Policy preview

Policy routing (`ip rule`) can select alternate tables. Out of scope for depth here; know it exists when "ip route get" surprises you with marks/fwmasks.

```
ip rule list
```

Predict → observe → fix drills

Drill: missing return route

Two routers between hosts; only one direction has statics.

Symptom: requests arrive, replies die.

Fix: install return route.

Harden: verify.sh pings both ways.

Drill: wrong gateway MAC path

Shut gateway interface; host neigh becomes FAILED.

```
ip neigh show
ping -c 2 <offlink>
```

Drill: more-specific shadow

Add junk more-specific blackhole:

```
ip route add 10.2.0.10/32 via 192.0.2.254
# 192.0.2.254 dead
ping 10.2.0.10  # fail despite broader working route
ip route del 10.2.0.10/32
```

Cheatsheet

```
ip route
ip route get 1.2.3.4
ip route add 10.0.0.0/8 via 192.168.0.1 metric 100
ip route del 10.0.0.0/8 via 192.168.0.1
ip neigh show
sysctl net.ipv4.ip_forward
```

Summary

- Longest prefix match drives selection
- Connected, static, and dynamic routes feed RIB→FIB

- Next hop must resolve at L2
- Hosts mostly default; routers interconnect prefixes
- Bidirectional reachability is the real goal

Next: **static routing labs**—triangles, floating statics, and deliberate failover.

Static Routing Labs

Static Routing Labs

Static routes are the control plane you can see with your eyeballs. Master them in Containerlab with FRR or Linux before OSPF/BGP hide the decisions inside protocols.

Learning goals

By the end of this chapter you can:

- Build a multi-router static topology as code
- Install and verify routes in Linux and FRR
- Implement floating static failover with metrics
- Break and restore return paths deliberately
- Write verify scripts that catch one-way routing

Topology: triangle with hosts

```
h1--r1-------r2--h2
     \       /
      \     /
        r3
```

Link	Subnet
r1–r2	10.0.12.0/24
r2–r3	10.0.23.0/24
r3–r1	10.0.13.0/24
h1 LAN	192.168.1.0/24 (r1 gateway)
h2 LAN	192.168.2.0/24 (r2 gateway)
r3 LAN optional	192.168.3.0/24

Loopbacks: r1 `1.1.1.1/32`, r2 `2.2.2.2/32`, r3 `3.3.3.3/32`.

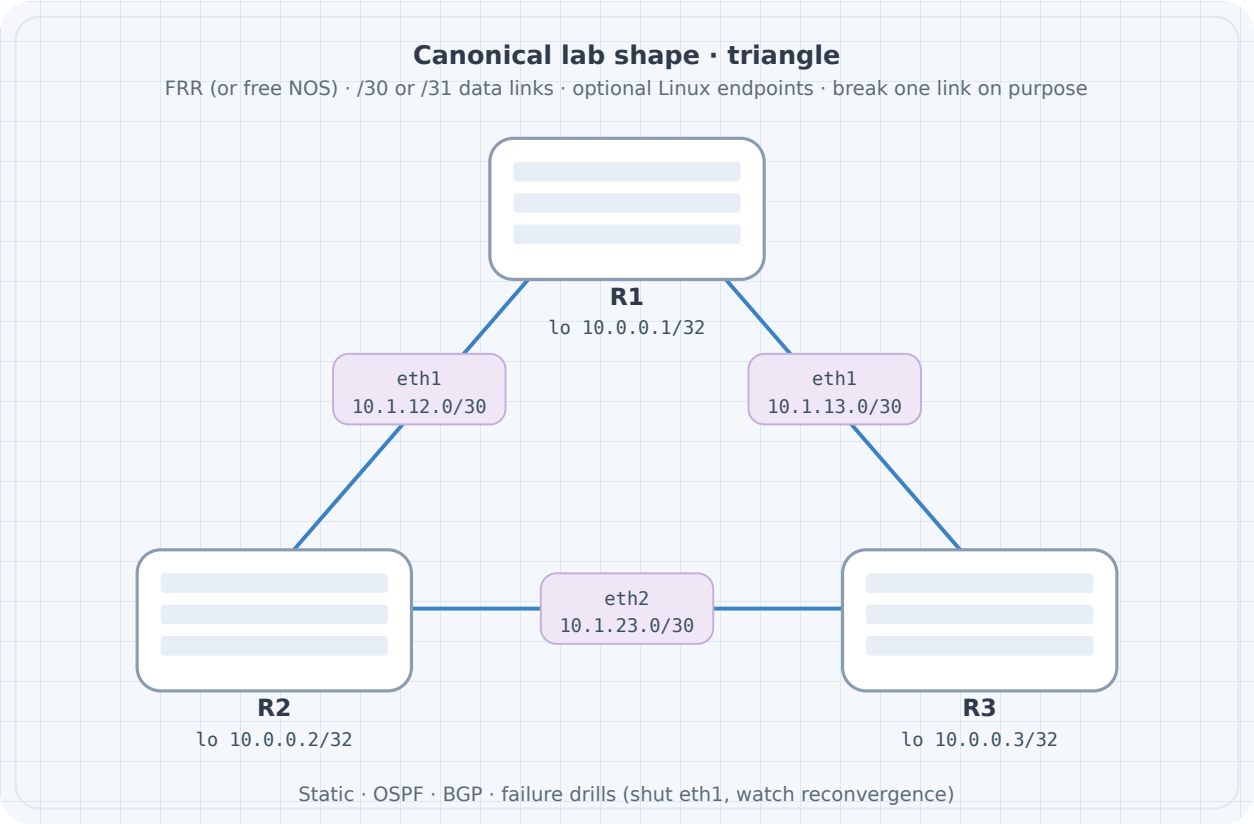


Figure 1: Triangle

Containerlab topology

```
name: static-tri

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth3"]
    - endpoints: ["h2:eth1", "r2:eth3"]
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
```

FRR config sketches

Enable `zebra` in `daemons`. Example `frr.conf` for **r1**:

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface lo
 ip address 1.1.1.1/32
!
interface eth1
 ip address 10.0.12.1/24
!
interface eth2
 ip address 10.0.13.1/24
!
interface eth3
 ip address 192.168.1.1/24
!
ip route 192.168.2.0/24 10.0.12.2
ip route 2.2.2.2/32 10.0.12.2
ip route 3.3.3.3/32 10.0.13.2
!
line vty
```

r2 mirrors with LAN `192.168.2.1/24`, route to `192.168.1.0/24` via `10.0.12.1`, etc.

r3 needs routes to both LANs if used as alternate path:

```
ip route 192.168.1.0/24 10.0.13.1
ip route 192.168.2.0/24 10.0.23.1
```

Adjust interface IPs for r3 on `10.0.13.2` and `10.0.23.2`.

> Exact FRR container startup (vtysh config load) depends on image—follow image docs for `/etc/frr` layout (`frr.conf`, `daemons`, permissions).

Linux-only alternative

If FRR image friction appears, pure Alpine routers work:

```
sysctl -w net.ipv4.ip_forward=1
ip addr add ...
ip route add 192.168.2.0/24 via 10.0.12.2
```

Deploy and baseline

```
sudo containerlab deploy -t static-tri.clab.yml
docker exec clab-static-tri-r1 vtysh -c 'show ip route'
docker exec clab-static-tri-h1 ping -c 3 192.168.2.10
docker exec clab-static-tri-h1 traceroute -n 192.168.2.10
```

Predict

- Path h1→h2 is h1–r1–r2–h2 (direct static)
- Loopback 2.2.2.2 reachable from r1

Observe

```
docker exec clab-static-tri-r1 ip route get 192.168.2.10
docker exec clab-static-tri-r2 ip route get 192.168.1.10
```

Floating statics

On r1, prefer r2 path, backup via r3:

```
ip route 192.168.2.0/24 10.0.12.2
ip route 192.168.2.0/24 10.0.13.2 200
```

In Linux:

```
ip route add 192.168.2.0/24 via 10.0.12.2 metric 10
ip route add 192.168.2.0/24 via 10.0.13.2 metric 200
```

Caveat: Floating statics failover when the **route is withdrawn**—often when the interface or next hop tracking says down. A silent next hop that stays "up" but blackholes may **not** fail over without tracking/BFD (advanced). In labs, shut the interface to force failover.

Failover drill

```
docker exec clab-static-tri-h1 ping -c 100 192.168.2.10 &
docker exec clab-static-tri-r1 ip link set eth1 down
# observe loss then recovery via r3 if floating installed end-to-end
docker exec clab-static-tri-r1 ip route get 192.168.2.10
docker exec clab-static-tri-r1 ip link set eth1 up
```

Ensure r3 and r2 have consistent return paths for the backup trajectory.

Return path drill (mandatory)

Inject on r2:

```
# remove route to h1 LAN
vtysh -c 'configure terminal' -c 'no ip route 192.168.1.0/24 10.0.12.1' -c 'end'
# or linux: ip route del 192.168.1.0/24
```

Predict: One-way behavior; h1→h2 may fail depending on path; debugging must check reverse.

Restore from config files + reload, not memory folklore.

Blackhole and reject

```
ip route add blackhole 10.66.0.0/16
# or
ip route add unreachable 10.66.0.0/16
```

Useful when summarizing: explicit sink for unallocated space you advertise.

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-static-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-static-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"
docker exec clab-static-tri-r1 vtysh -c 'show ip route' | grep -q 192.168.2.0 || fail "r1 ro

echo OK
```

Addressing.md excerpt

```
| Node | Iface | IP |
|------|-------|-----|
| r1 | eth1 | 10.0.12.1/24 |
| r2 | eth1 | 10.0.12.2/24 |
| r1 | eth2 | 10.0.13.1/24 |
```

```
| r3 | eth2 | 10.0.13.2/24 |
| r2 | eth2 | 10.0.23.1/24 |
| r3 | eth1 | 10.0.23.2/24 |
| r1 | eth3 | 192.168.1.1/24 |
| r2 | eth3 | 192.168.2.1/24 |
| h1 | eth1 | 192.168.1.10/24 |
| h2 | eth1 | 192.168.2.10/24 |
```

Default routes at edge

Simulating "internet":

```
# on r1
ip route 0.0.0.0/0 10.0.12.2
```

Only if r2 has a path onward. Avoid default loops (r1 defaults to r2, r2 defaults to r1).

IPv6 statics (optional parallel)

```
ipv6 route 2001:db8:2::/64 2001:db8:12::2
```

```
ip -6 route add 2001:db8:2::/64 via 2001:db8:12::2
```

Dual-stack the triangle when comfortable.

Common mistakes

Mistake	Symptom
Missing return route	One-way ping
Wrong next hop IP	Route present, unresolved neigh
Floating without interface down	No failover on silent blackhole
Overlapping LAN IPs	Chaos
Forgetting ip_forward	Router becomes host

Summary

- Static labs teach bidirectional design and verification

- Triangle + floating statics build failover intuition
- Interface shutdown is the honest failover inject
- FRR and Linux both work; keep configs in git
- verify.sh must test **both** directions

Next: **ICMP and path MTU**—when small pings lie and large packets die.

ICMP & Path MTU

ICMP & Path MTU

Small pings succeed while large transfers hang: classic **MTU / PMTUD** failure. This chapter treats ICMP as a control helper and shows you how to find black holes with deliberate packet sizes.

Learning goals

By the end of this chapter you can:

- Explain key ICMP types used in operations
- Test reachability with controlled sizes and DF bit
- Predict PMTUD behavior when ICMP "fragmentation needed" is filtered
- Clamp MTU on a lab link and measure the symptom
- Document a harden checklist for MTU in topologies

ICMP essentials (IPv4)

Type	Role
Echo request/reply	ping
Destination unreachable	no route, port unreachable, **frag needed**
Time exceeded	TTL expired (traceroute)
Redirect	host guidance (often ignored/noisy)

```
ping -c 3 10.0.0.1
ping -c 3 -s 1400 10.0.0.1
ping -M do -s 1472 10.0.0.1   # DF set; 1472 + 28 = 1500
```

`-M do` (Linux) sets Don't Fragment. Adjust payload so total IP packet hits MTU boundary.

IPv6 uses ICMPv6 extensively (ND, PMTUD); similar lab methods with `ping -6`.

Path MTU discovery (idea)

1. Sender assumes path MTU (often interface MTU)
2. Sends DF packets
3. Hop with smaller MTU sends ICMP frag needed + next-hop MTU
4. Sender caches smaller PMTU

If step 3 is **filtered**, large packets blackhole; small pings work.

Lab: MTU mismatch

```
h1 -- r1 ======== r2 -- h2
         MTU 1400 on r1-r2 link
```

Topology

```
name: mtu-lab

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iptables
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 192.168.1.1/24 dev eth1
        - ip addr add 10.0.0.1/24 dev eth2
        - ip link set eth1 up
        - ip link set eth2 up
        - ip link set eth2 mtu 1400
        - ip route add 192.168.2.0/24 via 10.0.0.2
    r2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iptables
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 192.168.2.1/24 dev eth1
        - ip addr add 10.0.0.2/24 dev eth2
        - ip link set eth1 up
```

```
      - ip link set eth2 up
      - ip link set eth2 mtu 1400
      - ip route add 192.168.1.0/24 via 10.0.0.1
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.1.10/24 dev eth1
      - ip link set eth1 up
      - ip route add default via 192.168.1.1
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.2.10/24 dev eth1
      - ip link set eth1 up
      - ip route add default via 192.168.2.1

links:
  - endpoints: ["h1:eth1", "r1:eth1"]
  - endpoints: ["h2:eth1", "r2:eth1"]
  - endpoints: ["r1:eth2", "r2:eth2"]
```

Predict

- `ping -c 2 192.168.2.10` small default works
- Large DF ping across path may fail or trigger PMTUD depending on sizes
- Capture may show ICMP unreachable frag needed if not filtered

Observe

```
docker exec clab-mtu-lab-h1 ping -c 2 192.168.2.10
docker exec clab-mtu-lab-h1 ping -M do -s 1472 -c 2 192.168.2.10
docker exec clab-mtu-lab-h1 ping -M do -s 1200 -c 2 192.168.2.10
docker exec clab-mtu-lab-r1 ip link show eth2
```

Inject: drop ICMP unreachable on r1

```
docker exec clab-mtu-lab-r1 iptables -A FORWARD -p icmp --icmp-type fragmentation-needed -j
# also output path variants may need INPUT/OUTPUT depending on generation point
docker exec clab-mtu-lab-r1 iptables -A OUTPUT -p icmp --icmp-type fragmentation-needed -j D
```

Retest large DF pings / large TCP if iperf available.

Predict: hard blackhole for large DF packets.

Restore

```
docker exec clab-mtu-lab-r1 iptables -F
```

Harden

- Match MTUs both ends of a link
- Do not filter all ICMP on transit routers
- For TCP, MSS clamping on edges sometimes used as workaround—prefer fixing MTU/ICMP

```
# example TCP MSS clamp (awareness; use carefully)
iptables -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1360
```

Traceroute and ICMP filters

```
traceroute -n 192.168.2.10
```

If time-exceeded filtered, traceroute incomplete while data works. Journal both ICMP classes separately: **PMTUD** vs **traceroute**.

Capture workflow

```
docker exec clab-mtu-lab-r1 tcpdump -ni eth2 -c 40 icmp or '(ip[6:2] & 0x1fff) != 0'
docker exec clab-mtu-lab-h1 ping -M do -s 1472 -c 3 192.168.2.10
```

Look for:

- Echo requests size

- ICMP type 3 code 4 (frag needed)
- Whether sender reduces size after

iperf3 large transfers

```
# on h2
apk add iperf3
iperf3 -s
# on h1
iperf3 -c 192.168.2.10
```

Compare before/after MTU clamp and ICMP drop. TCP may stall if PMTUD blackhole.

IPv6 notes

IPv6 routers do not fragment in the same way; PMTUD is even more critical. Minimum link MTU is 1280.

```
ping -6 -M do -s 1400 <dst>
ip -6 route get <dst>
```

Operational checklist

Check	Command / action
Interface MTU	`ip link`
End-to-end small ping	`ping -c 3`
DF large ping	`ping -M do -s ...`
ICMP allowed	policy review
TCP test	iperf3 / curl large
Tunnel overhead	account for encapsulation

Tunnels (later chapters) reduce effective MTU—plan overhead.

Predict worksheet

Path MTU 1500 except one hop 1400; ICMP filtered.

1. Ping default size 56 payload — ?

2. DF packet size 1500 — ?

3. TCP bulk transfer — ?

Answers: (1) usually ok; (2) blackhole; (3) often hang or extreme slowness depending on stack workarounds.

verify.sh additions

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-mtu-lab-h1 ping -c 2 -W 1 192.168.2.10
# expect success for moderate DF size under 1400 path
docker exec clab-mtu-lab-h1 ping -M do -s 1200 -c 2 -W 1 192.168.2.10
echo OK
```

Optional: assert large 1472 DF fails when MTU 1400—can be a negative test in failure mode only.

Summary

- ICMP is part of healthy networks (PMTUD, traceroute), not pure “attack surface” to zero
- Size-controlled pings expose MTU issues small pings hide
- Filtering frag-needed creates blackholes
- Match link MTUs; account for encapsulation overhead
- Always pair ping with a bulk TCP test when performance is in question

Next part: **interior routing**—OSPF and BGP basics plus policy/filters on free FRR images.

Interior Routing

OSPF Basics

OSPF Basics

OSPF is a **link-state IGP**: routers flood topology information within an area, run SPF, and install routes. This chapter builds a single-area triangle on FRR in Containerlab—the highest-ROI dynamic routing lab in the book.

Learning goals

By the end of this chapter you can:

- Contrast link-state vs distance-vector at a model level
- Bring up OSPF adjacencies on FRR and read neighbor states
- Advertise loopbacks and LANs into OSPF
- Predict reconvergence when a link costs out or dies
- Verify RIB/FIB and traceroute paths

Concepts

Distance-vector vs link-state

	Distance-vector (idea)	Link-state (OSPF)
What is shared	Vectors to prefixes	Topology (LSAs)
Loop risk	Needs careful rules	SPF on common map
Visibility	Partial	Area-wide topology view
Example	Classic RIP	OSPF, IS-IS

OSPF adjacency state machine (short)

Down → Init → 2-Way → ExStart → Exchange → Loading → **Full**

For learning: **Full** with expected neighbors on each transit link. Stuck states often mean MTU mismatch, authentication, layer-2, or network type mismatch.

Areas

Single-area (0) is enough here. Multi-area adds ABR summarization later in your journey; do not rush.

Cost

OSPF picks lowest cost path. Cost often derives from reference bandwidth / interface bandwidth. In FRR you can set `ip ospf cost` on interfaces for drills.

Network types (awareness)

Point-to-point vs broadcast affects adjacency count and DR/BDR. P2P links in labs are simple; Ethernet multipoint may elect DR.

Addressing plan

Node	lo	eth1 (to peer)	eth2	LAN eth3
r1	1.1.1.1/32	10.0.12.1/24 (r2)	10.0.13.1/24 (r3)	192.168.1.1/24
r2	2.2.2.2/32	10.0.12.2/24	10.0.23.1/24	192.168.2.1/24
r3	3.3.3.3/32	10.0.23.2/24	10.0.13.2/24	optional

Hosts: `192.168.1.10`, `192.168.2.10` with defaults to local r.

Method A: The Fast Netlab Workflow (Recommended)

Using Netlab (with `provider: clab`), you declare only the protocol intent and node interconnections. Netlab automatically plans IP subnets, allocates loopbacks, writes the FRR configurations, and stands up the Containerlab environment:

```
# topology.yml
---
provider: clab
defaults.device: frr
module: [ ospf ]

nodes:
  r1:
  r2:
```

```
  r3:
  h1:
    device: linux
  h2:
    device: linux

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:
  - h1:
    r1:
  - h2:
    r2:

validate:
  ospf_neighbors:
    description: "Verify all routers reach Full OSPF adjacency with 2 neighbors"
    nodes: [ r1, r2, r3 ]
    plugin: ospf_neighbor(count=2, state='Full')
  end_to_end_ping:
    description: "Verify end-to-end connectivity between endpoints h1 and h2"
    nodes: [ h1 ]
    exec:
      ping_h2: ping -c 3 10.0.0.5
```

Deploy and Validate with Netlab

```
# Compile configurations and bring up the Containerlab environment
netlab up

# Run automated OSPF adjacency and ping assertions
netlab validate

# Connect to any router CLI directly without memorizing container IDs
netlab connect r1
```

Method B: Raw Containerlab & Manual FRR Configurations

If you are inspecting low-level container plumbing or deploying on a system without Python/Netlab, you can author the Containerlab topology and FRR configurations by hand.

Topology YAML (`ospf-tri.clab.yml`)

```yaml
name: ospf-tri

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
    - endpoints: ["h1:eth1", "r1:eth3"]
```

```
    - endpoints: ["h2:eth1", "r2:eth3"]
```

FRR Daemons File (`config/r1/daemons`)

`daemons` file must enable:

```
zebra=yes
ospfd=yes
```

FRR OSPF Config (`config/r1/frr.conf`)

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface lo
 ip address 1.1.1.1/32
 ip ospf area 0
!
interface eth1
 ip address 10.0.12.1/24
 ip ospf area 0
!
interface eth2
 ip address 10.0.13.1/24
 ip ospf area 0
!
interface eth3
 ip address 192.168.1.1/24
 ip ospf area 0
!
router ospf
 ospf router-id 1.1.1.1
!
line vty
```

Advertise by putting interfaces in area 0 (common FRR style). Equivalent network statements exist; be consistent across nodes.

r2/r3: unique router-ids `2.2.2.2`, `3.3.3.3`; matching interface IPs; LANs in OSPF so remote LANs appear in RIB.

Deploy and Verify Manually

```
sudo containerlab deploy -t ospf-tri.clab.yml

docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf route'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf database'

docker exec clab-ospf-tri-h1 ping -c 3 192.168.2.10
docker exec clab-ospf-tri-h1 traceroute -n 192.168.2.10
```

Predict

- Each router: 2 neighbors Full (triangle)
- RIB contains remote LAN and loopbacks via OSPF
- Path prefers lower cost sides

Neighbor not Full?

Check:

```
ip link
ip addr
# MTU match
vtysh -c 'show ip ospf interface'
# timers, area mismatch, passive interfaces accidental
```

Cost change drill

```
# on r1 eth1 increase cost so traffic prefers other way
docker exec -it clab-ospf-tri-r1 vtysh
configure terminal
interface eth1
 ip ospf cost 1000
end
write
# promote back to frr.conf in git after experiment
```

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route 192.168.2.0/24'
docker exec clab-ospf-tri-h1 traceroute -n 192.168.2.10
```

Predict: path shifts if alternate exists with lower total cost.

Link failure drill

```
docker exec clab-ospf-tri-r1 ip link set eth1 down
sleep 5
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-ospf-tri-h1 ping -c 20 192.168.2.10
docker exec clab-ospf-tri-r1 ip link set eth1 up
```

Journal reconvergence behavior and whether ping loss matches expectations.

Passive interfaces / LAN notes

Sometimes you put a LAN in OSPF for advertisement but want **no** adjacency on user ports:

```
router ospf
 passive-interface eth3
```

Loopbacks are often passive. Transit links remain non-passive.

Verification checklist

Check	Command
Neighbors	`show ip ospf neighbor`
Database	`show ip ospf database`
OSPF RIB	`show ip ospf route`
Kernel/FIB	`show ip route` / `ip route`
Data plane	ping / traceroute

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }
```

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor' | grep -q Full \
  || fail "no Full neighbor on r1"
docker exec clab-ospf-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-ospf-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"
echo OK
```

Common mistakes

Mistake	Symptom
ospfd not enabled	no neighbors
Mismatched areas	stuck
Different IP subnets on link	no adjacency
MTU mismatch	ExStart/Exchange issues
router-id collisions	confusing DB
Forgetting LAN in OSPF	hosts missing remote routes

Beyond this chapter

- Multi-area + summarization at ABR
- Stub/NSSA ideas
- OSPFv3 for IPv6
- Authentication

Master single-area Full adjacencies and failover first.

Summary

- OSPF floods link state, computes SPF, installs routes
- FRR triangle lab is the default IGP workout
- Always verify neighbors Full **and** FIB **and** ping
- Cost and link-down drills teach reconvergence
- Keep configs in binds; promote experimental vtysh changes to git

Next: **BGP basics**—sessions, path selection process, and a dual-router lab without vendor exam noise.

BGP Basics

BGP Basics

BGP is the inter-domain workhorse and also appears inside modern DC fabrics. This chapter stays vendor-agnostic and free-image-only: **eBGP and iBGP ideas**, sessions, advertisements, and path selection at a practitioner level—implemented with FRR.

Learning goals

By the end of this chapter you can:

- Explain ASNs, eBGP vs iBGP roles
- Bring up a BGP session on FRR to Established
- Advertise a prefix and see it on the peer
- Apply the path selection process at a high level
- Break a session and observe withdrawal

Concepts

Autonomous systems

An **AS** is a policy domain with an ASN. Labs use private ASNs:

Range	Notes
64512–65534	16-bit private
4200000000+	32-bit private space (use docs for exact ranges)

Example: AS65001 and AS65002 as "two sites" or "customer and ISP."

	eBGP	iBGP

eBGP vs iBGP

	eBGP	iBGP
Between	Different ASNs	Same ASN
Typical TTL	1 (direct)	multi-hop ok with care
Default next-hop	Peer address	Needs next-hop-self often for reachability
Scale	Edge policy	Route reflection / confed for large iBGP

Session state

Idle → Connect → Active → OpenSent → OpenConfirm → **Established**

Only Established exchanges routes.

Path selection overview

BGP is policy-heavy. Simplified order of preference ideas:

1. Prefer higher **local preference** (inbound influence)
2. Prefer shorter **AS path**
3. Origin, MED, eBGP over iBGP, IGP metric to next hop, router-id, etc.

You will not memorize every tie-breaker today—**know that policy beats pure shortest path**.

NLRI and attributes

Routes carry attributes (AS_PATH, NEXT_HOP, LOCAL_PREF, MED, communities…). Filters rewrite attributes; that is policy.

Lab A: two AS eBGP

```
h1--r1 (AS65001) -------- r2 (AS65002)--h2
         10.0.0.1/30   10.0.0.2/30
```

Advertise LANs (or loopbacks) across eBGP.

Topology

```
name: bgp-ebgp

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["h1:eth1", "r1:eth2"]
    - endpoints: ["h2:eth1", "r2:eth2"]
```

daemons

```
zebra=yes
bgpd=yes
```

r1 frr.conf

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.0.0.1/30
!
interface eth2
 ip address 192.168.1.1/24
!
interface lo
 ip address 1.1.1.1/32
!
router bgp 65001
 bgp router-id 1.1.1.1
 no bgp ebgp-requires-policy
 neighbor 10.0.0.2 remote-as 65002
 !
 address-family ipv4 unicast
  network 192.168.1.0/24
  network 1.1.1.1/32
  neighbor 10.0.0.2 activate
 exit-address-family
!
line vty
```

> `no bgp ebgp-requires-policy` eases labs on modern FRR that otherwise require explicit export policy. In real life, **prefer explicit policies** (next chapter).

r2

ASN 65002, IP `10.0.0.2/30`, LAN `192.168.2.0/24`, networks advertised symmetrically, neighbor `10.0.0.1 remote-as 65001`.

Verify session

```
sudo containerlab deploy -t bgp-ebgp.clab.yml
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp ipv4 unicast'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show ip route'
docker exec clab-bgp-ebgp-h1 ping -c 3 192.168.2.10
```

Predict

- `show bgp summary` state Established, prefixes received > 0
- Kernel has remote LAN via BGP next hop 10.0.0.2
- Ping works both ways

Lab B: iBGP sketch (optional same day)

Two routers same ASN, loopbacks as session endpoints, IGP or statics for loopback reachability first.

```
router bgp 65001
 neighbor 2.2.2.2 remote-as 65001
 neighbor 2.2.2.2 update-source lo
 neighbor 2.2.2.2 next-hop-self
```

Rule: iBGP needs **underlying reachability** to the session address. OSPF underlay + iBGP overlay is a common pattern (underlay vs overlay planes!).

Session failure drill

```
docker exec clab-bgp-ebgp-r1 ip link set eth1 down
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show ip route 192.168.1.0'
# routes withdraw after hold timers - observe timing
docker exec clab-bgp-ebgp-r1 ip link set eth1 up
```

Predict: session drops; prefixes withdrawn; ping fails until re-established.

Optional: `neighbor x shutdown` in vtysh for cleaner control-plane inject.

Next-hop reachability drill

Install BGP route whose next hop is unreachable (misconfigured).

Symptom: BGP table shows path but not installed in FIB / not usable.

```
vtysh -c 'show bgp ipv4 unicast'
vtysh -c 'show ip route'
```

Compare “known in BGP” vs “installed.”

Dual-home teaser

Customer with two eBGP uplinks steers with local-pref / AS-path prepend—full treatment lives in policy chapter. Topology idea:

```
      isp1
     /
cust
     \
      isp2
```

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary' | grep -q Established \
  || fail "session not Established"
docker exec clab-bgp-ebgp-h1 ping -c 2 -W 1 192.168.2.10 || fail "ping"
docker exec clab-bgp-ebgp-h2 ping -c 2 -W 1 192.168.1.10 || fail "reverse ping"
echo OK
```

Safety habits

- Do not experiment with BGP on real Internet edges without change control
- Private ASNs in labs only unless you know what you are doing
- Explicit export policy in anything beyond throwaway labs
- Cap lab prefixes; never leak real full tables into tiny FRR VMs by accident

Common mistakes

Mistake	Result
wrong remote-as	session fails
no underlay to iBGP peer	idle/active
missing activate / AF	Established but no routes
ebgp-requires-policy deny	no prefixes without route-maps
next hop unreachable	BGP path not used

Summary

- BGP sessions must be Established before routes flow
- eBGP links ASNs; iBGP needs underlay reachability
- Path selection is policy-centric, not pure SPF
- FRR lab: advertise LANs, ping through, shut session
- Prefer explicit policies as you leave pure lab mode

Next: **policy and filters**—prefix lists, route-maps-as-idea, and preventing loops/leaks.

Policy & Filters

Policy & Filters

Routing without policy is a demo; routing with policy is operations. This chapter covers **prefix filters, distribute ideas, route-maps as a pattern, and loop-prevention instincts** using FRR—still free and vendor-agnostic.

Learning goals

By the end of this chapter you can:

- Write prefix-lists that match intended space only
- Apply import/export filters on BGP neighbors
- Use route-map style logic (match → set → permit/deny)
- Predict leaks and loops from redistribution mistakes
- Build a lab that rejects unauthorized prefixes

Why policy exists

Goal	Mechanism ideas
Do not accept bogons / your own space from customers	Inbound prefix filter
Do not advertise internal-only nets to Internet	Outbound filter
Prefer one uplink	local-pref on import
De-prefer backup path	AS-path prepend on export
Redistribute without feedback loops	Tags, filters, distance

Prefix lists (FRR)

```
ip prefix-list OWN-NETS seq 10 permit 192.168.1.0/24
ip prefix-list OWN-NETS seq 20 permit 1.1.1.1/32
ip prefix-list OWN-NETS seq 30 deny any
```

```
ip prefix-list CUST-ALLOW seq 10 permit 198.51.100.0/24
ip prefix-list CUST-ALLOW seq 20 deny any
```

Order matters: first match wins. Always know your implicit/default at the end.

Route-maps as idea

Regardless of vendor syntax, the pattern is:

```
if match prefix-list X:
  set local-preference 200
  permit
else deny
```

FRR example:

```
route-map FROM-CUST permit 10
 match ip address prefix-list CUST-ALLOW
 set local-preference 200
route-map FROM-CUST deny 99
!
route-map TO-CUST permit 10
 match ip address prefix-list OWN-NETS
```

Attach:

```
router bgp 65002
 neighbor 10.0.0.1 route-map FROM-CUST in
 neighbor 10.0.0.1 route-map TO-CUST out
```

Lab: filter unauthorized prefixes

Extend eBGP lab: r2 (ISP) should **only** accept `192.168.1.0/24` from r1 (customer), not a hijack prefix.

Customer r1 advertises extra junk

```
address-family ipv4 unicast
 network 192.168.1.0/24
 network 203.0.113.0/24
```

ISP r2 policy

```
ip prefix-list CUST-R1 seq 5 permit 192.168.1.0/24
ip prefix-list CUST-R1 seq 10 deny any
!
route-map FROM-R1 permit 10
 match ip address prefix-list CUST-R1
route-map FROM-R1 deny 20
!
router bgp 65002
 neighbor 10.0.0.1 route-map FROM-R1 in
```

Predict

- ISP installs `192.168.1.0/24`
- `203.0.113.0/24` is rejected
- `show bgp` on r2 lacks the junk

Observe

```
docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show ip route 203.0.113.0'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp ipv4 unicast neighbors 10.0.0.2 advertised-r
```

Harden

Return to explicit export on customer too—only advertise owned space.

```
route-map EXPORT-OWN permit 10
 match ip address prefix-list OWN-NETS
!
neighbor 10.0.0.2 route-map EXPORT-OWN out
```

Remove `no bgp ebgp-requires-policy` when policies are in place if you want FRR to enforce the discipline.

Local-pref steer drill

Two ISPs: isp1 and isp2; customer prefers isp1 for inbound to customer… actually:

- **Local pref** (on customer) influences **outbound** exit from customer AS
- **AS prepend / MED / communities** influence how others send **inbound**

Outbound preference (customer)

```
route-map PREFER-ISP1 permit 10
 set local-preference 300
route-map PREFER-ISP2 permit 10
 set local-preference 100
!
neighbor <isp1> route-map PREFER-ISP1 in
neighbor <isp2> route-map PREFER-ISP2 in
```

Predict: traffic leaves via isp1 when both paths exist.

```
vtysh -c 'show ip route 0.0.0.0'
vtysh -c 'show bgp ipv4 unicast'
```

Redistribution loop awareness

Redistributing OSPF BGP (or static OSPF) without filters can:

- Re-inject prefixes with changed metrics
- Create loops or suboptimal paths
- Explode tables

Rules of thumb:

1. Redistribute **on purpose**, on as few points as possible
2. Tag routes; filter on tags
3. Never blindly mutual redistribute two IGPs
4. Prefer BGP for policy domains, IGP for underlay topology

Example static→OSPF with filter (sketch)

```
ip prefix-list STATICS seq 5 permit 192.168.9.0/24
route-map REDIST-STATIC permit 10
 match ip address prefix-list STATICS
!
router ospf
 redistribute static route-map REDIST-STATIC
```

ACL-style packet filters vs routing policy

Do not confuse:

Routing policy	Packet filter
Which **routes** enter RIB	Which **packets** pass
BGP/OSPF route-maps	nftables/iptables/ACLs
Control plane	Data plane

Both appear in hardening. A permit route does not mean packets are allowed through a firewall mid-path.

Distribute-list / filter-list ideas

Older or alternate forms filter by ACL or AS-path lists:

```
bgp as-path access-list 1 permit _65001$
neighbor x filter-list 1 in
```

Same intent: **accept only what policy allows**.

Failure drills

Drill 1 — over-broad permit any

Export `permit any` to a lab "ISP."

Observe: internal lab nets leave the AS.

Fix: prefix-list OWN only.

Harden: default-deny export mindset.

Drill 2 — accidental deny all

Route-map deny first line matches everything.

Symptom: Established but zero prefixes.

Debug: `show bgp neighbors ... routes` / policy counters if available; simplify map.

Drill 3 — next-hop-self missing (iBGP)

Learned eBGP route reflected iBGP with remote next hop unreachable.

Symptom: BGP has path; FIB lacks usable route.

Fix: `next-hop-self` or IGP reachability to next hop.

verify.sh policy checks

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast' | grep -q '192.168.1.0/24' \
  || fail "missing customer prefix"
if docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast' | grep -q '203.0.113.0/24';
  fail "hijack prefix should be filtered"
fi
echo OK
```

Policy design worksheet

Before applying:

```
## Neighbor: r1-r2 eBGP
Import accept: ...
Import deny: ...
Export advertise: ...
Export never: ...
Preference intent: ...
Failure mode if map wrong: ...
```

Summary

- Policy decides which routes and with which attributes
- Prefix-lists + route-maps are the portable pattern
- Default-deny posture on Internet edges
- Redistribution needs filters/tags or it will bite
- Verify with BGP table **and** negative tests for rejected prefixes

Next part: **ops & automation**—observability, lab-as-code automation, and a capstone outline that pulls the book together.

BFD — Fast Failure Detection

BFD — Fast Failure Detection

Routing protocols are good at choosing paths and bad at noticing that a path died. When the failure is **not** a local link-down — a switch in the middle, a provider handoff, a hung line card, a VM whose host froze — BGP waits for its hold timer and OSPF for its dead interval. That is **minutes** of blackholed traffic with every session looking healthy. **Bidirectional Forwarding Detection (BFD)** is the boring fix: a tiny, protocol-independent hello that proves the forwarding path works and tells its clients (BGP, OSPF, static routes) within a second when it does not. This chapter runs the failure first without BFD, then with it, on two FRR routers behind a Linux bridge — and measures both.

Learning goals

By the end of this chapter you can:

- Explain why carrier loss on *one* side does not reach the other side through a switch
- Predict BGP failure detection time from keepalive/hold timers (and why it is a range)
- Describe BFD control packets, the session states, and the detection-time formula
- Enable BFD for an FRR BGP neighbor with a profile and verify the negotiated timers
- Show that the **slower side wins** timer negotiation
- Attach BFD to a static route so it is withdrawn when the next hop dies
- Choose timers that do not turn CPU hiccups into routing flaps

Why carrier is not enough

```
          r1-eth1                        r2-eth1
r1 (AS65001)      sw (bridge)      r2 (AS65002)   h2
10.0.12.1/24        sw-r1      sw-r2       10.0.12.2/24     203.0.113.10/24

                    failure here: r2 sees NO-CARRIER,
                    r1's port to the switch stays UP
```

Direct cable: either end losing light usually drops the link at both ends, and routing reacts to the interface event in milliseconds. Through a switch, a media converter, a provider's L2 service, or a virtual switch, **each side only sees its own port**. r2 notices instantly and tears its session down; r1 still has carrier, still has an Established BGP session, and still forwards toward a next hop that will never answer.

Mechanism	Default (FRR "traditional")	Detects
Local carrier	Immediate	Only *your* link
BGP hold timer	keepalive 60 s, hold 180 s	Peer silent for hold time
OSPF dead interval	hello 10 s, dead 40 s	Neighbor silent for dead interval
BFD (async)	300 ms × 3 = 900 ms	Forwarding path silent for detect time

Tuning BGP to keepalive 3 / hold 9 helps, but it puts liveness inside a TCP session handled by a busy control-plane process, it is per-protocol, and you still get several seconds. BFD exists so that every protocol can share **one** cheap, sub-second liveness check.

Concepts — BFD in one page

What it is. RFC 5880 defines BFD; RFC 5881 maps it onto single-hop IPv4/IPv6 (UDP destination port **3784**, sent with TTL/hop-limit **255** and checked on receipt so only a directly connected peer can inject packets); RFC 5883 covers multihop sessions (UDP **4784**, e.g., between loopbacks); RFC 5882 describes how clients such as BGP and OSPF should react.

Asynchronous mode. Both sides send control packets periodically. Each packet carries the sender's *My Discriminator* and the receiver's *Your Discriminator* (session IDs), the state, a diagnostic code, and three timer values: **Desired Min TX**, **Required Min RX**, and **Detect Mult**.

States. `AdminDown` → `Down` → `Init` → `Up`. A three-way handshake brings a session up; while not Up, packets are sent at most once per second (slow start), so a flapping peer cannot generate a burst.

Timer negotiation — the slower side wins.

```
r1 transmit interval = max(r1 Desired Min TX, r2 Required Min RX)
r1 detection time    = r2 Detect Mult × max(r1 Required Min RX, r2 Desired Min TX)
```

Each side declares what it *wants* to send and what it *can* receive; nobody is forced to receive faster than it said. Transmit times are jittered (75–100% of the interval) so sessions do not synchronize.

Diagnostics. When a session goes down the reason is in the packet and in `show bfd peer`: `control detection time expired` (no packets in time — the classic failure), `neighbor signaled session down`, `administratively down` (the peer disabled it on purpose — RFC 5882 says clients should *not* treat this as a path failure), and others.

Echo mode (optional) sends packets that the peer loops back in its forwarding plane, testing the data path rather than the peer's BFD process. Single-hop only. In FRR it works only when the peer is also FRR, unless you have hardware-offloaded BFD.

Clients. BFD itself changes nothing in the RIB. A client registers interest in a peer; when BFD declares it Down, the client acts — BGP closes the session and withdraws routes, OSPF drops the adjacency, a static route is removed. Several clients share one session to the same peer.

Addressing plan (BFD lab)

Node	Interface	Address	Role
r1	r1-eth1	10.0.12.1/24	AS 65001
r2	r2-eth1	10.0.12.2/24	AS 65002
r2	r2-eth2	203.0.113.1/24	Customer LAN gateway
r2	lo	198.51.100.1/24	Static-route target (Drill 4)
h2	h2-eth0	203.0.113.10/24	Customer host
sw	br0 (sw-r1, sw-r2)	—	L2 switch

Lab build (Linux namespaces + FRR)

This lab needs no container images: four network namespaces, veth pairs, a Linux bridge as the switch, and one FRR instance per router namespace using FRR's **pathspace** option (`-N`), which gives each instance its own sockets and PID files under `/var/run/frr/<name>/` and its own `vtysh -N <name>`. It was run on **Debian 13 with the distro `frr` package (FRR 10.3)**, as root. Outputs below are copied from those runs; ages, IDs, and PIDs will differ on yours.

The same `frr.conf` snippets drop unchanged into the FRR containers used in other labs in this book — add `bfdd=yes` to their `daemons` file.

Install FRR and helpers (Debian/Ubuntu), and make sure the system-wide FRR service is not running (it would use the default pathspace, not these):

```
sudo apt-get install -y frr iputils-ping bc
```

```
#!/usr/bin/env bash
# lab.sh - r1 and r2 (FRR) behind a Linux-bridge "switch", h2 on r2's LAN
# usage: sudo ./lab.sh up | down
set -euo pipefail
FRR=/usr/lib/frr
DAEMONS="mgmtd zebra bgpd bfdd staticd"

up() {
  for ns in r1 r2 sw h2; do ip netns add "$ns"; ip -n "$ns" link set lo up; done
```

```
  # the switch: one bridge, two access ports
  ip -n sw link add br0 type bridge
  ip -n sw link set br0 up
  for r in r1 r2; do
    ip link add "$r-eth1" netns "$r" type veth peer name "sw-$r" netns sw
    ip -n sw link set "sw-$r" master br0 up
    ip -n "$r" link set "$r-eth1" up
  done
  ip -n r1 addr add 10.0.12.1/24 dev r1-eth1
  ip -n r2 addr add 10.0.12.2/24 dev r2-eth1

  # r2's customer LAN with one host
  ip link add r2-eth2 netns r2 type veth peer name h2-eth0 netns h2
  ip -n r2 addr add 203.0.113.1/24 dev r2-eth2
  ip -n h2 addr add 203.0.113.10/24 dev h2-eth0
  ip -n r2 link set r2-eth2 up
  ip -n h2 link set h2-eth0 up
  ip -n h2 route add default via 203.0.113.1
  ip netns exec r2 sysctl -qw net.ipv4.ip_forward=1
  ip netns exec r1 sysctl -qw net.ipv4.ip_forward=1

  for ns in r1 r2; do
    install -d -o frr -g frr "/var/run/frr/$ns" "/etc/frr/$ns"
    : > "/etc/frr/$ns/vtysh.conf"
    for d in $DAEMONS; do
      ip netns exec "$ns" "$FRR/$d" -d -N "$ns" -F traditional \
        -u frr -g frr --limit-fds 1024 >/dev/null 2>&1
    done
    vtysh -N "$ns" -f "$ns.conf" >/dev/null
  done
}

down() {
  for ns in r1 r2; do
    for d in $DAEMONS; do
      pidf="/var/run/frr/$ns/$d.pid"
      [ -f "$pidf" ] && kill "$(cat "$pidf")" 2>/dev/null || true
    done
  done
  for ns in r1 r2 sw h2; do ip netns del "$ns" 2>/dev/null || true; done
}

"$@"
```

Router configs — plain eBGP first, **no BFD**:

```
! r1.conf - AS 65001, plain eBGP (no BFD yet)
hostname r1
router bgp 65001
 bgp router-id 10.0.12.1
 no bgp ebgp-requires-policy
 neighbor 10.0.12.2 remote-as 65002
```

```
! r2.conf - AS 65002, originates the customer LAN
hostname r2
router bgp 65002
 bgp router-id 10.0.12.2
 no bgp ebgp-requires-policy
 neighbor 10.0.12.1 remote-as 65001
 address-family ipv4 unicast
  network 203.0.113.0/24
 exit-address-family
```

The failure drill — it fails r2's switch port, confirms r1's port is still up, and times how long r1 keeps the dead route in its kernel FIB:

```
#!/usr/bin/env bash
# drill.sh - fail r2's switch port, time how long r1 keeps a dead route
# usage: sudo ./drill.sh [prefix]   (default 203.0.113.0/24)
set -euo pipefail
PFX=${1:-203.0.113.0/24}

ip -n r1 route show "$PFX" | grep -q . || { echo "no route to $PFX on r1"; exit 1; }
echo "before: $(ip -n r1 route show "$PFX")"

start=$(date +%s.%N)
ip -n sw link set sw-r2 down            # r2 loses carrier; r1's port stays up
echo "r1-eth1 state after failure: $(ip -n r1 -br link show r1-eth1 | awk '{print $2}')"

while ip -n r1 route show "$PFX" | grep -q .; do sleep 0.05; done
end=$(date +%s.%N)
printf 'route withdrawn on r1 after %.2f s\n' "$(echo "$end - $start" | bc)"

ip -n sw link set sw-r2 up              # heal for the next run
```

Bring it up and check the baseline:

```
chmod +x lab.sh drill.sh
sudo ./lab.sh up
sleep 10
sudo vtysh -N r1 -c 'show ip route bgp'
sudo ip netns exec r1 ping -c2 -W1 203.0.113.10
```

Output (tails):

```
IPv4 unicast VRF default:
B>* 203.0.113.0/24 [20/0] via 10.0.12.2, r1-eth1, weight 1, 00:00:08
2 packets transmitted, 2 received, 0% packet loss, time 1023ms
```

Each daemon prints a `[pid|daemon] done` line as `vtysh -f` applies configuration; that is normal.

Drill 1 — the silent failure (no BFD)

Predict

r1's hold time is 180 s and keepalives arrive every 60 s. The failure happens at a random point in the keepalive cycle. How long will r1 keep the route?

```
sudo vtysh -N r1 -c 'show ip bgp neighbors 10.0.12.2' | grep -i 'hold time'
```

```
  Hold time is 180 seconds, keepalive interval is 60 seconds
  Configured hold time is 180 seconds, keepalive interval is 60 seconds
```

Observe

```
sudo ./drill.sh
```

While it waits, in a second terminal:

```
sudo ip netns exec r1 ping -c3 -W1 203.0.113.10 | tail -2
sudo vtysh -N r1 -c 'show ip bgp summary' | grep -A1 Neighbor
sudo vtysh -N r2 -c 'show ip bgp summary' | grep -A1 Neighbor
```

```
3 packets transmitted, 0 received, 100% packet loss, time 2039ms

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.0.12.2       4      65002         4         5        1    0    0 00:00:35            1
Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.0.12.1       4      65001         4         4        0    0    0 00:00:13       Active
```

r2 already knows (its port lost carrier). r1 shows **Established with 1 prefix** and drops every packet. The drill finishes:

```
before: 203.0.113.0/24 nhid 6 via 10.0.12.2 dev r1-eth1 proto bgp metric 20
r1-eth1 state after failure: UP
route withdrawn on r1 after 158.87 s
```

Explain

Not 180 s: the hold timer restarts on every keepalive, so detection is anywhere from **hold − keepalive** (120 s) to **hold** (180 s), depending on when the last keepalive landed. Two and a half minutes of blackhole with a green dashboard.

Drill 2 — add BFD and repeat

Define a profile on each router and attach it to the BGP neighbor. The profile asks for 100 ms intervals with multiplier 3 (300 ms detection). FRR's defaults are 300 ms × 3.

```
! r1-bfd.conf - add a BFD profile and attach it to the BGP neighbor
bfd
 profile fast
  detect-multiplier 3
  receive-interval 100
  transmit-interval 100
 exit
exit
router bgp 65001
 neighbor 10.0.12.2 bfd profile fast
```

```
! r2-bfd.conf - add a BFD profile and attach it to the BGP neighbor
bfd
 profile fast
  detect-multiplier 3
  receive-interval 100
  transmit-interval 100
 exit
exit
router bgp 65002
 neighbor 10.0.12.1 bfd profile fast
```

```
sudo vtysh -N r1 -f r1-bfd.conf
sudo vtysh -N r2 -f r2-bfd.conf
sleep 3
sudo vtysh -N r1 -c 'show bfd peers brief'
sudo vtysh -N r1 -c 'show bfd peer 10.0.12.2'
sudo vtysh -N r1 -c 'show ip bgp neighbors 10.0.12.2' | grep -A3 'BFD'
```

Output (session IDs will differ):

```
Session count: 1
SessionId  LocalAddress                             PeerAddress
=========  ============                             ===========
```

```
3526470594 10.0.12.1                                    10.0.12.2
```

```
BFD Peer:
    peer 10.0.12.2 local-address 10.0.12.1 vrf default interface r1-eth1
        ID: 3526470594
        Remote ID: 1869221485
        Active mode
        Status: up
        Uptime: 2 second(s)
        Diagnostics: ok
        Remote diagnostics: ok
        Peer Type: dynamic
        RTT min/avg/max: 0/0/0 usec
        Local timers:
            Detect-multiplier: 3
            Receive interval: 100ms
            Transmission interval: 100ms
            Echo receive interval: 50ms
            Echo transmission interval: disabled
        Remote timers:
            Detect-multiplier: 3
            Receive interval: 100ms
            Transmission interval: 100ms
            Echo receive interval: 50ms
```

```
  BFD: Type: single hop
  Detect Multiplier: 3, Min Rx interval: 300, Min Tx interval: 300
  Status: Up, Last update: 0:00:00:02
```

(That last block was captured right after the first apply, before the 100 ms profile values were in place — BGP's neighbor view shows what the session had at registration. Trust `show bfd peer` for live timers.) `Peer Type: dynamic` means a client (BGP) created the session; you did not declare a `peer` under `bfd`.

Run the drill again, a few times:

```
sudo ./drill.sh
```

```
before: 203.0.113.0/24 nhid 6 via 10.0.12.2 dev r1-eth1 proto bgp metric 20
r1-eth1 state after failure: UP
route withdrawn on r1 after 0.38 s
```

Repeated runs: `0.33 s`, `0.33 s`, `0.33 s` — the 300 ms detection time plus BGP and zebra doing their work. After a failure the session records why:

```
sudo vtysh -N r1 -c 'show bfd peer 10.0.12.2' | grep -E 'Status|Diagnostics|Downtime'
```

```
        Status: down
        Downtime: 0 second(s)
        Diagnostics: control detection time expired
```

From 158.87 s to 0.33 s, with the same BGP timers.

Drill 3 — the slower side wins

Put r2 back to 300 ms while r1 stays at 100 ms:

```
sed 's/interval 100/interval 300/g' r2-bfd.conf > r2-slow.conf
sudo vtysh -N r2 -f r2-slow.conf
sleep 3
sudo vtysh -N r1 -c 'show bfd peer 10.0.12.2 json' | python3 -m json.tool | grep -E 'interva
```

```
    "receive-interval": 100,
    "transmit-interval": 100,
    "echo-receive-interval": 50,
    "echo-transmit-interval": 0,
    "detect-multiplier": 3,
    "remote-receive-interval": 300,
    "remote-transmit-interval": 300,
    "remote-echo-receive-interval": 50,
    "remote-detect-multiplier": 3,
```

Predict

r1 detection time = r2 Detect Mult × max(r1 Required Min RX, r2 Desired Min TX) = 3 × max(100, 300) = **900 ms**.

Observe

```
sudo ./drill.sh | tail -1
```

```
route withdrawn on r1 after 0.86 s
```

Configuring one end aggressively buys nothing; detection follows the slower declaration. Restore r2 with `sudo vtysh -N r2 -f r2-bfd.conf`.

Drill 4 — BFD for a static route

Static routes are the other place silent failures hurt: a static next hop "stays up" as long as the interface does. FRR's `staticd` can make a static route a BFD client — installed only while the session is Up.

```
sudo ip -n r2 addr add 198.51.100.1/24 dev lo
```

```
! r1-static.conf - a static route that is only installed while BFD says the next hop is aliv
ip route 198.51.100.0/24 10.0.12.2 bfd profile fast
```

```
! r2-static.conf - answer BFD for r1 even without a routing protocol
bfd
 peer 10.0.12.1 interface r2-eth1
  profile fast
 exit
exit
```

r2 has no routing protocol for this prefix, so it gets an explicit `peer` under `bfd` to answer the session (in this lab BGP's session to the same address already exists and is shared — one session, two clients).

```
sudo vtysh -N r1 -f r1-static.conf
sudo vtysh -N r2 -f r2-static.conf
sleep 3
sudo vtysh -N r1 -c 'show ip route 198.51.100.0/24'
sudo ./drill.sh 198.51.100.0/24
```

```
Routing entry for 198.51.100.0/24
  Known via "static", distance 1, metric 0, best
  Last update 00:00:03 ago
  * 10.0.12.2, via r1-eth1, weight 1
```

```
before: 198.51.100.0/24 nhid 6 via 10.0.12.2 dev r1-eth1 proto static metric 20
r1-eth1 state after failure: UP
route withdrawn on r1 after 0.27 s
```

A few seconds after the drill heals the port, both the static and the BGP route are back. While a monitored static is uninstalled it disappears from `show ip route`; use `show bfd static route` to see it. This is what makes a floating static (a backup with a higher distance) actually float when the primary next hop dies behind a switch.

Drill 5 — one-sided BFD (predict, then try)

Tear the lab down and up again, and apply `r1-bfd.conf` **only on r1**.

```
sudo ./lab.sh down && sudo ./lab.sh up
sudo vtysh -N r1 -f r1-bfd.conf
sleep 10
sudo vtysh -N r1 -c 'show bfd peers brief'
sudo vtysh -N r1 -c 'show ip bgp summary'
```

Predict before you look: does BGP come up? Is the BFD session Up? Does the drill now take 0.3 s or ~150 s? (FRR's default BGP–BFD integration reacts to a session going **down**; a session that never came up does not hold BGP back. Newer FRR releases add `neighbor … bfd strict`, which refuses to bring BGP up until BFD is Up — check `neighbor 10.0.12.2 bfd ?` on your build.) The lesson either way: BFD must be configured on **both** ends, and your verification must check BFD state, not just BGP state.

Clean up when done:

```
sudo ./lab.sh down
```

Choosing timers

Environment	Starting point	Why
Physical routers, hardware/offloaded BFD	50–100 ms × 3	Packets handled off the main CPU
Software routers (FRR on servers)	300 ms × 3 (default)	Survives scheduler hiccups and GC pauses
VMs / containers / cloud	300–1000 ms × 3–5	Noisy neighbors, live migration, steal time
Over a provider L2/L3 service	Agree with the provider	Their policers may drop small UDP

Rules of thumb:

- **Detection time is the number you choose**; interval and multiplier are how you get there. 900 ms that never false-positives beats 150 ms that flaps every afternoon.
- A false BFD-down is not free: it tears down BGP, withdraws routes, and triggers re-convergence across the network. Combine with BGP **graceful restart** awareness (`bfd check-control-plane-failure` in FRR) when routers restart their control planes.

- Protect BFD from your own filters and QoS: allow UDP 3784/4784 (and the echo port 3785 if used) in control-plane policy, and do not let it queue behind bulk traffic.
- Use **multihop** BFD (RFC 5883) for loopback-to-loopback iBGP; single-hop rules (TTL 255) do not apply, so set `minimum-ttl` sensibly.

Verification checklist

Check	Command	Intent
Session up on both ends	`show bfd peers brief` (r1 and r2)	No one-sided config
Live timers	`show bfd peer <ip>`	Negotiated values, not just configured
Client attached	`show ip bgp neighbors <ip>` → `BFD:` block	BGP actually registered
Monitored statics	`show bfd static route`	Uninstalled statics are visible here
Last failure reason	`show bfd peer <ip>` → `Diagnostics`	Detection expired vs admin down
Filters	Control-plane ACL permits UDP 3784/4784	BFD not dropped by your own policy
Drill	`drill.sh` or equivalent	Measured, not assumed, detection time

Common mistakes

Mistake	Symptom
BFD on one side only	BFD never Up; failures still take hold-timer time
Aggressive timers on VMs	Random BGP flaps under load; `control detection time expired` with no link event
Tuning only one end	Detection follows the slower side (Drill 3)
Filtering UDP 3784 in control-plane policy	BFD Down, BGP Up — or worse, BGP reset by strict modes
Treating `AdminDown` as a failure	Maintenance on the peer causes an outage
Single-hop BFD between loopbacks	Session never forms; needs multihop
Trusting `show ip bgp summary` alone	"Established" while forwarding is dead (Drill 1)

Summary

- Through a switch or L2 service, carrier loss is **local**: the far router keeps its port, its session, and its route

- BGP detection is **hold — keepalive ... hold** (120–180 s by default) — measured here at **158.87 s**
- BFD is a shared sub-second liveness check; clients (BGP, OSPF, statics) act on its Down event — measured here at **0.33 s**
- Detection time = remote multiplier × max(local Required Min RX, remote Desired Min TX): **the slower side wins**
- Configure both ends, verify negotiated timers, size timers for your platform, and drill the failure

Sources

- RFC 5880 — Bidirectional Forwarding Detection (BFD): https://www.rfc-editor.org/rfc/rfc5880
- RFC 5881 — BFD for IPv4 and IPv6 (Single Hop): https://www.rfc-editor.org/rfc/rfc5881
- RFC 5882 — Generic Application of BFD: https://www.rfc-editor.org/rfc/rfc5882
- RFC 5883 — BFD for Multihop Paths: https://www.rfc-editor.org/rfc/rfc5883
- FRRouting user guide — BFD (`bfdd`, profiles, BGP and static integration): https://docs.frrouting.org/en/latest/bfd.html
- FRRouting user guide — BGP neighbor timers: https://docs.frrouting.org/en/latest/bgp.html

RPKI — Route Origin Validation

RPKI — Route Origin Validation

BGP believes whatever its neighbors say. If a peer announces your prefix with someone else's AS at the end of the path, your router installs it, prefers it if it is more specific, and forwards your customers' traffic into the wrong network. Prefix lists help for the handful of customers you know; they do nothing for the 1,000,000 routes arriving from a transit provider. **RPKI route origin validation (ROV)** is the boring, widely deployed answer: address holders publish signed statements of *which AS may originate which prefix, up to which length*, a validator turns those into a flat table, and your router labels every route **Valid**, **Invalid**, or **NotFound**. Policy then drops the Invalids. This chapter builds the whole chain on one Linux box — a cache, a validating FRR router, and a misbehaving peer — and then breaks it the three ways it breaks in production.

Learning goals

By the end of this chapter you can:

- Explain the chain ROA → validator → VRP → RTR → router, and what each piece is trusted for
- Apply the RFC 6811 algorithm by hand: covering VRPs, origin AS, maxLength
- Enable RPKI in FRR, connect it to an RTR cache, and read validation state per route
- Show that validation without policy changes **nothing** about forwarding
- Write an inbound route-map that drops Invalid and de-prefers NotFound
- Explain why a loose `maxLength` turns a sub-prefix hijack into a **Valid** route
- Predict what happens to routing when the cache is unreachable, and when VRPs change underneath accepted routes

Concepts — RPKI in one page

Who signs what. The Resource Public Key Infrastructure (RFC 6480) mirrors the address allocation hierarchy: the five RIRs hold trust anchors, they certify the address blocks they hand out, and a holder can publish a **Route Origin Authorization (ROA)** for its space. A ROA says: *prefix P, origin AS A, maximum length M.* It says nothing about the rest of the AS path.

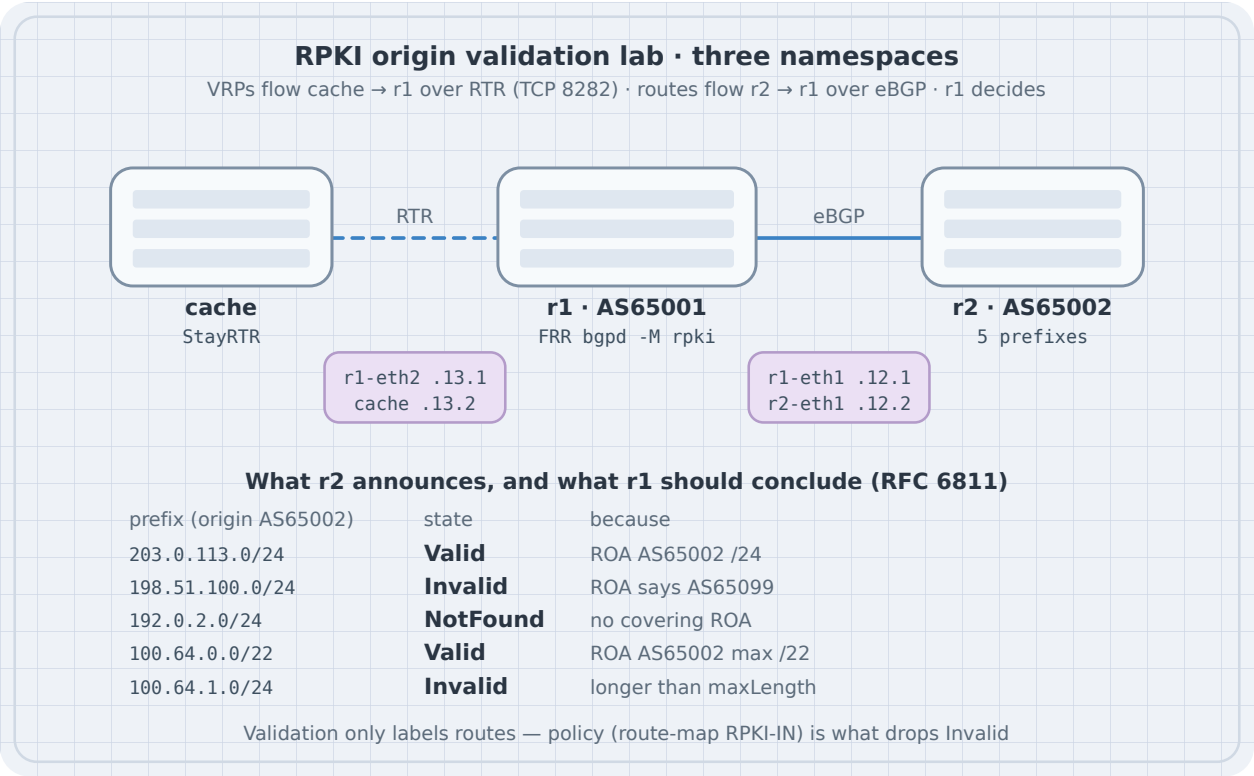


Figure 1: RPKI ROV lab

Who validates. A **relying-party validator** (Routinator, rpki-client, FORT, OctoRPKI…) fetches every repository, checks every signature and expiry, and emits a flat list of **Validated ROA Payloads (VRPs)**: `(prefix, maxLength, ASN)` tuples. Your routers never touch certificates.

How the router learns them. The validator (or a small cache in front of it, such as StayRTR) serves VRPs to routers over the **RPKI-to-Router protocol** (RTR, RFC 8210 for version 1), normally TCP port 8282 or 323. It is a serial-numbered, incremental feed: routers get a full table once, then diffs.

The algorithm (RFC 6811). For a route with prefix R and origin AS O (the last AS in the path):

1. Collect the **covering** VRPs: those whose prefix contains R.
2. None → **NotFound**. Nobody made a claim about this space.
3. Any covering VRP with ASN $= O$ **and** length(R) maxLength → **Valid**.
4. Covering VRPs exist but none match → **Invalid**. Either the wrong AS, or a longer prefix than the holder authorised.

What the router does with the label. Nothing, until you write policy. RFC 7115 describes operational practice; the common baseline today is *drop Invalid, accept NotFound (much of the table still has no ROAs), accept Valid.*

Term	Meaning
ROA	Signed object: prefix, maxLength, origin ASN
VRP	One validated `(prefix, maxLength, ASN)` tuple
RP / validator	Software that turns the RPKI into VRPs
RTR	Protocol that carries VRPs from cache to router
Covering VRP	A VRP whose prefix contains the route's prefix
ROV	Labelling routes by RFC 6811; policy acts on the label

The algorithm fits in a page of Python. Feeding it the lab's VRPs first makes the router output later look familiar:

```
#!/usr/bin/env python3
# rov.py - RFC 6811 route origin validation, nothing else
import ipaddress, json, sys

def load_vrps(path):
    with open(path) as f:
        roas = json.load(f)["roas"]
    return [(ipaddress.ip_network(r["prefix"]), r["maxLength"],
             int(str(r["asn"]).removeprefix("AS"))) for r in roas]

def validate(route, origin, vrps):
    route = ipaddress.ip_network(route)
    covering = [v for v in vrps
                if v[0].version == route.version and route.subnet_of(v[0])]
    if not covering:
        return "NotFound", []
    for pfx, maxlen, asn in covering:
        if asn == origin and route.prefixlen <= maxlen:
            return "Valid", [(str(pfx), maxlen, asn)]
    return "Invalid", [(str(p), m, a) for p, m, a in covering]

if __name__ == "__main__":
    vrps = load_vrps(sys.argv[1])
    for line in sys.stdin:
        route, origin = line.split()
        state, why = validate(route, int(origin), vrps)
        print(f"{route:<18} AS{origin:<6} {state:<9} {why}")
```

```
{
  "metadata": { "buildtime": "2026-10-08T00:00:00Z" },
  "roas": [
    { "prefix": "203.0.113.0/24", "maxLength": 24, "asn": "AS65002" },
    { "prefix": "198.51.100.0/24", "maxLength": 24, "asn": "AS65099" },
    { "prefix": "100.64.0.0/22",  "maxLength": 22, "asn": "AS65002" }
```

```
  ]
}
```

Save the JSON as `vrps.json` (StayRTR reads the same file later) and run:

```
printf '%s\n' "203.0.113.0/24 65002" "198.51.100.0/24 65002" "192.0.2.0/24 65002" \
  "100.64.0.0/22 65002" "100.64.1.0/24 65002" "100.64.1.0/24 65099" \
  | python3 rov.py vrps.json
```

```
203.0.113.0/24     AS65002  Valid     [('203.0.113.0/24', 24, 65002)]
198.51.100.0/24    AS65002  Invalid   [('198.51.100.0/24', 24, 65099)]
192.0.2.0/24       AS65002  NotFound  []
100.64.0.0/22      AS65002  Valid     [('100.64.0.0/22', 22, 65002)]
100.64.1.0/24      AS65002  Invalid   [('100.64.0.0/22', 22, 65002)]
100.64.1.0/24      AS65099  Invalid   [('100.64.0.0/22', 22, 65002)]
```

Read the last two lines carefully: the /24 inside the /22 is Invalid *whoever* originates it, because the holder only authorised the /22. That is the property that makes sub-prefix hijacks fail — and the property a loose `maxLength` gives away (Drill 3).

Addressing plan (RPKI lab)

Node	Interface	Address	Role
r1	r1-eth1	10.0.12.1/24	AS 65001, validating router
r1	r1-eth2	10.0.13.1/24	Management link to the cache
r2	r2-eth1	10.0.12.2/24	AS 65002, announces five prefixes
cache	cache-eth0	10.0.13.2/24	StayRTR, RTR on TCP 8282

Prefix announced by r2	VRP	Expected state
203.0.113.0/24	203.0.113.0/24-24 AS65002	Valid
198.51.100.0/24	198.51.100.0/24-24 AS65099	Invalid (wrong origin — the “hijack”)
192.0.2.0/24	none	NotFound
100.64.0.0/22	100.64.0.0/22-22 AS65002	Valid
100.64.1.0/24	(covered by the /22)	Invalid (too specific)

Lab build (Linux namespaces + FRR + StayRTR)

No container images and no Internet RPKI: three network namespaces, two veth pairs, one FRR instance per router namespace (FRR's pathspace option `-N`, exactly like the BFD chapter's lab), and **StayRTR** serving the `vrps.json` file above as an RTR cache. A real deployment points the same router configuration at a validator such as Routinator or rpki-client; only the cache address changes.

It was run on **Debian 13 with the distro packages FRR 10.3 (`frr`, `frr-rpki-rtrlib`) and StayRTR 0.6.2**, as root. Outputs below are copied from those runs; timestamps and PIDs will differ on yours.

```
sudo apt-get install -y frr frr-rpki-rtrlib stayrtr
sudo systemctl disable --now frr      # the lab starts its own instances
```

`frr-rpki-rtrlib` ships the bgpd module `bgpd_rpki.so`; bgpd only loads it when started with `-M rpki`. Without the package bgpd refuses to start with that flag.

```
#!/usr/bin/env bash
# lab.sh - r1 (validating FRR) peers with r2 (announcing FRR); "cache" runs StayRTR
# usage: sudo ./lab.sh up | down
set -euo pipefail
FRR=/usr/lib/frr
DAEMONS="mgmtd zebra bgpd staticd"
HERE=$(cd "$(dirname "$0")" && pwd)

up() {
  for ns in r1 r2 cache; do ip netns add "$ns"; ip -n "$ns" link set lo up; done

  # eBGP link r1 <-> r2
  ip link add r1-eth1 netns r1 type veth peer name r2-eth1 netns r2
  ip -n r1 addr add 10.0.12.1/24 dev r1-eth1
  ip -n r2 addr add 10.0.12.2/24 dev r2-eth1
  ip -n r1 link set r1-eth1 up; ip -n r2 link set r2-eth1 up

  # management link r1 <-> RPKI cache (RTR runs over plain TCP here)
  ip link add r1-eth2 netns r1 type veth peer name cache-eth0 netns cache
  ip -n r1 addr add 10.0.13.1/24 dev r1-eth2
  ip -n cache addr add 10.0.13.2/24 dev cache-eth0
  ip -n r1 link set r1-eth2 up; ip -n cache link set cache-eth0 up

  # the relying-party cache: StayRTR serving VRPs from a local JSON file
  ip netns exec cache stayrtr -bind 10.0.13.2:8282 -cache "$HERE/vrps.json" \
    -checktime=false -refresh 5 -metrics.addr "" >"$HERE/stayrtr.log" 2>&1 &
  echo $! > "$HERE/stayrtr.pid"

  for ns in r1 r2; do
```

```
    install -d -o frr -g frr "/var/run/frr/$ns" "/etc/frr/$ns"
    : > "/etc/frr/$ns/vtysh.conf"
    for d in $DAEMONS; do
      extra=""; [ "$d" = bgpd ] && [ "$ns" = r1 ] && extra="-M rpki"
      ip netns exec "$ns" "$FRR/$d" -d -N "$ns" -F traditional $extra \
        -u frr -g frr --limit-fds 1024 >/dev/null 2>&1
    done
    vtysh -N "$ns" -f "$HERE/$ns.conf" >/dev/null
  done
}

down() {
  for ns in r1 r2; do
    for d in $DAEMONS; do
      pidf="/var/run/frr/$ns/$d.pid"
      [ -f "$pidf" ] && kill "$(cat "$pidf")" 2>/dev/null || true
    done
  done
  [ -f "$HERE/stayrtr.pid" ] && kill "$(cat "$HERE/stayrtr.pid")" 2>/dev/null || true
  for ns in r1 r2 cache; do ip netns del "$ns" 2>/dev/null || true; done
}

"$@"
```

StayRTR flags used: `-cache` accepts a URL *or* a local file; `-checktime=false` stops it rejecting our hand-written file for having no fresh validator timestamp; `-refresh 5` re-reads the file every five seconds so the drills move quickly (the default is 600 s).

r2 announces all five prefixes; the blackhole statics only exist so that `network` statements have something in the RIB to match:

```
! r2.conf - AS 65002, announces four prefixes with four different RPKI outcomes
hostname r2
ip route 203.0.113.0/24 blackhole
ip route 198.51.100.0/24 blackhole
ip route 192.0.2.0/24 blackhole
ip route 100.64.0.0/22 blackhole
ip route 100.64.1.0/24 blackhole
router bgp 65002
 bgp router-id 10.0.12.2
 no bgp ebgp-requires-policy
 neighbor 10.0.12.1 remote-as 65001
 address-family ipv4 unicast
  network 203.0.113.0/24
  network 198.51.100.0/24
  network 192.0.2.0/24
  network 100.64.0.0/22
```

```
  network 100.64.1.0/24
 exit-address-family
```

r1 connects to the cache and validates — but has **no RPKI policy yet**:

```
! r1.conf - AS 65001, validates origins against the RPKI cache (no policy yet)
hostname r1
rpki
 rpki cache tcp 10.0.13.2 8282 preference 1
exit
router bgp 65001
 bgp router-id 10.0.12.1
 no bgp ebgp-requires-policy
 neighbor 10.0.12.2 remote-as 65002
```

`preference` orders several caches: the router uses the lowest-numbered group it can reach. Production routers should have at least two caches on different hosts.

Bring it up and check the RTR session:

```
chmod +x lab.sh
sudo ./lab.sh up
sleep 12
sudo vtysh -N r1 -c 'show rpki cache-connection'
sudo vtysh -N r1 -c 'show rpki prefix-table'
```

```
Connected to group 1
rpki tcp cache 10.0.13.2 8282 pref 1 (connected)
RPKI/RTR prefix table
Prefix                                  Prefix Length  Origin-AS
100.64.0.0                                 22 -  22   65002
203.0.113.0                                24 -  24   65002
198.51.100.0                               24 -  24   65099
Number of IPv4 Prefixes: 3
Number of IPv6 Prefixes: 0
```

`22 - 22` is "prefix length 22, maxLength 22". You can ask the cache directly what it serves, from r1's namespace, with StayRTR's companion tool:

```
sudo ip netns exec r1 rtrdump -connect 10.0.13.2:8282 -file /dev/stdout
```

```
{"metadata":{"vrps":3,"bgpsec_pubkeys":0,"sessionid":16344,"serial":0},"roas":[{"prefix":"19
```

Drill 1 — validation is a label, not a filter

Predict

r1 knows that 198.51.100.0/24 belongs to AS65099 and that nothing more specific than /22 should exist inside 100.64.0.0/22. Which of r2's five routes will r1 install?

Observe

```
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo ip netns exec r1 ip route show proto bgp
```

```
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
I*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0             0 65002 i
I*>  198.51.100.0/24  10.0.12.2                0             0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 5 routes and 5 total paths
100.64.0.0/22 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
100.64.1.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
192.0.2.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
198.51.100.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
203.0.113.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
```

The per-route view names the state explicitly:

```
sudo vtysh -N r1 -c 'show bgp ipv4 unicast 100.64.1.0/24'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast rpki invalid'
```

```
BGP routing table entry for 100.64.1.0/24, version 2
Paths: (1 available, best #1, table default)
  Advertised to non peer-group peers:
  10.0.12.2
  65002
    10.0.12.2 from 10.0.12.2 (10.0.12.2)
      Origin IGP, metric 0, valid, external, best (First path received), rpki validation-sta
...
     Network          Next Hop            Metric LocPrf Weight Path
I*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
I*>  198.51.100.0/24  10.0.12.2                0             0 65002 i
```

```
Displayed 2 routes and 5 total paths
```

Explain

All five are best (`*>`) and all five are in the kernel FIB, Invalids included. FRR's documentation says so plainly: enabling RPKI does not change best-path selection. Note the two meanings of "valid" in one line — `valid, external, best` is BGP's next-hop/attribute validity, `rpki validation-state: invalid` is origin validation. Do not grep for the wrong one.

Drill 2 — drop Invalid, de-prefer NotFound

```
! r1-policy.conf - drop Invalid, de-prefer NotFound, keep Valid
route-map RPKI-IN deny 10
 match rpki invalid
exit
route-map RPKI-IN permit 20
 match rpki notfound
 set local-preference 50
exit
route-map RPKI-IN permit 30
exit
router bgp 65001
 address-family ipv4 unicast
  neighbor 10.0.12.2 soft-reconfiguration inbound
  neighbor 10.0.12.2 route-map RPKI-IN in
 exit-address-family
```

`soft-reconfiguration inbound` keeps an unmodified Adj-RIB-In copy of what the peer sent, including routes your policy rejected. FRR's RPKI documentation requires it for cache updates to be applied to routes; Drill 4 shows why you want it anyway.

```
sudo vtysh -N r1 -f r1-policy.conf
sudo vtysh -N r1 -c 'clear bgp ipv4 unicast 10.0.12.2 soft in'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast neighbors 10.0.12.2 received-routes'
sudo ip netns exec r1 ip route show proto bgp
```

```
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 3 routes and 3 total paths
```

```
    Network          Next Hop            Metric LocPrf Weight Path
 *> 100.64.0.0/22    10.0.12.2                0             0 65002 i
 *> 100.64.1.0/24    10.0.12.2                0             0 65002 i
 *> 192.0.2.0/24     10.0.12.2                0             0 65002 i
 *> 198.51.100.0/24  10.0.12.2                0             0 65002 i
 *> 203.0.113.0/24   10.0.12.2                0             0 65002 i

Total number of prefixes 5 (2 filtered)
100.64.0.0/22 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
192.0.2.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
203.0.113.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
```

The hijacked 198.51.100.0/24 is gone. The too-specific 100.64.1.0/24 is gone, but traffic to 100.64.1.x still flows via the legitimate /22 — dropping an Invalid more-specific does not blackhole the space, it just stops the more-specific from winning longest-match. `received-routes` proves the peer still sends all five and the filter is doing the work.

Why `set local-preference 50` on NotFound rather than dropping it: a large share of the Internet table still has no ROAs. Dropping NotFound would cut you off from it. De-preferring means a Valid path wins whenever one exists.

Drill 3 — the maxLength trap

Predict

The holder of 100.64.0.0/22 decides to "save time" and publishes its ROA with `maxLength 24`, so it can announce /24s later without touching RPKI. It still announces only the /22. What happens to r2's 100.64.1.0/24?

Edit `vrps.json`, changing only the /22 line to `"maxLength": 24`. StayRTR notices within five seconds and bumps its serial:

```
time="2026-10-08T09:03:09+05:30" level=info msg="New update (3 uniques, 3 total prefixes, 0
time="2026-10-08T09:03:09+05:30" level=info msg="Update added, new serial 1"
```

```
sudo vtysh -N r1 -c 'show rpki prefix-table'
sudo vtysh -N r1 -c 'clear bgp ipv4 unicast 10.0.12.2 soft in'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
```

```
Prefix                                  Prefix Length  Origin-AS
100.64.0.0                                 22 -  24   65002
...
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
V*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
```

```
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i
```

Explain

The /24 is now **Valid**. In the lab r2 really is AS65002, but on the Internet the attacker does not need to be: a **forged-origin** hijack announces `100.64.1.0/24` with an AS path ending in `65002`. ROV checks only the last AS, so the forged route validates, and because it is more specific than the real /22 it attracts all the traffic for that quarter of the block. With `maxLength 22`, the same forgery is Invalid and dropped by every ROV network.

RFC 9319 (BCP 185) turns this into the rule: **do not use maxLength unless every prefix it authorises is actually announced**; publish one ROA per announced prefix instead ("minimal ROAs"). Put the setting back to 22 before the next drill.

Drill 4 — state changes under accepted routes

Predict

You tighten the ROA back to `maxLength 22`. The cache updates; r1's prefix-table updates. Is 100.64.1.0/24 withdrawn from r1's FIB?

Observe

About ten seconds after the edit, and again twenty seconds later:

```
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo ip netns exec r1 ip route show 100.64.1.0/24
```

```
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
I*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 4 routes and 4 total paths
100.64.1.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 proto bgp metric 20
```

The label flipped to `I` — and the route stayed best and installed. On this FRR 10.3 build, the new validation state was recorded but the inbound route-map was not re-run for an already-accepted route. Re-running inbound policy fixes it:

```
sudo vtysh -N r1 -c 'clear bgp ipv4 unicast 10.0.12.2 soft in'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo ip netns exec r1 ip route show 100.64.1.0/24
```

```
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 3 routes and 3 total paths
```

(no FIB line — the /24 is gone). The same happened in the other direction in Drill 3: the /24 rejected under `maxLength 22` did not reappear when the ROA was loosened until the soft clear.

Explain

FRR's RPKI guide says cache updates are applied and path selection updated, provided soft reconfiguration is enabled. Whether a particular release re-applies *policy* (not just the label) on every VRP change is exactly the kind of thing to **measure on your version** rather than assume. Operational answers that work everywhere: keep `soft-reconfiguration inbound` on RPKI-filtered sessions, alert on `show bgp ipv4 unicast rpki invalid` being non-empty, and schedule a periodic `clear bgp * soft in` (cheap with an Adj-RIB-In) until you have proved your release does it for you.

Drill 5 — the cache goes away (predict, then try)

Stop the cache and look at r1:

```
sudo kill "$(cat stayrtr.pid)"
sleep 8
sudo vtysh -N r1 -c 'show rpki cache-connection'
sudo vtysh -N r1 -c 'show rpki prefix-table' | tail -2
```

```
Connected to group 1
rpki tcp cache 10.0.13.2 8282 pref 1
Number of IPv4 Prefixes: 3
Number of IPv6 Prefixes: 0
```

The `(connected)` marker is gone but the VRPs remain: RTR data lives until the **expire interval** (StayRTR advertises 7200 s by default; FRR has `rpki expire_interval`). Routing continues unchanged. After expiry, every route becomes NotFound — Invalids included — and your "drop Invalid" policy silently stops dropping anything. ROV **fails open** by design: a dead validator must not take the network down, but it does take your protection away. Hence two caches, on different hosts, monitored. Newer FRR releases add `neighbor PEER rpki strict` to hold a session down until a cache is connected; FRR 10.3 rejects that command (`% Unknown command`), so check your version.

Checking real-world origins

The same three states exist on the live Internet, and RIPEstat exposes a validator over HTTPS. A real ROA, then a deliberately wrong origin:

```
curl -s 'https://stat.ripe.net/data/rpki-validation/data.json?resource=AS13335&prefix=1.1.1.
  | python3 -c "import json,sys;d=json.load(sys.stdin)['data'];print(json.dumps({k:d[k] for
curl -s 'https://stat.ripe.net/data/rpki-validation/data.json?resource=AS64500&prefix=1.1.1.
  | python3 -c "import json,sys;d=json.load(sys.stdin)['data'];print(d['resource'],d['prefix
```

```
{
 "resource": "13335",
 "prefix": "1.1.1.0/24",
 "status": "valid",
 "validating_roas": [
  {
   "origin": "13335",
   "prefix": "1.1.1.0/24",
   "validity": "valid",
   "max_length": 24
  }
 ]
}
64500 1.1.1.0/24 invalid_asn
```

Use it to check your own prefixes before you publish a ROA, and after: the holder can make its own routes Invalid with a mistake, and every ROV network will then drop them.

Deploying it for real

Step	What	Why
1	Publish ROAs for your own space (RIR portal, hosted RPKI) — one per announced prefix, `maxLength` = prefix length	Others can protect you; avoids Drill 3
2	Run two validators/caches (e.g., Routinator and rpki-client) on separate hosts	Fail-open is fine only if it is rare (Drill 5)
3	Point every border router at both (`rpki cache … preference 1/2`)	Cache failover
4	Start with **monitor-only**: label, do not filter; review `rpki invalid`	Find your customers' broken ROAs first

Step	What	Why
5	Enable `deny invalid` on transit and peer sessions, then on customers	The RFC 7115 path
6	Re-run inbound policy on VRP changes, or prove your release does (Drill 4)	No stale accepts
7	Alert on cache disconnect, VRP count drops, and serial not advancing	Validator outages are silent

ROV protects against wrong *origins*. It does not stop route **leaks** (valid origin, wrong path) or forged-origin hijacks of space with loose ROAs; path validation (ASPA, BGPsec) is the next layer and is outside this chapter.

Verification checklist

Check	Command	Intent
Cache connected	`show rpki cache-connection`	`(connected)` on at least one group
VRPs loaded	`show rpki prefix-table`	Non-zero, plausible count
State per route	`show bgp ipv4 unicast <prefix>`	`rpki validation-state:` line
Invalids accepted	`show bgp ipv4 unicast rpki invalid`	Should be empty with policy on
Policy attached	`show running-config` → `route-map RPKI-IN in`	Labels mean nothing without it
Filter doing work	`… neighbors <ip> received-routes`	`(N filtered)` matches expectations
Cache content	`rtrdump -connect <cache>:8282`	What routers actually receive

Common mistakes

Mistake	Symptom
RPKI enabled, no route-map	Invalids best and installed (Drill 1)
Dropping NotFound	Large parts of the Internet unreachable
`maxLength` wider than what you announce	Forged-origin sub-prefix hijack validates (Drill 3)
No soft-reconfiguration / no re-evaluation	Stale Invalids stay installed after VRP changes (Drill 4)
One cache	Validator outage → silent fail-open (Drill 5)

Mistake	Symptom
ROA for the aggregate only, then announcing a /24 for traffic engineering	Your own /24 is Invalid and dropped by others
Grepping for "valid" in `show bgp`	Confuses BGP path validity with RPKI state

Summary

- A ROA authorises **one origin AS for a prefix up to a maxLength**; validators flatten the RPKI into VRPs; routers receive VRPs over RTR
- RFC 6811: no covering VRP → NotFound; matching origin within maxLength → Valid; otherwise → Invalid
- Validation labels routes; **policy** drops them — without a route-map all five lab routes were installed
- Loose `maxLength` turned the 100.64.1.0/24 sub-prefix from Invalid into **Valid** (RFC 9319: don't)
- On FRR 10.3, VRP changes updated labels but did not re-run inbound policy until `clear … soft in`
- With the cache gone, VRPs persist until expiry, then everything becomes NotFound: ROV fails open

Sources

- RFC 6480 — An Infrastructure to Support Secure Internet Routing: https://www.rfc-editor.org/rfc/rfc6480
- RFC 6811 — BGP Prefix Origin Validation: https://www.rfc-editor.org/rfc/rfc6811
- RFC 7115 — Origin Validation Operation Based on the RPKI: https://www.rfc-editor.org/rfc/rfc7115
- RFC 8210 — The RPKI to Router Protocol, Version 1: https://www.rfc-editor.org/rfc/rfc8210
- RFC 9319 (BCP 185) — The Use of maxLength in the RPKI: https://www.rfc-editor.org/rfc/rfc9319
- RFC 9582 — A Profile for Route Origin Authorizations (ROAs): https://www.rfc-editor.org/rfc/rfc9582

- FRRouting user guide — RPKI: https://docs.frrouting.org/en/latest/bgp.html#prefix-origin-validation-using-rpki (source: https://github.com/FRRouting/frr/blob/master/doc/user/rpki.rst)

- StayRTR: https://github.com/bgp/stayrtr

- RIPEstat RPKI Validation data call: https://stat.ripe.net/docs/data-api/api-endpoints/rpki-validation

Segment Routing, SRv6, and TI-LFA

Segment Routing, SRv6, and TI-LFA

IGP shortest paths are easy to explain and slow to heal. Classic Loop-Free Alternates (LFA) and Remote LFA cover many failures but not all topologies. **Segment Routing (SR)** (RFC 8402) lets the ingress (or a repair point) encode an explicit path as a list of **segments**. On IPv6, that list lives in the **SR Header** and is programmed with **SRv6 network programming** (RFC 8986). **TI-LFA** (RFC 9855) uses those segments so a **Point of Local Repair (PLR)** can pre-install a loop-free backup that follows the **expected post-convergence** path — topology-independent coverage in two-connected IGP areas, without Targeted LDP tunnels. This chapter stays vendor-neutral and RFC-first.

Learning goals

By the end of this chapter you can:

- State the SR architecture: Prefix-SID / Adjacency-SID, SR-MPLS vs SRv6
- Explain SRv6 SID lists, active segment, and head-end behaviors at a PLR
- Define TI-LFA, P-space / Q-space, and Repair List (RL)
- Predict what the PLR pushes when the active segment is a node SID vs an adjacency SID
- Relate TI-LFA to BFD-scale detection: FRR is local and precomputed; IGP still converges

Thesis

TI-LFA is not a new IGP. It is a **local repair algorithm** that *expresses* post-failure shortest paths as SR segment lists the PLR can push the moment a link (or protected node/SRLG) dies — then withdraws that repair when the IGP SPF at the PLR catches up.

Mental model

```
Before failure (IGP SPF):     S  → F  → …  → D
                                \                ↑
After local repair (TI-LFA):      RL → P   Q
```

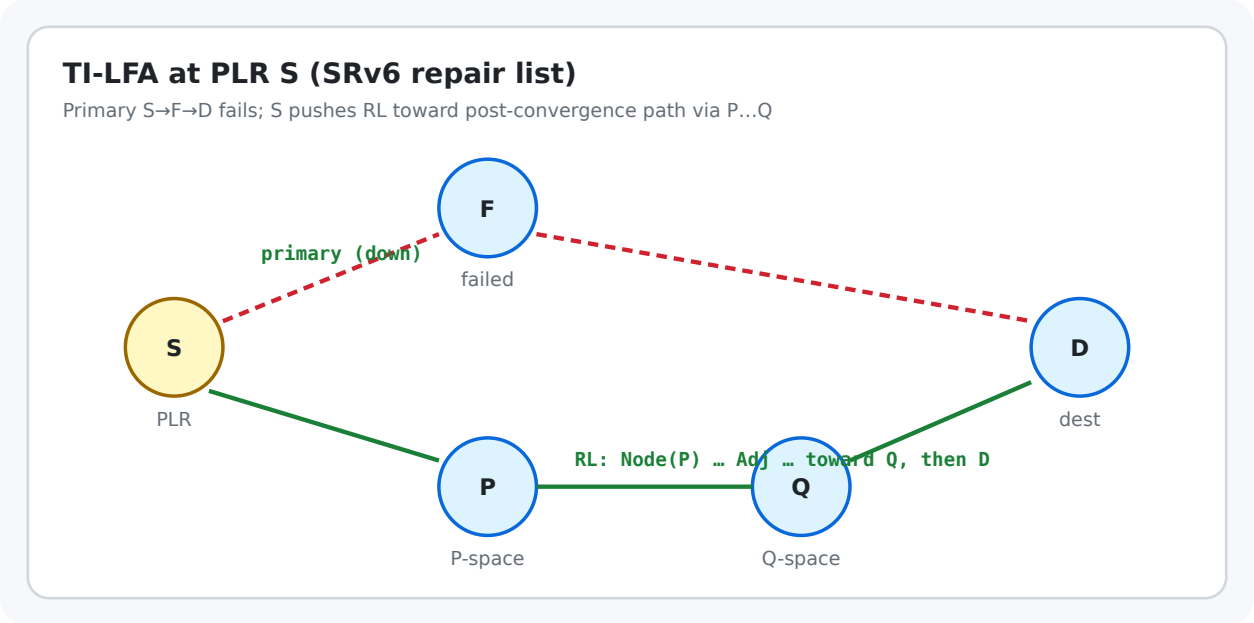


Figure 1: TI-LFA repair at the PLR

Term (RFC 9855 / 8402)	Meaning
PLR	Router that detects local failure and activates a precomputed backup
RL (Repair List)	Segment list encoding a loop-free path avoiding the protected resource
P-space	Nodes reachable from the PLR *without* traversing the failed resource
Q-space	Nodes that can reach the destination *without* traversing the failed resource
Post-convergence path	SPF path the PLR would use after the IGP absorbs the failure

SR data planes

Plane	How a segment is carried	Primary RFCs
SR-MPLS	MPLS label stack	RFC 8402, RFC 8660
SRv6	IPv6 destination + optional SRH (RFC 8754); behaviors in RFC 8986	RFC 8402, 8754, 8986

SRv6 **Adjacency SIDs** are often globally routed, so RLs can be shorter than SR-MPLS (less need to precede an Adj-SID with a Prefix-SID) — RFC 9855 §7.2.

Worked examples

Case 1: Architecture in one page (RFC 8402)

A **segment** is an instruction. Common kinds:

- **Prefix-SID (node segment)** — "go to this node via the IGP algorithm (usually SPF)"
- **Adjacency-SID** — "take this specific link / adjacency"

An **SR Policy** is an ordered list of segments. On SRv6, the **active** segment is the IPv6 Destination Address; `Segments Left` in the SRH walks the list (RFC 8402 §3.1.3 summary).

Operators instantiate SRv6 SIDs explicitly — ordinary interface addresses are **not** SIDs by default.

Case 2: TI-LFA repair flavours (RFC 9855 §5)

Situation along post-convergence path	RL shape
Direct neighbor in P Q	Empty RL; just change next hop (post-convergence LFA)
Remote PQ node	Single node segment to that PQ
Adjacent P then Q	Node-SID(P) then Adj-SID(P→Q)
Distant P and Q	Node-SID(P) + explicit path EP(P,Q) (Adj-SID list or optimized mix)

Protection modes: **link**, **node**, or **SRLG** failure assumptions when computing the backup.

Case 3: What the PLR does to the packet (RFC 9855 §6–7)

Active segment is a node SID. Keep it; **add** the RL so the node SID becomes the first segment *after* the repair list.

Active segment is an adjacency SID S–F (protected). Apply NEXT on that Adj-SID, then PUSH an RL that restores traffic toward F (or toward a Q whose path to the following node SID avoids the failure) — details in §6.2.1–6.2.2.

SRv6 transit PLR. Add the SID list using RFC 8986 **head-end** behaviors (e.g. `H.Encaps` / SRH push as implemented). **Endpoint PLR:** run endpoint processing first, then protect as transit.

USD flavor (RFC 8986) is often cited so the last repair endpoint can **decapsulate** the outer header the PLR added and forward the inner packet — useful for TI-LFA encapsulation on P routers.

Case 4: Numbers from RFC 9855 Appendix B (SR-MPLS measurements)

On real SP/enterprise topologies measured in the RFC:

- Link / local SRLG protection: **1 SID** repair path covers **>99%** of prefixes in the data set
- Node protection: **2 SIDs** yields about **99%** coverage

Exact hardware MSD limits still matter; implementations may optimize RL length.

The trap

- Expecting TI-LFA to protect **inter-domain** exits or non-IGP destinations — RFC 9855 limits scope to link-state IGP areas.
- Forgetting **micro-loops** during distributed convergence: TI-LFA is complementary to loop-free convergence methods (RFC 5715, RFC 8333, etc.), not a full substitute.
- Building RLs from **protected** Adj-SIDs and then suffering **double FRR** if a repair hop fails (RFC 9855 §9); some implementations prefer unprotected Adj-SIDs in RLs.
- Assuming SR Policies with custom constraints stay on-policy during local repair — TI-LFA does not know head-end TE constraints; use policy-level protection or Flexible Algorithm-aware TI-LFA (RFC 9855 §8) when constraints matter.
- Confusing **detection** (BFD, carrier) with **repair encoding** (TI-LFA): you need both for fast restoration.

The boring rule

Run a link-state IGP with SR enabled; precompute TI-LFA RLs toward post-convergence paths; detect failures quickly; keep RL depth inside MSD; let SPF release the repair after convergence.

Try this

1. Read RFC 8402 §§1–3 (architecture) and RFC 8986 §5 (head-end behaviors) skim.
2. Walk RFC 9855 §5 and Figure 1: compute P-space / Q-space intersection with the post-convergence path on paper.

3. On a lab with FRR or another open IGP+SR stack (when you already have it), enable SR and TI-LFA and compare `show` repair SIDs before/after shutting a link — no new packages required for reading RFCs alone.
4. Related in this book: BFD for *detection* timers; this chapter for *where packets go* during the repair window.

Sources

- RFC 8402 — Segment Routing Architecture. https://www.rfc-editor.org/rfc/rfc8402
- RFC 8986 — SRv6 Network Programming. https://www.rfc-editor.org/rfc/rfc8986
- RFC 8754 — IPv6 Segment Routing Header (SRH). https://www.rfc-editor.org/rfc/rfc8754
- RFC 9855 — Topology Independent Fast Reroute Using Segment Routing (TI-LFA). https://www.rfc-editor.org/rfc/rfc9855
- RFC 8660 — SR-MPLS (for the MPLS twin of the same RL algorithms)

Ops and Automation

Observability for Networks

Observability for Networks

If you cannot see it, you cannot operate it. This chapter turns lab verification habits into a minimal **observability stack**: interface signals, routing state, captures, logs, and a path toward metrics/telemetry—without requiring paid NMS.

Learning goals

By the end of this chapter you can:

- Build a layered observability checklist for any lab or small network
- Use Linux and FRR show/counters for incident triage
- Capture and filter packets productively
- Sketch metrics that matter (link, session, error)
- Write a short runbook entry from an observed failure

Observability layers

Layer	Questions	Tools
L1/L2 link	Up? errors? flaps?	`ip -s link`, ethtool, MAC/FDB
L3 reachability	Path? loss?	ping, mtr, traceroute
Control plane	Neighbors/sessions?	vtysh shows, logs
Data plane path	Which hop?	route get, capture
Services	App timeouts?	ss, curl, synthetic checks
Continuity	When did it start?	logs, metrics, journals

Baseline: what "healthy" looks like

Before incidents, save baselines:

```
mkdir -p baseline
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor' > baseline/r1-ospf-nbr.txt
```

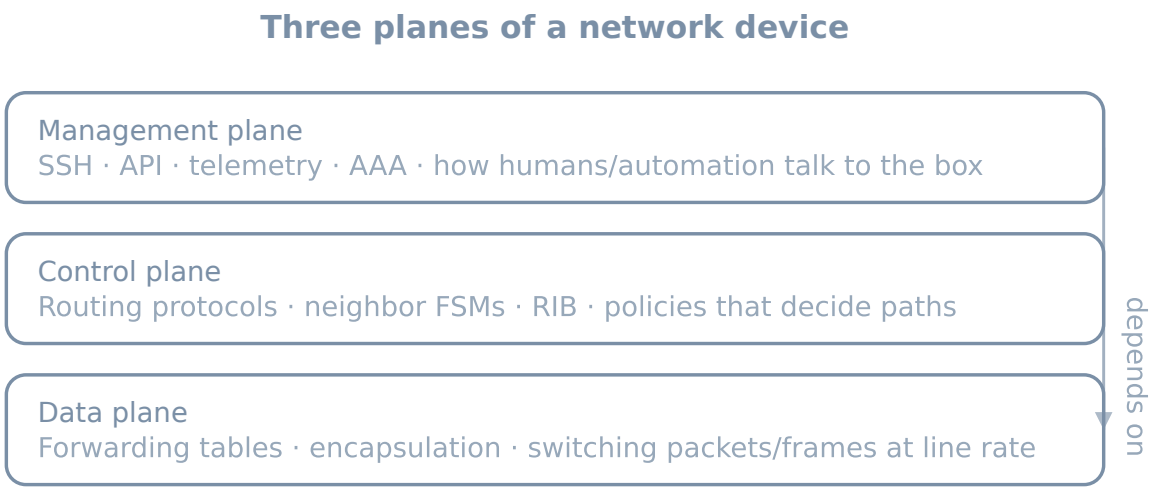


Figure 1: Planes

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route' > baseline/r1-route.txt
docker exec clab-ospf-tri-r1 ip -s link > baseline/r1-link.txt
```

Diff during incidents:

```
diff -u baseline/r1-ospf-nbr.txt <(docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neigh
```

Interface and error signals

```
ip -s -s link show eth1
# Look at RX/TX errors, drops, carrier changes
```

Drill: bounce a link; confirm counters and oper state track reality.

```
ip monitor link
```

Control-plane signals

OSPF

```
vtysh -c 'show ip ospf neighbor'
vtysh -c 'show ip ospf interface'
vtysh -c 'show log'   # if logging configured
```

BGP

```
vtysh -c 'show bgp summary'
vtysh -c 'show bgp neighbors 10.0.0.2'
vtysh -c 'show bgp neighbors 10.0.0.2 routes'
```

Watch for: session flaps, prefix count drops to zero, unexpected AS paths.

Path quality

```
ping -c 50 -i 0.2 192.168.2.10
mtr -rwzc 100 192.168.2.10
traceroute -n 192.168.2.10
```

Interpret:

Pattern	Suspect
Loss from first hop	Local segment
Loss starting mid-path	Transit / congestion / ACL
High latency one hop jump	That hop or link serialization
Jittery wireless-like	Lab CPU contention sometimes!

On shared lab hosts, **CPU starvation** fakes network loss—check `docker stats`.

Packet capture practice

```
tcpdump -ni eth1 -w /tmp/inc.pcap not port 22
# reproduce
tcpdump -r /tmp/inc.pcap -nn
tshark -r /tmp/inc.pcap -Y 'icmp || bgp || arp' -V | less
```

Capture points

Question	Where to capture
Did host send?	Host interface
Did router rewrite L2?	Router egress
Did peer receive?	Peer ingress
Policy drop?	Both sides + counters

Logging

FRR

Enable meaningful logging to file or syslog inside the node (image-dependent):

```
log file /var/log/frr/frr.log
log timestamp precision 3
```

```
docker exec clab-ospf-tri-r1 tail -f /var/log/frr/frr.log
```

Kernel

```
dmesg -T | tail
# martians, neighbor table full, etc.
```

Metrics sketch (Prometheus-shaped thinking)

Even if you do not run Prometheus yet, define signals:

Metric idea	Type	Labels
`link_up`	gauge	node, iface
`interface_errors_total`	counter	node, iface, dir
`bgp_session_up`	gauge	node, peer
`bgp_prefixes_received`	gauge	node, peer
`ospf_neighbors_full`	gauge	node
`ping_loss_ratio`	gauge	src, dst

Synthetic ping exporter + node exporter covers a surprising amount for home/lab NetOps.

Minimal blackbox idea

```
#!/usr/bin/env bash
# probe.sh - exit 1 on failure for CI/cron
ping -c 3 -W 1 192.168.2.10 >/dev/null
```

Cron or systemd timer + alert on fail is valid ops.

Telemetry awareness (2026)

Modern NOS (e.g. SR Linux community) emphasize streaming telemetry (gNMI). FRR/Linux labs may stick to show + exporters. Skill transfer:

- **Subscribe** to interface + protocol state
- Prefer push metrics over SSH scraping when scale grows
- Still keep packet capture for truth

Incident timeline template

```
## Incident
Start (UTC):
Detect signal:
Impact:
## Timeline
- T0:
- T+2m:
## Hypotheses
1.
2.
## Checks performed
## Root cause
## Fix
## Follow-ups
- monitor:
- prevent:
```

Lab drill: observe a BGP flap

1. Baseline `show bgp summary`
2. `neighbor shutdown` or link down
3. Record timestamps of state change and route withdrawal
4. Restore; measure time to Established and ping recovery
5. Write incident note with detection idea (“alert if bgp_session_up==0 for 60s”)

Lab drill: silent MTU blackhole

From the MTU chapter: large ping fails, small works.

Observability angle: which metric would catch this? (not link_down). Need synthetic large-packet probe or TCP service check.

Runbook snippet library

Keep short runbooks next to labs:

```
## Runbook: OSPF neighbors down
1. Check link state ip link
2. Check addressing same subnet
3. show ip ospf interface MTU
4. capture hello packets
5. check area / auth / passive
```

Privacy and safety

- Captures may contain payloads—store carefully
- Do not tcpdump production spans without authorization
- Redact journals before sharing

Summary

- Observability is layered: link → path → control → packet → service
- Baselines make incidents measurable
- Counters + mtr + captures beat vibes
- Define metrics before you buy platforms
- Runbooks convert personal heroics into repeatable ops

Next: **automation / lab-as-code**—pushing configs, smoke deploys, and CI-shaped checks.

Automation & Lab-as-Code

Automation & Lab-as-Code

A lab you cannot rebuild is a rumor. This chapter automates the Containerlab lifecycle, config push patterns, and smoke verification so your networking skill compounds in git.

Learning goals

By the end of this chapter you can:

- Wrap deploy/destroy/verify in repeatable scripts
- Organize inventories for multi-node labs
- Push or render configs with light Ansible or shell
- Sketch CI that smoke-tests a topology
- Treat secrets and image pins correctly

Principles

1. **Topology and config in git**
2. **Idempotent as practical** — re-run without fear
3. **Verify automatically** — exit codes matter
4. **No click-ops source of truth**
5. **Small steps** — shell first, Ansible when it pays off

Directory layout (automation-ready)

```
labs/ospf-tri/
  topology.clab.yml
  addressing.md
  README.md
```

```
  Makefile
  verify.sh
  up.sh
  down.sh
  config/
    r1/frr.conf
    r1/daemons
    r2/...
  ansible/
    inventory.yml
    playbooks/deploy-config.yml
```

Makefile pattern

```
TOPO := topology.clab.yml

.PHONY: up down verify smoke

up:
    sudo containerlab deploy -t $(TOPO)

down:
    sudo containerlab destroy -t $(TOPO) --cleanup

verify:
    ./verify.sh

smoke: up verify
    @echo smoke ok
```

```
make smoke
make down
```

up.sh / down.sh

```
#!/usr/bin/env bash
# up.sh
set -euo pipefail
cd "$(dirname "$0")"
sudo containerlab deploy -t topology.clab.yml
./verify.sh
```

```
#!/usr/bin/env bash
# down.sh
set -euo pipefail
cd "$(dirname "$0")"
sudo containerlab destroy -t topology.clab.yml --cleanup
```

Config strategies

Strategy	Pros	Cons
Bind-mount `/etc/frr`	Simple, git-native	Image load quirks
`containerlab` startup-config	Kind-supported	Kind-specific
Ansible after deploy	Flexible templating	Extra tool
Render templates → bind	Pure git artifacts	Build step

Prefer bind + pure files for FRR until pain appears.

Safe interactive explore → promote

```
docker exec -it clab-ospf-tri-r1 vtysh
# make change, show run
# copy running config back into config/r1/frr.conf
# redeploy clean to prove
```

Light Ansible (optional)

`ansible/inventory.yml`:

```
all:
  children:
    routers:
      hosts:
        r1:
          ansible_host: clab-ospf-tri-r1
          ansible_connection: docker
        r2:
          ansible_host: clab-ospf-tri-r2
          ansible_connection: docker
        r3:
          ansible_host: clab-ospf-tri-r3
          ansible_connection: docker
```

Playbook sketch:

```
- name: Push FRR config files
  hosts: routers
  gather_facts: false
  tasks:
    - name: Copy frr.conf
      copy:
        src: "{{ playbook_dir }}/../../config/{{ inventory_hostname }}/frr.conf"
        dest: /etc/frr/frr.conf
    - name: Reload FRR
      shell: vtysh -c 'write' || true
      # better: proper frr reload unit - image dependent
```

```
ansible-playbook -i ansible/inventory.yml ansible/playbooks/deploy-config.yml
```

Python with scrapli/netmiko is equally valid; pick one tool family and go deep later.

Templating addressing: DIY vs. Netlab

The DIY Approach (Jinja2 + custom scripts)

```
# addressing.yml
links:
  r1_r2: { r1: 10.0.12.1/24, r2: 10.0.12.2/24 }
lans:
  h1: { gw: 192.168.1.1/24, host: 192.168.1.10/24 }
```

You can render `frr.conf.j2` → `config/r1/frr.conf` with a custom Python/Jinja script or Ansible template module. **Generated files can be committed** for diff visibility or generated in CI.

The Netlab Declarative Approach

Writing custom Python Jinja compilers and manual IPAM spreadsheets becomes difficult to maintain when topologies exceed 3–4 nodes. Netlab provides a turnkey, declarative engine designed specifically for network labs:

```
# topology.yml
---
provider: clab
defaults.device: frr
module: [ ospf, bfd ]

nodes:
```

```
  r1:
  r2:
  r3:

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:

validate:
  ospf_full:
    description: "Verify all OSPF adjacencies reach Full"
    nodes: [ r1, r2, r3 ]
    plugin: ospf_neighbor(count=2, state='Full')
```

Running `netlab up` calculates IPAM, renders Jinja2 templates for each device, stands up the Containerlab environment, and configures the routers. Running `netlab validate` executes the validation suite automatically.

Verification: verify.sh vs. netlab validate

Custom Shell Script (`verify.sh`)

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

need_full() {
  local node=$1
  docker exec "$node" vtysh -c 'show ip ospf neighbor' | grep -q Full \
    || fail "$node missing Full neighbor"
}

need_full clab-ospf-tri-r1
need_full clab-ospf-tri-r2
need_full clab-ospf-tri-r3

docker exec clab-ospf-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-ospf-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"

echo "OK $(date -Is)"
```

Netlab Automated Validation

With Netlab, assertions are declared directly in YAML and executed with exit codes suitable for CI:

```
# Returns exit code 0 on pass, non-zero on failure
netlab validate
```

CI smoke (GitHub Actions sketch)

Conceptual job (runner must support Docker + Containerlab—often self-hosted):

```
# conceptual - adapt to your runners
jobs:
  lab-smoke:
    runs-on: self-hosted-linux
    steps:
      - uses: actions/checkout@v4
      - name: deploy+verify
        working-directory: books/Networking/labs/ospf-tri
        run: |
          sudo containerlab deploy -t topology.clab.yml
          ./verify.sh
      - name: cleanup
        if: always()
        working-directory: books/Networking/labs/ospf-tri
        run: sudo containerlab destroy -t topology.clab.yml --cleanup
```

This book's monorepo CI renders Quarto; **lab smoke is optional local/self-hosted**. Do not assume GitHub-hosted runners allow nested Containerlab without setup.

Image pins in automation

```
# .env or Makefile
FRR_IMAGE=quay.io/frrouting/frr:10.2.1
ALPINE_IMAGE=alpine:3.20
```

Topology can be generated or documented to match. Record digests for serious CI.

Idempotency drills

1. `make smoke` twice without `down` — should not corrupt

2. `make down && make smoke` — clean path
3. Break config, `make smoke` fails exit 0
4. Fix config, smoke passes

Secret hygiene

- No production passwords in lab repos
- Lab default creds documented as lab-only
- Use `.gitignore` for private env files

Graph and inventory export

```
sudo containerlab inspect -t topology.clab.yml --format json > inspect.json
sudo containerlab graph -t topology.clab.yml || true
```

Feed inspect outputs into your own tools later.

Predict → observe → fix

Predict: A peer clones the lab folder, runs `make smoke`, gets OK without chatting you.

Observe: Remove tribal knowledge; only README + scripts.

Fix: Missing apk packages, wrong container names, unpinned images.

Harden: Add `make smoke` badge or CI on self-hosted when ready.

Anti-patterns

Anti-pattern	Replace with
Only screenshot proof	verify.sh
Snowflake node edits	bind + git
`latest` tags	pins
Huge mono playbook day one	Makefile smoke
CI without cleanup	always destroy

Summary

- Lab-as-code = topology + config + verify in git
- Start with Makefile/scripts; add Ansible when templating hurts
- Exit codes turn labs into testable systems
- CI is optional but shaping; cleanup always
- Promote working state from nodes back into the repo

Next: **capstone outline**—projects that prove the whole journey.

Capstone Outline

Capstone Outline

This chapter is a **map of finishing projects**, not a single mandatory mega-lab. Pick at least one capstone and run it to the book's definition of done: design, code, break, prove, operate.

Learning goals

By the end of this chapter you can:

- Choose a capstone that matches your goals
- Scope addressing, topology, and success metrics
- Apply models + verification + automation end-to-end
- Produce a portfolio-quality lab folder
- Identify natural next topics beyond this spine

Definition of done (all capstones)

☐ Written design (1–2 pages): goals, non-goals, addressing, failure modes

☐ `topology.clab.yml` + configs in git

☐ `addressing.md` + diagram

☐ `verify.sh` green on clean deploy

☐ 3 failure drills with incident notes

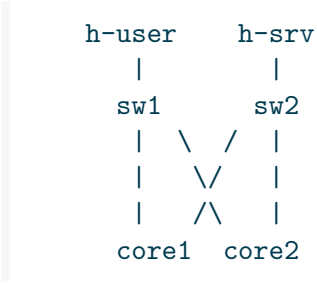
☐ One automation wrapper (`make smoke` or equivalent)

☐ Short retrospective: what model changed in your head

Capstone C1 — Campus-style dual core (open images)

Intent

Two access switches (or Linux bridges), dual core routers, dual-homed access, VLANs for users/servers, OSPF underlay between cores, static or OSPF toward access.



Skills exercised

- VLANs/trunks
- L2/L3 boundary
- OSPF or static redundancy
- Failure: core link down, trunk VLAN missing

Success metrics

- Hosts in VLAN 10 communicate across access
- Inter-VLAN via cores
- One core or one uplink loss recovers within your documented budget

Suggested stack

FRR cores + Alpine hosts + Linux bridge “switches” or SR Linux if you want NOS practice.

Capstone C2 — Dual-homed site to provider edge

Intent

Customer edge (CE) with two eBGP uplinks to two "ISP" PE routers; prefer primary; failover to backup; filter what you advertise/accept.

```
        PE1 (AS65010)
       /
  CE (AS65001)
       \
        PE2 (AS65020)
```

Skills exercised

- eBGP sessions
- Prefix filters / route-maps
- Local-pref / prepend
- Session loss drill

Success metrics

- Only owned prefixes exported
- ISP rejects hijack test prefix
- Traffic shifts on primary PE failure

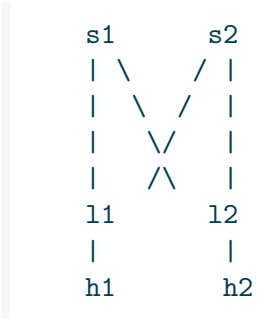
verify extras

Negative test: unauthorized prefix must not appear on PE.

Capstone C3 — Mini fabric snack

Intent

2 spine + 2 leaf (or 2×2) IP fabric with OSPF or eBGP underlay; endpoints on leaves; break a leaf uplink.



Skills exercised

- Underlay design
- ECMP behavior
- Scale of configs (templating pays off)
- Observability under failure

Stretch (optional)

Add a simple tunnel/overlay between leaves **after** underlay is solid—never before.

Capstone C4 — Incident week

Intent

Take any working lab (C1–C3 or static triangle). Each day for five days inject a **hidden** fault (or swap with a peer) and practice detect → diagnose → fix → prevent.

Day	Example inject
1	Wrong VLAN on one access port
2	Missing return route
3	MTU clamp + ICMP filter
4	OSPF cost surprise / link flap
5	BGP filter too strict

Skills exercised

- Observability under time pressure
- Incident notes
- Avoiding random change storms

Success metrics

- Each day: root cause written with evidence
- verify.sh restored green
- One new automated check added by end of week

How to pick

Goal	Pick
Campus / enterprise ops	C1
Edge / multihoming	C2
DC-ish fabric thinking	C3
Diagnostic mastery	C4 (optionally + another)

Time box: **1–2 weeks evenings** per capstone at 5–8h/week pace.

Project template (copy)

```
capstones/c2-dual-home/
  DESIGN.md
  README.md
  topology.clab.yml
  addressing.md
  verify.sh
  Makefile
  config/
  journals/
    drill-1.md
    drill-2.md
    drill-3.md
```

```
  diagrams/
  RETRO.md
```

DESIGN.md skeleton

```
# Design: Dual-homed CE

## Goals
## Non-goals
## Topology
## Addressing
## Control plane
## Policy
## Failure modes
## Observability
## Test plan
```

RETRO.md prompts

- Which plane failed most often in your drills?
- What did you overcomplicate?
- Which verify checks caught real bugs?
- What would you template next?

Integration with the book spine

Book part	Capstone use
Models	Language in DESIGN.md
Lab craft	Containerlab hygiene
L2	C1 trunks/VLANs
L3	All (addressing, static edges)
Interior routing	C1 OSPF, C2 BGP, C3 underlay
Ops automation	Makefile, verify, journals

Beyond this book (next horizons)

When capstones feel comfortable:

- Multi-area OSPF / IS-IS
- EVPN/VXLAN on free images where feasible
- QoS classification labs
- Stronger automation (CI self-hosted, config generate)
- Streaming telemetry on SR Linux
- Traffic engineering and more advanced BGP

Stay open-tools-first unless your job provides licensed platforms.

Final checkpoint: "expert" as defined in the syllabus

You can:

- Design multi-site L2/L3 with clear underlay roles
- Implement in Containerlab with free images
- Diagnose control vs data failures with tables + captures
- Apply routing policy without creating loops
- Automate lab lifecycle and basic validation
- Operate with observability and recovery habits

If any bullet is weak, return to that part's drills—not to random new features.

Summary

- Capstones prove integration, not trivia
- C1 campus, C2 dual-home BGP, C3 fabric snack, C4 incident week
- Same definition of done for all: design, code, verify, break, document
- Automation and negative tests separate portfolio work from toy demos
- The journey map continues; the habits stay

Return to the syllabus for the full long-range map (overlays, QoS, multi-area depth). Your next commit should be a capstone folder, not another unread PDF.

Edge and Services

First-Hop Redundancy

First-Hop Redundancy

Hosts rarely run a routing protocol. They send traffic to a **default gateway**. If that gateway dies, the LAN dies with it—unless you give the LAN a **virtual first hop** that can move between routers. This chapter builds the model and a free-stack lab: Linux + FRR + Containerlab (mid-2026 open path).

Learning goals

By the end of this chapter you can:

- Explain why a single gateway IP is a single point of failure
- Contrast VRRP-class first-hop redundancy with floating statics and ECMP
- Deploy a dual-router VRRP pair on FRR for a shared virtual IP (VIP)
- Predict host behavior when the master fails and when it recovers
- Verify with `ip neigh`, pings, and FRR VRRP state—not hope

Concepts

The problem

```
h1 ---- [LAN] ---- gw 10.0.0.1  (one box)
           |
        Internet-ish
```

Host config: `default via 10.0.0.1`. When that box reboots, ARP still points at a MAC that is gone until the host relearns—or traffic blackholes forever if nothing answers.

First-hop redundancy idea

Two (or more) routers share:

Object	Role
Virtual IP (VIP)	Address hosts use as gateway

Object	Role
Virtual MAC (often)	Stable L2 identity for the VIP
Master / backup roles	Who owns the VIP right now
Hello / advertise	"I am alive and I claim priority"

Hosts keep one gateway IP. The **active forwarder** changes under them.

Protocol family (vendor-agnostic)

Name	Notes
VRRP	Open standard idea; FRR supports VRRP
HSRP / GLBP	Proprietary dialects of the same problem class
CARP	OpenBSD-family cousin

This book uses **VRRP on FRR** because it is free, documented, and works in Containerlab without licenses. Optional: Nokia SR Linux community images have their own FHRP configuration surface—same model, different CLI.

Priority, preempt, timers

- **Priority:** higher wins master election (within rules).
- **Preempt:** when a higher-priority router returns, does it take the VIP back?
- **Advertisement interval:** how fast backups notice silence.
- **Skew / master-down interval:** how long before backup promotes.

What FHRP is *not*

Not this	Because
Full routing protocol	Hosts do not learn remote prefixes via VRRP
Load balancing of arbitrary flows (unless designed)	Classic VRRP is active/standby for a VIP
Multihoming policy	That is BGP/statics + dual default; separate chapter lab

FHRP protects the **LAN's first hop**. Dual-homing the site edge is complementary (see dual-home lab later in this part).

ARP and the VIP

When master changes, hosts may still have the old MAC in the neighbor cache briefly. Good implementations use a **shared virtual MAC** or gratuitous ARP so clients rebind quickly. In labs, clear neighbor cache deliberately when debugging:

```
# Linux host
ip neigh flush dev eth1
# or
ip neigh del 10.0.0.254 dev eth1
```

Addressing plan

Node	Role	Interface	Address
r1	VRRP priority high	eth1 (LAN)	10.10.10.2/24
r2	VRRP priority low	eth1 (LAN)	10.10.10.3/24
VIP	gateway for hosts	—	10.10.10.254
h1	client	eth1	10.10.10.10/24 via VIP
r1 eth2	uplink	eth2	172.16.1.1/30
r2 eth2	uplink	eth2	172.16.2.1/30
pe	“upstream”	eth1/eth2	172.16.1.2, 172.16.2.2

Upstream `pe` can be a simple FRR or Linux box that advertises a sink for drills (`203.0.113.1` loopback).

Topology YAML

```
name: vrrp-edge

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    pe:
```

```
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 10.10.10.254

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["r1:eth1", "r2:eth1"]   # same LAN: use bridge or multipoint
    - endpoints: ["r1:eth2", "pe:eth1"]
    - endpoints: ["r2:eth2", "pe:eth2"]
```

LAN wiring note: two point-to-point links do not make a shared segment. Prefer a Linux bridge node as "LAN switch," or Containerlab multipoint / bridge patterns from your lab-craft notes:

```
    lan:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 bridge-utils
        - ip link add name br0 type bridge
        - ip link set br0 up
  # links: h1, r1, r2 all into lan with bridge enslavement in exec
```

Simpler teaching pattern: **one host + two routers on a bridge** so ARP and VRRP multicast behave like a campus LAN.

FRR daemons

```
zebra=yes
vrrpd=yes
# staticd or bgpd if you route beyond the VIP
staticd=yes
```

Check mid-2026 FRR image docs for exact daemon names (`vrrpd` may be integrated differently by version—`vtysh show vrrp` is the operator check).

FRR VRRP sketch (r1)

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.10.10.2/24
!
interface eth2
 ip address 172.16.1.1/30
!
interface eth1
 vrrp 1 ip 10.10.10.254
 vrrp 1 priority 200
 vrrp 1 advertisement-interval 1000
!
ip route 0.0.0.0/0 172.16.1.2
!
line vty
```

r2: real IP `10.10.10.3/24`, priority `100`, same VIP and VRID `1`, default via its own uplink.

Syntax varies slightly by FRR minor version—always validate with:

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r1 vtysh -c 'show interface eth1'
```

Linux alternative (keepalived)

Some operators run keepalived on Linux instead of NOS VRRP. Same objects: VIP, priority, advert. In pure Alpine labs you may:

```
# illustrative - package availability varies
# apk add keepalived
# configure vrrp_instance with virtual_ipaddress
```

For this book's default path, **FRR VRRP** keeps one control-plane stack with your OSPF/BGP labs.

Deploy and baseline

```
sudo containerlab deploy -t vrrp-edge.clab.yml

docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 ip route
docker exec clab-vrrp-edge-h1 ip neigh
docker exec clab-vrrp-edge-h1 ping -c 3 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 3 172.16.1.2
```

Predict (baseline)

- r1 is **Master**, r2 is **Backup** (priority 200 > 100)
- h1 ARP for VIP resolves to master's virtual/real MAC pattern for your stack
- Ping VIP succeeds; traffic to upstream uses r1's uplink

Observe

Record: VRRP state, VIP owner interface counters, `ip neigh` on h1, traceroute if upstream is reachable.

Drill 1 — Master failure

```
# Simulate master death (stop data plane or whole node)
docker exec clab-vrrp-edge-r1 ip link set eth1 down
# or: docker stop clab-vrrp-edge-r1

sleep 3
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 ip neigh show 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 10 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 5 203.0.113.1   # if pe has this
```

Predict → observe → fix

Predict	Observe	Fix if wrong
r2 becomes Master after down-timer	`show vrrp`	timers, VRID mismatch, not same LAN

Predict	Observe	Fix if wrong
Short ping loss then recovery	count drops	ARP stuck—flush neigh; check VIP MAC
Upstream still works via r2	traceroute	r2 default / pe return routes

Restore:

```
docker exec clab-vrrp-edge-r1 ip link set eth1 up
# or docker start + wait FRR ready
sleep 5
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
```

Preempt on: r1 reclaims Master. **Preempt off:** r2 may stay Master—document which you configured.

Drill 2 — Split brain / LAN partition (concept)

If both routers believe they are alone (broken LAN bridge), **two masters** can both answer the VIP → intermittent ARP thrash.

Predict

Duplicate VIP answers; flapping neighbor cache; odd packet loss.

Observe

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
# tcpdump for VRRP (IP protocol 112) on the LAN
docker exec clab-vrrp-edge-h1 sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 proto 112
```

Fix / harden

- Ensure single L2 domain for the VRID
- BFD or link tracking to uplinks (advanced) so a “master” without upstream demotes
- Monitoring on “two masters seen” is an ops smell

Drill 3 — Priority change live

```
docker exec -it clab-vrrp-edge-r2 vtysh
configure terminal
interface eth1
 vrrp 1 priority 250
end
write
```

Predict

If preempt allowed, r2 becomes Master; h1 may re-ARP; path to upstream shifts to r2's uplink.

Observe

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 traceroute -n 172.16.2.2
```

Revert priorities in git-mounted configs after the experiment.

Return path and asymmetric edges

VIP failover only moves the **outbound first hop**. If upstream routing always returns via r1 while r2 is master, you get asymmetric paths or blackholes. Pair FHRP with:

- Shared upstream design, or
- Dynamic routing so return traffic follows the active edge, or
- State-aware NAT only on the active path (harder—prefer avoid state at edge in labs first)

Verification checklist

Check	Intent
`show vrrp`	One Master per VRID on the LAN
Host default route	Points at VIP only
`ping VIP`	L2/L3 ownership of first hop
Kill master	Backup promotes; host recovers
Capture proto 112	Hellos present on LAN
Upstream ping during fail	Data plane + return path

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-vrrp-edge-h1 ping -c 2 -W 1 10.10.10.254 || fail "VIP"
# Exactly one master is harder in shell-parse show vrrp or accept soft check:
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp' | grep -qi master \
  || docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp' | grep -qi master \
  || fail "no master"
echo OK
```

Common mistakes

Mistake	Symptom
Routers not same L2 domain	Never form master/backup correctly
Different VRID or VIP	Two independent groups
Host points at real IP not VIP	No redundancy
No return routes via backup	Failover pings die mid-path
Priority tie without tie-break awareness	Unstable election
Forgetting to enable vrrp daemon	Empty `show vrrp`

Optional SR Linux note (community)

SR Linux community Containerlab kinds expose interface and IRP/FHRP-related configuration in a different tree (YANG/CLI). **Do not memorize CLI here**—map objects: VIP, group id, priority, interface, preempt. Same predict→observe drills.

Summary

- Hosts need a stable **first hop**; VRRP-class protocols move a VIP between routers
- FRR + Containerlab is enough to learn Master/Backup, preempt, and failure timers
- Always verify **host ARP**, **VRRP state**, and **end-to-end path** after failover
- FHRP is not multihoming policy—combine later with dual-home BGP/statics
- Keep configs in binds; promote live priority experiments back to git

Next: **DHCP relay and NAT**—how edge services attach to this LAN model without becoming mysterious middleboxes.

DHCP Relay and NAT

DHCP Relay and NAT

Edges do more than forward packets. They **hand out addresses** (or relay DHCP) and often **translate** private space toward a provider. This chapter keeps both topics vendor-agnostic and lab-first: Linux services + FRR routing in Containerlab (mid-2026 free path).

Learning goals

By the end of this chapter you can:

- Explain DHCP discover/offer/request/ack across a relay
- Configure a simple DHCP server and a relay path in a lab
- Contrast source NAT (masquerade), static NAT, and "why not NAT everything forever"
- Implement Linux `nftables`/`iptables` MASQUERADE on an edge
- Predict breakage when relays, helpers, or return translations fail

Concepts — DHCP

Why not static everything?

Labs love static host IPs. Real access LANs use DHCP for:

Benefit	Trade-off
Central policy (lease, DNS, gateway)	Dependency on server/relay
Fast readdressing	Lease timers and "sticky" clients
Audit of who got what	Need logging and hygiene

Four-way dance (v4)

```
Client          Relay (optional)         Server
  |-- Discover ------(unicast to server)-->|
  |<-- Offer -------------------------------|
  |-- Request ------------------------------>|
```

```
   |<-- Ack --------------------------------|
```

On a flat LAN, broadcast Discover reaches the server. Across subnets, a **relay agent** (ip helper / DHCP relay) listens on the client VLAN and unicasts to the server, inserting **giaddr** (gateway IP of the client interface) so the server knows which pool to use.

Critical fields

Field / idea	Why it matters
Client MAC / Client-ID	Lease binding
giaddr	Selects pool / subnet
Options: router, DNS, domain	Host becomes usable
Lease time	How long address is "owned"

Relay failure modes

- Wrong giaddr → wrong pool or no offer
- ACL blocking UDP 67/68
- Server unreachable from relay
- Multiple relays without design → duplicate offers chaos

Concepts — NAT family

Kind	Meaning
SNAT / MASQUERADE	Rewrite source as packets leave (common "PAT")
DNAT	Rewrite destination (port forward / publish service)
1:1 static NAT	Stable mapping for a host
NAT64 / CLAT	v6/v4 transition tools (awareness only here)

NAT is **state + rewrite**. It is not a security model by itself. Stateful firewalls may sit with NAT; keep roles clear.

When NAT is appropriate in this book

- Lab edge simulating consumer/ISP CPE
- Hiding RFC1918 behind a single lab "public"

- Temporary access during dual-stack migration

Prefer **real global or ULA design** for multi-site fabrics later—NAT at every hop becomes undebugable.

Addressing plan (combined edge lab)

```
  h-client (DHCP)          dhcp-srv
       |                      |
      lan ---- edge-rtr ---- pe (public-ish)
                  |
              NAT outside
```

Net	Use
10.20.30.0/24	Client LAN (DHCP pool)
10.20.30.1	Edge LAN IP + VRRP VIP optional
172.31.0.0/30	Edge–server link for DHCP server placement
198.51.100.0/30	Edge–PE “public” lab space (TEST-NET)
Pool	10.20.30.100–200, GW 10.20.30.1, DNS 10.20.30.1 or public lab DNS

Topology YAML

```
name: dhcp-nat-edge

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      # or FRR image if you co-locate routing; Alpine for pure Linux NAT/relay
      exec:
        - apk add --no-cache iproute2 iptables iputils dhcrelay dnsmasq
    pe:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 198.51.100.2/30 dev eth1
        - ip link set eth1 up
        - ip addr add 203.0.113.1/32 dev lo
        - ip link set lo up
    dhcpsrv:
```

```
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dnsmasq iputils
        - ip addr add 172.31.0.2/30 dev eth1
        - ip link set eth1 up
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dhclient iputils
    lan:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2
        - ip link add br0 type bridge && ip link set br0 up

  links:
    - endpoints: ["client:eth1", "lan:eth1"]
    - endpoints: ["edge:eth1", "lan:eth2"]
    - endpoints: ["edge:eth2", "dhcpsrv:eth1"]
    - endpoints: ["edge:eth3", "pe:eth1"]
```

Bridge enslavement (run after deploy or in richer `exec` scripts):

```
# on lan node - enslaves data ports into br0
for i in eth1 eth2; do ip link set $i master br0; ip link set $i up; done
```

Edge addressing and forward

```
# edge LAN toward clients
docker exec clab-dhcp-nat-edge-edge ip addr add 10.20.30.1/24 dev eth1
docker exec clab-dhcp-nat-edge-edge ip link set eth1 up
# toward DHCP server
docker exec clab-dhcp-nat-edge-edge ip addr add 172.31.0.1/30 dev eth2
docker exec clab-dhcp-nat-edge-edge ip link set eth2 up
# toward PE
docker exec clab-dhcp-nat-edge-edge ip addr add 198.51.100.1/30 dev eth3
docker exec clab-dhcp-nat-edge-edge ip link set eth3 up
docker exec clab-dhcp-nat-edge-edge sysctl -w net.ipv4.ip_forward=1
docker exec clab-dhcp-nat-edge-edge ip route add default via 198.51.100.2
```

Bake these into startup scripts for real lab-as-code.

DHCP server (dnsmasq) on dhcpsrv

```
docker exec clab-dhcp-nat-edge-dhcpsrv sh -c 'cat >/etc/dnsmasq.conf <<EOF
interface=eth1
dhcp-range=10.20.30.100,10.20.30.200,12h
dhcp-option=option:router,10.20.30.1
dhcp-option=option:dns-server,10.20.30.1
log-dhcp
EOF'
docker exec clab-dhcp-nat-edge-dhcpsrv dnsmasq
```

Server is **not** on 10.20.30.0/24—the relay's giaddr must make the pool selection work. For dnsmasq/simple labs, operators often put the server **on** the client subnet first (no relay). Teach both:

1. **Same subnet:** client broadcast reaches server—no relay.

2. **Off-subnet:** relay required; server must understand the client subnet (shared-network / giaddr-aware config).

Minimal same-subnet lab (sanity first)

Place dnsmasq on edge's LAN IP or a server on the bridge:

```
dhcp-range=10.20.30.100,10.20.30.200,12h
dhcp-option=option:router,10.20.30.1
```

Relay path (concept + command)

```
# edge: relay client-facing interface toward server
docker exec clab-dhcp-nat-edge-edge dhcrelay -i eth1 172.31.0.2
```

Predict (DHCP)

- Client sends Discover; server logs Offer for 10.20.30.x

- Client installs default via 10.20.30.1

- Lease appears in server logs

Observe

```
docker exec clab-dhcp-nat-edge-client ip link set eth1 up
docker exec clab-dhcp-nat-edge-client dhclient eth1 -v
docker exec clab-dhcp-nat-edge-client ip addr show eth1
docker exec clab-dhcp-nat-edge-client ip route
docker exec clab-dhcp-nat-edge-dhcpsrv cat /var/lib/misc/dnsmasq.leases 2>/dev/null || true
```

Fix

Symptom	Checks
No offer	bridge, interface up, firewall, wrong interface in dnsmasq
Offer but wrong GW	dhcp-option router
Relay silent	UDP 67/68, giaddr, server route

NAT — MASQUERADE on edge

Clients use private 10.20.30.0/24; PE only knows 198.51.100.0/30.

```
docker exec clab-dhcp-nat-edge-edge iptables -t nat -A POSTROUTING \
  -o eth3 -j MASQUERADE
# nftables equivalent pattern:
# nft add table ip nat
# nft add chain ip nat postrouting { type nat hook postrouting priority 100 \; }
# nft add rule ip nat postrouting oifname "eth3" masquerade
```

PE needs a way to answer (connected route to 198.51.100.1 is enough for return of MASQUERADE traffic).

Predict

- `client → 203.0.113.1` works
- On pe, sources appear as 198.51.100.1, not 10.20.30.x
- Without MASQUERADE, pe has no return path to 10.20.30.0/24

Observe

```
docker exec clab-dhcp-nat-edge-client ping -c 3 203.0.113.1
docker exec clab-dhcp-nat-edge-pe sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 -c 5 i
docker exec clab-dhcp-nat-edge-edge iptables -t nat -L -n -v
```

Fix

- `ip_forward=1`
- Correct `-o` egress interface
- Default route on client and edge
- Conntrack full / flush during experiments: `conntrack -F` if tool present

Drill — break DHCP then NAT

1. Stop dnsmasq → new client gets nothing; journal "dependency."
2. Remove MASQUERADE → pings to PE fail; add static route on PE for 10.20.30.0/24 via edge **as a teaching alternate** (routing vs NAT).
3. DNAT publish (optional): map 198.51.100.1:8080 → client:80 and curl from pe.

```
docker exec clab-dhcp-nat-edge-edge iptables -t nat -A PREROUTING \
  -i eth3 -p tcp --dport 8080 -j DNAT --to-destination 10.20.30.100:80
```

Only works if a client actually holds that lease/IP and listens.

FRR on the edge (optional co-location)

Many designs run FRR for BGP/OSPF and Linux for NAT/DHCP. Pattern:

- FRR owns routing (`vtysh`)
- Linux netfilter owns NAT
- DHCP server may be centralized; FRR/host runs relay

Do not assume FRR "is" the NAT engine—check which process owns rewrite.

Verification checklist

Check	Command / intent
Lease	client has /24 + default
Options	DNS/GW match design
Relay	server log shows giaddr path
NAT	capture shows public source
Forward	sysctl ip_forward
Rules	iptables/nft list nat

Common mistakes

Mistake	Symptom
DHCP server wrong subnet logic	No lease or wrong mask
Client static leftover IP	"DHCP broken" false positive
NAT without hairpin plan	Local services odd
Forgetting return routing when NAT removed	One-way pings
Blocking bootp/dhcp in filter	Silent failure

Summary

- DHCP centralizes host addressing; **relay** extends it across L3
- **giaddr** and pool design matter as much as the daemon
- NAT rewrites and tracks state—use deliberately, verify with captures
- Linux edge + Containerlab is enough to practice both without licenses
- Prefer lab scripts that start dnsmasq, relay, and NAT rules idempotently

Next: **name, time, and log hygiene**—the quiet services that make edge failures diagnosable.

Name, Time, and Log Hygiene

Name, Time, and Log Hygiene

Broken DNS, skewed clocks, and silent logs turn simple outages into multi-hour mysteries. This chapter treats **name resolution, time sync, and logging** as first-class edge services—still on free Linux tooling in Containerlab (mid-2026).

Learning goals

By the end of this chapter you can:

- Design a minimal lab DNS path (stub resolver → recursive or authoritative)
- Explain why certificate and log correlation depend on clock sync
- Run chrony/ntp-style time sync in a lab and detect skew
- Centralize or at least structure logs so incidents are replayable
- Apply a hygiene checklist to any edge or fabric lab

Why this chapter exists

Failure	Symptom people mis-blame
DNS	"The network is down" (TCP works, names fail)
Time skew	TLS fails, Kerberos fails, log order nonsense
No logs	Every fix is guesswork; no postmortem

NetOps competence includes **service dependencies of the control and management planes**.

Concepts — naming

Layers of DNS use

```
Application
    → stub resolver (/etc/resolv.conf)
        → recursive resolver (lab or ISP)
            → authoritative servers
```

In labs you often run:

Role	Example tool
Authoritative for `lab.example`	dnsmasq, unbound, knot (as available)
Recursive	unbound, dnsmasq
Stub on nodes	`nameserver` in resolv.conf

Records you actually need

Type	Lab use
A/AAAA	Host and VIP names
PTR	Reverse for traceroute readability (optional)
SRV	Only when practicing real apps
CNAME	Alias hygiene—avoid chains

Split horizon awareness

Internal names may not match external. Document which view a resolver serves. Mis-split horizon looks like “works on my jump host.”

Concepts — time

- **UTC everywhere** in configs and logs when possible
- **NTP/chrony/ptp** family: clients step or slew toward sources
- Auth and TLS validate **notBefore/notAfter** against local clock
- Distributed tracing and log merge need comparable timestamps

Skew of minutes is enough to break modern auth. Skew of seconds confuses incident timelines.

Concepts — logs

Plane	What to capture
Device/OS	link up/down, daemon start, OOM
Routing	neighbor flaps, policy denies (where logged)
Security	filter drops if logged
Lab harness	deploy/destroy, verify.sh results

Structured fields beat walls of free text: `timestamp host facility message`.

Addressing plan (hygiene lab)

```
  client ---- edge ---- dns-ntp
                 \
                  pe
```

Node	IP	Role
edge eth1	10.40.0.1/24	client GW
client	10.40.0.10/24	stub resolver → edge or dns
dns-ntp	10.40.0.53/24	dnsmasq + chrony server
edge eth2	198.51.100.1/30	optional upstream

Topology YAML

```yaml
name: hygiene-edge

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iptables iputils bind-tools chrony
    dnsntp:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dnsmasq chrony iputils bind-tools
    client:
      kind: linux
```

```
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools
    pe:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["client:eth1", "edge:eth1"]
    - endpoints: ["dnsntp:eth1", "edge:eth2"]
    - endpoints: ["edge:eth3", "pe:eth1"]
```

Use a small bridge if you want client and dnsntp on one LAN with edge as L3 gateway only—either design is fine if documented.

Bring-up addressing (example)

```
# edge: two LANs + pe
docker exec clab-hygiene-edge-edge sh -c '
  ip addr add 10.40.0.1/24 dev eth1; ip link set eth1 up
  ip addr add 10.40.1.1/24 dev eth2; ip link set eth2 up
  ip addr add 198.51.100.1/30 dev eth3; ip link set eth3 up
  sysctl -w net.ipv4.ip_forward=1
'
docker exec clab-hygiene-edge-dnsntp sh -c '
  ip addr add 10.40.1.53/24 dev eth1; ip link set eth1 up
  ip route add default via 10.40.1.1
'
docker exec clab-hygiene-edge-client sh -c '
  ip addr add 10.40.0.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.40.0.1
  echo nameserver 10.40.1.53 > /etc/resolv.conf
'
# edge route to dns LAN is connected; client needs path to 10.40.1.53
docker exec clab-hygiene-edge-edge ip route add 10.40.1.0/24 dev eth2 2>/dev/null || true
```

Ensure client can reach 10.40.1.53 (forwarding + reverse path).

DNS lab — dnsmasq authoritative snack

```
docker exec clab-hygiene-edge-dnsntp sh -c 'cat >/etc/dnsmasq.conf <<EOF
interface=eth1
no-resolv
address=/vip.lab.example/10.40.0.254
address=/edge.lab.example/10.40.0.1
address=/client.lab.example/10.40.0.10
log-queries
EOF'
docker exec clab-hygiene-edge-dnsntp dnsmasq
```

Predict

```
docker exec clab-hygiene-edge-client nslookup vip.lab.example 10.40.1.53
# or
docker exec clab-hygiene-edge-client dig +short vip.lab.example @10.40.1.53
```

Returns `10.40.0.254`.

Observe / fix

Symptom	Fix
timeout	routing, firewall, dnsmasq not listening
NXDOMAIN	typo in address= or wrong server
uses wrong resolver	resolv.conf

Drill — DNS outage looks like network outage

```
docker exec clab-hygiene-edge-dnsntp pkill dnsmasq || true
docker exec clab-hygiene-edge-client ping -c 2 vip.lab.example   # may fail resolve
docker exec clab-hygiene-edge-client ping -c 2 10.40.0.1         # IP still works
```

Predict → observe → fix

- Name fails, IP works → **name plane**, not forwarding
- Restart dnsmasq; retest dig
- Harden: secondary resolver IP in resolv.conf (even if both lab-local)

```
nameserver 10.40.1.53
nameserver 10.40.1.54
```

Time sync — chrony sketch

On dnsntp as a simple lab time source (isolated labs often use local orphan mode or host chrony):

```
docker exec clab-hygiene-edge-dnsntp sh -c 'cat >/etc/chrony/chrony.conf <<EOF
local stratum 8
allow 10.40.0.0/16
makestep 1.0 3
EOF'
docker exec clab-hygiene-edge-dnsntp chronyd
```

Client/edge:

```
docker exec clab-hygiene-edge-client sh -c '
  apk add --no-cache chrony
  echo "server 10.40.1.53 iburst" > /etc/chrony/chrony.conf
  chronyd
'
docker exec clab-hygiene-edge-client chronyc tracking
docker exec clab-hygiene-edge-client chronyc sources -v
```

Package paths vary (`/etc/chrony.conf` vs `/etc/chrony/chrony.conf`)—adjust to the image.

Skew drill

```
# induce skew on client (lab only)
docker exec clab-hygiene-edge-client date -s "2020-01-01 00:00:00"
docker exec clab-hygiene-edge-client chronyc -a makestep
docker exec clab-hygiene-edge-client date
docker exec clab-hygiene-edge-dnsntp date
```

Predict

After makestep/slew, client approaches server time; `chronyc tracking` shows low offset.

Observe

Record offset before/after. Correlate with a fake "TLS" check: if you generate certs with short validity in lab tools, wrong dates fail verification.

Logging hygiene

Local journal pattern

```
# example: log dnsmasq and link events to a file on edge
docker exec clab-hygiene-edge-edge sh -c '
  mkdir -p /var/log/lab
  echo "$(date -Is) lab start" >> /var/log/lab/edge.log
'
```

syslog-style forward (concept)

```
device/app → rsyslog/syslog-ng → central lab collector
```

In Containerlab, mounting a host directory for `/var/log/lab` keeps evidence after destroy:

```
    edge:
      kind: linux
      image: alpine:3.20
      binds:
        - ./logs/edge:/var/log/lab
```

What to log for network incidents

Event	Why
Link down/up	Correlates with neighbor loss
Daemon restart	Control plane blip
DHCP ACK / VRRP transition	Edge role changes
Filter deny (sampled)	Policy vs outage
verify.sh pass/fail	Automation truth

FRR logging (when routers are FRR)

```
log file /var/log/frr/frr.log
log timestamp precision 3
```

```
docker exec clab-...-r1 vtysh -c 'show log'
# or tail the bound log file
```

Bind `/var/log/frr` into the lab folder for postmortems.

Hygiene checklist (use every serious lab)

- ☐ Addressing plan names every important VIP and service IP
- ☐ DNS A records for VIP, routers, servers (even if only lab zone)
- ☐ Clients have 1 working resolver; document fallback
- ☐ Clocks within seconds of a known source (or explicitly isolated)
- ☐ Logs land on durable storage (bind mount / central)
- ☐ Timezone policy stated (UTC recommended)
- ☐ Incident notes include timestamps from the **same** clock domain

Predict → observe → fix mini-scenarios

Scenario A — "Git clone fails" in a lab VM

- Predict: DNS or proxy
- Observe: `ping 1.1.1.1` vs `ping github.com`
- Fix: resolv.conf / reachability to recursive resolver

Scenario B — "Certificate expired" on brand-new lab CA

- Predict: node year is wrong
- Observe: `date -u` on client and server
- Fix: chrony; disable fake date experiments

Scenario C — "It failed an hour ago" with no trace

- Predict: no logs retained
- Observe: empty `/var/log/lab`, container destroyed
- Fix: binds + verify.sh archival

Optional: name the lab graph

Containerlab graph exports help humans; DNS helps apps. Keep **hostname = inventory name = DNS label** when you can:

```
clab-hygiene-edge-edge     edge.lab.example
```

Common mistakes

Mistake	Symptom
resolv.conf points at host DNS unreachable from namespace	all names fail
No forward path to DNS subnet	intermittent based on which node
Logging only to container overlay	evidence vanishes on destroy
Mixing local timezones in notes	impossible timelines
NTP blocked by ACL you added "for hardening"	chronic skew

Summary

- **Name, time, logs** are edge/management dependencies, not optional polish
- Practice DNS outage vs forwarding outage until the difference is reflex
- Sync clocks before debugging TLS or distributed logs
- Bind-mount lab logs; structure enough fields to postmortem
- Add this hygiene checklist to dual-home and fabric labs later

Next: **edge dual-home lab**—tie FHRP, routing preference, and these services into one site edge workout.

Edge Dual-Home Lab

Edge Dual-Home Lab

This chapter is a **full edge workout**: one customer site, two uplinks to two providers, first-hop redundancy on the LAN, filtered eBGP, and failure drills. Stack: Containerlab + FRR + Alpine (mid-2026 free path). Optional SR Linux as a PE substitute for CLI diversity.

Learning goals

By the end of this chapter you can:

- Design dual-homed CE addressing and AS layout
- Prefer one uplink with local-preference; backup with AS-path prepend
- Combine VRRP VIP for hosts with dual CE or single CE dual-uplink patterns
- Prove failover with ping bursts, BGP tables, and traceroute
- Ship a lab folder that meets the book's definition of done

Design choices (pick one primary)

Pattern A — Single CE, two eBGP uplinks (simpler)

```
      pe1 (AS65010)
     /
ce (AS65001) ---- LAN ---- hosts (GW = ce LAN IP or VIP if HA pair)
     \
      pe2 (AS65020)
```

Pattern B — Dual CE + VRRP (more realistic campus edge)

```
      pe1
     /
ce1 === VRRP VIP === LAN --- hosts
     \   /
      ce2
```

```
        \
         pe2
```

This chapter implements **Pattern B** as the main lab and notes how to simplify to A.

Addressing plan

Object	Value
Site LAN	192.168.50.0/24
VIP gateway	192.168.50.254
ce1 LAN	192.168.50.2/24
ce2 LAN	192.168.50.3/24
host	192.168.50.10/24 via VIP
ce1–pe1	172.16.10.0/30 (ce1 .1, pe1 .2)
ce2–pe2	172.16.20.0/30 (ce2 .1, pe2 .2)
Site prefix	192.168.50.0/24 (advertise)
pe1 lo	1.1.1.1/32 (AS65010)
pe2 lo	2.2.2.2/32 (AS65020)
Hijack test	203.0.113.0/24 must **not** be accepted from CE

Topology YAML

```yaml
name: dualhome

topology:
  nodes:
    ce1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/ce1:/etc/frr
    ce2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/ce2:/etc/frr
    pe1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe1:/etc/frr
    pe2:
      kind: linux
```

```
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe2:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.50.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.50.254
    lan:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2
        - ip link add br0 type bridge
        - ip link set br0 up

  links:
    - endpoints: ["h1:eth1", "lan:eth1"]
    - endpoints: ["ce1:eth1", "lan:eth2"]
    - endpoints: ["ce2:eth1", "lan:eth3"]
    - endpoints: ["ce1:eth2", "pe1:eth1"]
    - endpoints: ["ce2:eth2", "pe2:eth1"]
```

Enslave LAN ports to `br0` in a deploy hook:

```
for n in eth1 eth2 eth3; do
  docker exec clab-dualhome-lan ip link set $n master br0
  docker exec clab-dualhome-lan ip link set $n up
done
```

FRR daemons

```
zebra=yes
bgpd=yes
staticd=yes
vrrpd=yes
```

CE1 config sketch

```
frr version 10.2.1
frr defaults traditional
hostname ce1
!
interface lo
 ip address 10.0.0.1/32
!
interface eth1
 ip address 192.168.50.2/24
 vrrp 1 ip 192.168.50.254
 vrrp 1 priority 200
!
interface eth2
 ip address 172.16.10.1/30
!
router bgp 65001
 bgp router-id 10.0.0.1
 no bgp ebgp-requires-policy
 neighbor 172.16.10.2 remote-as 65010
 !
 address-family ipv4 unicast
  network 192.168.50.0/24
  neighbor 172.16.10.2 route-map FROM-PE in
  neighbor 172.16.10.2 route-map TO-PE out
 exit-address-family
!
ip prefix-list SITE seq 5 permit 192.168.50.0/24
ip prefix-list SITE seq 10 deny any
!
ip prefix-list DEFAULT-OK seq 5 permit 0.0.0.0/0
ip prefix-list DEFAULT-OK seq 10 deny any
!
route-map TO-PE permit 10
 match ip address prefix-list SITE
route-map TO-PE deny 20
!
route-map FROM-PE permit 10
 match ip address prefix-list DEFAULT-OK
 set local-preference 200
route-map FROM-PE deny 20
!
line vty
```

`no bgp ebgp-requires-policy` is a lab convenience on some FRR versions—prefer explicit policies always in "production-shaped" labs.

CE2 config differences

- LAN IP `.3`, VRRP priority `100`
- Uplink `172.16.20.1/30` to pe2
- **Outbound prepend** so pe2 is less preferred by others if both advertise:

```
route-map TO-PE permit 10
 match ip address prefix-list SITE
 set as-path prepend 65001 65001
```

- **Inbound** local-pref `100` (lower than ce1's 200) if both learn default:

```
route-map FROM-PE permit 10
 match ip address prefix-list DEFAULT-OK
 set local-preference 100
```

For dual-CE, each CE may only have one uplink—**VRRP** chooses which CE hosts use; each CE's BGP prefers its own default. Advanced: iBGP between ce1/ce2—optional stretch goal.

PE1 / PE2 sketch

```
hostname pe1
!
interface lo
 ip address 1.1.1.1/32
!
interface eth1
 ip address 172.16.10.2/30
!
router bgp 65010
 bgp router-id 1.1.1.1
 neighbor 172.16.10.1 remote-as 65001
 !
 address-family ipv4 unicast
  network 0.0.0.0/0
  network 1.1.1.1/32
  neighbor 172.16.10.1 route-map FROM-CE in
  neighbor 172.16.10.1 route-map TO-CE out
 exit-address-family
!
ip prefix-list CE-ALLOW seq 5 permit 192.168.50.0/24
ip prefix-list CE-ALLOW seq 10 deny any
!
```

```
route-map FROM-CE permit 10
 match ip address prefix-list CE-ALLOW
route-map FROM-CE deny 20
!
ip prefix-list ORIGINATE seq 5 permit 0.0.0.0/0
ip prefix-list ORIGINATE seq 10 permit 1.1.1.1/32
route-map TO-CE permit 10
 match ip address prefix-list ORIGINATE
!
line vty
```

pe2 mirrors with AS 65020, `2.2.2.2`, `172.16.20.2`.

Hijack drill: temporarily add `network 203.0.113.0/24` on CE—PE must **not** install it.

Deploy and baseline verify

```
sudo containerlab deploy -t dualhome.clab.yml
# bridge hook as above

docker exec clab-dualhome-ce1 vtysh -c 'show vrrp'
docker exec clab-dualhome-ce2 vtysh -c 'show vrrp'
docker exec clab-dualhome-ce1 vtysh -c 'show bgp summary'
docker exec clab-dualhome-ce1 vtysh -c 'show ip route'
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni'
docker exec clab-dualhome-h1 ping -c 3 192.168.50.254
docker exec clab-dualhome-h1 ping -c 3 1.1.1.1
docker exec clab-dualhome-h1 traceroute -n 1.1.1.1
```

Predict (baseline)

- ce1 VRRP Master
- h1 reaches VIP and pe1 loopback via ce1
- pe1 has 192.168.50.0/24 only from CE
- pe2 may also have site prefix if ce2 session up

Drill 1 — Primary PE session loss

```
docker exec clab-dualhome-ce1 ip link set eth2 down
sleep 2
docker exec clab-dualhome-ce1 vtysh -c 'show bgp summary'
docker exec clab-dualhome-h1 ping -c 20 2.2.2.2
docker exec clab-dualhome-h1 traceroute -n 2.2.2.2
```

Predict → observe → fix

Predict	If false
Host still uses ce1 VIP (VRRP unchanged)	—
ce1 loses default → traffic blackholes unless ce2 takes VIP or ce1 has alternate	Add static floating / iBGP / kill VRRP master

Important teaching moment: if only ce1 is master and its uplink dies, **VRRP does not care** unless you track the uplink (BFD/link tracking) or lower priority. Options:

1. Shut ce1 LAN to force VRRP failover (crude).
2. Configure track/priority decrement when eth2 down (if supported).
3. Run dual default via both CEs with host-side multipath (rare).

Document which design you chose. Professional edges **tie FHRP priority to uplink health**.

Drill 2 — Force VRRP failover

```
docker exec clab-dualhome-ce1 ip link set eth1 down
sleep 3
docker exec clab-dualhome-ce2 vtysh -c 'show vrrp'
docker exec clab-dualhome-h1 ip neigh flush dev eth1
docker exec clab-dualhome-h1 ping -c 10 2.2.2.2
docker exec clab-dualhome-h1 traceroute -n 2.2.2.2
```

Predict

ce2 Master; path via pe2; short disruption.

Drill 3 — Policy rejection

```
# on ce1 temporarily
docker exec -it clab-dualhome-ce1 vtysh
# network 203.0.113.0/24 under bgp + dummy interface address if needed
```

```
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni 203.0.113.0/24'
```

Predict

Empty / not best—filter works. If accepted, fix FROM-CE prefix-list.

Drill 4 — Name/time smoke (hygiene)

Optional: add dnsmasq A record `vip.site.lab → 192.168.50.254` and dig from h1; confirm `date -u` roughly matches host.

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-dualhome-h1 ping -c 2 -W 1 192.168.50.254 || fail "vip"
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni' | grep -q '192.168.50.0' \
  || fail "pe1 missing site"
# hijack should be absent
if docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni' | grep -q '203.0.113.0'; then
  fail "hijack accepted"
fi
echo OK
```

Definition of done (this lab)

- ☐ Topology + configs in git
- ☐ Addressing plan + diagram
- ☐ VRRP master/backup documented

☐ eBGP policies on CE and PE

☐ Three drills with notes (PE loss, VRRP fail, hijack)

☐ `verify.sh` green on clean deploy

☐ Written note: how uplink loss interacts with VRRP

Optional SR Linux PE

Replace pe1 with community SR Linux kind in Containerlab; keep CE on FRR. Same BGP objects: neighbor, import prefix allow-list, export default. Compare `show bgp` dialects in your journal—models stay identical.

Common mistakes

Mistake	Symptom
No bridge on LAN	VRRP never forms
Host uses .2 not VIP	No HA
PE accepts any prefix	Hijack succeeds
Uplink down but VRRP stays on blind CE	Blackhole
Asymmetric return without pe routes	One-way

Summary

- Dual-home labs combine **multihoming policy** and **first-hop HA**—design their interaction
- FRR eBGP + prefix-lists + VRRP are enough for a portfolio-quality edge
- Always test **policy negatives** (hijack) and **uplink vs VIP** failures separately
- Prefer link tracking or clear runbooks when master loses upstream
- This lab maps to capstone C2 in the ops part

Next part: **policy, QoS vocabulary, and hardening**—filters beyond BGP, control-plane protection, and checklists.

DNS Authoritative and Recursive

DNS Authoritative and Recursive

Name resolution fails more often than forwarding. This chapter treats **DNS roles**—stub, recursive, authoritative—as lab-first architecture: who answers from zone data, who walks the tree, what gets cached, and how operators verify SOA/NS/glue and DNSSEC with `dig`. Tooling stays free Linux containers in Containerlab (mid-2026 path): CoreDNS for authoritative lab zones, Unbound for recursion/validation, `bind-tools` / `dig` for truth.

Learning goals

By the end of this chapter you can:

- Draw the stub → recursive → authoritative path and predict failure symptoms
- Distinguish authoritative answers (AA) from referrals and cached recursed answers
- Explain SOA, NS, glue, and why in-bailiwick names need A/AAAA glue
- Run a two-node lab: CoreDNS authoritative + Unbound recursive
- Verify with `dig` (+trace, +norecurse, DNSSEC flags) and a short checklist
- State what DNSSEC validates for an operator (and what it does not)

Why roles matter

Role	Answers from	Typical lab tool
Stub resolver	Asks one or two configured servers	`/etc/resolv.conf`
Recursive resolver	Cache, else iterative queries toward authority	Unbound
Authoritative server	Local zone data (primary/secondary)	CoreDNS, BIND, Knot, PowerDNS

Mixing roles on one daemon is possible (classic BIND "do everything") but **lab clarity** comes from splitting: one box owns `lab.example`, another recurses for clients.

```
Application
  → stub (/etc/resolv.conf)
      → recursive (Unbound)
          → root / TLD / child authoritative (or lab fake root)
```

On the public Internet the recursive walks from the **root**. In a closed lab you either:

- Forward Unbound to a reachable upstream recursive, or
- Serve a private root / stub zones that point at your authoritative.

This chapter uses a **private zone** plus Unbound **stub-zone** / forward to the CoreDNS auth IP—no need for a full root mirror.

Concepts — recursive vs authoritative

Authoritative

Configured for a set of zones. Answers from local data **without** querying others for those names. Flags **AA** (Authoritative Answer) when the response is from that zone's data.

Primary vs secondary: secondaries still answer authoritatively once they hold a zone copy (IXFR/AXFR). Delegation in the *parent* does not by itself make a server authoritative—you must configure the zone.

Recursive

Receives queries from stubs (RD=1). Satisfies from **cache** when possible; otherwise starts at a configured root or forwarder and follows **referrals** until an authoritative answer is obtained, then returns it to the client (usually with AA clear on the final client-facing message, AD set if DNSSEC validation succeeded—see below).

Stub

Library or OS resolver: almost no policy. Points at recursive IPs. Failures here look like "DNS is down" even when the path to 8.8.8.8 is fine but `resolv.conf` points at a black hole inside the namespace.

Record	Operator meaning

Concepts — SOA, NS, glue

Record	Operator meaning
SOA	Apex start-of-authority: primary name, contact, serial, refresh/retry/expire/minimum (negative TTL flavor)
NS	Nameserver hostnames for the zone (apex) or delegation (parent)
Glue	A/AAAA for NS hostnames that lie **inside** the child zone (in-bailiwick)—parent must supply them in additional data or resolution cannot start

```
Parent (.example) referral for lab.example:
  NS ns1.lab.example.
  Additional: ns1.lab.example. A 10.50.0.53   ← glue

Child apex (authoritative):
  lab.example. SOA ...
  lab.example. NS ns1.lab.example.
  ns1.lab.example. A 10.50.0.53
```

Parent NS/glue and child apex NS should agree. Mismatches are a classic "works from some resolvers" outage (parent-centric vs child-centric resolver behavior differs; treat consistency as hygiene).

Concepts — caching

Recursors cache positive answers and negative answers (NXDOMAIN / NODATA) within TTL / SOA minimum policies. Operator effects:

- Lower TTL → faster change pickup, more query load on auth
- Raise TTL carefully on stable records
- Flush cache after lab zone edits when testing (`unbound-control flush` or restart)

Authoritative servers may also cache *nothing* for their own zones—they serve configured data. Do not expect editing a zone file on auth to update a recursive client until TTL expires or cache is flushed.

Concepts — DNSSEC at operator level

DNSSEC adds authenticity and integrity (not confidentiality). Chain of trust:

```
Trust anchor (root DS/DNSKEY in validator)
  → DS at parent points to child DNSKEY
    → RRSIGs over RRSets in the child
```

What to verify as an operator:

Check	Tooling idea
Validator has a current trust anchor	Unbound `auto-trust-anchor-file` / system anchors
`AD` bit on answers you care about	`dig +dnssec example.com` from the recursive
Broken chain symptoms	`SERVFAIL` from validating resolver; dig from non-validating still "works"
Delegation consistency	Parent **DS** matches child **DNSKEY**; NS/glue still required

Lab note: signing a private `lab.example` needs a trust anchor you install in Unbound, or you practice DNSSEC against public zones from a recursive that can reach the Internet. Do not confuse **AD** (Authenticated Data from a validator) with **AA** (Authoritative Answer from a zone server).

Addressing plan (DNS roles lab)

```
client ---- (L2) ---- recur ---- (L2) ---- auth
               10.50.0.0/24
```

Node	IP	Role
auth	10.50.0.53/24	CoreDNS authoritative for `lab.example`
recur	10.50.0.54/24	Unbound recursive; stub-zone → auth
client	10.50.0.10/24	Stub → 10.50.0.54

Topology YAML

```yaml
name: dns-roles

topology:
  nodes:
    auth:
      kind: linux
```

```
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools
        # Install coredns binary via your preferred lab method
        # (apk package, static binary bind-mount, or distro package).
    recur:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools unbound
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools
  links:
    - endpoints: ["auth:eth1", "recur:eth1"]
    - endpoints: ["recur:eth2", "client:eth1"]
```

Adjust images if you standardize on a CoreDNS-bearing image. The verification culture below is what matters more than the exact package name.

Authoritative zone (CoreDNS sketch)

Zone file `lab.example.db`:

```
$ORIGIN lab.example.
$TTL 60
@   IN SOA ns1.lab.example. hostmaster.lab.example. (
        2026100301 ; serial
        300        ; refresh
        60         ; retry
        86400      ; expire
        60         ; minimum
)
@       IN NS  ns1.lab.example.
ns1     IN A   10.50.0.53
www     IN A   10.50.0.80
api     IN A   10.50.0.81
```

Corefile:

```
lab.example:53 {
    file /etc/coredns/lab.example.db
    log
```

```
    errors
}
```

Bring-up sketch:

```
docker exec clab-dns-roles-auth ip addr add 10.50.0.53/24 dev eth1
docker exec clab-dns-roles-auth ip link set eth1 up
# copy zone + Corefile, then:
docker exec clab-dns-roles-auth coredns -conf /etc/coredns/Corefile
```

Direct auth checks (no recursion)

```
dig @10.50.0.53 www.lab.example A +norecurse
dig @10.50.0.53 lab.example SOA +norecurse
dig @10.50.0.53 lab.example NS +norecurse
```

Expect **AA** on answers for names in zone. Querying a name outside the zone should fail closed (REFUSED/NXDOMAIN per config)—auth should **not** recurse unless you deliberately enable it.

Recursive (Unbound) sketch

`/etc/unbound/unbound.conf` fragment:

```
server:
  interface: 0.0.0.0
  access-control: 10.50.0.0/24 allow
  do-ip6: no
  # For public DNSSEC labs, keep trust anchors enabled.
  # auto-trust-anchor-file: "/usr/share/dnssec-root/root.key"

stub-zone:
  name: "lab.example"
  stub-addr: 10.50.0.53

forward-zone:
  name: "."
  # Optional: only if auth/private root is not enough and egress exists
  # forward-addr: 1.1.1.1
```

```
docker exec clab-dns-roles-recur ip addr add 10.50.0.54/24 dev eth1
docker exec clab-dns-roles-recur ip link set eth1 up
docker exec clab-dns-roles-recur unbound -d -c /etc/unbound/unbound.conf &
```

Client stub:

```
docker exec clab-dns-roles-client sh -c 'cat >/etc/resolv.conf <<EOF
nameserver 10.50.0.54
search lab.example
EOF'
```

dig verification workflows

1. Happy path via recursive

```
docker exec clab-dns-roles-client dig www.lab.example A +noall +answer +comments
```

Predict: answer `10.50.0.80`, from recursive (AA typically unset on the client-facing message).

2. Prove auth is the source of truth

```
dig @10.50.0.53 www.lab.example A +norecurse
dig @10.50.0.54 www.lab.example A
```

Change the zone A record / bump SOA serial / reload CoreDNS, then:

```
# may still see old data until TTL expires
dig @10.50.0.54 www.lab.example A
# flush and retest (command name varies by package)
docker exec clab-dns-roles-recur unbound-control flush www.lab.example
```

3. Trace mindset (public names)

When the recursive has real root access:

```
dig example.com +trace
```

Shows iterative referrals. In a closed lab, `+trace` may fail at root—use stub-zone labs instead of forcing a fake public trace.

4. Glue / NS consistency

```
dig @10.50.0.53 lab.example NS +norecurse
dig @10.50.0.53 ns1.lab.example A +norecurse
```

If you later add a parent zone, inspect **additional** section glue on the referral.

5. DNSSEC operator glance (public)

```
dig @10.50.0.54 cloudflare.com A +dnssec
# Look for ad flag when validation works; SERVFAIL when chain breaks
```

Compare with a non-validating query path if you temporarily disable validation—difference isolates DNSSEC vs reachability.

Predict → observe → fix

Scenario A — client NXDOMAIN for `www.lab.example`

- Predict: wrong stub IP, auth down, or typo in zone
- Observe: `dig @10.50.0.54`, then `dig @10.50.0.53 +norecurse`
- Fix: restore auth listening; fix zone; fix resolv.conf

Scenario B — auth works, recursive SERVFAIL

- Predict: stub-zone missing, auth unreachable from recur, or DNSSEC failure
- Observe: `dig @10.50.0.53` OK; `dig @10.50.0.54` SERVFAIL; check Unbound log
- Fix: routing between recur auth; stub-addr; trust anchor only if validating private zones incorrectly

Scenario C — "update not visible"

- Predict: recursive cache / TTL
- Observe: auth shows new A; recur shows old
- Fix: wait TTL, lower lab TTL, or flush cache

Scenario D — AA vs AD confusion

- Predict: operator expects AD on auth dig
- Observe: `dig @auth` has AA, not AD
- Fix: AD comes from a **validating recursive**, not from AA

Verification checklist

Check	Intent
`dig @auth SOA`	AA + expected serial
`dig @auth NS` + glue A	Apex NS resolve
`dig @recur name`	Client path works
Cache flush drill	TTL behavior understood
resolv.conf	Stub points at recur, not auth (unless intentional)
Logs	CoreDNS/Unbound query logs on failure
DNSSEC (optional)	AD on public name; document trust anchors

Common mistakes

Mistake	Symptom
Pointing stubs at authoritative only	Public names fail; private names OK
Enabling recursion on Internet-exposed auth	Abuse / cache poisoning risk
Missing glue for in-bailiwick NS	Delegation black hole
Forgetting SOA serial bump on secondaries	Stale secondaries (when you add transfers later)
Assuming AA means DNSSEC-valid	Conflates authority with authenticity
Lab resolv.conf → Docker host DNS unreachable	Intermittent or total failure inside netns

Summary

- **Authoritative** servers answer from zone data (AA); **recursives** cache and walk or stub toward authority
- Stubs are configuration—wrong `nameserver` lines create false "network down" stories
- SOA/NS/glue consistency is operational hygiene; dig `+norecurse` against auth is your ground truth
- DNSSEC: verify AD via a validating recursive and keep DS/DNSKEY aligned—do not confuse with AA
- Split CoreDNS (auth) and Unbound (recur) in Containerlab until the roles are muscle memory

Next edge labs can reuse this DNS path whenever VIP names and dual-home failovers need human-readable verification.

QUIC Version 2 and Compatible Version Negotiation

QUIC Version 2 and Compatible Version Negotiation

QUIC version 1 (RFC 9000) won the Internet. That success is also a risk: middleboxes that only ever see `0x00000001` in the long-header Version field, or that hard-code the version-1 Initial salt, will quietly break the next version. **QUIC version 2** (RFC 9369) is deliberately almost identical to version 1 — same streams, same TLS, same congestion control — with a handful of wire-image changes whose only job is to break that ossification and to exercise the negotiation machinery. **Compatible version negotiation** (RFC 9368) is how endpoints switch between v1 and v2 (or any pair of compatible versions) without paying an extra round trip. This chapter teaches the Version field, Compatible VN versus Incompatible VN, when v2 actually matters on the wire, and how to observe it in a modern browser stack.

Learning goals

By the end of this chapter you can:

- State what QUICv2 changes (and what it deliberately does *not* change)
- Decode a long-header Version field: `0x00000001` vs `0x6b3343cf`
- Explain Compatible VN vs Incompatible VN, and why v1 v2 is Compatible
- Predict which packets keep the Original Version and which must use the Negotiated Version
- Name the `version_information` transport parameter (0x11) and why v2 endpoints MUST use it
- Observe negotiated version in Firefox (Nightly 154+, all channels from 155) and reason about server support

Concepts — why a "version 2" that does nothing new?

RFC 9369 is explicit: QUICv2 exists to mitigate ossification and to exercise version negotiation. Application capabilities are unchanged. ALPN tokens defined for QUICv1 (`h3`, `doq`, …) also work over v2. Extensions that work with v1 work with v2 unless they say otherwise.

What *does* change:

Knob	QUICv1 (RFC 9000 / 9001)	QUICv2 (RFC 9369)
Long-header Version field	`0x00000001`	`0x6b3343cf`
Long-header Type bits	Initial `0b00`, 0-RTT `0b01`, Handshake `0b10`, Retry `0b11`	Initial `0b01`, 0-RTT `0b10`, Handshake `0b11`, Retry `0b00`
Initial salt	v1 salt	`0x0dede3def700a6db819381be6e269dcbf9bd2ed9`
HKDF labels	`quic key` / `quic iv` / `quic hp` / `quic ku`	`quicv2 key` / `quicv2 iv` / `quicv2 hp` / `quicv2 ku`
Retry Integrity Tag keying	v1 secret	v2 secret / key / nonce (RFC 9369 §3.3.3)

The version number is *not* the integer 2. It is the first four bytes of `sha256("QUICv2 version number")`, chosen randomly on purpose so middleboxes cannot guess "the next number". A provisional draft codepoint `0x709a50c4` also sits in the IANA registry for historical draft traffic.

```
Long header (RFC 8999 invariants):
  Header Form (1) = 1
  Fixed Bit    (1) = 1
  Long Packet Type (2)   ← different encoding in v2
  Type-Specific (4)
  Version (32)           ← 0x00000001 or 0x6b3343cf
  DCID Len / DCID / SCID Len / SCID / …
```

Concepts — Compatible VN vs Incompatible VN

RFC 9000 gave servers a Version Negotiation packet that says "I do not speak the version you chose", but it never defined a *safe* client reaction. RFC 9368 fills that hole with two mechanisms:

Incompatible VN. Server cannot parse the client's first flight. It sends a Version Negotiation packet listing Offered Versions. Client starts a *new* connection attempt with a mutually supported version. Cost: one extra RTT. Works between any versions.

Compatible VN. Server *can* parse the first flight, extracts the client's Version Information, and may switch to another version that (1) it supports and (2) it knows is compatible with the client's Chosen Version. It converts the first flight and replies as if that converted flight had arrived. Cost: zero extra RTT, but only when the versions share a convertible first-flight format.

RFC 9369 states that v2 is compatible with v1 and v1 is compatible with v2. Endpoints that speak both SHOULD use Compatible VN.

```
Compatible VN (happy path, client offers v1+v2, prefers v1 as Chosen):

  Client                                          Server
  Chosen=v1, Available=(v1,v2)  ---------------->
                               (server prefers v2, converts first flight)
                               <---------------- CRYPTO in v2 long headers
  (client sees Version 0x6b3343cf   Chosen) → Negotiated = v2
  subsequent Initial / Handshake / 1-RTT use v2
```

Version Information (transport parameter 0x11)

```
Version Information {
  Chosen Version (32),
  Available Versions (32) ...,
}
```

- **Client Available Versions:** every version this first flight is compatible with, preference order descending. Chosen Version MUST appear in the list.
- **Server Available Versions:** the deployment's Fully Deployed Versions (may be empty during rollouts; ordering carries no meaning).
- QUICv2 endpoints **MUST** send, process, and validate this parameter (downgrade defence). QUICv1 endpoints that only ever speak v1 may omit it; RFC 9368 §8 special-cases that.

Packet-version rules after a Compatible switch (v1 v2)

Packet class	Rule
Retry	Always Original Version; client must not switch version in the next Initial that carries the Retry token
Server Initial before client TP processed	May use Original Version
All CRYPTO frames from the server	Negotiated Version
Handshake and 1-RTT	MUST be Negotiated Version; other versions dropped
Client 0-RTT	MUST stay on Original Version even after seeing Negotiated Version

Packet class	Rule
Session tickets / NEW_TOKEN	Version-scoped; do not resume v1 tickets on a v2 connection (and vice versa). After Compatible VN, issued tokens belong to the *Negotiated* version

The client learns the Negotiated Version by watching the first long-header Version field that differs from the Original Version. If the server answers with CRYPTO still in the Original Version, negotiation landed on Original == Negotiated.

When v2 matters vs when it does not

Situation	Prefer
Fighting middleboxes that ossify on v1 Initial crypto / Version bytes	Offer v2; some clients start with v2 when they know the server supports it (accepts Incompatible VN cost on miss)
Maximising reach to old clients / Alt-Svc `h3` without version signalling	Keep speaking v1; RFC 9369 says HTTP servers SHOULD support multiple versions
Measuring deployment health	Log negotiated version per handshake (Firefox Glean: `networking.http_3_version`)
Application features (streams, datagrams, HTTP/3)	Irrelevant — v2 adds none

v2 does **not** deprecate v1. Treating "we enabled v2" as "we can drop v1" breaks clients that only offer v1, and Alt-Svc still does not distinguish versions.

Worked observation — Firefox and Compatible VN

As of mid-2026, Mozilla's neqo stack can offer QUICv2. Bug 2040450 enabled Compatible VN on **Firefox Nightly 154** via the pref `network.http.http3.version_negotiation.enabled` (Nightly-only at first). Bug 2059947 later set that pref to `true` on **all channels** (Firefox 155). When the pref is true, the client offers `Version::all()` (v2 preferred, then v1); when false, v1 only. The *server* still chooses:

```
Client offers: [v2, v1]     Server speaks only v1  → connection uses v1
Client offers: [v2, v1]     Server prefers v2        → Compatible VN → v2
Client offers: [v1]         Server prefers v2        → stays v1 (client never offered v2)
```

Try this (browser)

1. Firefox 155+ (or Nightly 154+ with the pref true).
2. Visit an HTTP/3 origin you control that advertises QUICv2 (or a public origin known to prefer v2).
3. `about:networking#httpconn` (or your TLS/QUIC debugger) — confirm HTTP/3 and note the negotiated QUIC version if exposed.
4. On Nightly builds that still gate the pref, toggle `network.http.http3.version_negotiation.enabled` and compare.

Server-side (conceptual checklist, library-specific):

- Advertise both version codepoints in Compatible VN.
- Emit `version_information` on every v2 (and dual-stack) handshake.
- Log Original vs Negotiated Version.
- Keep issuing version-scoped session tickets.
- Roll versions through Acceptable → Offered → Fully Deployed with 1 MSL between steps (RFC 9368 §5) so downgrade detection does not false-positive during fleet upgrades.

Libraries commonly cited for dual-version support include **quiche**, **quic-go**, and browser stacks built on **neqo** / NSS-adjacent HTTP/3 plumbing. Always check the *current* release notes for your pin — Compatible VN and v2 landed at different times in different stacks.

Mental model — one diagram

```
                      Incompatible VN (VN packet, +1 RTT)
first flight
(Chosen Version)      Compatible VN (convert flight, 0 RTT)


                     Negotiated Version
                     (Handshake + 1-RTT live here)
```

The trap

- **Assuming the Version field integer "2".** The wire value is `0x6b3343cf`. Fingerprinting scripts that look for `version == 2` will miss every real v2 packet.

- **Enabling v2 only on the client.** Without servers that select v2 from Available Versions, you negotiate v1 forever and learn nothing.
- **Reusing tickets across versions.** A v1 session ticket on a v2 attempt must be rejected (full handshake, no 0-RTT).
- **Skipping `version_information` on a v2 endpoint.** Mandatory; without it you lose downgrade protection and violate RFC 9369.
- **Expecting application-visible differences.** If your HTTP/3 metrics are identical on v1 and v2, that is success — the change is at the transport wire image.

The boring rule

Offer QUICv1 and QUICv2 together. Use Compatible VN so the upgrade is free when both sides agree. Keep v1 around for reachability. Authenticate negotiation with `version_information`. Measure which version you actually land on.

Sources

- RFC 9369 — QUIC Version 2: https://www.rfc-editor.org/rfc/rfc9369.html
- RFC 9368 — Compatible Version Negotiation for QUIC: https://www.rfc-editor.org/rfc/rfc9368.html
- RFC 9000 — QUIC: https://www.rfc-editor.org/rfc/rfc9000.html
- RFC 8999 — Version-Independent Properties of QUIC: https://www.rfc-editor.org/rfc/rfc8999.html
- Firefox Nightly 154 notes (Bug 2040450): https://www.firefox.com/en-US/firefox/154.0a1/releasenotes/
- Bug 2040450 — Enable QUIC version negotiation: https://bugzilla.mozilla.org/show_bug.cgi?id=2040450
- Bug 2059947 — Enable QUIC v2 VN on all channels (Firefox 155): https://bugzilla.mozilla.org/show_bug.cgi?id=2059947

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft)

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft)

A dual-stack name resolves to both AAAA and A records. If the client tries them one at a time, a single broken AAAA record (a typo in the zone, a blackholed prefix, a dead load balancer) costs every user the full TCP SYN timeout before IPv4 is even tried. **Happy Eyeballs** fixes this on the client: start with the preferred address, and if it has not answered within a short delay, start the next one *in parallel*. The first handshake to finish wins. Version 2 is RFC 8305 (2017). **Version 3** (`draft-ietf-happy-happyeyeballs-v3-04`, July 2026) folds in SVCB/HTTPS records, so the race now also chooses *protocols* (QUIC vs TCP, ECH or not), not just address families. This chapter builds the failure in a two-namespace lab, measures the race, and explains what v3 changes.

Learning goals

- Reproduce a broken AAAA record and measure its cost with and without Happy Eyeballs
- Read the race in a packet capture: one SYN, a 200–250 ms gap, a second SYN
- Tell a *silent* failure (drop) from a *loud* one (RST), and explain why only the silent one needs racing
- Name the v3 timers (Resolution Delay, Connection Attempt Delay, and its bounds) and say what SVCB adds
- Explain why Happy Eyeballs **hides** IPv6 outages, and add a per-family health check

Lab platform: Linux 6.12, iproute2, nftables, curl 8.14.1, Python 3.13 (`asyncio`), tcpdump. All commands need root. Every output below is pasted from a run.

Concepts in one page

```
client (cli)  2001:db8::20 / 192.0.2.20          server (srv)  one veth
                                                  2001:db8::10   healthy AAAA
         "DNS" (hosts)                            2001:db8::bad  SYN silently DROPPED
         www.lab AAAA 2001:db8::bad               2001:db8::dead SYN answered with RST
         www.lab A    192.0.2.10                  192.0.2.10     healthy A
                                                  http.server :8080 on [::] (dual-stack)

HEv2/v3 connection race:   t=0     SYN → preferred (IPv6)
                           t=Δ     no answer yet → SYN → next family (IPv4)   Δ   250 ms (re
                           first SYN-ACK wins, the others are cancelled
```

Build the lab

```
#!/usr/bin/env bash
# lab.sh - a dual-stack server whose second IPv6 address silently eats SYNs
set -euo pipefail
ip netns add srv; ip netns add cli
ip link add s0 netns srv type veth peer name c0 netns cli
ip -n srv addr add 192.0.2.10/24 dev s0
ip -n srv addr add 2001:db8::10/64 dev s0 nodad      # healthy AAAA
ip -n srv addr add 2001:db8::bad/64 dev s0 nodad     # broken AAAA (blackholed below)
ip -n srv addr add 2001:db8::dead/64 dev s0 nodad    # refused AAAA (RST below)
ip -n cli addr add 192.0.2.20/24 dev c0
ip -n cli addr add 2001:db8::20/64 dev c0 nodad
for n in srv cli; do ip -n $n link set lo up; done
ip -n srv link set s0 up; ip -n cli link set c0 up
ip netns exec srv /usr/sbin/nft -f - <<'NFT'
table inet lab {
  chain input {
    type filter hook input priority 0; policy accept;
    ip6 daddr 2001:db8::bad tcp dport 8080 drop
    ip6 daddr 2001:db8::dead tcp dport 8080 reject with tcp reset
  }
}
NFT
ip netns exec srv python3 -m http.server 8080 --bind :: >/dev/null 2>&1 &
sleep 1
```

```
# ./lab.sh
# mkdir -p /etc/netns/cli
# printf '2001:db8::bad www.lab\n192.0.2.10 www.lab\n' > /etc/netns/cli/hosts
```

`ip netns exec cli` bind-mounts `/etc/netns/cli/hosts` over `/etc/hosts`, so inside the client namespace `www.lab` resolves to the broken AAAA first and the healthy A second. That is the same order RFC 6724 address selection gives for a real dual-stack name.

Drill 1: three kinds of AAAA, one client that races

curl implements Happy Eyeballs with a default 200 ms head start for the first family. `--resolve` lets us swap the AAAA without touching the hosts file:

```
# C() { ip netns exec cli curl -s -o /dev/null -w '%{remote_ip} connect=%{time_connect}s\n'
# for a in 2001:db8::10 2001:db8::bad 2001:db8::dead; do echo "== AAAA $a"; $C --resolve "ww
== AAAA 2001:db8::10
2001:db8::10 connect=0.000232s
== AAAA 2001:db8::bad
```

```
192.0.2.10 connect=0.200484s
== AAAA 2001:db8::dead
192.0.2.10 connect=0.000122s
```

A healthy AAAA wins at once. A **refused** AAAA (RST) fails immediately, so IPv4 starts without waiting. A **silent** AAAA costs exactly the head start, about 200 ms, and then IPv4 wins. That cost is the price of preferring IPv6, and it is bounded.

The head start is a tunable, and it is the whole cost:

```
# $C --happy-eyeballs-timeout-ms 1000 --resolve 'www.lab:8080:[2001:db8::bad],192.0.2.10' ht
192.0.2.10 connect=1.001154s
```

Drill 2: the same name, a client that does not race

```
# seq.py - what a client without Happy Eyeballs does: try each address in order
import socket, time
addrs = [("2001:db8::bad", 8080), ("192.0.2.10", 8080)]
t0 = time.monotonic()
for host, port in addrs:
    fam = socket.AF_INET6 if ":" in host else socket.AF_INET
    s = socket.socket(fam, socket.SOCK_STREAM); s.settimeout(5)
    try:
        s.connect((host, port)); print(f"connected {host} after {time.monotonic()-t0:.3f}s")
    except OSError as e:
        print(f"{host}: {e!r} after {time.monotonic()-t0:.3f}s")
    finally:
        s.close()
```

```
# ip netns exec cli python3 seq.py
2001:db8::bad: TimeoutError('timed out') after 5.005s
connected 192.0.2.10 after 5.006s
```

The sequential client pays its *whole* connect timeout. Here that is 5 s because the script sets it. A client with no timeout pays the kernel's SYN retry budget (`net.ipv4.tcp_syn_retries`, about two minutes by default). This is the outage users report as "the site is slow, but only for some people".

`asyncio` ships both behaviours. Only one of them is on by default:

```
# he.py - asyncio with and without Happy Eyeballs against the broken name
import asyncio, socket, sys, time

async def attempt(delay):
    t0 = time.monotonic()
    try:
```

```
        r, w = await asyncio.wait_for(
            asyncio.open_connection("www.lab", 8080, happy_eyeballs_delay=delay), 3)
        peer = w.get_extra_info("peername")[0]
        w.close(); await w.wait_closed()
        return f"connected to {peer} in {time.monotonic()-t0:.3f}s"
    except TimeoutError:
        return f"gave up after {time.monotonic()-t0:.3f}s"

async def main():
    for _, _, _, _, sa in socket.getaddrinfo("www.lab", 8080, type=socket.SOCK_STREAM):
        print("resolver order:", sa[0])
    print("happy_eyeballs_delay=None :", await attempt(None))
    print("happy_eyeballs_delay=0.25 :", await attempt(0.25))

asyncio.run(main())
```

```
# ip netns exec cli python3 he.py
resolver order: 2001:db8::bad
resolver order: 192.0.2.10
happy_eyeballs_delay=None : gave up after 3.004s
happy_eyeballs_delay=0.25 : connected to 192.0.2.10 in 0.252s
```

`happy_eyeballs_delay=None` (the default for `open_connection` and `loop.create_connection`) is sequential. Passing `0.25`, the RFC 8305 recommended value, turns the race on. If you write connection code on top of a socket API, check which of the two you are getting.

Drill 3: read the race on the wire

```
# ip netns exec cli tcpdump -i c0 -n -tt 'tcp[tcpflags] & tcp-syn != 0 or (ip6 and ip6[53] &
# ip netns exec cli curl -s -o /dev/null http://www.lab:8080/
+0.000s IP6 2001:db8::20.43750 > 2001:db8::bad.8080: Flags [S], seq 329770494, win 64800, …
+0.201s IP 192.0.2.20.58954 > 192.0.2.10.8080: Flags [S], seq 1595341819, win 64240, …
+0.201s IP 192.0.2.10.8080 > 192.0.2.20.58954: Flags [S.], seq 694257496, ack 1595341820, …
```

(Timestamps are rebased to the first SYN.) One IPv6 SYN goes out and gets no reply. 201 ms later the IPv4 SYN goes out and is answered in microseconds. When you suspect Happy Eyeballs is masking something, this three-line pattern is what you look for in a client-side capture.

What version 3 changes

HEv2 raced *addresses*. The v3 draft races *endpoints*: an address **plus** the protocol and security properties learned from SVCB/HTTPS records (RFC 9460). The draft's main mechanics are:

Stage	v3 behaviour	Default
Queries	Send HTTPS/SVCB, then AAAA, then A, back to back	—
Resolution Delay	After A arrives first, wait briefly for AAAA and SVCB	50 ms
Grouping	Group by ALPN/ECH, then by SVCB priority, then sort by RFC 6724 and RTT history, interleaving families	Preferred Address Family Count 1
Connection Attempt Delay	Gap between attempts, ideally the first SYN/Initial retransmit time	250 ms (min 100 ms, never < 10 ms; max 2 s)
Success	TCP-only: handshake done. TCP+TLS: wait for TLS. QUIC: its one handshake	—
IPv6-only	Synthesise NAT64 addresses from PREF64 (RFC 8781) for IPv4 literals and A answers	Last-resort synthesis delay 2 s

Two practical consequences for operators. First, an HTTPS record with `alpn="h3"` and `ipv6hint` lets a v3 client start a QUIC attempt *before* the AAAA answer arrives. A wrong hint is therefore a new way to publish a broken address. Second, with encrypted or DNSSEC-validated DNS and ECH support, clients **must wait** for the SVCB answer before sending a ClientHello. A slow HTTPS-record path then becomes connection latency, not just a missed optimisation.

The trap

Happy Eyeballs makes a broken IPv6 path *look healthy*. Drill 1 is a 100% IPv6 outage that shows up as a 200 ms slowdown. RFC 8305 and the v3 draft both warn that the race hides operational problems. Monitoring that connects "to the name" will stay green. Probe each family on its own:

```
# ip netns exec cli curl -s -o /dev/null --connect-timeout 3 -6 http://www.lab:8080/; echo "
exit=28
# ip netns exec cli curl -s -o /dev/null --connect-timeout 3 -4 http://www.lab:8080/; echo "
exit=0
```

Exit 28 is curl's timeout. Alert on `-6` and `-4` separately.

The boring rule

Clients race: use a library that implements Happy Eyeballs, or turn it on (`happy_eyeballs_delay=0.25`). Servers publish only addresses they can serve, *including* SVCB `ipv6hint`/`ipv4hint`. Monitoring never trusts the race and probes IPv6 and IPv4 separately. A silent drop is the failure to design for, because an RST is already fast.

Try this

1. Change the nftables rule for `2001:db8::bad` from `drop` to `reject with icmpv6 type admin-prohibited`. Predict the curl `connect=` time first, then run it.
2. Put two broken AAAA records before the A record in `--resolve`. Does curl try the second IPv6 address before falling back to IPv4? Compare with the v3 "interleave families" rule.
3. Set `net.ipv4.tcp_syn_retries=1` in `cli` and rerun `seq.py` with `settimeout(None)`. How long does the sequential client wait now?
4. Write the HTTPS record you would publish for `www.lab` with `alpn="h3,h2"` and both hints. Which v3 group does each address land in?

Sources

- RFC 8305, *Happy Eyeballs Version 2: Better Connectivity Using Concurrency*: https://www.rfc-editor.org/rfc/rfc8305
- draft-ietf-happy-happyeyeballs-v3-04 (July 2026): https://datatracker.ietf.org/doc/draft-ietf-happy-happyeyeballs-v3/
- RFC 9460, SVCB and HTTPS resource records: https://www.rfc-editor.org/rfc/rfc9460
- RFC 6724, Default Address Selection for IPv6: https://www.rfc-editor.org/rfc/rfc6724
- curl `--happy-eyeballs-timeout-ms`: https://curl.se/docs/manpage.html
- Python `asyncio loop.create_connection(happy_eyeballs_delay=…)`: https://docs.python.org/3/library/asyncio-eventloop.html

QUIC Load Balancing with nftables: Routing on Connection IDs

QUIC Load Balancing with nftables: Routing on Connection IDs

A TCP load balancer can lean on the kernel. Pick a backend on the SYN, DNAT it, and conntrack keeps every later segment of that 4-tuple on the same backend. QUIC breaks both assumptions behind that recipe. A QUIC connection is named by its **connection ID** (CID), not by its 4-tuple. A client can change its address and port mid-connection (migration, or a NAT rebinding it never notices). Two connections can also share one 4-tuple. A load balancer that routes on the 4-tuple sends a migrated connection to a server that has never heard of it. This chapter builds a QUIC load balancer from nothing but nftables rules: no proxy, no kernel module, no eBPF. It routes **handshakes** by client address and **established connections** by a server ID that each backend writes into every CID it issues. Then it drains a backend, deregisters it, and shows the one return-path rule that makes the whole thing look fine in a lab and fail at the edge.

Learning goals

- Read the QUIC wire image a load balancer is allowed to rely on (RFC 8999) and find the CID with `@th` raw payload matches
- Explain why the ruleset needs **conntrack zones**, not just DNAT, and show what breaks without them
- Encode a server ID in the CID the way QUIC-LB lays it out, and route on it
- Drain and deregister a backend with nothing but map element edits, and know what the client sees afterwards
- Explain why every backend must share the **stateless reset** key
- Put the return-path zone rule in the right hook, and prove it with an edge filter

Lab platform: Linux 6.12, nftables 1.1.3, iproute2, Python 3.13, Go 1.27.2 with quic-go v0.63.0, tcpdump 4.99.5. All commands need root. Every output below is the captured output of one run of the listings shown.

Mental model

```
clients 198.51.100.10-17          load balancer (qlb)                    backends (qbe)
              l0     VIP 198.51.100.1:443     l1     be1 10.0.0.11:8080  server ID 0x01
                                                                    be2 10.0.0.12:8081  serv
                                                                    be3 10.0.0.13:8082  serv
                                                                    be4 10.0.0.14:8083  (joi
```

```
prerouting, priority raw (-300): decide the backend, put it in the packet mark, then in a c
  long header (first byte & 0x80)       → @flows[client IP]  or  random slot → @slots → bac
  short header, DCID[0] & 0xe0 == 0     → @sids[DCID[1]]  (server ID)       → backend ID
  short header, unknown server ID       → treated like a new flow
  from a backend (iif l1)               → @replies[saddr . sport]           → backend ID
  ct zone = backend ID
prerouting, conntrack (-200): look up / create the entry *inside that zone*
prerouting, priority dstnat (-100): new entry → dnat to @backends[zone]
postrouting, srcnat (100): replies get the VIP back as their source
```

Three facts carry the design:

1. **What a middlebox may parse.** RFC 8999 (QUIC invariants) fixes, for every QUIC version, the high bit of the first byte (1 = long header, 0 = short header), the long header's version and CID length fields, and the fact that a short header's Destination CID starts right after the first byte. A short header does *not* carry the CID length. RFC 9000 §5.1 says a load balancer that routes on CIDs has to agree on a fixed length or an encoding with the servers. Nothing else in the packet is readable: the rest is encrypted.
2. **Who chooses CIDs.** The client's first Initial carries a random DCID of at least 8 bytes (RFC 9000 §7.2). The server answers with a CID *it* chose, and from then on the client sends to the server's CIDs. So handshake packets cannot be routed by CID. Everything after the handshake can be, if the server encodes "it's me" into its CIDs.
3. **Why zones.** NAT is decided once, on the first packet of a conntrack entry. Every later packet with the same 4-tuple follows that entry, whatever its CID says. Putting each backend in its own conntrack zone gives the same 4-tuple one entry *per backend*, so the CID decides again on every packet. A reply has to be looked up in the same zone, which is why the return path needs its own rule.

In nftables, `@th,OFFSET,LEN` reads raw bits relative to the transport header. A UDP header is 8 bytes (64 bits), so:

Match	Bits	What it is in a short-header packet
`@th,64,8`	UDP payload byte 0	QUIC first byte; `& 0x80` is the header form
`@th,72,8`	payload byte 1	DCID byte 0 (QUIC-LB "first octet": 3 config rotation bits + 5 bits)
`@th,80,8`	payload byte 2	DCID byte 1: the server ID

The CID layout

QUIC-LB (`draft-ietf-quic-load-balancers-21`, the QUIC working group's draft for routable CIDs) reserves the first CID octet. Its top three bits are a **config rotation** codepoint, and `0b111` means "unroutable" (§3). The server ID follows, then a nonce of at least 4 bytes (§5.2). This lab uses the draft's *unencrypted* layout with an 8-byte CID: `[000 + 5 random bits][server ID][6`

`random bytes]`. The load balancer reads one byte at a fixed offset. That is exactly what nftables is good at.

Build the lab

```
#!/usr/bin/env bash
# lab.sh - clients, one load balancer, and the backends, in three namespaces
set -euo pipefail
for n in qcli qlb qbe; do ip netns add $n; ip -n $n link set lo up; done
ip link add c0 netns qcli type veth peer name l0 netns qlb       # client side
ip link add l1 netns qlb  type veth peer name b0 netns qbe       # backend side
for a in 10 11 12 13 14 15 16 17; do ip -n qcli addr add 198.51.100.$a/24 dev c0; done
ip -n qlb addr add 198.51.100.1/24 dev l0                        # the VIP
ip -n qlb addr add 10.0.0.1/24 dev l1
for a in 11 12 13 14; do ip -n qbe addr add 10.0.0.$a/24 dev b0; done   # be1..be4
ip -n qcli link set c0 up; ip -n qlb link set l0 up; ip -n qlb link set l1 up; ip -n qbe lin
ip -n qcli route add default via 198.51.100.1
ip -n qbe route add default via 10.0.0.1
ip netns exec qlb sysctl -qw net.ipv4.ip_forward=1
```

The load balancer's whole configuration is one file. Six maps hold every per-backend fact, so adding, draining and removing a backend are element edits, never rule edits:

```
# lb-quic.nft - route QUIC by connection ID with nftables alone
table ip lb-quic {
    map slots {                     # new connections: random slot -> backend ID
        typeof meta mark : meta mark
        elements = { 0 : 1, 1 : 2, 2 : 3 }
    }

    map sids {                      # server ID byte in the DCID -> backend ID
        typeof @th,80,8 : meta mark
        elements = { 0x01 : 1, 0x02 : 2, 0x03 : 3 }
    }

    map flows {                     # client IP -> backend ID while handshakes run
        typeof ip saddr : meta mark
        size 65536
        flags dynamic,timeout
        timeout 5s
    }

    map backends {                  # conntrack zone -> backend address
        typeof ct zone : ip daddr . udp dport
        elements = { 1 : 10.0.0.11 . 8080, 2 : 10.0.0.12 . 8081, 3 : 10.0.0.13 . 8082 }
```

```
    }

    map replies {                   # backend address -> backend ID (return path)
        typeof ip saddr . udp sport : meta mark
        elements = { 10.0.0.11 . 8080 : 1, 10.0.0.12 . 8081 : 2, 10.0.0.13 . 8082 : 3 }
    }

    map zone-of {                   # backend ID -> conntrack zone
        typeof meta mark : ct zone
        elements = { 1 : 1, 2 : 2, 3 : 3, 4 : 4, 5 : 5, 6 : 6 }
    }

    chain raw {
        type filter hook prerouting priority raw; policy accept;
        iifname "l1" meta mark set ip saddr . udp sport map @replies
        iifname "l0" udp dport 443 @th,64,8 & 0x80 == 0x00 jump short-header
        iifname "l0" udp dport 443 @th,64,8 & 0x80 == 0x80 jump new-flow
        ct zone set meta mark map @zone-of
    }

    chain short-header {            # DCID = [config rotation 000|5 bits][server ID][nonce]
        @th,72,8 & 0xe0 != 0x00 goto new-flow
        meta mark set @th,80,8 map @sids return
        goto new-flow
    }

    chain new-flow {                # long header, or a CID no registered backend owns
        meta mark set ip saddr map @flows return
        meta mark set numgen random mod 3
        meta mark set meta mark map @slots update @flows { ip saddr : meta mark }
    }

    chain to-backend {
        type nat hook prerouting priority dstnat; policy accept;
        meta l4proto udp dnat ip to ct zone map @backends
    }
}
```

Two details are deliberate:

- `short-header` starts with `goto`, not `jump`, for unroutable config rotation bits. A `jump` would return into `short-header` and keep evaluating its rules.
- The packet mark carries the **backend ID**, and `zone-of` turns it into a zone at the end of `raw`. Handshake pins in `@flows` store the backend ID, not the random slot, so a pinned handshake keeps its backend while the slots are remapped.

Two probe scripts stand in for clients and servers before real QUIC arrives. `qsend.py` sends one QUIC-shaped datagram: a long-header Initial (version 1, padded to 1200 bytes like a real client

Initial) or a short header with a chosen DCID. `backend.py` answers with what it saw. They exercise the steering, not QUIC itself.

```
# qsend.py - send one QUIC-shaped datagram to the VIP and print the reply
#   python3 qsend.py long|short DCID_HEX [SRC_IP] [SRC_PORT]
import socket, sys
kind, dcid = sys.argv[1], bytes.fromhex(sys.argv[2])
src = (sys.argv[3] if len(sys.argv) > 3 else "198.51.100.10",
       int(sys.argv[4]) if len(sys.argv) > 4 else 0)
if kind == "long":   # Initial, version 1, DCID, empty SCID
    pkt = bytes([0xc0]) + (1).to_bytes(4, "big") + bytes([len(dcid)]) + dcid + b"\x00"
else:                # 1-RTT: fixed bit set, then the DCID the server chose
    pkt = bytes([0x40]) + dcid
pkt += b"\x00" * (1200 - len(pkt))           # QUIC pads client Initials to 1200
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(src); s.settimeout(1)
s.sendto(pkt, ("198.51.100.1", 443))
try:
    data, peer = s.recvfrom(2048)
    print(f"{s.getsockname()[1]} <- {peer[0]}:{peer[1]}  {data.decode()}")
except TimeoutError:
    print(f"{s.getsockname()[1]} <- (no reply)")
```

```
# backend.py - a UDP "QUIC server" that only reports what it received
import socket, sys
name, addr, port = sys.argv[1], sys.argv[2], int(sys.argv[3])
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind((addr, port))
while True:
    data, peer = s.recvfrom(2048)
    if data[0] & 0x80:                       # long header (RFC 8999 §5.1)
        dcid = data[6:6 + data[5]]
        kind = "long "
    else:                                    # short header (RFC 8999 §5.2)
        dcid = data[1:9]
        kind = "short"
    s.sendto(f"{name} got {kind} dcid={dcid.hex()} from {peer[0]}:{peer[1]}".encode(), peer)
```

Real QUIC uses one Go program as backend and client. A backend generates every CID with its server ID in byte 1 and shares one stateless reset key with the others. The client can migrate to a new local port, or idle and then send again:

```
// main.go - a quic-go backend that puts its server ID in every CID, and a client
//
//  qlb server ID ADDR:PORT      e.g. qlb server 1 10.0.0.11:8080
//  qlb client [-src IP] [-migrate] [-pause 7s [-pad 200]] VIP:PORT
package main
```

```
import (
    "context"
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/tls"
    "crypto/x509"
    "crypto/x509/pkix"
    "flag"
    "fmt"
    "io"
    "log"
    "math/big"
    "net"
    "os"
    "strconv"
    "strings"
    "time"

    "github.com/quic-go/quic-go"
)

// QUIC-LB unencrypted layout (draft-ietf-quic-load-balancers-21 §5.2):
// [config rotation 000 + 5 random bits][server ID][6-byte nonce]
type lbCIDs struct{ id byte }

func (g lbCIDs) ConnectionIDLen() int { return 8 }
func (g lbCIDs) GenerateConnectionID() (quic.ConnectionID, error) {
    b := make([]byte, 8)
    if _, err := rand.Read(b); err != nil {
        return quic.ConnectionID{}, err
    }
    b[0] &= 0x1f // config rotation codepoint 0
    b[1] = g.id  // server ID
    return quic.ConnectionIDFromBytes(b), nil
}

// Every backend shares this key, so any of them can reset any connection.
var resetKey = quic.StatelessResetKey{0x6c, 0x61, 0x62}

func server(id byte, addr string, ownKey bool) {
    udp, err := net.ListenUDP("udp4", mustAddr(addr))
    check(err)
    key := resetKey
    if ownKey { // the mistake: every backend invents its own reset key
        rand.Read(key[:])
```

```
    }
    tr := &quic.Transport{Conn: udp, ConnectionIDGenerator: lbCIDs{id}, StatelessResetKey: &
    ln, err := tr.Listen(selfSigned(), &quic.Config{})
    check(err)
    for {
        conn, err := ln.Accept(context.Background())
        check(err)
        go func() {
            for {
                str, err := conn.AcceptStream(context.Background())
                if err != nil {
                    return
                }
                msg, _ := io.ReadAll(str)
                fmt.Fprintf(str, "be%d: %s (client %s)", id, msg, conn.RemoteAddr())
                str.Close()
            }
        }()
    }
}

func ask(conn *quic.Conn, msg string) string {
    str, err := conn.OpenStreamSync(context.Background())
    check(err)
    str.Write([]byte(msg))
    str.Close()
    reply, err := io.ReadAll(str)
    if err != nil {
        return "error: " + err.Error()
    }
    return string(reply)
}

func client(vip, src string, migrate bool, pause time.Duration, pad int) {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    tr := &quic.Transport{Conn: listen(src)}
    conn, err := tr.Dial(ctx, mustAddr(vip), &tls.Config{InsecureSkipVerify: true, NextProto
        &quic.Config{MaxIdleTimeout: 10 * time.Second})
    check(err)
    fmt.Printf("port %d: %s\n", port(tr), ask(conn, "hello"))
    if migrate { // new UDP socket = new 4-tuple, same connection
        tr2 := &quic.Transport{Conn: listen(src)}
        path, err := conn.AddPath(tr2)
        check(err)
        check(path.Probe(ctx))
        check(path.Switch())
```

```
        fmt.Printf("port %d: %s\n", port(tr2), ask(conn, "after migration"))
    }
    if pause > 0 { // meanwhile the backend that owns this connection goes away
        time.Sleep(pause)
        t0 := time.Now()
        reply := ask(conn, "still there?"+strings.Repeat(".", pad))
        fmt.Printf("port %d after %v: %s (%.1fs)\n", port(tr), pause, reply, time.Since(t0).
    }
    conn.CloseWithError(0, "")
}

func main() {
    migrate := flag.Bool("migrate", false, "switch to a new local port mid-connection")
    pause := flag.Duration("pause", 0, "wait, then use the connection again")
    ownKey := flag.Bool("own-key", false, "server: use a private stateless reset key")
    src := flag.String("src", "", "client: source IP")
    pad := flag.Int("pad", 0, "client: bytes of padding on the request sent after -pause")
    if len(os.Args) < 2 {
        log.Fatal("usage: qlb server ID ADDR:PORT | qlb client [-src IP] [-migrate] [-pause
    }
    flag.CommandLine.Parse(os.Args[2:]) // flags follow the subcommand
    a := append([]string{os.Args[1]}, flag.Args()...)
    switch {
    case len(a) == 3 && a[0] == "server":
        id, err := strconv.Atoi(a[1])
        check(err)
        server(byte(id), a[2], *ownKey)
    case len(a) == 2 && a[0] == "client":
        client(a[1], *src, *migrate, *pause, *pad)
    default:
        log.Fatal("usage: qlb server ID ADDR:PORT | qlb client [-src IP] [-migrate] [-pause
    }
}

func port(t *quic.Transport) int { return t.Conn.LocalAddr().(*net.UDPAddr).Port }

func listen(src string) *net.UDPConn {
    c, err := net.ListenUDP("udp4", &net.UDPAddr{IP: net.ParseIP(src)})
    check(err)
    return c
}

func mustAddr(s string) *net.UDPAddr {
    a, err := net.ResolveUDPAddr("udp4", s)
    check(err)
    return a
}
```

```
func selfSigned() *tls.Config { // lab only: throwaway key, client skips verification
    key, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    tmpl := &x509.Certificate{SerialNumber: big.NewInt(1), Subject: pkix.Name{CommonName: "l
        NotBefore: time.Now().Add(-time.Hour), NotAfter: time.Now().Add(24 * time.Hour)}
    der, err := x509.CreateCertificate(rand.Reader, tmpl, tmpl, &key.PublicKey, key)
    check(err)
    return &tls.Config{Certificates: []tls.Certificate{{Certificate: [][]byte{der}, PrivateK
}

func check(err error) {
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
}
```

```
# build (quic-go v0.63.0 needs Go 1.26 or newer; GOTOOLCHAIN fetches it)
cd qgo                                  # main.go lives here
export GOTOOLCHAIN=go1.27.2
go mod init qlb
go get github.com/quic-go/quic-go@v0.63.0
go mod tidy
go build -o ../qlb .
```

```
# uname -r
6.12.94+
# nft --version
nftables v1.1.3 (Commodore Bullmoose #4)
# go version -m qlb | grep -E '^qlb|quic-go'
qlb: go1.27.2
    dep github.com/quic-go/quic-go  v0.63.0 h1:LIFGHI4PFUhhw2dDD1ARHdCff143ffMHwZtbnbuJ78A=
# python3 --version
Python 3.13.5
```

Why not paste the published ruleset?

The design here follows Marten Seemann's April 2026 post (see Sources). He installs his rules through the Go `google/nftables` netlink library and warns that the `nft` CLI cannot display them. The CLI cannot load his listing either. Saved as `lb-udp-seemann.nft` and checked with `nft -c`:

```
# ./lab.sh
# ip netns exec qlb nft -c -f lb-udp-seemann.nft
lb-udp-seemann.nft:30:78-80: Error: syntax error, unexpected map, expecting '}'
        ct zone set meta mark map @backend-zones add @flows { ip saddr : meta mark map @back
```

```
                                                                                ^^^
lb-udp-seemann.nft:28:15-33: Error: datatype mismatch: expected integer, expression has type
        ct zone set ip saddr map @flows return
        ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
lb-udp-seemann.nft:35:33-36: Error: transport protocol mapping is only valid after transport
        ct zone 3 dnat to 192.168.0.3:8082
                  ~~~~                ^^^^
lb-udp-seemann.nft:36:33-36: Error: transport protocol mapping is only valid after transport
        ct zone 2 dnat to 192.168.0.2:8081
                  ~~~~                ^^^^
lb-udp-seemann.nft:37:33-36: Error: transport protocol mapping is only valid after transport
        ct zone 1 dnat to 192.168.0.1:8080
                  ~~~~                ^^^^
# ip netns exec qlb nft -f lb-quic.nft
```

Published listing (installed over netlink)	This chapter (loads with `nft -f`)	Why
maps whose value type is `mark` feeding `ct zone set`	`typeof … : ct zone` for zone-valued maps	CLI type check: "expected integer, expression has type packet mark"
`add @flows { ip saddr : meta mark map @backend-zones }`	backend ID travels in the mark, `@zone-of` maps it to a zone	The CLI grammar has no map lookup inside a set element. In this lab, writing `ct zone` itself as the element value loaded fine and inserted nothing, so the mark carries it
`ct zone 3 dnat to 192.168.0.3:8082`, one rule per backend	`meta l4proto udp dnat ip to ct zone map @backends`	A port in `dnat to` needs a transport-protocol match. A map turns N rules into one
per-backend rules in the short-header and output chains	element maps `@sids`, `@backends`, `@replies`	Register, drain and deregister become element edits applied atomically by `nft -f`
server ID in CID byte 0	config rotation in byte 0, server ID in byte 1	QUIC-LB reserves the first octet
return path set in an `output` hook chain	return path set in `raw` prerouting, keyed on the backend address	The `output` hook only sees packets the LB host generates. Backends on their own hosts reply *through* the LB (see The trap)

Drill 1: handshakes stick to a client address

```
# for i in 1 2 3; do ip netns exec qbe python3 backend.py be$i 10.0.0.1$i 808$((i-1)) & done
# S() { ip netns exec qcli python3 qsend.py "$@"; }
# for i in 1 2 3; do S long $(openssl rand -hex 8); done
55715 <- 198.51.100.1:443  be2 got long  dcid=4248938fa0c5ae57 from 198.51.100.10:55715
34938 <- 198.51.100.1:443  be2 got long  dcid=a9807a6bd375cecc from 198.51.100.10:34938
53655 <- 198.51.100.1:443  be2 got long  dcid=7d4a699acc7a4368 from 198.51.100.10:53655
# for i in 1 2 3; do S long $(openssl rand -hex 8) 198.51.100.11; done
60861 <- 198.51.100.1:443  be3 got long  dcid=19b6d724a3379218 from 198.51.100.11:60861
45629 <- 198.51.100.1:443  be3 got long  dcid=4b1e25e33ff2ddd5 from 198.51.100.11:45629
58044 <- 198.51.100.1:443  be3 got long  dcid=323209b7e60d3201 from 198.51.100.11:58044
# ip netns exec qlb nft list map ip lb-quic flows
table ip lb-quic {
    map flows {
        typeof ip saddr : meta mark
        size 65536
        flags dynamic,timeout
        timeout 5s
        elements = { 198.51.100.10 expires 4s872ms : 0x00000002,
                     198.51.100.11 expires 4s940ms : 0x00000003 }
    }
}
```

Every long-header packet from `198.51.100.10` reached be2 and every one from `.11` reached be1, each with a fresh random DCID. The first packet from each address took the random slot, and the `update` stored the result in `@flows`. Later packets found the pin and returned early. A real handshake sends several long-header packets (Initial, Handshake) before the client switches to the server's CID, and all of them have to reach the server that holds the handshake state. The 5-second timeout is the handshake budget. It also means two clients behind one NAT address that start handshakes within 5 s land on the same backend. Every reply came from `198.51.100.1:443`: conntrack undid the DNAT on the way out.

Drill 2: established packets follow the server ID

```
# for sid in 01 02 03; do S short 07${sid}a1b2c3d4e5f6; done
45811 <- 198.51.100.1:443  be1 got short dcid=0701a1b2c3d4e5f6 from 198.51.100.10:45811
44651 <- 198.51.100.1:443  be2 got short dcid=0702a1b2c3d4e5f6 from 198.51.100.10:44651
38289 <- 198.51.100.1:443  be3 got short dcid=0703a1b2c3d4e5f6 from 198.51.100.10:38289
# S short 0704a1b2c3d4e5f6      # server ID 4: nobody registered it
34962 <- 198.51.100.1:443  be2 got short dcid=0704a1b2c3d4e5f6 from 198.51.100.10:34962
# S short e702a1b2c3d4e5f6      # config rotation 0b111: unroutable by definition
60745 <- 198.51.100.1:443  be2 got short dcid=e702a1b2c3d4e5f6 from 198.51.100.10:60745
```

The server ID byte picked the backend, regardless of source port. Server ID `0x04` has no `@sids` entry, so the `sids` lookup failed and `goto new-flow` treated the packet like a handshake. It found the pin `198.51.100.10 → be2` from Drill 1, still inside its 5 s. Config rotation `0b111` (`0xe7`) never reached the `@sids` lookup and took the same fallback. QUIC-LB defines `0b111` as unroutable (§3.2), so the load balancer falls back instead of trusting byte 1.

Drill 3: two connections, one 4-tuple, and why zones exist

```
# S short 0701000000000001 198.51.100.10 40000
40000 <- 198.51.100.1:443  be1 got short dcid=0701000000000001 from 198.51.100.10:40000
# S short 0703000000000002 198.51.100.10 40000
40000 <- 198.51.100.1:443  be3 got short dcid=0703000000000002 from 198.51.100.10:40000
# ip netns exec qlb grep 'sport=40000 ' /proc/net/nf_conntrack
ipv4     2 udp      17 29 src=198.51.100.10 dst=198.51.100.1 sport=40000 dport=443 src=10.0.
ipv4     2 udp      17 29 src=198.51.100.10 dst=198.51.100.1 sport=40000 dport=443 src=10.0.
```

Same source port, same VIP, two CIDs, two backends. Conntrack holds two entries with identical original tuples, kept apart only by `zone=1` and `zone=3`. Now the same steering without zones:

```
# nozone.nft - the same CID steering, minus conntrack zones
table ip nozone {
    chain raw {
        type filter hook prerouting priority raw; policy accept;
        iifname "lo" udp dport 443 @th,80,8 0x01 meta mark set 1
        iifname "lo" udp dport 443 @th,80,8 0x03 meta mark set 3
    }
    chain to-backend {
        type nat hook prerouting priority dstnat; policy accept;
        meta l4proto udp meta mark 1 dnat to 10.0.0.11:8080
        meta l4proto udp meta mark 3 dnat to 10.0.0.13:8082
    }
}
```

```
# ip netns exec qlb nft delete table ip lb-quic
# ip netns exec qlb nft -f nozone.nft
# S short 0701000000000001 198.51.100.10 40001
40001 <- 198.51.100.1:443  be1 got short dcid=0701000000000001 from 198.51.100.10:40001
# S short 0703000000000002 198.51.100.10 40001
40001 <- 198.51.100.1:443  be1 got short dcid=0703000000000002 from 198.51.100.10:40001
# ip netns exec qlb nft delete table ip nozone
# ip netns exec qlb nft -f lb-quic.nft
```

The second packet carried be3's server ID and still went to be1. The mark said 3, but the 4-tuple already had a conntrack entry, and an existing entry is never handed to the NAT chain again. With zones, the CID chooses on every packet. Without them, the first packet chooses for the whole 4-tuple.

Drill 4: real QUIC, including migration

The probes go away and real quic-go servers take their ports:

```
# pkill -f backend.py
# for i in 1 2 3; do ip netns exec qbe ./qlb server $i 10.0.0.1$i:808$((i-1)) & done; sleep
# Q() { ip netns exec qcli ./qlb client "$@" 198.51.100.1:443; }
# for a in 10 11 12 13; do Q -src 198.51.100.$a; done
port 39756: be2: hello (client 198.51.100.10:39756)
port 43470: be1: hello (client 198.51.100.11:43470)
port 52985: be2: hello (client 198.51.100.12:52985)
port 35223: be2: hello (client 198.51.100.13:35223)
# Q -src 198.51.100.14 -migrate
port 33196: be3: hello (client 198.51.100.14:33196)
port 48439: be3: after migration (client 198.51.100.14:48439)
```

Each handshake was steered by `@flows`. After it, every 1-RTT packet carried a server-chosen CID with that server's ID in byte 1. The `-migrate` run opened a second UDP socket, so the connection got a new 4-tuple. It probed the path (`PATH_CHALLENGE`/`PATH_RESPONSE`, RFC 9000 §9), switched, and sent again. The new 4-tuple had no conntrack entry. `@sids` sent it to the same backend anyway, and be3 saw the client's new port.

Compare the TCP recipe applied to UDP, choosing a backend per 4-tuple:

```
# naive.nft - the TCP recipe applied to UDP: pick a backend per 4-tuple
table ip naive {
    map backends {
        type mark : ipv4_addr . inet_service
        flags constant
        elements = { 0 : 10.0.0.11 . 8080, 1 : 10.0.0.12 . 8081, 2 : 10.0.0.13 . 8082 }
    }
    chain to-backend {
        type nat hook prerouting priority dstnat; policy accept;
        udp dport 443 dnat ip to numgen random mod 3 map @backends
    }
}
```

```
# ip netns exec qlb nft delete table ip lb-quic
# ip netns exec qlb nft -f naive.nft
# for a in 10 11 12 13 14 15; do Q -src 198.51.100.$a -migrate; done
port 36487: be2: hello (client 198.51.100.10:36487)
error: context deadline exceeded
port 58578: be3: hello (client 198.51.100.11:58578)
port 44403: be3: after migration (client 198.51.100.11:44403)
port 60557: be3: hello (client 198.51.100.12:60557)
error: context deadline exceeded
port 57019: be2: hello (client 198.51.100.13:57019)
```

```
error: context deadline exceeded
port 45910: be2: hello (client 198.51.100.14:45910)
port 41195: be2: after migration (client 198.51.100.14:41195)
port 35883: be2: hello (client 198.51.100.15:35883)
error: context deadline exceeded
# ip netns exec qlb nft delete table ip naive
# ip netns exec qlb nft -f lb-quic.nft
```

Four of six migrations failed. The new path was a new 4-tuple, so `numgen` picked a backend at random. Two times out of six it happened to be the same server. The other four times the `PATH_CHALLENGE` landed on a backend with no state for that connection, and the probe ran out of its 5 s budget. On the Internet you don't choose when this happens: a NAT rebinding changes the client's port without asking.

Drill 5: drain be3, add be4, deregister be3, reset its clients

```
# drain-be3.nft - be4 joins; slot 2 now means be4, so be3 gets no new connections
delete element ip lb-quic slots { 2 }
add element ip lb-quic slots { 2 : 4 }
add element ip lb-quic sids { 0x04 : 4 }
add element ip lb-quic backends { 4 : 10.0.0.14 . 8083 }
add element ip lb-quic replies { 10.0.0.14 . 8083 : 4 }
```

```
# deregister-be3.nft - be3 is gone: its CIDs are no longer routable
delete element ip lb-quic sids { 0x03 }
delete element ip lb-quic backends { 3 }
delete element ip lb-quic replies { 10.0.0.13 . 8082 }
```

```
# sleep 6      # let earlier handshake pins in @flows expire
# ip netns exec qlb nft add element ip lb-quic flows '{ 198.51.100.10 : 3 }'   # pin the nex
# Q -src 198.51.100.10 -pause 7s -pad 200 & qa=$!; sleep 1
port 46079: be3: hello (client 198.51.100.10:46079)
# ip netns exec qbe ./qlb server 4 10.0.0.14:8083 & sleep 1
# ip netns exec qlb nft -f drain-be3.nft
# for a in 11 12 13 14 15 16; do Q -src 198.51.100.$a; done
port 57472: be4: hello (client 198.51.100.11:57472)
port 37147: be2: hello (client 198.51.100.12:37147)
port 36258: be2: hello (client 198.51.100.13:36258)
port 58225: be2: hello (client 198.51.100.14:58225)
port 34076: be2: hello (client 198.51.100.15:34076)
port 49176: be1: hello (client 198.51.100.16:49176)
# ip netns exec qlb nft -f deregister-be3.nft; pkill -f 'qlb server 3'
# wait $qa
port 46079 after 7s: error: received a stateless reset (0.0s)
```

The client on `198.51.100.10` was pinned to be3 before the drain (the first `add element` exists only to make the drill deterministic). `drain-be3.nft` swapped slot 2 from be3 to be4 in one transaction. The six handshakes that followed went to be1, be2 and be4, never be3, while be3 kept serving its connection. In this ruleset a client address already pinned to be3 in `@flows` would still reach it for up to 5 s, so give a drain at least the pin timeout before deregistering. Deregistering removed be3's server ID, address and return path. The pinned client's next packet then carried an unknown server ID, fell through to `new-flow`, and reached a live backend. That backend has no state for the CID, so it answered with a **stateless reset** (RFC 9000 §10.3). The client checked the reset token and closed at once: `received a stateless reset (0.0s)`.

The reset token is derived from the CID and a key. quic-go computes it as HMAC-SHA256 keyed with `Transport.StatelessResetKey`, and picks a random key if none is set (`stateless_reset.go`). Another backend can only produce the right token if it has the same key. Here is the same drill with every backend inventing its own key:

```
# pkill -f 'qlb server'; sleep 1
# for i in 1 2 3 4; do ip netns exec qbe ./qlb server -own-key $i 10.0.0.1$i:808$((i-1)) & d
# ip netns exec qlb nft delete table ip lb-quic; ip netns exec qlb nft -f lb-quic.nft
# ip netns exec qlb nft add element ip lb-quic flows '{ 198.51.100.10 : 3 }'
# Q -src 198.51.100.10 -pause 7s -pad 200 & qa=$!; sleep 1
port 54876: be3: hello (client 198.51.100.10:54876)
# ip netns exec qlb nft -f drain-be3.nft
# ip netns exec qlb nft -f deregister-be3.nft; pkill -f 'qlb server -own-key 3'
# wait $qa
port 54876 after 7s: error: timeout: no recent network activity (10.0s)
```

The reset still arrived, but its token didn't verify, so the client ignored it. The client retransmitted into the void until its 10 s idle timeout. A real client would hang for the negotiated idle timeout (quic-go's default is 30 s). `-pad 200` is in both drills because quic-go v0.63.0 does not answer packets of 42 bytes or less with a reset (`MinStatelessResetSize` in `transport.go`). RFC 9000 §10.3 permits answering those with a reset one byte shorter. An unpadded 41-byte request makes the outcome depend on packet sizes rather than on the key.

The trap

The return path looks optional because the lab can pass without it. Flush `@replies`, the equivalent of keeping the zone-restoring rule in an `output` chain while the backends live on other hosts:

```
# edge.nft - BCP 38 at the upstream: nothing leaves l0 with a source we don't own
table inet edge {
    chain egress {                  # after srcnat, so it sees the translated source
        type filter hook postrouting priority srcnat + 10; policy accept;
        oifname "l0" ip saddr != 198.51.100.0/24 counter drop
    }
}
```

```
# pkill -f 'qlb server'; sleep 1
# for i in 1 2 3; do ip netns exec qbe ./qlb server $i 10.0.0.1$i:808$((i-1)) & done; sleep
# ip netns exec qlb nft delete table ip lb-quic; ip netns exec qlb nft -f lb-quic.nft
# ip netns exec qlb nft flush map ip lb-quic replies      # "the output chain handles replie
# ip netns exec qcli tcpdump -tqlni c0 -c 1 'udp and src net 10.0.0.0/24' 2>/dev/null & td=$
# Q -src 198.51.100.17; wait $td
port 50391: be2: hello (client 198.51.100.17:50391)
IP 10.0.0.12.8081 > 198.51.100.17.50391: UDP, length 1280
# ip netns exec qlb grep 'src=198.51.100.17 ' /proc/net/nf_conntrack | sort
ipv4     2 udp      17 29 src=10.0.0.12 dst=198.51.100.17 sport=8081 dport=50391 [UNREPLIED]
ipv4     2 udp      17 29 src=198.51.100.17 dst=198.51.100.1 sport=50391 dport=443 [UNREPLIE
# ip netns exec qlb nft -f edge.nft
# Q -src 198.51.100.16
error: context deadline exceeded
# ip netns exec qlb nft add element ip lb-quic replies '{ 10.0.0.11 . 8080 : 1, 10.0.0.12 .
# Q -src 198.51.100.16
port 42633: be2: hello (client 198.51.100.16:42633)
# ip netns exec qlb nft list chain inet edge egress
table inet edge {
    chain egress {
        type filter hook postrouting priority srcnat + 10; policy accept;
        oifname "l0" ip saddr != 198.51.100.0/24 counter packets 16 bytes 12520 drop
    }
}
```

Read it in order:

1. The handshake **succeeded**, and that is the trap. The backend's reply came in on `l1` with no mark, so it was looked up in zone 0. It didn't match the zone-2 entry, so conntrack created a *new* zone-0 entry instead, `[UNREPLIED]` in both directions. Nothing undid the DNAT. The client received `10.0.0.12.8081 > 198.51.100.17`, a packet from the backend's private address. quic-go demultiplexes by CID on an unconnected socket, so it accepted the packet.
2. On a real network, that packet leaves the load balancer with a source address the site doesn't own. Egress filtering (BCP 38 / RFC 2827), uRPF upstream, or the client's own NAT, which only admits replies from `198.51.100.1:443`, drops it. `edge.nft` models that. Its filter sits after `srcnat` in postrouting because the reply's source is only translated back to the VIP at that hook. A filter in `forward` would also drop correct replies. With the edge filter on, the same setup times out.
3. With `@replies` restored, replies carry the request's zone, find their entry, and leave as `198.51.100.1:443`. The edge filter passes them. Its counter records the leaked replies it dropped.

Rule: the zone for replies must be set before conntrack's lookup (prerouting priority −200) on the interface the replies actually *arrive* on. Backends on the LB host reply through `output`. Backends anywhere else reply through `prerouting`. Test it with an egress filter, not with a client that accepts anything.

Notes and pitfalls

- **A plaintext server ID is visible to everyone.** Any observer can tell which backend owns a connection, and can link a migrating client's old and new paths by that byte. That is the linkability RFC 9000 §5.1 and §9.5 try to prevent. QUIC-LB's answer is to encrypt the server ID with AES, which nftables cannot do. Use this design where that leak is acceptable, such as internal fleets or few clients per server, or move CID decoding to software that can decrypt (a userspace or eBPF load balancer).
- **The packet mark is now taken.** Policy routing (`ip rule fwmark`) or other firewall code that uses marks will collide with it. Mask a field of the mark if you need to share.
- **IPv4 only.** `table ip` and `ip saddr` keys. An `inet` or `ip6` version needs IPv6-keyed `@flows`/`@replies` and IPv6 `@backends`.
- **Server IDs are one byte.** That allows 255 backends. A 2-byte ID is `@th,80,16` and a wider `@sids` key.
- **The flows timeout is a trade-off.** Too short and slow handshakes split across backends. Too long and a draining backend keeps receiving new handshakes from pinned addresses.

The boring rule

Route handshakes by client address and everything else by a server ID the backends write into their CIDs. Separate backends with conntrack zones, and restore the zone on the path replies actually take. Keep every backend fact in a map and change it with atomic `nft -f` element edits: drain by remapping slots, wait out the pin timeout, then deregister. Give all backends one stateless reset key. Prove the return path with an egress filter.

Try this

1. Change `lbCIDs` to use a 2-byte server ID (`b[1:3]`), switch the match to `@th,80,16`, and register a backend with ID `0x0102`.
2. Run `nft monitor` in `qlb` during Drill 5 and watch the element changes land as one transaction.
3. Put a NAT between the client and the VIP (a fourth namespace with `masquerade`). Delete its conntrack entry mid-connection (`conntrack -D`) to force a rebinding, and confirm the connection survives under `lb-quic.nft` but not under `naive.nft`.
4. Set `@flows`' timeout to `1s` and add 400 ms of delay to `l1` with `tc qdisc add dev l1 root netem delay 400ms`. Find the point where handshakes start to fail.
5. Rewrite the table as `table inet` with dual-stack `@flows` and `@replies`, then repeat Drill 4 over IPv6.

Sources

- RFC 8999 — Version-Independent Properties of QUIC: https://www.rfc-editor.org/rfc/rfc8999.html

- RFC 9000 — QUIC (§5.1 connection IDs, §7.2 negotiating CIDs, §9 migration, §9.5 privacy, §10.3 stateless reset): https://www.rfc-editor.org/rfc/rfc9000.html
- QUIC-LB: Generating Routable QUIC Connection IDs, draft-ietf-quic-load-balancers-21: https://datatracker.ietf.org/doc/draft-ietf-quic-load-balancers/
- nftables wiki — Netfilter hooks and priorities: https://wiki.nftables.org/wiki-nftables/index.php/Netfilter_hooks
- nftables wiki — Configuring chains (priority keywords): https://wiki.nftables.org/wiki-nftables/index.php/Configuring_chains
- nftables wiki — Setting packet connection tracking metainformation (`ct zone`): https://wiki.nftables.org/wiki-nftables/index.php/Setting_packet_connection_tracking_metainformation
- nft(8) manual (raw payload `@th`, maps, `numgen`, dynamic sets): https://www.netfilter.org/projects/nftables/manpage.html
- quic-go v0.63.0 `Transport` (`ConnectionIDGenerator`, `StatelessResetKey`, reset size floor): https://github.com/quic-go/quic-go/blob/v0.63.0/transport.go
- quic-go v0.63.0 reset token derivation: https://github.com/quic-go/quic-go/blob/v0.63.0/stateless_reset.go
- RFC 2827 (BCP 38) — Network Ingress Filtering: https://www.rfc-editor.org/rfc/rfc2827.html
- Marten Seemann, "Using nftables for QUIC load balancing" (2026-04-08), the design this chapter adapts: https://seemann.io/posts/2026-04-08---nftables-quic-load-balancing/

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499

A user closes a tab, a mobile client races a new search, an SDK hits its read timeout. The HTTP request is over for the client. Whether the origin finds out — and how fast — depends on the transport. **HTTP/1.1** has no per-request abort: the only signal is closing the TCP connection, which also kills keep-alive and any pipelined work on that socket. **HTTP/2** can abort one stream with `RST_STREAM` and error code `CANCEL` (0x08) while the TLS connection stays up. **HTTP/3** splits that into two QUIC frames on independent stream halves: `STOP_SENDING` (stop the peer sending) and `RESET_STREAM` (I am done sending), usually carrying `H3_REQUEST_CANCELLED` (0x010c). This chapter is the mapping, the proxy duty (propagate immediately, log 499, do not wait for origin 200), and a localhost HTTP/3 lab that captures the frames.

Learning goals

- Name the abort signal on HTTP/1.1, HTTP/2, and HTTP/3, and say what happens to sibling requests
- Read a QUIC bidirectional stream as two state machines, and say when `STOP_SENDING` vs `RESET_STREAM` is the right frame (RFC 9000 §3, §19.4, §19.5)
- Pick `H3_REQUEST_CANCELLED` (0x010c) vs `H3_NO_ERROR` (0x0100) vs `H3_REQUEST_REJECTED` (0x010b) from RFC 9114 §4.1.1 and §8.1
- Write the proxy rule: on downstream cancel, stop the origin stream *now* and record a 499-style "client closed request"
- Design an origin so a cancelled GET stops work, and a cancelled POST does not double-apply on retry
- Run a localhost HTTP/3 server and client, cancel mid-response, and point at the qlog frames

Lab platform: Python 3.13.5, aioquic 1.7.0 from PyPI, OpenSSL 3.5.7, Linux 6.12. Bind `127.0.0.1:4433`. No root. Every listing output below is pasted from one run.

This chapter does not reteach QUIC version negotiation or CID-based load balancing.

Mental model

A cancel is "this request is no longer of interest." The transport has to carry that to the peer without taking down every other request that happens to share a connection.

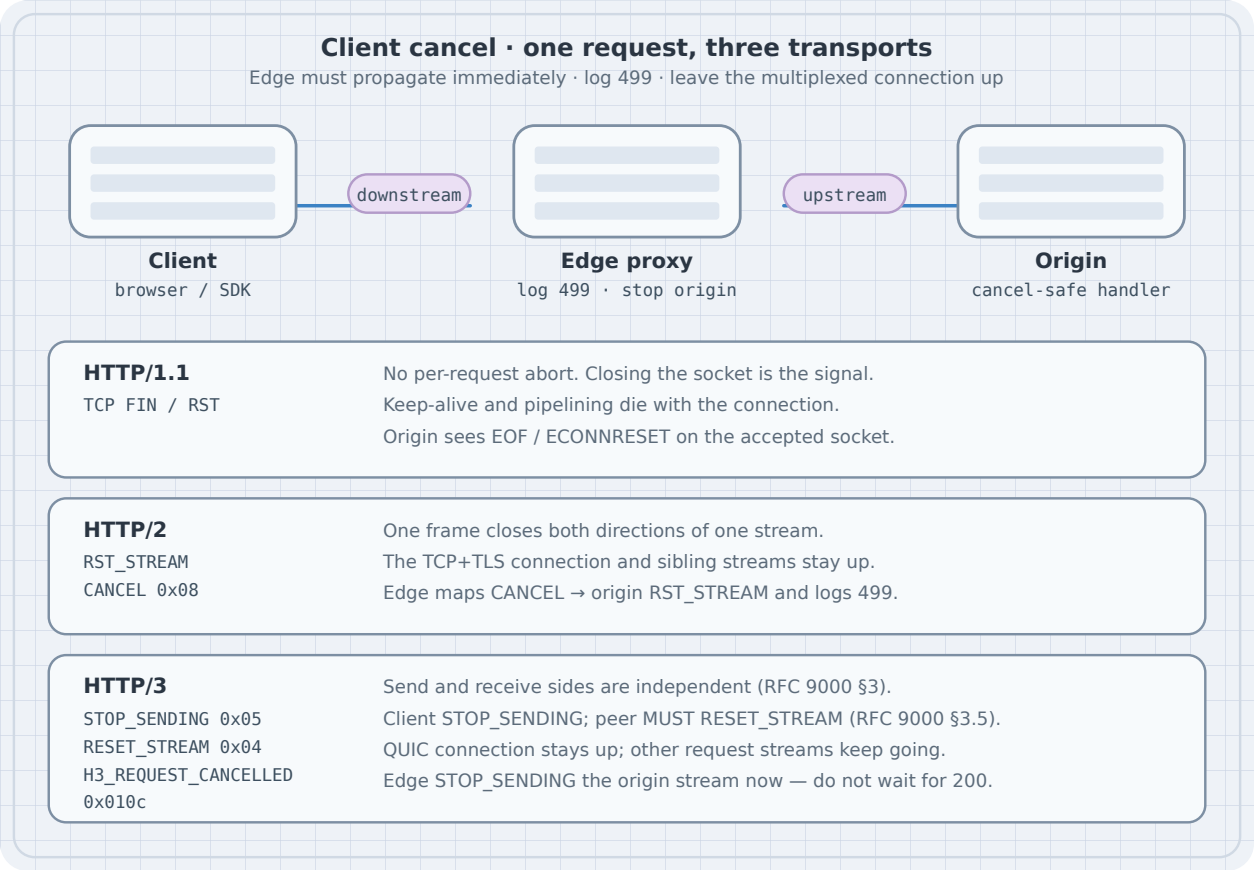


Figure 1: Client, edge proxy, and origin, with the abort signal on HTTP/1.1, HTTP/2, and HTTP/3

	HTTP/1.1 (RFC 9112)	HTTP/2 (RFC 9113)	HTTP/3 (RFC 9114 on RFC 9000)
Abort signal	Close the TCP connection (FIN or RST)	`RST_STREAM` with `CANCEL` (0x08)	`STOP_SENDING` and/or `RESET_STREAM` with `H3_REQUEST_CANCELLED` (0x010c)
Scope	The whole connection	One stream; both directions	One stream; send and receive halves independently
Sibling requests	Die with the socket	Keep going	Keep going
What the origin must notice	`read()` returns 0 / `ECONNRESET`	Stream reset	`StopSendingReceived` / `StreamReset` (library events), then stop the handler

The edge is not a spectator. RFC 9114 Appendix A.4.1 says an intermediary *may* pass client cancellations indicated by `H3_REQUEST_CANCELLED`. In production that "may" is the difference between origin CPU spent on a tab that already closed and a 499 in the log. nginx invented status **499** ("client closed request") for exactly this: the downstream went away before a response was sent. It is a log code, not an IETF status the client ever sees.

Concepts — two halves of a QUIC stream

HTTP/3 puts each request/response on one client-initiated **bidirectional** QUIC stream (RFC 9114 §2, §6.1). QUIC treats that stream as two independent parts (RFC 9000 §3): a sending part and a receiving part. Closing one does not close the other.

```
Client-initiated bidir stream (id 0, 4, 8, …)

  client send    STREAM / STREAM+FIN / RESET_STREAM     server recv
  client recv    STREAM / STREAM+FIN / RESET_STREAM    server send
                 STOP_SENDING travels the opposite way
                 of the data it wants to stop
```

Application operations (RFC 9000 §2.4):

- **Reset sending** → `RESET_STREAM` (type 0x04). "I will not send more on this stream."
- **Abort reading** → `STOP_SENDING` (type 0x05). "Stop sending to me on this stream."

Frame layouts (RFC 9000 §19.4, §19.5):

```
RESET_STREAM Frame {
  Type (i) = 0x04,
  Stream ID (i),
  Application Protocol Error Code (i),
```

```
  Final Size (i),
}

STOP_SENDING Frame {
  Type (i) = 0x05,
  Stream ID (i),
  Application Protocol Error Code (i),
}
```

The coupling that operators forget is RFC 9000 §3.5: an endpoint that receives `STOP_SENDING` **MUST** send `RESET_STREAM` if its sending part is in "Ready" or "Send". It SHOULD copy the error code. STREAM frames that still arrive after `STOP_SENDING` still count toward flow control even if the application discards them.

So a client that has already finished its GET (HEADERS + FIN) and then loses interest in the response does this:

1. `STOP_SENDING` with `H3_REQUEST_CANCELLED` — abort reading.
2. `RESET_STREAM` only if the send side is still open (a POST body still in flight). If the GET is already acknowledged, `RESET_STREAM` is a no-op.
3. The server MUST `RESET_STREAM` the response. The client then sees a stream reset with the same application error code.

That pair is the HTTP/3 equivalent of HTTP/2's single `RST_STREAM`. HTTP/2 streams are one state machine (RFC 9113 §5.1): `RST_STREAM` moves the stream to "closed" in both directions at once.

Concepts — which HTTP/3 error code

RFC 9114 §4.1.1: implementations SHOULD cancel by resetting any sending parts that are still open and aborting reading on receiving parts. The codes (RFC 9114 §8.1):

Code	Value	When
`H3_NO_ERROR`	0x0100	Close with no error. A server that has sent a complete response and does not need the rest of the request SHOULD abort reading with this code (RFC 9114 §4.1). Also used for graceful connection close.
`H3_REQUEST_REJECTED`	0x010b	Server refused the request **without** application processing. The client may retry as if the request never happened. Servers MUST NOT use this after passing data to a higher layer.

Code	Value	When
`H3_REQUEST_CANCELLED`	0x010c	The request or its response is cancelled. Clients SHOULD use this. A server that abandons a *partially processed* response SHOULD use this, not `H3_REQUEST_REJECTED`.

RFC 9114 Appendix A.4 maps HTTP/2 `CANCEL` (0x08) to `H3_REQUEST_CANCELLED`. Treat them as the same operator event: "this request is no longer needed."

Do not send `H3_NO_ERROR` when the user cancelled. The stream is torn down either way, but `0x0100` tells the peer "nothing went wrong", so metrics and origin cancellation hooks that key off `0x010c` will miss it.

Case — what a proxy must do

Downstream cancel is not "the origin can finish and we will drop the body." The body is the expensive part.

```
Client                        Edge                          Origin

    GET /report             GET /report
                                                          (query running)
    STOP_SENDING 0x010c
                                  log 499
                                  STOP_SENDING 0x010c
                                                          abort handler
        RESET_STREAM 0x010c        RESET_STREAM 0x010c
```

RFC 9114 Appendix A.4.1: it is beneficial to pass client cancellations indicated by `H3_REQUEST_CANCELLED`. The operational translation:

1. On downstream `STOP_SENDING` / `RESET_STREAM` / HTTP/2 `RST_STREAM CANCEL` / HTTP/1.1 connection close, **stop the origin stream immediately**. Do not wait for origin headers.
2. Log **499** (nginx `NGX_HTTP_CLIENT_CLOSED_REQUEST`; Cloudflare uses the same number in analytics and logs). The client never receives a 499 status line — they already left.
3. Exclude 499 from server-error rates. It is a client abort, not a 5xx. A jump in 499s after you start honouring HTTP/3 cancels is reporting catching up, not a new outage.
4. Leave `proxy_ignore_client_abort` (and equivalents) off for reads. Turn it on only for work that *must* finish without a client, and only with an idempotency story.

Cloudflare's 9 October 2026 changelog records the HTTP/3-shaped version of this bug: the edge did not always stop an HTTP/3 request when the client cancelled the stream, so some requests continued to the origin and were logged with the upstream status instead of 499. The fix stops

those requests sooner and records 499. Operators then see more 499s on HTTP/3; the cancellations were already happening.

HTTP/1.1 through the same proxy is the same 499 with a worse blast radius: the only way the client can abort is to close TCP, so the proxy also loses connection reuse.

Case — cancel-safe origins

The transport can only deliver the signal. The handler has to listen.

- **Context.** In Go, `Request.Context()` is cancelled when the peer goes away. In Python asyncio, cancel the task that is writing the response (the lab below). In any language, the HTTP server must surface stream reset as cancellation, not as "write failed later."
- **Reads (GET, HEAD).** Abort the query, the upstream HTTP call, the file read. Retrying a partial GET is safe (RFC 9110: GET is idempotent).
- **Writes (POST, PUT, PATCH).** A cancel after the origin began processing is *not* "the write never happened." The client may retry. Use an idempotency key (client-supplied or derived) and make the handler safe to run twice. Do not use `H3_REQUEST_REJECTED` after you have touched storage — RFC 9114 §4.1.1 forbids it, because that code tells the client the request is retryable as if it never ran.
- **Do not finish work for a gone client** on the default path. The exception is a small class of must-complete actions (some payments, some audit writes), and those need their own queue, not a streaming HTTP response the client already abandoned.

A 200 that nobody reads still cost the database. A 499 that you treat as 5xx pages people at 3 a.m. for a user tapping Back.

Lab — cancel mid-response on localhost

Self-signed certificate for `localhost` and `127.0.0.1`, then aioquic 1.7.0 as both HTTP/3 server and client. The server streams 40 × 256-byte chunks with 50 ms between them (~2 s if nobody cancels). The client cancels after 512 bytes, then issues `GET /fast` on a new stream of the **same** connection to prove the abort was per-stream.

```
$ python3 -m venv /workspace/run-2026-10-17-am/venv-net-http3
$ /workspace/run-2026-10-17-am/venv-net-http3/bin/pip install aioquic
...
Successfully installed aioquic-1.7.0 attrs-26.1.0 certifi-2026.7.22 cffi-2.1.1 cryptography-
$ openssl version
OpenSSL 3.5.7 9 Jun 2026 (Library: OpenSSL 3.5.7 9 Jun 2026)
$ openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes \
    -days 1 -subj /CN=localhost \
    -addext subjectAltName=DNS:localhost,IP:127.0.0.1 \
    -keyout key.pem -out cert.pem
$ ls -l cert.pem key.pem
-rw-r--r-- 1 box box 615 Oct 17 08:47 cert.pem
```

```
-rw------- 1 box box 241 Oct 17 08:47 key.pem
```

```
# cancel_lab.py - HTTP/3 client cancel mid-response (aioquic, localhost)
import asyncio
import json
import ssl
import sys
import time
from typing import Optional

from aioquic.asyncio import connect, serve
from aioquic.asyncio.protocol import QuicConnectionProtocol
from aioquic.h3.connection import H3_ALPN, ErrorCode, H3Connection
from aioquic.h3.events import DataReceived, H3Event, HeadersReceived
from aioquic.quic.configuration import QuicConfiguration
from aioquic.quic.events import ProtocolNegotiated, QuicEvent, StopSendingReceived, StreamRe
from aioquic.quic.logger import QuicLogger

T0 = time.monotonic()
SERVER_CHUNKS_SENT = 0
SERVER_SEND_ABORTED = False
SERVER_SAW_STOP_SENDING: Optional[tuple[int, int]] = None
SERVER_SAW_RESET: Optional[tuple[int, int]] = None
CLIENT_BYTES = 0
CLIENT_SAW_RESET: Optional[tuple[int, int]] = None
CHUNK = b"x" * 256
N_CHUNKS = 40
CHUNK_PAUSE = 0.05


def stamp(role: str, msg: str) -> None:
    print(f"{time.monotonic() - T0:7.3f}s  {role:6}  {msg}", flush=True)


def dump_cancel_frames(label: str, logger: QuicLogger) -> None:
    kinds = {"reset_stream", "stop_sending"}
    print(f"\n# qlog {label}: STOP_SENDING / RESET_STREAM frames")
    n = 0
    for trace in logger.to_dict()["traces"]:
        vp = trace["vantage_point"]["type"]
        for ev in trace["events"]:
            for fr in ev.get("data", {}).get("frames") or []:
                if fr.get("frame_type") in kinds:
                    n += 1
                    extra = (
                        f"  final_size={fr.get('final_size')}"
```

```
                        if "final_size" in fr
                        else ""
                    )
                    print(
                        f"  {vp:6}  {ev['name']:28}  {fr['frame_type']}"
                        f"  stream={fr.get('stream_id')}"
                        f"  error_code={fr.get('error_code')}{extra}"
                    )
    if n == 0:
        print("  (none)")


class H3ServerProtocol(QuicConnectionProtocol):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._http: Optional[H3Connection] = None
        self._tasks: dict[int, asyncio.Task] = {}

    def quic_event_received(self, event: QuicEvent) -> None:
        global SERVER_SAW_STOP_SENDING, SERVER_SAW_RESET
        if isinstance(event, ProtocolNegotiated):
            stamp("server", f"ALPN={event.alpn_protocol}")
            if event.alpn_protocol in H3_ALPN:
                self._http = H3Connection(self._quic)
        if isinstance(event, StopSendingReceived):
            SERVER_SAW_STOP_SENDING = (event.stream_id, event.error_code)
            stamp(
                "server",
                f"StopSendingReceived stream={event.stream_id} "
                f"error_code=0x{event.error_code:x} ({event.error_code})",
            )
            task = self._tasks.pop(event.stream_id, None)
            if task is not None:
                task.cancel()
        if isinstance(event, StreamReset):
            SERVER_SAW_RESET = (event.stream_id, event.error_code)
            stamp(
                "server",
                f"StreamReset stream={event.stream_id} "
                f"error_code=0x{event.error_code:x} ({event.error_code})",
            )
        if self._http is not None:
            for http_event in self._http.handle_event(event):
                self._http_event(http_event)

    def _http_event(self, event: H3Event) -> None:
        if isinstance(event, HeadersReceived):
```

```
            headers = {k.decode(): v.decode() for k, v in event.headers}
            path = headers.get(":path", "?")
            stamp(
                "server",
                f"request {headers.get(':method')} {path} stream={event.stream_id}",
            )
            if path == "/fast":
                self._http.send_headers(
                    event.stream_id,
                    [(b":status", b"200"), (b"content-type", b"text/plain")],
                )
                self._http.send_data(event.stream_id, b"ok\n", end_stream=True)
                self.transmit()
                stamp("server", f"/fast complete stream={event.stream_id}")
            elif path == "/slow":
                self._http.send_headers(
                    event.stream_id,
                    [
                        (b":status", b"200"),
                        (b"content-type", b"application/octet-stream"),
                        (b"x-lab", b"slow-body"),
                    ],
                )
                self.transmit()
                self._tasks[event.stream_id] = asyncio.create_task(
                    self._slow_body(event.stream_id)
                )

    async def _slow_body(self, stream_id: int) -> None:
        global SERVER_CHUNKS_SENT, SERVER_SEND_ABORTED
        try:
            for i in range(N_CHUNKS):
                self._http.send_data(
                    stream_id, CHUNK, end_stream=(i == N_CHUNKS - 1)
                )
                self.transmit()
                SERVER_CHUNKS_SENT += 1
                if i == 0:
                    stamp(
                        "server",
                        f"started slow body stream={stream_id} chunk=0/{N_CHUNKS}",
                    )
                await asyncio.sleep(CHUNK_PAUSE)
            stamp(
                "server",
                f"slow body FIN stream={stream_id} chunks={SERVER_CHUNKS_SENT}",
            )
```

```
        except asyncio.CancelledError:
            SERVER_SEND_ABORTED = True
            stamp(
                "server",
                f"origin send cancelled stream={stream_id} "
                f"chunks_sent={SERVER_CHUNKS_SENT}/{N_CHUNKS}",
            )
        except RuntimeError as exc:
            SERVER_SEND_ABORTED = True
            stamp(
                "server",
                f"origin send RuntimeError: {exc} chunks_sent={SERVER_CHUNKS_SENT}",
            )


class H3ClientProtocol(QuicConnectionProtocol):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._http: Optional[H3Connection] = None
        self._slow_id: Optional[int] = None
        self._fast_id: Optional[int] = None
        self._cancelled = False
        self.done = asyncio.get_running_loop().create_future()

    def start_slow(self) -> None:
        self._slow_id = self._quic.get_next_available_stream_id()
        self._http.send_headers(
            self._slow_id,
            [
                (b":method", b"GET"),
                (b":scheme", b"https"),
                (b":authority", b"localhost"),
                (b":path", b"/slow"),
                (b"user-agent", b"cancel-lab/1.0"),
            ],
            end_stream=True,
        )
        self.transmit()
        stamp("client", f"GET /slow stream={self._slow_id}")

    def start_fast(self) -> None:
        self._fast_id = self._quic.get_next_available_stream_id()
        self._http.send_headers(
            self._fast_id,
            [
                (b":method", b"GET"),
                (b":scheme", b"https"),
```

```
                (b":authority", b"localhost"),
                (b":path", b"/fast"),
                (b"user-agent", b"cancel-lab/1.0"),
            ],
            end_stream=True,
        )
        self.transmit()
        stamp("client", f"GET /fast stream={self._fast_id}")

    def cancel_slow(self) -> None:
        if self._cancelled or self._slow_id is None:
            return
        self._cancelled = True
        code = ErrorCode.H3_REQUEST_CANCELLED
        self._quic.stop_stream(self._slow_id, code)
        self._quic.reset_stream(self._slow_id, code)
        self.transmit()
        stamp(
            "client",
            f"cancel stream={self._slow_id} "
            f"STOP_SENDING+RESET_STREAM error_code=0x{int(code):x} "
            f"({int(code)} H3_REQUEST_CANCELLED) bytes_so_far={CLIENT_BYTES}",
        )

    def quic_event_received(self, event: QuicEvent) -> None:
        global CLIENT_SAW_RESET
        if isinstance(event, ProtocolNegotiated):
            stamp("client", f"ALPN={event.alpn_protocol}")
            if event.alpn_protocol in H3_ALPN:
                self._http = H3Connection(self._quic)
                self.start_slow()
        if isinstance(event, StreamReset):
            CLIENT_SAW_RESET = (event.stream_id, event.error_code)
            stamp(
                "client",
                f"StreamReset stream={event.stream_id} "
                f"error_code=0x{event.error_code:x} ({event.error_code})",
            )
        if self._http is not None:
            for http_event in self._http.handle_event(event):
                self._http_event(http_event)

    def _http_event(self, event: H3Event) -> None:
        global CLIENT_BYTES
        if isinstance(event, HeadersReceived):
            status = dict(event.headers).get(b":status", b"?").decode()
            stamp(
```

```
                "client",
                f"headers :status={status} stream={event.stream_id} "
                f"ended={event.stream_ended}",
            )
        elif isinstance(event, DataReceived):
            CLIENT_BYTES += len(event.data)
            if (
                event.stream_id == self._slow_id
                and not self._cancelled
                and CLIENT_BYTES >= 512
            ):
                stamp("client", f"received {CLIENT_BYTES} bytes of /slow; cancelling")
                self.cancel_slow()
                asyncio.get_running_loop().call_later(0.15, self.start_fast)
            if event.stream_id == self._fast_id:
                stamp(
                    "client",
                    f"/fast data={event.data!r} ended={event.stream_ended}",
                )
                if event.stream_ended and not self.done.done():
                    self.done.set_result(True)


async def main() -> None:
    import aioquic

    print(f"python {sys.version.split()[0]}")
    print(f"aioquic {aioquic.__version__}")
    print(
        f"H3_REQUEST_CANCELLED = {int(ErrorCode.H3_REQUEST_CANCELLED)} "
        f"(0x{int(ErrorCode.H3_REQUEST_CANCELLED):x})"
    )
    print(
        f"H3_NO_ERROR          = {int(ErrorCode.H3_NO_ERROR)} "
        f"(0x{int(ErrorCode.H3_NO_ERROR):x})"
    )
    print()

    server_logger = QuicLogger()
    client_logger = QuicLogger()
    server_conf = QuicConfiguration(
        alpn_protocols=H3_ALPN, is_client=False, quic_logger=server_logger
    )
    server_conf.load_cert_chain("cert.pem", "key.pem")
    server = await serve(
        "127.0.0.1", 4433, configuration=server_conf, create_protocol=H3ServerProtocol
    )
```

```
    stamp("lab", "HTTP/3 server listening on 127.0.0.1:4433")

    client_conf = QuicConfiguration(
        alpn_protocols=H3_ALPN,
        is_client=True,
        server_name="localhost",
        quic_logger=client_logger,
    )
    client_conf.verify_mode = ssl.CERT_NONE
    try:
        async with connect(
            "127.0.0.1",
            4433,
            configuration=client_conf,
            create_protocol=H3ClientProtocol,
        ) as client:
            await asyncio.wait_for(client.done, timeout=5)
            await asyncio.sleep(0.2)
            client.close()
            await client.wait_closed()
    finally:
        server.close()

    print("\n# summary")
    print(f"server chunks sent     : {SERVER_CHUNKS_SENT}/{N_CHUNKS}")
    print(f"server send aborted    : {SERVER_SEND_ABORTED}")
    print(f"server StopSending     : {SERVER_SAW_STOP_SENDING}")
    print(f"server StreamReset     : {SERVER_SAW_RESET}")
    print(f"client bytes (end)     : {CLIENT_BYTES}")
    print(f"client StreamReset     : {CLIENT_SAW_RESET}")
    dump_cancel_frames("client", client_logger)
    dump_cancel_frames("server", server_logger)


if __name__ == "__main__":
    asyncio.run(main())
```

The listing above is the lab that ran, with one print-label tweak (`client bytes (end)`) so the summary is not misread as "bytes at cancel." aioquic 1.7.0 reports `H3_REQUEST_CANCELLED = 268`, which is 0x010c.

```
$ python cancel_lab.py
python 3.13.5
aioquic 1.7.0
H3_REQUEST_CANCELLED = 268 (0x10c)
H3_NO_ERROR          = 256 (0x100)
```

```
  0.003s  lab     HTTP/3 server listening on 127.0.0.1:4433
  0.016s  server  ALPN=h3
  0.017s  client  ALPN=h3
  0.018s  client  GET /slow stream=0
  0.019s  server  request GET /slow stream=0
  0.019s  server  started slow body stream=0 chunk=0/40
  0.020s  client  headers :status=200 stream=0 ended=False
  0.071s  client  received 512 bytes of /slow; cancelling
  0.071s  client  cancel stream=0 STOP_SENDING+RESET_STREAM error_code=0x10c (268 H3_REQUEST
  0.071s  server  StopSendingReceived stream=0 error_code=0x10c (268)
  0.071s  server  origin send cancelled stream=0 chunks_sent=2/40
  0.071s  client  StreamReset stream=0 error_code=0x10c (268)
  0.222s  client  GET /fast stream=4
  0.222s  server  request GET /fast stream=4
  0.223s  server  /fast complete stream=4
  0.223s  client  headers :status=200 stream=4 ended=False
  0.223s  client  /fast data=b'ok\n' ended=True

# summary
server chunks sent     : 2/40
server send aborted    : True
server StopSending     : (0, 268)
server StreamReset     : None
client bytes (end)     : 515
client StreamReset     : (0, 268)

# qlog client: STOP_SENDING / RESET_STREAM frames
  client  transport:packet_sent         stop_sending  stream=0  error_code=268
  client  transport:packet_received     reset_stream  stream=0  error_code=268  final_size=5

# qlog server: STOP_SENDING / RESET_STREAM frames
  server  transport:packet_received     stop_sending  stream=0  error_code=268
  server  transport:packet_sent         reset_stream  stream=0  error_code=268  final_size=5
```

Read it in order:

1. **ALPN `h3`.** Both sides negotiated HTTP/3. Stream 0 is the first client-initiated bidirectional stream.
2. **Cancel is `STOP_SENDING`, not a connection close.** qlog shows the client *sending* `stop_sending` with `error_code=268` (0x010c) and *not* sending `reset_stream`. The GET already had FIN; aioquic's `reset_stream()` is a no-op once the send side is finished (RFC 9000 §3.1: no `RESET_STREAM` from a terminal send state). RFC 9114 §4.1.1 still says to reset sending parts that are *still open* — here they were not.
3. **The server MUST answer `STOP_SENDING` with `RESET_STREAM`.** aioquic copies the error code (RFC 9000 §3.5). The client event is `StreamReset stream=0 error_code=0x10c`. qlog `final_size=555` is the STREAM offset the server had committed: two 256-byte DATA chunks plus the HEADERS framing (~43 bytes).
4. **The origin stopped.** 2 of 40 chunks. Without the `task.cancel()` on `StopSendingReceived`,

the handler would keep calling `send_data` until aioquic raised `RuntimeError: Cannot send data after the stream was reset`. That is the "proxy that does not propagate" failure, on one box.

5. **The connection survived.** `GET /fast` on stream 4 returned `200` and `ok\n` 150 ms later. That is the whole point of multiplexed abort. HTTP/1.1 cannot do this: the cancel would have been a TCP close, and `/fast` would have needed a new connection.
6. **515 vs 512.** The cancel fired at 512 body bytes. The summary's 515 is bytes counted through the end of the run — three extra bytes of STREAM data already in flight. RFC 9000 §3.5: STREAM frames received after `STOP_SENDING` still count toward flow control even when discarded.

nginx was not run here. Its 499 is a log-only code for "downstream closed while we still owed a response," documented by nginx and by Cloudflare's Error 499 page. The lab's `StopSendingReceived` is the HTTP/3 event that a proxy should turn into that log line *and* an upstream cancel.

The trap

- **Closing the QUIC connection to cancel one request.** That is the HTTP/1.1 move. It dumps every other stream, burns the handshake, and wrecks 0-RTT. Cancel the stream.
- **Waiting for origin 200 after downstream cancel.** The user is gone. Every extra millisecond is origin work you will throw away, then log as 200 and lie to your SLO.
- **Putting 499 in the 5xx burn rate.** 499 means the client left. After you start honouring HTTP/3 `STOP_SENDING`, 499 volume on HTTP/3 will rise. That is the metric becoming honest.
- **Sending `H3_REQUEST_REJECTED` after you processed the request.** RFC 9114 §4.1.1: that code means "safe to retry as if never sent." Use `H3_REQUEST_CANCELLED` once a handler has run.
- **Sending `H3_NO_ERROR` for a user abort.** 0x0100 is "no error" (complete response, leftover request body; graceful GOAWAY). User cancel is 0x010c.
- **Ignoring `STOP_SENDING` in the application.** The transport will `RESET_STREAM` for you. The database query will not, unless you cancel the task.

The boring rule

Cancel is a stream event, not a connection event. On HTTP/3 the client sends `STOP_SENDING` (and `RESET_STREAM` if it is still sending) with `H3_REQUEST_CANCELLED`. The peer MUST `RESET_STREAM`. The proxy copies that to the origin immediately and logs 499, excluded from server errors. The origin aborts the handler and treats writes as retryable only when they are actually idempotent. Sibling streams keep running.

Try this

1. Change the client to cancel with `ErrorCode.H3_NO_ERROR` (0x0100). Predict the qlog `error_code` on both `stop_sending` and the server's `reset_stream`, then run it. The origin

still stops; your cancel metrics that filter on 0x010c go silent.
2. Comment out `task.cancel()` on the server. How many extra chunks are sent, and do you hit `RuntimeError: Cannot send data after the stream was reset`?
3. Turn `/slow` into a POST whose body is sent in 20 chunks without FIN. Cancel after the first response byte. Confirm a client-originated `reset_stream` now appears in qlog alongside `stop_sending`.
4. Start two `/slow` requests in parallel, cancel only stream 0, and confirm stream 4 still delivers 40 chunks. That is the HTTP/1.1 counterfactual.

Sources

- RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport (§2.4 operations, §3 stream states, §3.5 solicited state transitions, §19.4 `RESET_STREAM`, §19.5 `STOP_SENDING`): https://www.rfc-editor.org/rfc/rfc9000.html
- RFC 9114 — HTTP/3 (§4.1 leftover request body / `H3_NO_ERROR`, §4.1.1 request cancellation and rejection, §8.1 error codes, Appendix A.4 mapping from HTTP/2, Appendix A.4.1 intermediaries): https://www.rfc-editor.org/rfc/rfc9114.html
- RFC 9113 — HTTP/2 (§5.1 stream states, §5.4.2 stream errors, §6.4 `RST_STREAM`, §7 `CANCEL` 0x08): https://www.rfc-editor.org/rfc/rfc9113.html
- RFC 9112 — HTTP/1.1 (§9 connection management; abort is connection close): https://www.rfc-editor.org/rfc/rfc9112.html
- RFC 9110 — HTTP Semantics (idempotent methods; GET/PUT/DELETE retry): https://www.rfc-editor.org/rfc/rfc9110.html
- IANA HTTP/3 error codes (`H3_NO_ERROR` 0x0100, `H3_REQUEST_REJECTED` 0x010b, `H3_REQUEST_CANCELLED` 0x010c): https://www.iana.org/assignments/http3-parameters
- aioquic 1.7.0 (`H3Connection`, `ErrorCode`, `stop_stream` / `reset_stream`, qlog): https://aioquic.readthedocs.io/en/latest/ and https://pypi.org/project/aioquic/1.7.0/
- aioquic `QuicConnection.stop_stream` / `reset_stream`: https://aioquic.readthedocs.io/en/latest/quic.html
- Cloudflare changelog, 9 October 2026 — Improved HTTP/3 client cancellation reporting: https://developers.cloudflare.com/changelog/post/2026-10-09-http3-499-reporting-improvement/
- Cloudflare Error 499 (nginx `499 Client Closed Request` in logs and analytics): https://developers.cloudflare.com/support/troubleshooting/http-status-codes/4xx-client-error/error-499/
- nginx source — `NGX_HTTP_CLIENT_CLOSED_REQUEST` 499 in `ngx_http_request.h`: https://hg.nginx.org/nginx/file/tip/src/http/ngx_http_request.h

Policy, QoS, and Hardening

ACL Thinking

ACL Thinking

Access control lists are not a pile of lines—they are a **decision policy** evaluated in order. This chapter builds vendor-agnostic ACL decision patterns and implements them with Linux nftables/iptables and FRR filters in Containerlab (mid-2026 free tools).

Learning goals

By the end of this chapter you can:

- Write allow/deny policies with explicit default stance
- Place filters at the right plane (data plane vs route policy vs management)
- Implement Linux netfilter rules that match a written policy
- Apply FRR prefix-lists/route-maps without confusing them with packet ACLs
- Predict asymmetric ACL failures and fix with evidence

Concepts

Three different "filters" people mix up

Kind	Acts on	Example
Packet ACL / firewall	Data-plane packets	nftables drop tcp/23
Route policy	Routes in RIB/advertisements	prefix-list on BGP
Control-plane protection	Traffic **to** the CPU/RE	CoPP / policers (next chapter)

Using a BGP prefix-list will **not** stop a host scan. Dropping packets will **not** stop a bad route install (unless you couple systems).

Policy statement template

Before typing rules, write:

```
Stance: default deny (or default allow-pick deliberately)
Allow:
  - established/related (if stateful)
  - DNS to resolver R from clients
  - HTTPS to internet from clients
  - BGP from peer P on interface I
Deny:
  - everything else (log sample)
```

First-match vs best-match

Most packet ACLs are **first match wins**. Order is part of the policy. Route filters also often first-match in lists. Never "add a deny at the top" without reading the whole list.

Stateful vs stateless

	Stateless	Stateful
Idea	Each packet alone	Track flows/conntrack
Return traffic	Must allow reverse explicitly	Often allow established
Lab tools	Simple iptables rules	iptables/nft conntrack

Direction and attachment

```
inbound on client LAN   inbound on uplink
```

Always specify **interface + direction + zone**. Diagram the edge:

```
[clients] --in--> | edge | --out--> [uplink]
                  |      |
                  +-- mgmt
```

Written policy for the lab

Edge role: small site router (Linux).

1. Default **drop** forwarded traffic not explicitly allowed.

2. Allow clients (`10.60.0.0/24`) to ping and HTTP to lab server `10.60.1.10`.

3. Allow clients DNS to `10.60.0.53`.
4. Allow SSH **only** from mgmt host `10.60.0.9` to edge.
5. Deny client access to edge SSH from other addresses.
6. Log dropped new TCP (rate-limited mentally—don't flood disks).

Topology YAML

```yaml
name: acl-think

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iptables iputils curl
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils curl
    mgmt:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils openssh-client
    srv:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils busybox-extras
        # tiny http: nc -l -p 80 ...

  links:
    - endpoints: ["client:eth1", "edge:eth1"]
    - endpoints: ["mgmt:eth1", "edge:eth1"]  # same LAN via bridge preferred
    - endpoints: ["edge:eth2", "srv:eth1"]
```

Prefer a bridge node so client + mgmt share `10.60.0.0/24`:

```yaml
    lan:
      kind: linux
```

```
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2
        - ip link add br0 type bridge && ip link set br0 up
  links:
    - endpoints: ["client:eth1", "lan:eth1"]
    - endpoints: ["mgmt:eth1", "lan:eth2"]
    - endpoints: ["edge:eth1", "lan:eth3"]
    - endpoints: ["edge:eth2", "srv:eth1"]
```

Addressing

Node	IP
edge eth1	10.60.0.1/24
edge eth2	10.60.1.1/24
client	10.60.0.10/24 via .1
mgmt	10.60.0.9/24 via .1
srv	10.60.1.10/24 via .1
DNS (optional)	10.60.0.53 on edge lo or dnsmasq

```
docker exec clab-acl-think-edge sh -c '
  sysctl -w net.ipv4.ip_forward=1
  ip addr add 10.60.0.1/24 dev eth1; ip link set eth1 up
  ip addr add 10.60.1.1/24 dev eth2; ip link set eth2 up
'
docker exec clab-acl-think-client sh -c '
  ip addr add 10.60.0.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.0.1
'
docker exec clab-acl-think-mgmt sh -c '
  ip addr add 10.60.0.9/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.0.1
'
docker exec clab-acl-think-srv sh -c '
  ip addr add 10.60.1.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.1.1
'
```

Implement with iptables (readable teaching form)

Flush carefully in lab only:

```
docker exec clab-acl-think-edge sh -c '
iptables -F
iptables -X
iptables -t nat -F
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# loopback + established
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# SSH to edge only from mgmt
iptables -A INPUT -p tcp -s 10.60.0.9 --dport 22 -j ACCEPT

# allow ping to edge from LAN (optional)
iptables -A INPUT -p icmp -s 10.60.0.0/24 -j ACCEPT

# forward: client/mgmt to server http + ping
iptables -A FORWARD -s 10.60.0.0/24 -d 10.60.1.10 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 10.60.0.0/24 -d 10.60.1.10 -p icmp -j ACCEPT

# explicit deny counters
iptables -A FORWARD -j LOG --log-prefix "FWD-DROP: " --log-level 4
iptables -A INPUT -j LOG --log-prefix "IN-DROP: " --log-level 4
'
```

If `conntrack` module unavailable in minimal Alpine, approximate with broad return allows **only for learning**, then move to an image with conntrack.

nftables sketch (same policy)

```
docker exec clab-acl-think-edge sh -c 'apk add --no-cache nftables; nft -f - <<EOF
flush ruleset
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    iif lo accept
    ct state established,related accept
    tcp dport 22 ip saddr 10.60.0.9 accept
    ip protocol icmp ip saddr 10.60.0.0/24 accept
  }
  chain forward {
    type filter hook forward priority 0; policy drop;
    ct state established,related accept
```

```
    ip saddr 10.60.0.0/24 ip daddr 10.60.1.10 tcp dport 80 accept
    ip saddr 10.60.0.0/24 ip daddr 10.60.1.10 ip protocol icmp accept
  }
  chain output {
    type filter hook output priority 0; policy accept;
  }
}
EOF'
```

Predict → observe → fix drills

Drill 1 — Allowed path

```
docker exec clab-acl-think-srv sh -c 'echo OK | nc -l -p 80 &'
docker exec clab-acl-think-client ping -c 2 10.60.1.10
docker exec clab-acl-think-client wget -qO- http://10.60.1.10 || \
  docker exec clab-acl-think-client nc -w2 10.60.1.10 80 </dev/null
```

Predict: success. **If fail:** forwarding, routes, rule order, server listen.

Drill 2 — Denied path

```
docker exec clab-acl-think-client sh -c 'nc -w2 10.60.1.10 22 || true'
docker exec clab-acl-think-edge iptables -L FORWARD -n -v
```

Predict: fail; counters/logs increment.

Drill 3 — SSH source restriction

```
# from mgmt should work if dropbear/sshd runs on edge
# from client should fail
docker exec clab-acl-think-client sh -c 'nc -w2 10.60.0.1 22 || echo blocked'
```

Drill 4 — Asymmetric paths

Allow only client→server tcp/80 but forget established return:

Predict: SYN works, data stalls if truly stateless wrong.
Observe: conntrack ESTABLISHED rule presence.
Fix: stateful allow or explicit reverse rule.

Drill 5 — Wrong plane

Install FRR on edge and **only** add a prefix-list denying `10.60.1.0/24`. Client ping to server still works if connected routes exist.

Predict: route policy packet filter.
Observe: `ip route` still forwards; iptables counters unchanged for that myth.
Fix: teach the table of three filter kinds again.

FRR: route policy is still "ACL thinking"

```
ip prefix-list BOGON seq 5 deny 0.0.0.0/8 le 32
ip prefix-list BOGON seq 10 deny 10.0.0.0/8 le 32
ip prefix-list BOGON seq 100 permit any
!
route-map IMPORT deny 10
 match ip address prefix-list BOGON
route-map IMPORT permit 20
```

Same discipline: **written intent → ordered rules → negative tests**.

Object-group idea (vendor-agnostic)

Group hosts and services even if the engine lacks object-groups:

```
CLIENTS = 10.60.0.0/24
SERVERS = 10.60.1.10/32
WEB     = tcp/80
```

Expand into concrete rules; keep the group doc next to the ruleset in git.

Change control for ACLs

1. Write policy diff in markdown.
2. Add rules in lab.
3. Positive + negative tests.
4. Only then promote.
5. Never "temporary allow any" without a ticket/expiry note.

Verification checklist

Check	Intent
Policy doc matches counters	Rules do what you think
Default action explicit	No surprise open
Negative tests	Deny paths proven
Management path	You cannot lock yourself out without console plan
Logs sampled	Drops are visible

Common mistakes

Mistake	Symptom
Allow after broad deny	Never hits
Filter only one direction	One-way connectivity
Open SSH to world in lab configs committed	Habit risk
Logging every drop at full rate	Disk / CPU melt
Confusing RIB filter with packet filter	False confidence

Summary

- ACLs are **ordered policies** with an explicit default stance
- Separate **packet**, **route**, and **control-plane** filtering in your head
- Linux netfilter in Containerlab is enough to practice real data-plane policy
- Always pair positive allows with **negative** tests
- Write the policy in prose before the rule engine dialect

Next: **control-plane protection**—stop the CPU from drowning while the data plane still forwards.

Control-Plane Protection

Control-Plane Protection

The data plane can forward millions of packets while a handful of exception packets **kill the control plane**. Control-plane protection (CoPP / CPPr / RE protect—names vary) rate-limits and classifies traffic **to the device CPU**. This chapter builds the model with Linux and FRR-facing labs (Containerlab, mid-2026 open stack).

Learning goals

By the end of this chapter you can:

- Distinguish transit traffic from traffic destined to the router
- List common control-plane protocols worth protecting
- Apply Linux rate limits / policers toward local services
- Design a CoPP policy matrix (class → police → action)
- Predict symptoms of over-tight vs over-loose protection

Concepts

Two paths through a router

```
Transit (data plane):  in → FIB lookup → out
Receive (to CPU):      in → "this is for me" → process
```

Examples to CPU	Examples transit
BGP, OSPF, SSH, SNMP, NTP	Customer flows through the box
ARP to router MAC	Switched L2 through (L2 device)
ICMP echo to router IP	ICMP through to remote host
TTL expiry messages	Normal forwarded packets

Why unprotected CPUs melt

- Routing protocol floods / session storms
- Management scans (SSH/HTTPS)
- ICMP floods to the router IP
- ARP storms
- Fragment / exception paths

Symptoms: neighbor flaps, slow CLI, missed hellos → **self-inflicted outage** even if bandwidth remains.

Policy matrix (start simple)

Class	Match idea	Police	Notes
Critical routing	OSPF/BGP from known peers	generous	Never starve hellos blindly
Management	SSH/HTTPS from mgmt net	moderate	Jump host only
Monitoring	SNMP/telemetry agents	moderate	
Normal ICMP to RE	echo to router	low	
Default to RE	everything else	very low / drop	

Vendors implement class-maps + policers. Linux: nftables meter/limit, tc, or firewall rate modules.

Hardening adjacent controls

CoPP is **not** a substitute for:

- Interface ACLs that drop garbage early
- Management plane VRF / out-of-band
- Routing policy
- Authentication on protocols

Defense in depth: each layer different failure mode.

Lab topology

```
  attacker ----|          |---- peer (OSPF/BGP)
  mgmt --------|  router  |
  client ------|          |---- transit target
```

```yaml
name: copp-lab

topology:
  nodes:
    rtr:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/rtr:/etc/frr
      # also need iptables in image or side package
    peer:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/peer:/etc/frr
    mgmt:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils openssh-client
    attacker:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    target:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["rtr:eth1", "peer:eth1"]
    - endpoints: ["rtr:eth2", "mgmt:eth1"]
    - endpoints: ["rtr:eth3", "attacker:eth1"]
```

```
    - endpoints: ["rtr:eth4", "client:eth1"]
    - endpoints: ["rtr:eth5", "target:eth1"]
```

Addressing

Link	Subnet
rtr–peer	10.0.0.0/30 (.1 rtr, .2 peer)
rtr–mgmt	10.255.0.0/30 (.1 rtr)
rtr–attacker	10.66.0.0/30
rtr–client	10.10.0.0/24 (.1 rtr, .10 client)
rtr–target	10.20.0.0/24 (.1 rtr, .10 target)
rtr lo	192.0.2.1/32

Enable forward for transit client→target; BGP or static for peer experiments.

Baseline routing (static snack)

```
docker exec clab-copp-lab-rtr sysctl -w net.ipv4.ip_forward=1
# addresses omitted-set on each node per table
docker exec clab-copp-lab-client ip route add default via 10.10.0.1
docker exec clab-copp-lab-target ip route add default via 10.20.0.1
```

Optional OSPF between rtr and peer on eth1 (reuse interior-routing habits).

Linux receive-path protection (teaching CoPP)

Protect **SSH to the router** and **ICMP to the router**, without blocking transit.

```
# On rtr (needs iptables in container-install if FRR image lacks it)
docker exec clab-copp-lab-rtr sh -c 'command -v iptables || (apt-get update && apt-get insta
  2>/dev/null || true
```

FRR images vary; if package install is painful, run CoPP rules on an Alpine "router" with `ip_forward` and FRR only when needed. Pattern matters more than package manager.

```
docker exec clab-copp-lab-rtr sh -c '
# INPUT chain = traffic to local stack (control/management plane analogue)
iptables -N COPP 2>/dev/null || iptables -F COPP
iptables -F COPP

# BGP from peer only (TCP 179)
```

```
iptables -A COPP -p tcp -s 10.0.0.2 --dport 179 -j ACCEPT
iptables -A COPP -p tcp -s 10.0.0.2 --sport 179 -j ACCEPT

# OSPF (IP proto 89) from peer subnet
iptables -A COPP -p 89 -s 10.0.0.0/30 -j ACCEPT

# SSH from mgmt only, rate-limit new connections
iptables -A COPP -p tcp -s 10.255.0.0/30 --dport 22 -m conntrack --ctstate NEW -m limit --li
iptables -A COPP -p tcp -s 10.255.0.0/30 --dport 22 -m conntrack --ctstate ESTABLISHED -j AC

# ICMP to self: low rate
iptables -A COPP -p icmp --icmp-type echo-request -m limit --limit 5/sec -j ACCEPT

# established to self
iptables -A COPP -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# drop other to-self
iptables -A COPP -j DROP

iptables -I INPUT 1 -j COPP
'
```

Critical: do **not** put transit in INPUT. FORWARD remains separate:

```
docker exec clab-copp-lab-rtr sh -c '
iptables -P FORWARD ACCEPT
# or tighter ACL from previous chapter
'
```

Predict → observe → fix

Drill 1 — Transit still works under "attack" to RE

```
# flood ping to router IP from attacker
docker exec clab-copp-lab-attacker sh -c 'ping -f -c 1000 10.66.0.1 || ping -c 200 10.66.0.1
# simultaneous transit
docker exec clab-copp-lab-client ping -c 20 10.20.0.10
```

Predict: transit mostly fine; many attacker pings dropped by limit; CLI may still respond if limits work.

Observe: `iptables -L COPP -n -v` counters.

Fix if transit dies: rules mistakenly in FORWARD; CPU still overloaded—tighten attacker path earlier (interface ACL).

Drill 2 — SSH from non-mgmt

```
docker exec clab-copp-lab-attacker sh -c 'nc -w2 10.66.0.1 22 || echo blocked'
docker exec clab-copp-lab-mgmt sh -c 'nc -w2 10.255.0.1 22 || true'
```

Predict: attacker blocked; mgmt allowed (if sshd up).

Drill 3 — Over-tight BGP protection

Intentionally rate-limit TCP 179 too aggressively:

```
iptables -I COPP 1 -p tcp --dport 179 -m limit --limit 1/hour -j ACCEPT
iptables -I COPP 2 -p tcp --dport 179 -j DROP
```

Predict: BGP session flaps or won't establish.
Observe: `vtysh -c 'show bgp summary'`.
Fix: remove bad rules; document that **routing classes need headroom**.

Drill 4 — Peer-only OSPF

Send spoofed or neighbor attempt from attacker interface (concept): OSPF from non-peer should not be accepted.

Observe: no adjacency from wrong interface; INPUT drops proto 89 from wrong sources.

FRR-side levers (awareness)

Lever	Role
`ttl-security` / GTSM	BGP hop check
MD5/TCP-AO (as available)	Session auth
Interface ACLs + passive-interface	Reduce attack surface
Max prefix	Limit RIB blowups
BFD timers sanity	Avoid micro-flap storms

These complement packet CoPP; they do not replace it.

Building a CoPP document

```
Device role: lab edge
Classes:
  - ROUTING_PEERS: list IPs/protos, police 1Mbps, prio high
```

```
  - MGMT: 10.255.0.0/30 tcp/22, 100kbps
  - ICMP_RE: 64kbps
  - DEFAULT_RE: 32kbps drop excess
Exceptions: console always
Change window: test in lab before prod
Rollback: saved ruleset file in git
```

Store as `copp-policy.md` beside topology.

IPv6 notes

Same model: traffic to link-local and global router addresses hits receive path. ND/RA are control-plane-adjacent—filter carefully or you break neighbor discovery.

Optional SR Linux / NOS CoPP

Commercial and community NOS expose CoPP/CPM filter policies in YANG. Map your matrix classes to their filter entries; validate with protocol sessions + flood tests. Free path still proves the **idea** on Linux.

Verification checklist

Check	Intent
Transit ping during RE flood	Data vs control separation
Known peer protocols up	Not over-policed
Unknown src management down	Surface reduced
Counters move on floods	Policy hit
Rollback file exists	Safety

Common mistakes

Mistake	Symptom
Policing FORWARD as “CoPP”	Kills customers, not RE
Starving BGP/OSPF	Random reconvergence
No mgmt exception	Locked out
Only cloud dashboards, no on-box	Outage of mgmt path blind
Copy-paste prod CoPP into lab scale	Lab protocols die

Summary

- Control-plane protection polices **traffic to the CPU**, not transit
- Build a **class matrix** before touching syntax
- Linux INPUT rate limits teach the model; NOS CoPP is the same idea
- Always retest **routing adjacencies** after tightening
- Combine with mgmt ACLs, auth, and max-prefix for defense in depth

Next: **QoS vocabulary**—how queues and marks express intent without vendor exam laundry lists.

QoS Vocabulary

QoS Vocabulary

Quality of Service is a pile of vendor features if you start from menus. Start from **vocabulary and intent**: classify → mark → queue → schedule → (maybe) shape/police. This chapter stays conceptual with small Linux `tc` experiments in Containerlab—no paid QoS licenses (mid-2026).

Learning goals

By the end of this chapter you can:

- Explain classification, marking, queuing, scheduling, policing, shaping
- Map DSCP/PHB ideas without memorizing every codepoint table
- Build a minimal Linux htb/fq_codel lab and measure fairness under load
- Predict what happens when you police vs shape
- Know what *not* to promise with lab QoS

Concepts — the pipeline

```
packets in

Classify  (match: 5-tuple, interface, DSCP, VLAN, app)

Mark      (set DSCP / internal class / MPLS TC / VLAN PCP)

Police    (drop/remark excess - hard limit)

Queue     (buffers per class)

Schedule  (which queue sends next: PQ, WRR, WFQ-like…)
```

```
Shape     (smooth egress to rate - buffer then send)


wire
```

Not every hop does every stage. Trust boundaries matter: **remark at edges**, honor marks in the core only if you trust them.

Vocabulary table

Term	Meaning
Classification	Deciding which class a packet belongs to
Marking	Writing a field others can see (e.g. DSCP)
PHB	Per-hop behavior—what a class *should* experience
Policing	Enforce rate; excess drop or remark (little buffering)
Shaping	Enforce rate by delaying (buffering) excess
Queue depth	How much can wait; latency vs drop trade-off
Congestion avoidance	ECN, RED/WRED-like early drop/mark
Scheduling	Priority vs sharing bandwidth among classes
LLQ / priority queue	Low-latency class that can starve others if unbounded
Fair queueing	Prevent one flow from dominating (fq / fq_codel ideas)
Trust boundary	Where you accept or rewrite marks

DSCP / PHB (operator view)

Intent class	Typical idea	Notes
Network control	Highest care for protocols	Protect, don't just "EF everything"
Expedited (EF-like)	Low latency, low loss	Strict admission; small volume
Assured / AF-like	Business data tiers	Drop precedence inside class
Default / BE	Best effort	Most traffic
Scavenger	Below BE	Backups, bulk

Exact codepoints are standards-listed; **do not** treat exam bingo as design. Document *your* map:

```
EF-like   → voice lab class (tiny)
AF31-like → interactive
CS6-like  → routing (if marked)
BE        → default
```

Policing vs shaping

	Police	Shape
Excess	Drop/remark	Delay in buffer
Latency	Can be lower for in-profile	Adds delay when congested
Use	Enforce contract, ingress	Match egress to circuit rate

Predict: shaping a 100 mbit stream to 10 mbit increases latency under load; policing 10 mbit drops packets and breaks TCP more brutally.

Linux lab topology

```
name: qos-vocab

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["r1:eth2", "h2:eth1"]
```

Addressing

Node	IP
h1	10.80.1.10/24 via 10.80.1.1
r1 eth1	10.80.1.1/24

Node	IP
r1 eth2	10.80.2.1/24
h2	10.80.2.10/24 via 10.80.2.1

```
docker exec clab-qos-vocab-r1 sysctl -w net.ipv4.ip_forward=1
# apply addresses on all nodes...
```

Experiment 1 — bottleneck without QoS

```
# limit r1 eth2 with a simple tbf shape to create congestion
docker exec clab-qos-vocab-r1 tc qdisc replace dev eth2 root tbf rate 5mbit burst 32kbit lat

docker exec -d clab-qos-vocab-h2 iperf3 -s
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -t 20 -P 4
```

Predict

Aggregate ~5 Mbit; latency rises; flows share roughly (TCP dynamics).

Observe

```
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -t 10 -J | head
docker exec clab-qos-vocab-r1 tc -s qdisc show dev eth2
```

Experiment 2 — two classes with HTB

Goal: bulk class 3 mbit ceiling; interactive class 2 mbit guaranteed-ish; total 5 mbit.

```
docker exec clab-qos-vocab-r1 sh -c '
tc qdisc del dev eth2 root 2>/dev/null || true
tc qdisc add dev eth2 root handle 1: htb default 20
tc class add dev eth2 parent 1: classid 1:1 htb rate 5mbit ceil 5mbit
tc class add dev eth2 parent 1:1 classid 1:10 htb rate 2mbit ceil 5mbit prio 1
tc class add dev eth2 parent 1:1 classid 1:20 htb rate 3mbit ceil 5mbit prio 2
tc qdisc add dev eth2 parent 1:10 handle 10: fq_codel
tc qdisc add dev eth2 parent 1:20 handle 20: fq_codel
# classify by destination port: "interactive" = 5202, bulk = 5201
tc filter add dev eth2 protocol ip parent 1:0 prio 1 u32 match ip dport 5202 0xffff flowid 1
tc filter add dev eth2 protocol ip parent 1:0 prio 2 u32 match ip dport 5201 0xffff flowid 1
'
```

Run two iperf servers/ports on h2:

```
docker exec -d clab-qos-vocab-h2 iperf3 -s -p 5201
docker exec -d clab-qos-vocab-h2 iperf3 -s -p 5202
docker exec -d clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -p 5201 -t 30 -P 2
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -p 5202 -t 20
```

Predict

Interactive class gets better latency / share under contention than pure bulk; total still capped ~5 mbit.

Observe

```
docker exec clab-qos-vocab-r1 tc -s class show dev eth2
docker exec clab-qos-vocab-r1 tc -s qdisc show dev eth2
```

Fix

Wrong filter → all traffic in default class. Check `flowid` and dport match (u32 host byte care).

Experiment 3 — police vs shape feel

```
# police ingress-ish on r1 eth1 (example)
docker exec clab-qos-vocab-r1 sh -c '
tc qdisc replace dev eth1 handle ffff: ingress
tc filter add dev eth1 parent ffff: protocol ip u32 match u32 0 0 \
  police rate 2mbit burst 20k drop flowid :1
'
```

Compare TCP goodput and retransmissions vs tbf shape on egress.

Predict: police drops → TCP sawtooth pain; shape delays → higher RTT, smoother rate.

Marking sketch (DSCP)

```
# set DSCP on packets from h1 (example iptables)
docker exec clab-qos-vocab-h1 sh -c '
apk add --no-cache iptables
iptables -t mangle -A OUTPUT -p tcp --dport 5202 -j DSCP --set-dscp 0x2e
'
```

```
# on r1 classify on DSCP instead of dport (tc flower/u32 match)
```

Predict

If r1 trusts and classifies EF-like mark into priority class, marked flows win under load.

Observe

```
docker exec clab-qos-vocab-r1 sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 -v -c 5 tc
```

What lab QoS cannot prove

- Hardware queue behavior of a specific ASIC
- Carrier hierarchical QoS at scale
- Exact voice MOS scores
- Multi-vendor PHB interoperability without careful design

It **can** prove: classification logic, bottlenecks, unfairness, and the difference between police/shape.

FRR / NOS note

FRR is not your main QoS engine. Real NOS (including SR Linux community) expose class-maps and queues in their models. Translate:

```
class VOICE → match DSCP EF → queue LLQ-like → police admit
class BULK  → match default → shaped remainder
```

Same vocabulary.

Design mini-checklist

- ☐ Trust boundary defined
- ☐ Classes 4–5 for first design
- ☐ Admission for low-latency class

- ☐ Capacity math: sum guarantees link
- ☐ Congestion metrics (drop, latency, ECN) collected
- ☐ Failure mode: QoS mis-class worse than no QoS?

Predict → observe → fix template

Step	Example
Predict	Bulk should not starve interactive under 5 mbit cap
Observe	iperf + `tc -s` + ping RTT during load
Fix	filters, class rates, default class, ECN/fq_codel

Common mistakes

Mistake	Symptom
Mark without classifying later	Marks are cosmetics
Guarantees sum > link	Unexpected drops
Priority unbounded	Starvation of BE
QoS to "fix" bandwidth shortage	Still need capacity
Policing management/routing	Self-outage (see CoPP)

Summary

- QoS is a **pipeline of intents**, not a single CLI feature
- Classify → mark → queue → schedule → police/shape
- Linux `tc` + iperf3 in Containerlab is enough to feel congestion policy
- Document trust boundaries and class maps in git
- Keep routing/management classes out of scavenger police

Next: **hardening checklists**—turn models into repeatable gates for labs and edges.

Hardening Checklists

Hardening Checklists

Hardening is not a one-time "secure the router" checkbox. It is a **repeatable set of gates** you run after every design change. This chapter consolidates edge, control-plane, ACL, and ops hygiene into checklists you can attach to Containerlab labs (mid-2026 open stack: FRR, Linux, optional SR Linux notes).

Learning goals

By the end of this chapter you can:

- Run role-based hardening checklists (edge, P, lab host)
- Turn checklist items into automated `verify-hard.sh` probes
- Balance security with operability (break-glass paths)
- Record exceptions with expiry and owner
- Apply the same gates to dual-home and fabric labs

How to use these lists

1. **Pick the role** of the device or lab.
2. Mark each item: Pass / Fail / N/A / Exception.
3. Failures become backlog; exceptions need owner + date.
4. Automate the boring Pass/Fail checks.
5. Re-run after every topology or policy change.

```
Design → Implement → Functional verify → Hardening verify → Document
```

Checklist 0 — Lab hygiene (every topology)

#	Gate	Pass looks like
0.1	Topology in git	`.clab.yml` + configs committed
0.2	Addressing plan	No overlapping accidental spaces
0.3	Secrets	No production passwords in repo
0.4	Destroy works	`containerlab destroy -c` clean
0.5	Logs bound	Evidence survives destroy
0.6	Images free	Main path needs no paid license
0.7	Resource note	RAM/CPU expected documented

Checklist 1 — Management plane

#	Gate	Pass looks like
1.1	SSH/API not on all interfaces blindly	Listen limited or filtered
1.2	Mgmt sources limited	ACL/CoPP allow jump net only
1.3	Auth	Keys or strong lab passwords; no default vendor admin left
1.4	Banner / MOTD	Optional but marks non-prod
1.5	Out-of-band story	Console or dual path to recover
1.6	Telemetry endpoints known	Who scrapes what
1.7	DNS for mgmt names	Optional but reduces fat-finger

Linux probes

```
# who listens?
docker exec clab-X-r1 ss -lntp
# INPUT policy not wide open without reason
docker exec clab-X-r1 iptables -L INPUT -n -v | head
```

Checklist 2 — Control plane & routing

#	Gate	Pass looks like
2.1	Passive interfaces on access LANs	No accidental OSPF on user ports
2.2	BGP peer ACLs / TTL security as applicable	Only intended peers
2.3	Max-prefix on eBGP	Limit configured + action known
2.4	Prefix filters in and out	Negative hijack test fails closed

#	Gate	Pass looks like
2.5	Router-IDs unique	Documented
2.6	Auth on IGP/BGP where in scope	Or explicit exception
2.7	CoPP/receive ACL present on edges	Matrix documented
2.8	Loop prevention	No rediscovery of classic redistrib loops

FRR probes

```
docker exec clab-X-r1 vtysh -c 'show ip ospf interface'
docker exec clab-X-r1 vtysh -c 'show bgp summary'
docker exec clab-X-r1 vtysh -c 'show run' | grep -E 'prefix-list|route-map|max-prefix|passiv
```

Checklist 3 — Data plane filters

#	Gate	Pass looks like
3.1	Written policy exists	Prose before rules
3.2	Default stance explicit	Deny or allow documented
3.3	Positive tests	Needed paths work
3.4	Negative tests	Forbidden paths fail
3.5	Anti-spoof (uRPF-like idea)	As scope allows
3.6	Directed broadcast / dangerous services off	No chargen era nonsense
3.7	IPv6 filters parity	Not “v6 open because forgot”

Checklist 4 — Edge services

#	Gate	Pass looks like
4.1	DHCP pools correct	giaddr/relay tested
4.2	NAT scoped	Only intended interfaces
4.3	VRRP/anycast VIP documented	Single master observed
4.4	DNS dependency known	Outage drill done once
4.5	Time sync	chrony/ntp status OK
4.6	Logging on role changes	VRRP/BGP flap visible

Checklist 5 — Secrets, users, supply chain

#	Gate	Pass looks like
5.1	Image sources pinned	Digests/tags known
5.2	No shared root SSH everywhere	Or accepted lab-only risk noted
5.3	CI does not print secrets	Logs clean
5.4	Third-party scripts reviewed	Especially curl

Checklist 6 — Resilience & abuse

#	Gate	Pass looks like
6.1	Link failure drill	Recover within budget
6.2	Peer failure drill	Documented
6.3	Flood to RE drill	Transit survives
6.4	Config rollback	Known good in git
6.5	Capacity headroom	Lab not at 100% RAM always

Role profiles (quick)

Lab host (Containerlab hypervisor)

- Disk for images; unused labs destroyed
- Access limited to operators
- sysctl baseline documented
- Docker/podman not exposed to world

FRR edge CE

- Checklists 1–4 heavily
- Dual-home policies + VRRP tracking story
- Export only owned space

FRR P / fabric leaf

- Minimal services listening
- Underlay filters; no customer ACLs confusion
- ECMP + BFD timer sanity
- CoPP light but present

Linux bridge “switch”

- No IP on data plane unless needed
- STP/loop story if L2 loops possible
- Do not run random services on bridge NS

Exception register template

ID	Control	Reason	Owner	Expiry	Review
E-001	BGP without TTL security	Lab only single hop	you	2026-12-01	

Exceptions without expiry become permanent holes.

Automating gates — `verify-hard.sh`

```
#!/usr/bin/env bash
set -euo pipefail
LAB=${LAB:-clab-dualhome}
fail() { echo "FAIL: $*" >&2; exit 1; }
pass() { echo "PASS: $*"; }

# 2.4 negative: hijack absent on pe1
if docker exec ${LAB}-pe1 vtysh -c 'show bgp ipv4 uni' 2>/dev/null | grep -q '203.0.113.0';
  fail "hijack prefix present"
else
  pass "no hijack prefix"
fi

# 1.2 example: ssh port not open to all - soft check listening interfaces
```

```
docker exec ${LAB}-ce1 ss -lntp | grep -q ssh && pass "sshd present" || pass "no sshd (ok if

# 4.3 VIP answers
docker exec ${LAB}-h1 ping -c1 -W1 192.168.50.254 >/dev/null && pass "VIP" || fail "VIP"

# 0.1 topology file exists
test -f dualhome.clab.yml && pass "topology file" || fail "missing topology"

echo "HARDENING SMOKE OK"
```

Wire into Makefile:

```
smoke: ; ./verify.sh
hard:  ; ./verify-hard.sh
```

Predict → observe → fix (meta-drill)

Break one control on purpose:

```
# remove FROM-CE filter on pe1 in lab
```

Predict | `verify-hard.sh` fails hijack test when you advertise junk |
Observe | script FAIL line |
Fix | restore route-map; re-run |

Never disable a gate silently—disable via exception register.

Mapping to prior chapters

Chapter theme	Checklist
ACL thinking	3.x
CoPP	2.7, 6.3
QoS	don't police control into death; document classes
Dual-home edge	2.x + 4.x
Name/time/log	4.4–4.6, 0.5

Optional SR Linux

Export checklist to NOS-specific show commands in an appendix note. Same gates: peer authenticity, prefix limits, mgmt ACL, logging. Free community image is for dialect practice, not a second security model.

Definition of done — hardened lab

- ☐ Role checklist completed with Pass/Fail
- ☐ Exceptions registered
- ☐ `verify.sh` + `verify-hard.sh` green
- ☐ One intentional fail demonstrated (gate works)
- ☐ Rollback path known

Common mistakes

Mistake	Symptom
Checklist once, never again	Drift
Automating only happy path	False green
Lockout without console	Unrecoverable lab (or prod!)
Copying bank-grade CoPP into tiny lab	Protocols die
Paper checklists not in repo	Lost tribal knowledge

Summary

- Hardening is **repeatable gates**, not a mood
- Use role-based checklists + exception register
- Automate negative tests (hijack, flood survival, mgmt limits)
- Re-run after every material change
- Operability (break-glass) is part of security

Next part: **overlays and tunnels**—why we encapsulate, and how free stacks practice VXLAN/EVPN-class ideas.

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow

A TCP server commits memory before it knows the client is real. The first SYN makes the kernel allocate a request socket, send a SYN-ACK, and wait. A **SYN flood** sends SYNs from spoofed addresses that will never send the final ACK, so those half-open entries pile up until the listener refuses real clients. Linux has two queues per listener, two different limits, and a stateless escape hatch, **SYN cookies**, that is on by default. Most "we tuned it and nothing changed" incidents come from mixing up which queue overflowed. This chapter separates them, measures each one with real counters, and runs an actual spoofed flood, without root, inside a throwaway network namespace.

Learning goals

- Draw the SYN queue (request sockets, `SYN_RECV`) and the accept queue (`ESTABLISHED`, waiting for `accept()`) and say which limit applies to each
- Explain why `net.ipv4.tcp_max_syn_backlog` does **not** size the SYN queue on a modern kernel, and what it actually does
- Read `ss -lnt Recv-Q/Send-Q` on a listener and the `TcpExt` counters (`ListenOverflows`, `ListenDrops`, `TCPReqQFullDrop`, `TCPReqQFullDoCookies`, `Syncookies*`)
- Say what a SYN cookie encodes, and what a cookie-built connection loses (MSS precision, and every option when timestamps are off)
- Tell a flood from an application that accepts too slowly, and fix the right one
- Reproduce a spoofed SYN flood safely with an unprivileged user + network namespace

Lab platform: Linux 6.12, iproute2 6.15 (`ss`, `ip`, `nstat`), Python 3.13.5, run as an unprivileged user. Each lab runs in its own `unshare -rn` namespace: root there, with its own sysctls, raw sockets, and veth, and no effect on the host. Every listing output below is pasted from those runs. Kernel behaviour is quoted from `net/ipv4/tcp_input.c`, `tcp_minisocks.c`, and `syncookies.c` in Linux 6.12.

This chapter covers the host's own defences. Upstream scrubbing, SYN proxies in front of the server (`nftables synproxy`, load balancers), and XDP drop paths are out of scope here.

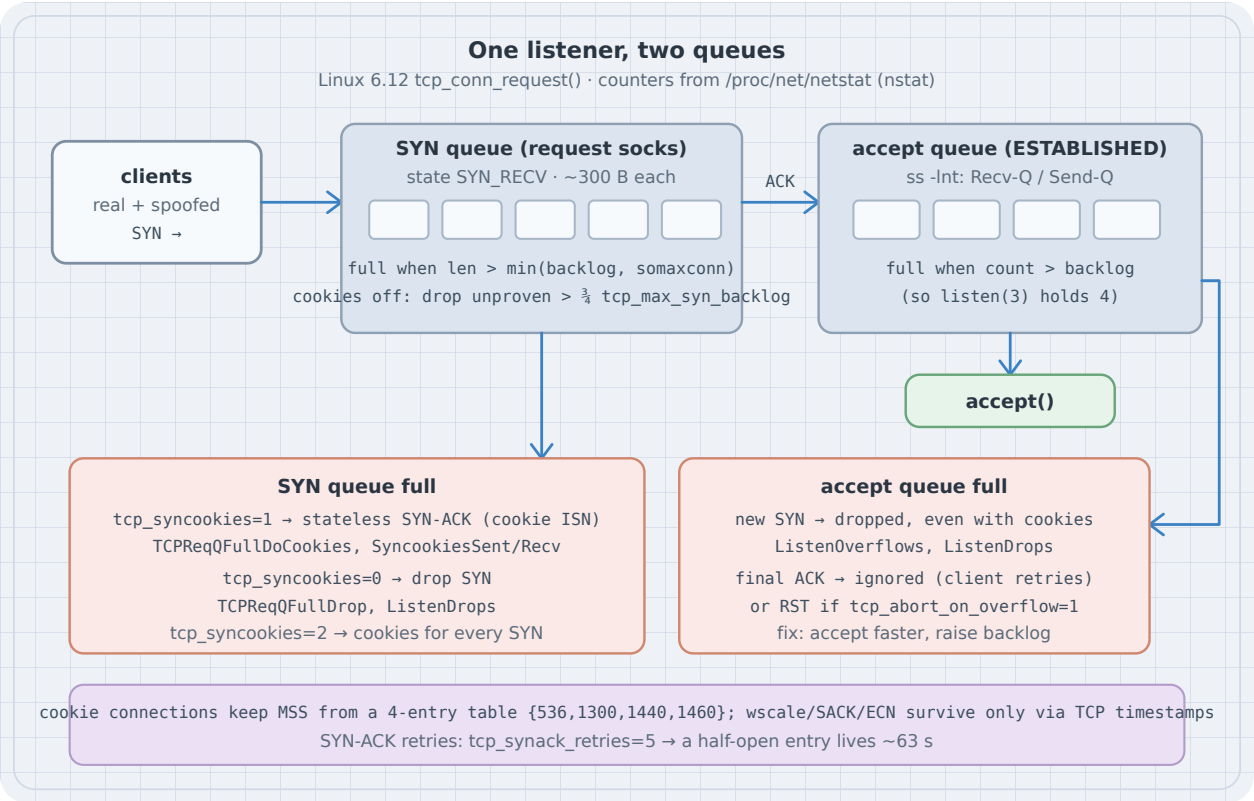


Figure 1: One listener, two queues

Mental model

```
              SYN                          final ACK                 accept()
client          [ SYN queue ]              [ accept queue ]      application
                     request socks, SYN_RECV          full sockets, ESTABLISHED
                     limit: len > min(backlog,         limit: count > min(backlog,
                            somaxconn)                        somaxconn)
                     overflow: cookie or drop         overflow: drop SYN; ignore ACK
                                                                 (RST if abort_on_overflow)
```

Both queues hang off the same `listen()` call. In Linux 6.12, `tcp_conn_request()` checks them in this order for every incoming SYN:

1. `tcp_syncookies == 2`, or the **SYN queue** is full (`inet_csk_reqsk_queue_len(sk) > sk_max_ack_backlog`)? Then `tcp_syn_flood_action()`: with cookies enabled, answer with a cookie (`TCPReqQFullDoCookies`); otherwise drop (`TCPReqQFullDrop`).
2. The **accept queue** is full (`sk_ack_backlog > sk_max_ack_backlog`)? Drop the SYN (`ListenOverflows`). Cookies do not help here.
3. Cookies off and the SYN queue more than three quarters of `tcp_max_syn_backlog`? Drop SYNs from peers the kernel has not seen before (`tcp_peer_is_proven()` is false). `ListenDrops` only.

4. Otherwise allocate a request socket (300 bytes), send the SYN-ACK, and retransmit it up to `tcp_synack_retries` times (5 → about 63 s before giving up).

`sk_max_ack_backlog` is the `backlog` you passed to `listen()`, silently capped at `net.core.somaxconn` (default 4096 since Linux 5.4). The `>` comparisons mean `listen(N)` holds **N + 1** entries.

Knob	Default here	What it really does on 6.12
`listen(fd, backlog)`	app-chosen	Sets **both** queue limits (`sk_max_ack_backlog`)
`net.core.somaxconn`	4096	Caps `backlog` without telling the application
`net.ipv4.tcp_syncookies`	1	0 = never, 1 = when the SYN queue is full, 2 = always
`net.ipv4.tcp_max_syn_backlog`	1024 (128, scales with RAM)	Only with cookies **off**: keep the last quarter for proven peers
`net.ipv4.tcp_synack_retries`	5	How long a half-open entry occupies a slot
`net.ipv4.tcp_abort_on_overflow`	0	Final ACK into a full accept queue: 0 = ignore it, 1 = send RST

All of these are per network namespace, which is what makes the labs below safe.

Concepts — what a SYN cookie is

With a cookie, the server keeps **no state**. It encodes the essentials in the 32-bit initial sequence number of its SYN-ACK: a keyed hash of the 4-tuple and a coarse timestamp, plus a 2-bit index into a table of four MSS values (`msstab = {536, 1300, 1440, 1460}` in `net/ipv4/syncookies.c`). When an ACK arrives with no request socket behind it, the kernel checks `ack - 1` against the hash (valid for two 60-second periods), rebuilds the connection, and counts `SyncookiesRecv` (or `SyncookiesFailed`).

The price is the TCP options. The ISN has no room for window scale, SACK, or ECN. Linux hides them in the low 6 bits of the TCP timestamp value if the client sent timestamps (`TS_OPT_WSCALE_MASK`, `TS_OPT_SACK`, `TS_OPT_ECN`). If timestamps are off, a cookie connection has no window scaling (64 KiB window), no SACK, and an MSS rounded down to the table. The kernel's own sysctl documentation is blunt: cookies are a fallback and "seriously violate TCP protocol"; if you see SYN-flood warnings from **legitimate** load, fix the backlog, not the cookies.

Lab — setup: a private network namespace with a void

`unshare -rn` gives an unprivileged user a new user namespace (mapped to root) and a new network namespace. The helper below adds a veth so the server has a real address (`10.9.0.1`) and routes

TEST-NET-2 (`198.51.100.0/24`, RFC 5737) to a fake next hop. SYN-ACKs to "spoofed" clients leave `v0` and are never answered, exactly like a real spoofed flood.

```
#!/bin/sh
# netns_lab.sh - run CMD inside a throwaway network namespace:
#   lo up, veth v0 = 10.9.0.1/24, and 198.51.100.0/24 (TEST-NET-2) routed into a void
# usage: unshare -rn ./netns_lab.sh CMD [ARGS...]
set -e
ip link set lo up
ip link add v0 type veth peer name v1
ip link set v0 up && ip link set v1 up
ip addr add 10.9.0.1/24 dev v0
ip route add 198.51.100.0/24 via 10.9.0.2 dev v0
# a fake next hop: SYN-ACKs to the "spoofed" clients leave v0 and are never answered
ip neigh replace 10.9.0.2 lladdr 02:00:00:00:00:02 dev v0 nud permanent
exec "$@"
```

```
#!/bin/sh
# listen_defaults.sh - the knobs that size and protect a TCP listener (this netns)
for k in net/ipv4/tcp_syncookies net/ipv4/tcp_max_syn_backlog \
         net/ipv4/tcp_synack_retries net/ipv4/tcp_abort_on_overflow net/core/somaxconn; do
  printf '%-32s %s\n' "$k" "$(cat /proc/sys/$k)"
done
nstat -az TcpExtListenOverflows TcpExtListenDrops TcpExtTCPReqQFullDrop \
          TcpExtTCPReqQFullDoCookies TcpExtSyncookiesSent TcpExtSyncookiesRecv
```

```
chmod +x netns_lab.sh listen_defaults.sh
./listen_defaults.sh                              # host namespace, read-only
unshare -rn ./netns_lab.sh ip route               # the lab namespace
```

```
net/ipv4/tcp_syncookies          1
net/ipv4/tcp_max_syn_backlog     1024
net/ipv4/tcp_synack_retries      5
net/ipv4/tcp_abort_on_overflow   0
net/core/somaxconn               4096
#kernel
TcpExtSyncookiesSent            0                  0.0
TcpExtSyncookiesRecv            0                  0.0
TcpExtListenOverflows           0                  0.0
TcpExtListenDrops               0                  0.0
TcpExtTCPReqQFullDoCookies      0                  0.0
TcpExtTCPReqQFullDrop           0                  0.0
```

```
10.9.0.0/24 dev v0 proto kernel scope link src 10.9.0.1
198.51.100.0/24 via 10.9.0.2 dev v0
```

The host counters are all zero, and they stay zero after every lab below: each `unshare -rn` namespace has its own `/proc/net/netstat` and its own sysctls.

`tcpext.py` is the shared counter reader (the same numbers `nstat` prints):

```
# tcpext.py - read TcpExt counters from /proc/net/netstat (what `nstat` reports)
KEYS = ("ListenOverflows", "ListenDrops", "TCPReqQFullDrop",
        "TCPReqQFullDoCookies", "SyncookiesSent", "SyncookiesRecv", "SyncookiesFailed")

def tcpext():
    lines = open("/proc/net/netstat").read().splitlines()
    for names, values in zip(lines[::2], lines[1::2]):
        if names.startswith("TcpExt:"):
            return dict(zip(names.split()[1:], map(int, values.split()[1:])))

def delta(before, after):
    return {k: after[k] - before[k] for k in KEYS if after[k] != before[k]}
```

Lab — accept queue: a server that never calls `accept()`

```
# accept_queue_lab.py - a listener that never calls accept(): what happens to client #5?
# usage: unshare -rn ./netns_lab.sh python3 accept_queue_lab.py BACKLOG CLIENTS ABORT_ON_OVE
import socket, subprocess, sys, time
from tcpext import tcpext, delta

backlog, clients, abort = map(int, sys.argv[1:4])
with open("/proc/sys/net/ipv4/tcp_abort_on_overflow", "w") as f:
    f.write(str(abort))

lsock = socket.socket()
lsock.bind(("10.9.0.1", 8080))
lsock.listen(backlog)                          # never accept()

before = tcpext()
socks = []
for i in range(clients):
    c = socket.socket()
    c.setblocking(False)
    c.connect_ex(("10.9.0.1", 8080))           # non-blocking: SYN goes out now
    socks.append(c)
    time.sleep(0.05)
time.sleep(1.5)                                # let one SYN retransmit (RTO 1 s) happen

print(f"listen({backlog}), {clients} clients, tcp_abort_on_overflow={abort}")
print(subprocess.run(["ss", "-ltn", "sport = :8080"], capture_output=True, text=True).stdout
```

```
states = subprocess.run(["ss", "-Htn", "dport = :8080"], capture_output=True, text=True).std
count = {}
for line in states.splitlines():
    count[line.split()[0]] = count.get(line.split()[0], 0) + 1
print("client-side states:", count)
srv = subprocess.run(["ss", "-Htn", "state", "syn-recv", "sport = :8080"],
                     capture_output=True, text=True).stdout
print("server SYN-RECV:", len(srv.splitlines()))
print("TcpExt deltas:", delta(before, tcpext()))
```

```
unshare -rn ./netns_lab.sh python3 accept_queue_lab.py 3 8 0
unshare -rn ./netns_lab.sh python3 accept_queue_lab.py 3 8 1
```

```
listen(3), 8 clients, tcp_abort_on_overflow=0
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 4      3           10.9.0.1:8080      0.0.0.0:*
client-side states: {'SYN-SENT': 4, 'ESTAB': 4}
server SYN-RECV: 0
TcpExt deltas: {'ListenOverflows': 8, 'ListenDrops': 8}

listen(3), 8 clients, tcp_abort_on_overflow=1
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 4      3           10.9.0.1:8080      0.0.0.0:*
client-side states: {'ESTAB': 4, 'SYN-SENT': 4}
server SYN-RECV: 0
TcpExt deltas: {'ListenOverflows': 8, 'ListenDrops': 8}
```

How to read it:

- On a **listening** socket, `ss` reuses the columns: `Recv-Q` is the current accept-queue length (4), `Send-Q` is its limit (3). `Recv-Q` = `Send-Q` + 1 is the "full" signature, because the check is `>`.
- Four clients completed the handshake and sit `ESTAB` in the queue. The application has not seen them, but the clients think they are connected and may already be sending.
- The other four never got a SYN-ACK. Their SYNs hit step 2 (accept queue full) and were dropped. Each client retransmitted once after the 1 s initial RTO, so $4 \times 2 = \mathbf{8}$ `ListenOverflows`. They stay in `SYN-SENT` and keep retrying with backoff; if the application wakes up and calls `accept()`, they get in.
- `tcp_abort_on_overflow=1` changed nothing. It acts only in `tcp_check_req()` when a **final ACK** arrives for a request whose accept queue filled in the meantime (`listen_overflow:` → `embryonic_reset`). Here the SYNs were refused before any handshake. In production it turns a short burst into connection resets for clients that would have recovered, which is why the kernel docs say to enable it only if the daemon really cannot keep up.
- No `SYN_RECV` entries and no cookie counters: this is not a flood. It is an application that does not accept fast enough, and the fix is in the application or its `listen()` backlog.

Lab — `somaxconn` silently caps the backlog

```
# somaxconn_lab.py - listen() backlog is silently capped by net.core.somaxconn
# usage: unshare -rn ./netns_lab.sh python3 somaxconn_lab.py
import socket, subprocess

def show(somaxconn, backlog):
    open("/proc/sys/net/core/somaxconn", "w").write(str(somaxconn))
    s = socket.socket()
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(("10.9.0.1", 8080))
    s.listen(backlog)
    line = subprocess.run(["ss", "-Hltn", "sport = :8080"], capture_output=True, text=True).
    print(f"somaxconn={somaxconn:<5} listen({backlog:<6}) -> ss Send-Q={line[2]}")
    s.close()

show(4096, 128)
show(4096, 65535)
show(128, 1024)
```

```
unshare -rn ./netns_lab.sh python3 somaxconn_lab.py
```

```
somaxconn=4096  listen(128   ) -> ss Send-Q=128
somaxconn=4096  listen(65535 ) -> ss Send-Q=4096
somaxconn=128   listen(1024  ) -> ss Send-Q=128
```

`listen()` never fails because the backlog is too large. `__sys_listen_socket()` in `net/socket.c` clamps it to `somaxconn`, and only `ss -lnt` `Send-Q` shows what the kernel actually used. Many servers have their own setting (nginx `listen … backlog=`, default 511 on Linux; HAProxy `backlog`; Go's `net` package passes `somaxconn` itself; Java's `ServerSocket` defaults to 50). Raising `somaxconn` without raising the application value does nothing, and the reverse is capped.

Lab — a spoofed SYN flood, three ways

The flood builds IPv4 + TCP SYN headers by hand on a raw socket (allowed in the namespace, where the user is root). Sources cycle through `198.51.100.1-254`. After the flood, one **real** client in the same namespace tries to connect with a 3 s timeout.

```
# synflood_lab.py - spoofed SYN flood against a listener, inside a private netns.
# usage: unshare -rn ./netns_lab.sh python3 synflood_lab.py COOKIES MAX_SYN_BACKLOG BACKLOG
import os, random, socket, struct, sys, time
from tcpext import tcpext, delta

cookies, max_syn_backlog, backlog, syns = map(int, sys.argv[1:5])
```

```
SERVER, PORT = "10.9.0.1", 8080

def sysctl(name, value):
    with open("/proc/sys/" + name.replace(".", "/"), "w") as f:
        f.write(str(value))

sysctl("net.ipv4.tcp_syncookies", cookies)
sysctl("net.ipv4.tcp_max_syn_backlog", max_syn_backlog)

lsock = socket.socket()
lsock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
lsock.bind((SERVER, PORT))
lsock.listen(backlog)

def csum(data):
    if len(data) % 2:
        data += b"\0"
    s = sum(struct.unpack(f"!{len(data)//2}H", data))
    s = (s >> 16) + (s & 0xFFFF)
    return ~(s + (s >> 16)) & 0xFFFF

def syn(src, sport):
    dst = socket.inet_aton(SERVER)
    seq = random.getrandbits(32)
    tcp = struct.pack("!HHIIBBHHH", sport, PORT, seq, 0, 5 << 4, 0x02, 64240, 0, 0)
    pseudo = socket.inet_aton(src) + dst + struct.pack("!BBH", 0, 6, len(tcp))
    tcp = tcp[:16] + struct.pack("!H", csum(pseudo + tcp)) + tcp[18:]
    ip = struct.pack("!BBHHHBBH4s4s", 0x45, 0, 20 + len(tcp), 0, 0, 64, 6, 0,
                     socket.inet_aton(src), dst)
    return ip + tcp

def syn_recv_count():
    n = 0
    with open("/proc/net/tcp") as f:
        next(f)
        for line in f:
            local, state = line.split()[1], line.split()[3]
            if state == "03" and local.endswith(f":{PORT:04X}"):   # 03 = SYN_RECV
                n += 1
    return n

before = tcpext()
raw = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_RAW)
for i in range(syns):
    src = f"198.51.100.{1 + i % 254}"
    raw.sendto(syn(src, 1024 + i), (SERVER, 0))
time.sleep(0.5)
```

```
print(f"tcp_syncookies={cookies} tcp_max_syn_backlog={max_syn_backlog} "
      f"listen({backlog}) spoofed SYNs={syns}")
print(f"  SYN_RECV entries on :{PORT}: {syn_recv_count()}")

c = socket.socket()
c.settimeout(3)
t0 = time.monotonic()
try:
    c.connect((SERVER, PORT))
    conn, _ = lsock.accept()
    print(f"  real client: connected in {1000*(time.monotonic()-t0):.1f} ms, accepted")
except OSError as e:
    print(f"  real client: FAILED after {time.monotonic()-t0:.1f} s ({e})")
print(f"  TcpExt deltas: {delta(before, tcpext())}")
```

```
unshare -rn ./netns_lab.sh python3 synflood_lab.py 0  128 4096 1000   # cookies off, small m
unshare -rn ./netns_lab.sh python3 synflood_lab.py 0 4096   64 1000   # cookies off, small l
unshare -rn ./netns_lab.sh python3 synflood_lab.py 1  128   64 1000   # default: cookies whe
unshare -rn ./netns_lab.sh python3 synflood_lab.py 2  128   64   10   # cookies always
```

```
tcp_syncookies=0 tcp_max_syn_backlog=128 listen(4096) spoofed SYNs=1000
  SYN_RECV entries on :8080: 97
  real client: FAILED after 3.0 s (timed out)
  TcpExt deltas: {'ListenDrops': 906}
tcp_syncookies=0 tcp_max_syn_backlog=4096 listen(64) spoofed SYNs=1000
  SYN_RECV entries on :8080: 65
  real client: FAILED after 3.0 s (timed out)
  TcpExt deltas: {'ListenDrops': 938, 'TCPReqQFullDrop': 938}
tcp_syncookies=1 tcp_max_syn_backlog=128 listen(64) spoofed SYNs=1000
  SYN_RECV entries on :8080: 65
  real client: connected in 0.2 ms, accepted
  TcpExt deltas: {'TCPReqQFullDoCookies': 936, 'SyncookiesSent': 936, 'SyncookiesRecv': 1}
tcp_syncookies=2 tcp_max_syn_backlog=128 listen(64) spoofed SYNs=10
  SYN_RECV entries on :8080: 0
  real client: connected in 0.1 ms, accepted
  TcpExt deltas: {'TCPReqQFullDoCookies': 11, 'SyncookiesSent': 11, 'SyncookiesRecv': 1}
```

Every number is accounted for:

1. **Cookies off, `listen(4096)`, `tcp_max_syn_backlog=128`.** The SYN queue is nowhere near the listen limit, but it stops at **97** entries: step 3 drops unproven peers once fewer than `128 >> 2 = 32` slots of `tcp_max_syn_backlog` remain (`128 - 97 < 32`). 1000 − 97 = 903 spoofed SYNs dropped, plus the real client's SYN and its two retransmits (at 1 s and 3 s) = **906** `ListenDrops`. This path does not count `TCPReqQFullDrop` because the queue is not "full", it is being rationed. The real client is a new, unproven peer, so it loses too.
2. **Cookies off, `listen(64)`, `tcp_max_syn_backlog=4096`.** Now the listen backlog is the limit: **65** entries (`> 64`), then step 1 drops: 935 + 3 client attempts = **938**, counted as both

`TCPReqQFullDrop` and `ListenDrops`. A large `tcp_max_syn_backlog` did not buy a single extra slot.

3. **Default cookies (1).** The same 65 entries fill, then every further SYN gets a cookie: 935 spoofed + 1 real = **936** `SyncookiesSent`. The real client's ACK validates (`SyncookiesRecv` 1) and it connects in 0.2 ms, with a queue full of junk.
4. **Cookies always (2).** No request sockets at all (`SYN_RECV` 0). Every SYN, including the real one, is answered statelessly. This is a test mode, and it costs every client its options (next lab).

The kernel also logged the event once per listener (ratelimited, `tcp_syn_flood_action()`), on the host's `dmesg`, which is shared by all namespaces:

```
TCP: request_sock_TCP: Possible SYN flooding on port 10.9.0.1:8080. Dropping request.
TCP: request_sock_TCP: Possible SYN flooding on port 10.9.0.1:8080. Sending cookies.
```

Case 1 logs nothing: step 3 drops are reported only through `pr_drop_req()` at debug level. A host can be shedding real clients with no warning in the log, which is one more reason to read the counters.

Lab — what a cookie connection keeps

```
# cookie_options_lab.py - what a SYN-cookie connection keeps of the client's TCP options
# usage: unshare -rn ./netns_lab.sh python3 cookie_options_lab.py
import socket, struct

TS, SACK, WSCALE = 1, 2, 4                  # TCPI_OPT_* bits in tcp_info.tcpi_options

def sysctl(name, value):
    with open("/proc/sys/" + name.replace(".", "/"), "w") as f:
        f.write(str(value))

def options(sock):
    raw = sock.getsockopt(socket.IPPROTO_TCP, socket.TCP_INFO, 104)
    opts, wscale = raw[5], raw[6]
    snd_mss = struct.unpack_from("I", raw, 16)[0]
    names = [n for bit, n in ((TS, "timestamps"), (SACK, "sack"), (WSCALE, "wscale")) if opt
    if opts & WSCALE:
        names[-1] += f"={wscale & 0xF}/{wscale >> 4}"
    return f"{' '.join(names) or '(no options)'}  snd_mss={snd_mss}"

def handshake(cookies, timestamps):
    sysctl("net.ipv4.tcp_syncookies", cookies)
    sysctl("net.ipv4.tcp_timestamps", timestamps)
    lsock = socket.socket()
    lsock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    lsock.bind(("10.9.0.1", 8080))
```

```
    lsock.listen(16)
    c = socket.socket()
    c.connect(("10.9.0.1", 8080))
    conn, _ = lsock.accept()
    print(f"syncookies={cookies} timestamps={timestamps}: server sees {options(conn)}")
    for s in (c, conn, lsock):
        s.close()

handshake(0, 1)   # normal handshake
handshake(2, 1)   # always cookies, timestamps on: options ride in the TS value
handshake(2, 0)   # always cookies, no timestamps: only an MSS bucket survives
```

```
unshare -rn ./netns_lab.sh python3 cookie_options_lab.py
```

```
syncookies=0 timestamps=1: server sees timestamps sack wscale=7/7  snd_mss=32768
syncookies=2 timestamps=1: server sees timestamps sack wscale=7/7  snd_mss=1448
syncookies=2 timestamps=0: server sees (no options)  snd_mss=1460
```

- A normal handshake negotiates timestamps, SACK, and window scale 7 in both directions. The MSS is large because the path is local (veth address over loopback, MTU 65536).
- With a cookie and timestamps, window scale and SACK survive (they rode in the timestamp's low bits). The MSS drops to **1448**: 1460 from `msstab` minus 12 bytes of timestamp option. A loopback-class path now uses Ethernet-sized segments.
- With a cookie and **no timestamps**, nothing survives: no window scaling (receive window capped at 64 KiB, which caps throughput at 64 KiB per RTT), no SACK (loss recovery without selective acknowledgements, one hole per RTT), MSS 1460. Some middleboxes and hosts disable TCP timestamps for privacy or legacy reasons; their users pay this price on every cookie connection.

That is why cookies are the default *fallback* (`1`), not the default *mode* (`2`).

Case — reading a production listener

The same counters tell three different stories. Take two samples a few seconds apart (`nstat` with no `-a` prints the delta since its last run):

You see	It is	Do
`ListenOverflows` rising, `Recv-Q` = `Send-Q` + 1 on `ss -lnt`, cookie counters flat	Accept queue full: the application is slow (GC pause, blocked event loop, too few workers) or the backlog is tiny	Fix the accept loop, raise the app's `listen` backlog **and** `somaxconn`
`TCPReqQFullDoCookies` / `SyncookiesSent` rising, many `SYN_RECV` from scattered sources, `SyncookiesRecv` small	SYN flood (or a burst far above the backlog), cookies are absorbing it	Nothing urgent on the host; raise the backlog if real clients also trigger it; push filtering upstream

You see	It is	Do
`SyncookiesRecv` `SyncookiesSent`, few distinct sources	Legitimate load above the SYN-queue limit, served by cookies	Raise the listen backlog; this is the case the sysctl docs warn about
`TCPReqQFullDrop` or `ListenDrops` rising with cookies off	Real clients are being refused	Turn cookies back on (`tcp_syncookies=1`)

```
ss -lnt '( sport = :443 )'                        # Recv-Q / Send-Q per listener
ss -Htn state syn-recv '( sport = :443 )' | wc -l  # half-open count
nstat TcpExtListenOverflows TcpExtListenDrops TcpExtTCPReqQFullDoCookies TcpExtSyncookiesSen
```

(Commands for a production host; their output here would be the zero counters shown in the setup.)

The trap

Trap: "raise `tcp_max_syn_backlog` to survive a SYN flood." On current kernels the SYN queue limit is the listen backlog (capped by `somaxconn`). `tcp_max_syn_backlog` matters only when cookies are **off**, and then it is a rationing threshold, not a size. Lab case 2 set it to 4096 and still got 65 slots. The `ip-sysctl` text still calls it a "maximal number of remembered connection requests"; the code in `tcp_conn_request()` is the authority.

Trap: "SYN cookies protect against a full accept queue." They do not. Step 2 drops SYNs when the accept queue is full, cookies or not. A slow `accept()` loop looks like a SYN flood to clients (timeouts in `SYN-SENT`) but shows `ListenOverflows`, not cookie counters.

Trap: "`listen(65535)` gives me a 64 K backlog." `somaxconn` clamps it without an error. Check `Send-Q` on the listener.

Trap: "turn on `tcp_abort_on_overflow` so clients fail fast." It only resets handshakes whose final ACK arrives into a full accept queue, and it turns a recoverable burst into hard `ECONNRESET`s. Fix the accept rate instead.

Trap: "`tcp_syncookies=2` is safer." It disables request sockets entirely, so every client gets cookie-quality options (MSS table, no options without timestamps). Leave it at `1`.

The boring rule

Size the listen backlog in the application and in `somaxconn` together, leave `tcp_syncookies=1`, and when clients time out, read `ss -lnt` and the `TcpExt` counters before touching a sysctl: `ListenOverflows` means accept faster, `SyncookiesSent` means you are being flooded and the host is coping.

Try this

1. Re-run the accept-queue lab, then add an `accept()` loop after 2 s. How long do the `SYN-SENT` clients take to get in, and why (SYN retransmit backoff 1, 2, 4 s …)?
2. In the flood lab, set `tcp_synack_retries=1` with cookies off. Re-run the real client after 5 s instead of immediately. What changed, and what does that tell you about how long a flood must be sustained?
3. Add IPv6 to `netns_lab.sh` (`2001:db8::/32` routed into the void) and repeat case 3. Which counters are shared between IPv4 and IPv6?
4. With `tcp_timestamps=0` on the **client** side only, make a cookie connection and send 10 MB over the veth. Compare throughput with a normal connection.
5. Put an `nft synproxy` rule in front of the lab listener (the namespace lets you load rules there) and compare which counters move during the flood.

Verification

- Linux **6.12.94+** (Debian 13), iproute2 **6.15.0**, Python **3.13.5**, unprivileged user `uid=1000`; each lab ran in its own `unshare -rn` user + network namespace. All outputs above are pasted from those runs.
- The flood was real (1000 hand-built SYN packets on a raw socket per case) but never left the namespace: spoofed sources are TEST-NET-2, SYN-ACKs went to a static fake neighbour on a veth with no peer stack. Host `TcpExt` counters stayed 0.
- Counter arithmetic was checked against `tcp_conn_request()`, `tcp_syn_flood_action()`, `inet_csk_reqsk_queue_is_full()`, `sk_acceptq_is_full()` (Linux 6.12) and the SYN retransmit timing of the real client (initial RTO 1 s).
- **Not shown:** the `tcp_abort_on_overflow` RST path (it needs a final ACK to race a filling accept queue, which local handshakes do not do); it is described from `tcp_check_req()` in `tcp_minisocks.c`. Production `ss`/`nstat` commands in the Case section were not run against a production host.
- No packages installed, no `sudo`, no host sysctl changed.

Sources

- Linux 6.12 `net/ipv4/tcp_input.c` — `tcp_conn_request()`, `tcp_syn_flood_action()`: https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/ipv4/tcp_input.c?h=linux-6.12.y
- Linux 6.12 `net/ipv4/syncookies.c` — `msstab`, timestamp option bits, cookie validity: https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/ipv4/syncookies.c?h=linux-6.12.y
- Linux 6.12 `net/ipv4/tcp_minisocks.c` — `tcp_check_req()` `listen_overflow` / `tcp_abort_on_overflow`
- Linux 6.12 `include/net/inet_connection_sock.h`, `include/net/sock.h` — queue-full checks; `net/socket.c` — `somaxconn` clamp
- Kernel docs — IP sysctl: `tcp_syncookies`, `tcp_max_syn_backlog`, `tcp_synack_retries`, `tcp_abort_on_overflow`

- man7 — `tcp(7)`, `listen(2)`, `ss(8)`, `user_namespaces(7)`
- RFC 4987 — TCP SYN Flooding Attacks and Common Mitigations
- RFC 9293 — Transmission Control Protocol (three-way handshake, §3.5)
- RFC 5737 — IPv4 Address Blocks Reserved for Documentation

Overlays and Tunnels

Why Encapsulate

Why Encapsulate

Encapsulation is not a fashion—it is how networks **reuse an underlay** while giving tenants or sites a different topology, address space, or policy domain. This chapter builds durable models before GRE/IPsec/VXLAN/EVPN syntax (Containerlab + FRR + Linux, mid-2026 free path).

Learning goals

By the end of this chapter you can:

- Separate underlay from overlay responsibilities
- List reasons to tunnel or encapsulate (and reasons not to)
- Trace a packet through outer/inner headers
- Predict MTU, ECMP entropy, and troubleshooting failure modes
- Map encapsulation choices to common design problems

The core picture

```
overlay topology (what apps believe)

site A    ·····    site B

     encapsulated path

   underlay IP fabric/WAN
```

Plane	Job
Underlay	Deliver outer packets between tunnel endpoints (IP reachability, IGP/BGP, ECMP)
Overlay	Present virtual links, L2 segments, or VRFs to tenants/sites

Plane	Job
Control (overlay)	Who is where (EVPN-class, controller, static maps)
Data (overlay)	Encapsulation format (VXLAN, GRE, Geneve, IPsec…)

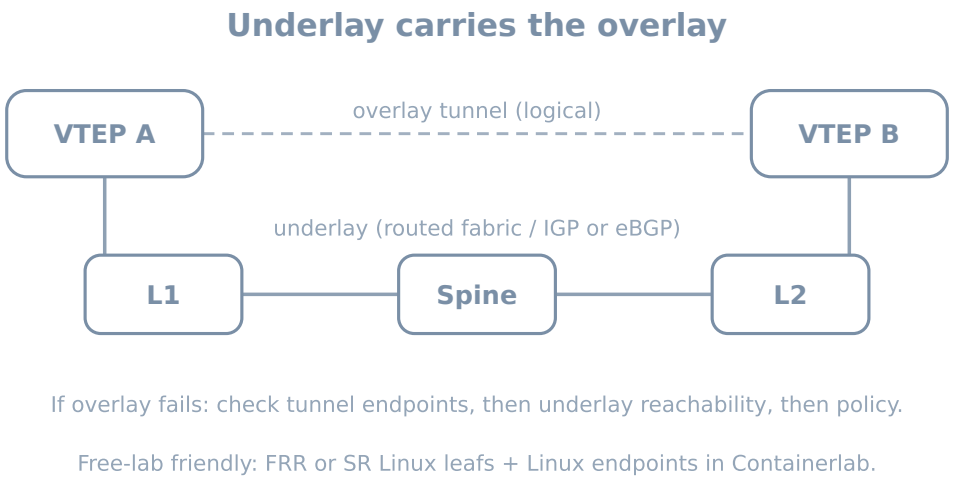


Figure 1: Underlay vs overlay

Why people encapsulate

Driver	What encapsulation buys
Overlap addresses	Two tenants both use 10.0.0.0/8
L2 adjacency over L3	VM mobility, some clusters, legacy apps
Policy domains	Separate routing tables / VRFs
Traffic engineering	Steer logical topology physical
Multi-site stretch	Same segment in two DCs (careful!)
Security boundary	IPsec provides crypto + integrity
Abstraction	Apps ignore underlay renumbering

Why *not* to encapsulate

Temptation	Cost
Tunnel everything by default	MTU tax, state, debug opacity

Temptation	Cost
Stretch L2 globally	Failure domains explode
Overlay without underlay skill	“EVPN down” actually IGP down
Double NAT + tunnel + hairpin	Undebuggable piles

Rule: prefer plain IP routing until a requirement forces an overlay.

Encapsulation as Russian dolls

Example mental stack (VXLAN-ish):

```
[ Eth ][ IP underlay ][ UDP ][ VXLAN ][ Eth inner ][ IP inner ][ TCP ][ payload ]
```

Header	Owned by
Outer IP	Underlay endpoints (VTEPs / tunnel routers)
Outer L4	Often UDP for ECMP entropy (VXLAN)
Overlay header	VNI / keys / flags
Inner frame	Tenant

GRE classic:

```
[ Eth ][ IP ][ GRE ][ IP inner ][ ...]
```

IPsec tunnel mode adds crypto headers and often transports inner IP.

Identifiers you must name

Identifier	Role
Tunnel endpoint IP	Underlay address of VTEP/router
VNI / VPN id / GRE key	Separates segments
VRF	Routing table isolation
Route target / RD (EVPN/MPLS-ish)	Control-plane membership
Mapping	End-host IP/MAC → endpoint

Without a clear mapping system (static, flood-and-learn, EVPN), overlays become amateur radio.

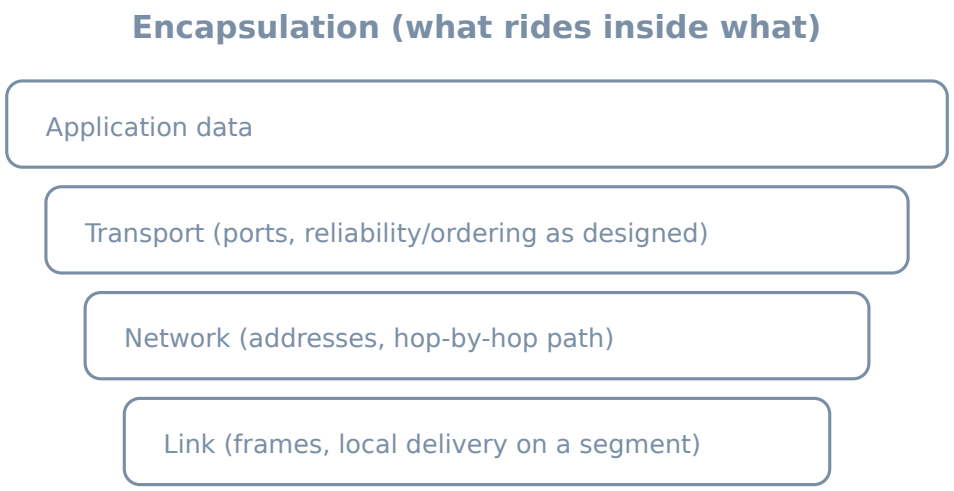


Figure 2: Encapsulation

Failure domains

```
Underlay down → all overlays using it down
One VTEP down → locals on that leaf isolated from fabric overlay
VNI mis-map → blackhole or cross-tenant leak (severe)
MTU blackhole → big packets die, small pings live
```

Always ask: **is this underlay, overlay data, or overlay control?**

MTU tax

Every header steals bytes. If underlay MTU is 1500 and you add ~50–100 bytes, inner needs lower MTU or underlay needs jumbo.

Symptom	Likely
Ping works, TCP hangs	PMTUD broken + encapsulation
Only large transfers fail	MTU
DF set drops	Need clamp or larger underlay

Lab habit: set explicit MTU on overlay interfaces and test large pings.

```
ping -M do -s 1472 10.0.0.1   # classic Ethernet payload probe
# inside overlay, lower size until success; compute header tax
```

ECMP and entropy

Underlays load-balance on outer five-tuple. If outer UDP/TCP ports are fixed, many inner flows hash to **one** path → polarization.

Design	Entropy
VXLAN UDP source port from inner flow hash	Good
GRE without entropy helpers	Often poor
IPsec without cleverness	Can polarize

When debugging "uneven spines," check **outer** headers, not only inner apps.

Control plane vs data plane (again)

	Data plane only	With control plane
Static GRE	Manual who-to-whom	—
Flood-and-learn VXLAN	Dynamic MAC via flood	Limited scale
EVPN-class	Advertises reachability	Mapping distributed

This book's later chapters: site tunnels (mostly static/service), VXLAN data plane, EVPN intro where free images allow.

Mini lab — see encapsulation with tcpdump

```
name: encap-why

topology:
  nodes:
    a:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
    b:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
    under:
      kind: linux
      image: alpine:3.20
```

```
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
        - sysctl -w net.ipv4.ip_forward=1

  links:
    - endpoints: ["a:eth1", "under:eth1"]
    - endpoints: ["b:eth1", "under:eth2"]
```

GRE between a and b across under

```
# addresses: a 10.0.0.1/24, under eth1 10.0.0.254; b 10.0.1.1/24, under eth2 10.0.1.254
# underlay route a b via under

docker exec clab-encap-why-a sh -c '
  ip tunnel add gre1 mode gre remote 10.0.1.1 local 10.0.0.1 ttl 64
  ip link set gre1 up
  ip addr add 172.16.0.1/30 dev gre1
'
docker exec clab-encap-why-b sh -c '
  ip tunnel add gre1 mode gre remote 10.0.0.1 local 10.0.1.1 ttl 64
  ip link set gre1 up
  ip addr add 172.16.0.2/30 dev gre1
'
```

Predict → observe → fix

```
docker exec clab-encap-why-a ping -c 2 172.16.0.2
docker exec clab-encap-why-under tcpdump -ni eth1 -c 5 proto gre
# or: tcpdump -ni eth1 proto 47
```

Predict: underlay capture shows outer IP a→b with GRE; inner ICMP not as plain on underlay without decap.
Observe: header stack.
Fix: if no gre, check tunnel remote/local and underlay ping 10.0.1.1 from a.

Design questions checklist

Before choosing an overlay:

1. What problem is pure IP routing failing to solve?

2. L2 stretch or L3 VPN?

3. Who allocates VNIs / VRFs?
4. What is the underlay routing design?
5. MTU strategy?
6. How do we debug (mirror, drop counters, inner/outer captures)?
7. Failure domain acceptable?

Mapping to the journey map

```
Models (planes, encap)
  → underlay skill (L2/L3/IGP/BGP)
  → edge services
  → overlays (this part)
  → fabrics (scale the underlay + overlay)
```

Skipping underlay competence makes overlay chapters mystical.

Common mistakes

Mistake	Symptom
Troubleshooting only inner	Miss underlay loss
Ignoring MTU	Mystery TCP issues
One giant L2 VNI	Broadcast storms / failure blast
No inventory of VNIs	Collisions and leaks
Encrypt + fragment chaos	Performance cliff

Summary

- Encapsulation reuses underlays to build different logical networks
- Always name **underlay, overlay data, overlay control**, and **identifiers**
- MTU and ECMP entropy are not footnotes
- Prefer routing until requirements force tunnels
- A tiny GRE lab + tcpdump makes the model real

Next: **site tunnels**—practical GRE/IPsec-style site-to-site patterns on free Linux/FRR labs.

Site Tunnels

Site Tunnels

Site-to-site tunnels connect islands of private addressing across a routed underlay (lab "Internet" or WAN). This chapter implements **GRE and policy-based / route-based IPsec-style** patterns with Linux in Containerlab, optionally fronted by FRR for routing (mid-2026 free tools).

Learning goals

By the end of this chapter you can:

- Build a GRE tunnel and route site prefixes across it
- Explain route-based vs policy-based VPN approaches
- Run a simple WireGuard or LibreSwan/strongSwan-style lab path (tool availability permitting)
- Fail underlay links and predict overlay behavior
- Clamp MTU and verify with large pings and tcpdump

When site tunnels are the right tool

Use	Not the first tool for
Two branches over untrusted WAN	DC fabric east-west (use fabric overlay)
Quick lab multi-site	Massive multi-tenant scale (EVPN fabrics)
Crypto requirement	Cleartext GRE alone on hostile nets

Topology — two sites + “Internet”

```
h1--siteA--\                /--siteB--h2
           +---- inet ----+
```

```yaml
name: site-tun

topology:
  nodes:
    sitea:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump iptables wireguard-tools || true
    siteb:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump iptables wireguard-tools || true
    inet:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
        - sysctl -w net.ipv4.ip_forward=1
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["h1:eth1", "sitea:eth1"]
    - endpoints: ["h2:eth1", "siteb:eth1"]
    - endpoints: ["sitea:eth2", "inet:eth1"]
    - endpoints: ["siteb:eth2", "inet:eth2"]
```

Addressing plan

Object	Address
Site A LAN	192.168.10.0/24 (sitea .1, h1 .10)
Site B LAN	192.168.20.0/24 (siteb .1, h2 .10)
sitea WAN	198.51.100.1/30
inet eth1	198.51.100.2/30
siteb WAN	203.0.113.1/30
inet eth2	203.0.113.2/30
GRE outer	WAN IPs
GRE inner	10.255.255.0/30 (.1 a, .2 b)

```
# site A
docker exec clab-site-tun-sitea sh -c '
  sysctl -w net.ipv4.ip_forward=1
  ip addr add 192.168.10.1/24 dev eth1; ip link set eth1 up
  ip addr add 198.51.100.1/30 dev eth2; ip link set eth2 up
  ip route add 203.0.113.0/30 via 198.51.100.2
'
docker exec clab-site-tun-h1 sh -c '
  ip addr add 192.168.10.10/24 dev eth1; ip link set eth1 up
  ip route add default via 192.168.10.1
'
# site B + h2 analogous with 192.168.20.0/24 and 203.0.113.1
docker exec clab-site-tun-inet sh -c '
  ip addr add 198.51.100.2/30 dev eth1; ip link set eth1 up
  ip addr add 203.0.113.2/30 dev eth2; ip link set eth2 up
'
```

Lab 1 — GRE + static routes

```
docker exec clab-site-tun-sitea sh -c '
  ip tunnel add gre1 mode gre remote 203.0.113.1 local 198.51.100.1 ttl 64
  ip link set gre1 up mtu 1400
  ip addr add 10.255.255.1/30 dev gre1
  ip route add 192.168.20.0/24 dev gre1
'
docker exec clab-site-tun-siteb sh -c '
  ip tunnel add gre1 mode gre remote 198.51.100.1 local 203.0.113.1 ttl 64
  ip link set gre1 up mtu 1400
  ip addr add 10.255.255.2/30 dev gre1
  ip route add 192.168.10.0/24 dev gre1
'
```

Predict

- Underlay: sitea pings 203.0.113.1
- Overlay: h1 pings h2
- tcpdump on inet shows GRE (proto 47), not plain inner ICMP as clear tenant IP without decap context

Observe

```
docker exec clab-site-tun-sitea ping -c 2 203.0.113.1
docker exec clab-site-tun-h1 ping -c 3 192.168.20.10
docker exec clab-site-tun-inet tcpdump -ni eth1 -c 8 proto 47
docker exec clab-site-tun-sitea ip route get 192.168.20.10
```

Fix

Symptom	Check
No underlay	inet addrs, routes
Underlay OK overlay fail	gre remote/local swap, routes to LAN via gre
One-way	missing return route

Lab 2 — MTU blackhole drill

```
docker exec clab-site-tun-h1 ping -c 2 -M do -s 1472 192.168.20.10
docker exec clab-site-tun-h1 ping -c 2 -M do -s 1372 192.168.20.10
```

Predict

Large DF pings fail; smaller succeed if gre mtu 1400.
Fix: lower inner interface MTU, TCP MSS clamp, or raise underlay MTU.

```
docker exec clab-site-tun-sitea iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN
```

Lab 3 — Underlay failure

```
docker exec clab-site-tun-inet ip link set eth2 down
docker exec clab-site-tun-h1 ping -c 5 192.168.20.10
docker exec clab-site-tun-inet ip link set eth2 up
```

Predict

Overlay dies with underlay; no magic. Multi-homed underlay would need dual tunnels + routing.

Route-based vs policy-based VPN

Model	Idea	Lab analogue
Route-based	Tunnel is an interface; RIB steers prefixes into it	GRE, WireGuard `wg0`, VTI
Policy-based	SPD selects traffic by ACL selectors for crypto	classic "interesting traffic" IPsec

Route-based composes better with dynamic routing (run OSPF/BGP over tunnel—carefully).

Lab 4 — WireGuard route-based (if packages available)

```
# Generate keys on both sides (persist in lab folder for real work)
docker exec clab-site-tun-sitea sh -c '
  apk add --no-cache wireguard-tools
  wg genkey | tee /tmp/a.key | wg pubkey > /tmp/a.pub
'
# exchange pubs out of band in lab notes
```

```
docker exec clab-site-tun-sitea sh -c '
  ip link add wg0 type wireguard
  ip addr add 10.255.255.1/30 dev wg0
  wg set wg0 private-key /tmp/a.key listen-port 51820 \
    peer B_PUBLIC_KEY endpoint 203.0.113.1:51820 allowed-ips 192.168.20.0/24,10.255.255.2/32
  ip link set wg0 up
  ip route add 192.168.20.0/24 dev wg0
'
```

Mirror on siteb with `allowed-ips 192.168.10.0/24`. Open UDP 51820 on path (inet forwards by default).

Predict → observe

```
docker exec clab-site-tun-h1 ping -c 3 192.168.20.10
docker exec clab-site-tun-inet tcpdump -ni eth1 -c 5 udp port 51820
docker exec clab-site-tun-sitea wg show
```

If `wireguard-tools` or kernel module missing in container, treat as **optional**—GRE still teaches site tunnels. Host-level WireGuard between namespaces is an alternate craft path from lab-craft chapters.

FRR over GRE (dynamic optional)

Enable `zebra` + `ospfd` or `bgpd` on FRR images replacing Alpine sites:

```
interface gre1
 ip ospf area 0
router ospf
 ospf router-id 1.1.1.1
```

Predict: LAN prefixes appear via OSPF over tunnel.
Risk: if underlay also runs OSPF without careful design, you can create recursive routing (tunnel endpoint learned *via* tunnel). **Keep underlay and overlay routing domains separate.**

Policy-based IPsec awareness

```
Selectors: src 192.168.10.0/24 dst 192.168.20.0/24
Peer: 203.0.113.1
```

Tools: strongSwan, LibreSwan—configs are verbose; for this book, understand selectors + SA state:

```
# illustrative
ip xfrm state
ip xfrm policy
```

Debug with state/policy tables + outer ESP captures (proto 50).

Security notes (vendor-agnostic)

- GRE alone is **not** confidential
- Authenticate/encrypt on untrusted underlays
- Filter what enters the tunnel (no free transit for world)
- Manage keys as secrets (not committed plaintext in public repos)

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-site-tun-h1 ping -c2 -W1 192.168.20.10
docker exec clab-site-tun-h2 ping -c2 -W1 192.168.10.10
echo OK
```

Common mistakes

Mistake	Symptom
Tunnel up, no LAN routes	Only gre peer pings
Recursive routing	Flapping tunnel endpoint
MTU ignored	App failures
NAT on underlay without keepalive	Idle VPN dies
Asymmetric interesting traffic	Policy VPN one-way

Summary

- Site tunnels extend private sites across a routed underlay
- GRE is the clearest teaching tool; WireGuard is a modern route-based crypto option
- Separate underlay reachability from overlay routing
- Always test **MTU** and **underlay failure**
- Prefer route-based models when combining with dynamic routing

Next: **VXLAN data plane**—L2 segments over L3 underlays with VNI encapsulation.

WireGuard: Cryptokey Routing and the Pre-Shared Key

WireGuard: Cryptokey Routing and the Pre-Shared Key

The site-tunnel chapter used WireGuard as one route-based VPN option among several. This chapter opens it up. WireGuard has two ideas you must understand before you run it in production. **Cryptokey routing** means that a peer's `AllowedIPs` list is both its routing table and its source-address filter. The optional **pre-shared key (PSK)** is a 32-byte symmetric secret mixed into every handshake. In 2026 the PSK is getting attention as a cheap **post-quantum hedge**: VPN providers now generate it by default for router and native-client configs. Both ideas are easy to misconfigure, and both fail *silently*. This chapter makes them fail on purpose in a two-namespace lab and reads the evidence.

Learning goals

By the end of this chapter you can:

- Build a WireGuard tunnel between two Linux network namespaces with `wg` and `ip`, without config files
- Explain why `AllowedIPs` drops a correctly encrypted packet and where that drop is counted
- Recognise WireGuard's "no peer for this destination" behaviour
- Add a PSK, explain exactly what it does and does not protect against, and rotate it live
- Diagnose a PSK mismatch from a packet capture (both handshake messages are on the wire, yet the tunnel never comes up)

Lab platform: Debian 13, Linux 6.12, `wireguard-tools` v1.0.20210914 (the current Debian package), `iproute2`, `tcpdump` 4.99.5. All commands need root. Every output below is pasted from a run.

Concepts in one page



Direction	What `AllowedIPs` does
Outbound (a packet leaves via `wg0`)	Longest-prefix match of the *destination* against every peer's AllowedIPs picks the peer, and so the key and endpoint. No match means the packet is not sent, and the sender gets an error.
Inbound (a packet arrives and decrypts)	The decrypted inner *source* must fall inside the AllowedIPs of the peer whose key decrypted it. Otherwise it is dropped.

The kernel route only gets a packet *to* `wg0`. AllowedIPs decides *which peer*, if any.

The handshake is Noise **`IKpsk2`** (Curve25519, ChaCha20-Poly1305, BLAKE2s). There are two messages. The **initiation** (type 1, 148 bytes of UDP payload) carries the initiator's encrypted static key and a timestamp. The **response** (type 2, 92 bytes) carries the responder's ephemeral key and an empty AEAD box. The PSK is hashed into the chaining key while the **response** is being built. Without a PSK the protocol uses 32 zero bytes in its place.

What the PSK buys: if Curve25519 is broken later (the "harvest now, decrypt later" scenario with a large quantum computer), traffic recorded today stays confidential, *provided the PSK itself was never exposed*. What it does not buy: it is not a post-quantum key exchange, it does not replace the public keys, and a PSK that was sent over a classical channel (for example a TLS session protected by ECDHE) is only as strong as that channel. Protocols such as Rosenpass run a post-quantum key exchange alongside WireGuard and feed the result into the PSK slot, rotating it. That is the direction to take when this matters.

Lab build

```
#!/usr/bin/env bash
# lab-up.sh - two namespaces, a veth "internet", and a WireGuard tunnel between them
set -euo pipefail
ip netns add siteA; ip netns add siteB
ip link add vA netns siteA type veth peer name vB netns siteB
ip -n siteA addr add 198.51.100.1/24 dev vA; ip -n siteA link set vA up; ip -n siteA link se
ip -n siteB addr add 198.51.100.2/24 dev vB; ip -n siteB link set vB up; ip -n siteB link se

umask 077
for s in A B; do wg genkey | tee $s.key | wg pubkey > $s.pub; done
wg genpsk > ab.psk

# create wg0 inside each namespace: a WireGuard interface sends its UDP from the namespace i
ip -n siteA link add wg0 type wireguard
ip -n siteB link add wg0 type wireguard
ip netns exec siteA wg set wg0 private-key A.key listen-port 51820 \
   peer "$(cat B.pub)" preshared-key ab.psk endpoint 198.51.100.2:51820 allowed-ips 10.0.0.2
ip netns exec siteB wg set wg0 private-key B.key listen-port 51820 \
   peer "$(cat A.pub)" preshared-key ab.psk endpoint 198.51.100.1:51820 allowed-ips 10.0.0.1
ip -n siteA addr add 10.0.0.1/24 dev wg0; ip -n siteA link set wg0 up
```

```
ip -n siteB addr add 10.0.0.2/24 dev wg0; ip -n siteB link set wg0 up
# siteB also owns a LAN prefix that siteA routes into the tunnel
ip -n siteB addr add 192.0.2.1/24 dev lo
ip -n siteA route add 192.0.2.0/24 dev wg0
echo "lab up"
```

```
#!/usr/bin/env bash
# lab-down.sh - remove everything lab-up.sh created
ip netns del siteA 2>/dev/null; ip netns del siteB 2>/dev/null
rm -f A.key A.pub B.key B.pub ab.psk wrong.psk
echo "lab down"
```

`wg genpsk` produces 32 random bytes in base64, the same format as a private key. The PSK is configured **per peer** and must be identical on both ends. `umask 077` keeps the key files private. In production these values live in `/etc/wireguard/wg0.conf` (mode 600) or in your configuration management tool's secret store, never in a world-readable file.

```
$ sudo ./lab-up.sh
lab up
$ sudo ip netns exec siteA ping -c 2 10.0.0.2 | tail -2
2 packets transmitted, 2 received, 0% packet loss, time 1015ms
rtt min/avg/max/mdev = 0.547/0.590/0.633/0.043 ms
$ sudo ip netns exec siteA wg show
interface: wg0
  public key: SfKQSp7pB2JRf6CnHF048gHOIul0sUi0k9lCH/V9P2I=
  private key: (hidden)
  listening port: 51820

peer: SVXjcMNZVP6LGPEc+CYMNGGQ0/4KHY05bo34Oc5MuVg=
  preshared key: (hidden)
  endpoint: 198.51.100.2:51820
  allowed ips: 10.0.0.2/32, 192.0.2.0/24
  latest handshake: 1 second ago
  transfer: 348 B received, 404 B sent
$ sudo ip -n siteA link show wg0
3: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN mode DEFAULT gr
```

The handshake completed on the first ping (`latest handshake: 1 second ago`), and `wg show` reports the PSK as `(hidden)`. Its presence is visible and its value is not. The interface MTU is **1420** by default. That leaves room for the outer IPv6 header (40), UDP (8), and WireGuard's 32 bytes of data-message overhead (a 16-byte header plus a 16-byte Poly1305 tag) inside a 1500-byte path. Over IPv4 there are 20 bytes to spare.

Drill 1: cryptokey routing drops a perfectly encrypted packet

```
#!/usr/bin/env bash
# drill-cryptokey.sh - AllowedIPs is a routing table outbound and a source filter inbound
set -euo pipefail
ip -n siteA addr add 203.0.113.5/32 dev lo           # an address siteB never allowed for th
echo "== inside the allowed source (10.0.0.1 -> 10.0.0.2)"
ip netns exec siteA ping -c 2 -W 1 10.0.0.2 | tail -2
echo "== outside it (203.0.113.5 -> 10.0.0.2)"
ip netns exec siteA ping -c 3 -W 1 -I 203.0.113.5 10.0.0.2 | tail -2 || true
echo "== siteB wg0 counters (decrypted fine, then dropped: src not in peer's allowed-ips)"
ip -n siteB -s link show wg0 | sed -n '3,4p'
echo "== a route into wg0 that no peer's allowed-ips covers"
ip -n siteA route add 198.18.0.0/15 dev wg0
ip netns exec siteA ping -c 1 -W 1 198.18.0.1 2>&1 | head -2 || true
```

```
$ sudo ./drill-cryptokey.sh
== inside the allowed source (10.0.0.1 -> 10.0.0.2)
2 packets transmitted, 2 received, 0% packet loss, time 1028ms
rtt min/avg/max/mdev = 0.138/0.334/0.531/0.196 ms
== outside it (203.0.113.5 -> 10.0.0.2)
3 packets transmitted, 0 received, 100% packet loss, time 2040ms

== siteB wg0 counters (decrypted fine, then dropped: src not in peer's allowed-ips)
    RX:  bytes packets errors dropped  missed   mcast
           660       5      3       0       0       0
== a route into wg0 that no peer's allowed-ips covers
PING 198.18.0.1 (198.18.0.1) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
```

Read it in order:

1. From the allowed source, the ping works.
2. From `203.0.113.5`, siteA's kernel still routes the packet into `wg0`, because the destination `10.0.0.2` matches peer B's AllowedIPs. WireGuard encrypts it and siteB **decrypts it successfully**. Then siteB checks the inner source against peer A's AllowedIPs (`10.0.0.1/32`), finds no match, and drops it. The only trace is the **RX `errors`** counter on siteB's `wg0` (3 packets, one per ping). Nothing is logged, and no ICMP message goes back.
3. A destination that the kernel routes into `wg0` but that no peer allows (`198.18.0.0/15`) is never sent. WireGuard rejects it at transmit, and the sender sees `Destination Host Unreachable`.

That makes AllowedIPs a security boundary as well as a convenience. A peer that holds a valid key still cannot inject traffic from addresses it was not given. The flip side is that every new subnet behind a peer means editing AllowedIPs on **both** sides: the outbound list on one end, the inbound filter on the other.

Drill 2: a PSK mismatch, seen from the wire

```
#!/usr/bin/env bash
# drill-psk-mismatch.sh - same keypairs, different PSK on each side: the handshake never com
set -euo pipefail
umask 077; wg genpsk > wrong.psk
# re-adding the peer throws away its current session, forcing a new handshake
for ns in siteA siteB; do
  peer=$(ip netns exec $ns wg show wg0 peers)
  ep=$(ip netns exec $ns wg show wg0 endpoints | cut -f2)
  aips=$(ip netns exec $ns wg show wg0 allowed-ips | cut -f2 | tr ' ' ',')
  psk=ab.psk; [ $ns = siteB ] && psk=wrong.psk
  ip netns exec $ns wg set wg0 peer "$peer" remove
  ip netns exec $ns wg set wg0 peer "$peer" preshared-key $psk endpoint "$ep" allowed-ips "$
done
ip netns exec siteB timeout 8 tcpdump -lnni vB -c 4 udp port 51820 2>/dev/null &
sleep 1
ip netns exec siteA ping -c 3 -W 1 10.0.0.2 | tail -2 || true
wait
echo "siteA latest-handshake (0 = never):"; ip netns exec siteA wg show wg0 latest-handshake
echo "siteB latest-handshake (0 = never):"; ip netns exec siteB wg show wg0 latest-handshake
```

```
$ sudo ./drill-psk-mismatch.sh
03:40:05.224460 IP 198.51.100.1.51820 > 198.51.100.2.51820: UDP, length 148
03:40:05.224668 IP 198.51.100.2.51820 > 198.51.100.1.51820: UDP, length 92
3 packets transmitted, 0 received, 100% packet loss, time 2049ms

03:40:10.317113 IP 198.51.100.1.51820 > 198.51.100.2.51820: UDP, length 148
03:40:10.317315 IP 198.51.100.2.51820 > 198.51.100.1.51820: UDP, length 92
siteA latest-handshake (0 = never):
SVXjcMNZVP6LGPEc+CYMNGGQO/4KHYO5bo34Oc5MuVg=    0
siteB latest-handshake (0 = never):
SfKQSp7pB2JRf6CnHFO48gHOIul0sUiOk9lCH/V9P2I=    0
```

This is the most confusing failure in WireGuard, and the capture explains it:

- The **148-byte initiation** goes out, and siteB **accepts** it. The initiation is authenticated with the static keys and does not depend on the PSK, so siteB has no reason to be silent.
- siteB answers with a **92-byte response**. It mixes *its* PSK into the chaining key while building that response, so the empty AEAD box inside is sealed under a key that siteA cannot derive.
- siteA fails to open the response and discards it without logging. About five seconds later (`REKEY_TIMEOUT` plus jitter) it tries again. Note the timestamps.
- Neither side records a handshake (`0` means never).

So "packets in both directions on UDP 51820, tunnel down" points to a **PSK mismatch** (or a corrupted PSK). A wrong *public* key looks different: the responder cannot authenticate the initiation, so it sends nothing back, and you see only the 148-byte packets.

Drill 3: fix and rotate the PSK live

```
#!/usr/bin/env bash
# drill-psk-fix.sh - put the right PSK back on siteB (live, no restart) and watch the tunnel
set -euo pipefail
peer=$(ip netns exec siteB wg show wg0 peers)
ip netns exec siteB wg set wg0 peer "$peer" preshared-key ab.psk
ip netns exec siteA ping -c 8 -W 1 10.0.0.2 | tail -2
now=$(date +%s)
for ns in siteA siteB; do
  echo "$ns latest handshake: $(( now - $(ip netns exec $ns wg show wg0 latest-handshakes |
done
```

```
$ sudo ./drill-psk-fix.sh
8 packets transmitted, 8 received, 0% packet loss, time 7114ms
rtt min/avg/max/mdev = 0.268/1859.476/5006.915/1836.351 ms, pipe 5
siteA latest handshake: 2 s ago
siteB latest handshake: 2 s ago
$ sudo ./lab-down.sh
lab down
```

`wg set ... preshared-key` changes the PSK on a running interface. Existing sessions keep their keys until the next handshake, and new handshakes use the new PSK. Here the first ping waited for siteA's next retry (the 5-second maximum RTT), and after that the tunnel ran normally. To rotate a PSK in production, change both ends inside one rekey window (sessions rekey about every two minutes) so that at most one handshake attempt fails. Or let a key-exchange daemon do it for you.

Deploying it for real

- **Generate a separate PSK per peer pair** with `wg genpsk`. One PSK shared across a whole fleet becomes a single secret whose leak removes the hedge for everyone.
- **Distribute it like a private key**, over a channel you would trust in 2040: out of band, through a secret store, or from a post-quantum key exchange. A PSK pasted into a TLS-protected web form is protected only by that TLS session.
- **Store it with the private key**: `PresharedKey = ...` under `[Peer]` in `wg0.conf` (mode 600), or `PresharedKeyFile=` in a systemd-networkd `.netdev`.
- **Monitor the handshake age.** `wg show wg0 latest-handshakes` returns epoch seconds. Alert when the value is `0` or older than about three minutes for a peer that should be up.
- **Watch RX errors on `wg0`.** A rising count usually means AllowedIPs is filtering traffic someone expects to pass.

Verification checklist

- ☐ `wg show` lists `preshared key: (hidden)` on **both** ends for every peer that should have one
- ☐ `latest-handshakes` is recent (non-zero and under about 3 minutes) on both ends
- ☐ Capture shows both 148-byte initiations and 92-byte responses, and data packets follow
- ☐ Each remote prefix appears in the sender's AllowedIPs *and* in the receiver's AllowedIPs for the return path
- ☐ `ip -s link show wg0` RX errors are flat during normal traffic
- ☐ Large pings at your path MTU (`ping -M do -s 1392` over IPv4 with MTU 1420) succeed

Common mistakes

Mistake	Symptom	Fix
PSK set on one side only, or different on each side	Initiations and responses on the wire, no handshake	Same `wg genpsk` value on both peers
Subnet added to the route but not to AllowedIPs	`Destination Host Unreachable` from the local `wg0`	Add the prefix to the peer's AllowedIPs
Subnet added on the sender, not on the receiver	Silent loss, RX errors on the far `wg0`	Mirror AllowedIPs for the return direction
Overlapping AllowedIPs across peers	Traffic goes to the "wrong" peer (longest prefix wins, and a later `wg set` moves the prefix)	One owner per prefix; review `wg show allowed-ips`
Calling the PSK "post-quantum encryption"	False confidence	It is a symmetric hedge for recorded traffic. Distribute it safely or use a PQ key exchange
One fleet-wide PSK	One leak removes the hedge everywhere	One PSK per peer pair

Summary

WireGuard's configuration is small because AllowedIPs does two jobs: it chooses the peer on the way out and filters the source on the way in, and both outcomes are silent unless you read `wg show` and the interface counters. The PSK adds a symmetric secret to the Noise `IKpsk2` handshake. That protects recorded traffic against a future break of Curve25519, as long as the PSK itself stays secret. A PSK mismatch has a distinctive signature on the wire: both handshake messages appear and no session forms, because the responder cannot detect the mismatch and the initiator cannot open its reply. Rotate PSKs per peer pair, distribute them like private keys, and monitor handshake age.

Sources

- WireGuard, "Protocol & Cryptography": https://www.wireguard.com/protocol/. Noise `IKpsk2`, message layouts (148 and 92 bytes), where the PSK is mixed in, timers (`REKEY_TIMEOUT`).
- J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel", NDSS 2017 (whitepaper): https://www.wireguard.com/papers/wireguard.pdf. Cryptokey routing, PSK as a post-quantum hedge.
- `wg(8)` manual page (`genpsk`, `set ... preshared-key`, `show ... latest-handshakes`): https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8.
- `systemd.netdev(5)`, `[WireGuardPeer] PresharedKeyFile=`: https://www.freedesktop.org/software/systemd/man/latest/systemd.netdev.html.
- Rosenpass (post-quantum key exchange that supplies WireGuard's PSK): https://rosenpass.eu/.

OpenVPN Data Channel Offload (DCO)

OpenVPN Data Channel Offload (DCO)

Thesis

Classic OpenVPN runs the **control plane and the data plane in userspace**. Every encrypted packet crosses the kernel userspace boundary twice. That copy-and-context-switch tax is why OpenVPN used to lose throughput races against kernel-native VPNs. **Data Channel Offload (DCO)** moves encryption, decryption, and encapsulation into a Linux kernel module (`ovpn` since Linux **6.16**; earlier out-of-tree `ovpn-dco` / `ovpn-dco-v2`). Userspace OpenVPN keeps TLS, authentication, and option negotiation; the kernel keeps the hot path. The security model does not change — only *where* the ciphertext is processed. On modern hardware, published UDP benchmarks regularly show roughly **2×** legacy OpenVPN throughput, and independent tests put DCO in the same ballpark as other high-performance VPN designs.

Mental model

```
Before DCO (userspace data path)          With DCO (kernel data path)

NIC    kernel    openvpn(userspace)     NIC    kernel ovpn module
                encrypt / decrypt                        encrypt / decrypt
                encapsulate                              encapsulate

       kernel    NIC                                NIC (no userspace hop)

Control plane (TLS, auth, PUSH) stays in userspace either way.
```

Piece	Role
Control plane	TLS handshake, certificates, `--push` / NCP, peer lifecycle — always userspace
Data plane	AEAD encrypt/decrypt, tunnel encapsulation, peer forwarding — kernel with DCO
`ovpn` module	In-tree since Linux **6.16**; needs **OpenVPN 2.7+**
`ovpn-dco` / backports	Out-of-tree path for kernels < 6.16 or OpenVPN **2.6.x**
`--disable-dco`	Force the legacy userspace data path (benchmarks, unsupported options)

Cipher and topology constraints (by design). DCO is **L3 / `dev tun` only**. Supported data-channel ciphers are **AES-GCM-128/192/256** and **ChaCha20-Poly1305**. Compression framing is out; servers use **`--topology subnet`**. Older CBC-only peers force a non-DCO fallback. Routing to clients uses the **host routing table** (a client VPN IP as nexthop) rather than OpenVPN's internal iroute machinery — see the official `README.dco.md`.

Version pairing that matters in 2026:

Kernel / userspace	Module name	Notes
Linux 6.16 + OpenVPN 2.7	`ovpn` (in-tree)	Preferred; no DKMS on stock distro kernels
Older kernel or OpenVPN 2.6	`ovpn-dco` via ovpn-backports / distro DKMS	Still fine; package names differ by distro
Windows	`ovpn-dco-win`	Ships with official Windows builds since 2.6; multi-core workers improved further in 2026

Worked examples

Case 1: Confirm the kernel side before you touch configs

On a host that already runs Linux 6.16+ with OpenVPN 2.7+, you do not install a third-party module. Check what you have:

```
# dco-probe.sh - inventory for OpenVPN DCO (read-only; no packages installed here)
uname -r
modinfo ovpn 2>/dev/null | head -5 || echo "ovpn module metadata not present"
lsmod | grep -E '^ovpn' || echo "ovpn not loaded yet (normal until openvpn starts)"
openvpn --version 2>/dev/null | head -3 || echo "openvpn binary not on PATH"
```

Expected shape on a DCO-ready box (versions will differ):

```
$ uname -r
6.16.3-1-amd64
$ modinfo ovpn | head -3
filename:       /lib/modules/6.16.3-1-amd64/kernel/drivers/net/ovpn/ovpn.ko
description:    OpenVPN data channel offload
license:        GPL
```

If `modinfo ovpn` is missing on an older kernel, use the ovpn-backports project or your distro's `ovpn-dkms` / `kmod-ovpn` package — **not** covered as an install lab in this chapter (no package installs in the book environment).

Case 2: Let OpenVPN auto-enable DCO (and force it off for A/B)

OpenVPN **auto-detects** DCO when the module is present and the config is compatible. You only need an explicit flag to *disable* it.

```
# Minimal server sketch - AEAD + subnet topology so DCO can engage
# openvpn-server-dco.conf
port 1194
proto udp
dev tun
topology subnet
server 10.8.0.0 255.255.255.0
dh none
tls-groups X25519:secp256r1
data-ciphers AES-256-GCM:AES-128-GCM:CHACHA20-POLY1305
persist-key
keepalive 10 60
verb 3
# certificates omitted - use your existing PKI
```

```
# Run with DCO (default when available):
#   openvpn --config openvpn-server-dco.conf
# Force userspace data path for a throughput comparison:
#   openvpn --config openvpn-server-dco.conf --disable-dco
```

Log lines to look for:

Message	Meaning
DCO / `ovpn` enabled (wording varies by version)	Kernel data path is live
`Note: Kernel support for ovpn-dco missing, disabling data channel offload.`	Module absent or userspace too old for in-tree `ovpn`
Warning that an option disabled DCO	Config used a feature DCO cannot implement (compression, unsupported cipher, …)

Antonio Quartulli's workstation numbers (Ryzen 9 9950X, 25 Gbps NIC, iperf3 UDP, published via the OpenVPN DCO guide, Feb 2026): legacy userspace **3.61 Gbps**; DCO **7.15 Gbps** — about **2×** on the same box. Third-party reports (IPFire, consumer VPN vendors, router vendors) show similar order-of-magnitude gains; treat any single percentage as hardware-specific.

Case 3: One-sided DCO still helps

You do **not** need DCO on both ends for a win. A DCO server alone cuts server CPU and raises aggregate capacity; both ends DCO maximizes the path. Clients that speak OpenVPN 2.6+/Connect 3.4+ with AEAD are enough for the control plane to negotiate a DCO-capable data channel when the server supports it.

```
laptop (no DCO)  TLS   gateway (DCO / ovpn)

        ciphertext still         encrypt/decrypt + forward
        encrypted tunnel     stays in kernel on the gateway
```

Case 4: Routing mental shift under DCO

With DCO, "iroutes" become ordinary host routes whose nexthop is the client's VPN address. Client-to-client is no longer a special userspace hairpin: packets hit the tunnel interface and the **system routing table** decides the next peer.

```
# Example: send 192.0.2.0/24 to the client that owns 10.8.0.5
# ip route add 192.0.2.0/24 via 10.8.0.5
```

If you still need a blackhole when that client is offline, add an explicit companion route that only exists while the peer is connected — DCO will not invent OpenVPN's old internal routing table for you.

The trap

1. **CBC or compression "just for compatibility".** A single `--data-ciphers` entry that forces CBC, or leftover `--compress`, silently (or with a warning) drops you back to userspace. You keep the old config working and wonder why the benchmark looks unchanged.
2. **OpenVPN 2.6 + in-tree `ovpn`.** The Linux 6.16 `ovpn` module speaks the **2.7** netlink API. On 2.6 you still want the older `ovpn-dco` out-of-tree module (or upgrade userspace). Mixing them yields "kernel support missing" even though *a* DCO module is loaded.
3. **Assuming DCO changes cryptography.** It does not. TLS, certificates, and cipher *selection* stay in userspace; DCO only relocates AEAD packet processing. Threat modeling stays the same; capacity planning does not.
4. **Comparing protocols by blog headlines alone.** Marketing posts crown a winner every quarter. Measure *your* NIC, CPU crypto offload, and MTU. DCO removes OpenVPN's historical userspace tax; it does not erase the need for sound AEAD choice, MSS clamping, and CPU affinity on multi-gigabit paths.

The boring rule

Ship **OpenVPN 2.7** on **Linux 6.16** with **AES-GCM or ChaCha20-Poly1305**, `--topology subnet`, and no compression. Let DCO auto-enable. Keep `--disable-dco` in your lab toolkit for A/B tests. On older kernels, use the matching out-of-tree module — do not assume the in-tree `ovpn` name. Verify with `modinfo` / logs before you trust a speed claim.

Try this

1. On a 6.16+ lab host, run `modinfo ovpn` and `openvpn --version`. Confirm the 2.7 + `ovpn` pairing.
2. Bring up a two-namespace UDP tunnel (reuse your existing PKI). Capture a short iperf3 run with DCO on, then the same run with `--disable-dco`. Record throughput and `top` CPU for `openvpn`.
3. Intentionally add a DCO-incompatible option (e.g. force a CBC cipher). Confirm the log shows DCO disabled, then remove it.
4. Add a host route `via` a client VPN IP and traceroute from the server VRF/namespace to prove DCO is using the system FIB.

Sources

- OpenVPN DCO: The Definitive Guide (OpenVPN blog; updated 2026-10-08)
- OpenVPN `README.dco.md` (upstream limitations and enablement)
- Community wiki: DataChannelOffload
- ovpn-backports (out-of-tree `ovpn` for older kernels)

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch

The cryptokey-routing chapter showed one tunnel and one peer. Real laptops and edge routers often run **several** WireGuard interfaces at once: a full-tunnel VPN to work, a split-tunnel to a lab, maybe a site-to-site mesh peer. The hard part is not key exchange. It is **who owns** `0.0.0.0/0`, how **policy routing** and **fwmarks** keep ciphertext off the wrong table, and how a **kill switch** (which is *not* a WireGuard feature) drops plaintext when the tunnel is down.

This chapter makes those three ideas concrete. Lab notes use Debian 13 ideas and `wireguard-tools` **v1.0.20210914** as found on this host; privileged `ip netns` drills need root and are shown as scripts you run on a disposable machine. No packages are installed in the scout sandbox.

```
wg version
ip -V | head -1
```

```
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
ip utility, iproute2-6.15.0, libbpf 1.5.0
```

Learning goals

By the end of this chapter you can:

- Explain why two peers both listing `AllowedIPs = 0.0.0.0/0` cannot share one routing story without policy routing
- Set `fwmark` on a WireGuard interface and steer plaintext with `ip rule` / extra tables (the `wg-quick` pattern)
- Sketch a kill switch as **filter** rules that drop non-marked traffic, not as a WireGuard knob
- Know when network namespaces (wireguard.com/netns) beat classic route hacks

Concepts in one page

```
App traffic (no mark)    ip rule: lookup table 2468    default dev wg0
wg0 UDP ciphertext        socket marked fwmark 1234    main table / ISP

Second tunnel wg1 with its own mark + table
   rules choose table by mark, uid, or fwmark from cgroup/VRF
```

Piece	Job
`AllowedIPs`	Cryptokey routing: outbound peer selection **and** inbound source filter
Kernel route to `wg0`	Only gets the packet *onto* the interface
`wg set … fwmark`	Marks WireGuard's **own** UDP so policy routing can exempt it from the tunnel default
`ip rule` + custom table	Sends unmarked (or otherwise classified) traffic into the tunnel table
Kill switch	Firewall (`iptables`/`nftables`) drop of traffic that is neither local nor marked for a live tunnel
Netns physical/init split	Alternative full-tunnel design from the WireGuard site — ciphertext socket stays in the "physical" namespace

Worked examples

Case 1: Two tunnels, one default — the collision

```
# Bad idea on one host without policy routing:
# /etc/wireguard/wg0.conf  (work VPN)
[Peer]
AllowedIPs = 0.0.0.0/0, ::/0

# /etc/wireguard/wg1.conf  (personal VPN)
[Peer]
AllowedIPs = 0.0.0.0/0, ::/0
```

Both interfaces claim the entire Internet in cryptokey routing **and** (via `wg-quick`) typically install default routes. Whichever `wg-quick up` ran last wins the default route; the other peer's `AllowedIPs` still claim destinations they will never see. Symptoms: "VPN is up" in `wg show` but traffic leaks to the wrong peer or blackholes.

Boring fix: at most **one** peer on a host with `0.0.0.0/0`. Other tunnels get **split** prefixes (`10.0.0.0/8`, `192.0.2.0/24`, …). If you truly need two full tunnels, separate them with **namespaces**, **VRFs**, or **per-uid / per-mark** policy routing — not two defaults in `main`.

Case 2: `fwmark` + policy routing (single full tunnel)

This is the improved rule-based pattern from Routing & Network Namespaces — the same idea `wg-quick` uses:

```
#!/usr/bin/env bash
# wg-policy-up.sh - docs-faithful sketch (needs root; not run in scout sandbox)
set -euo pipefail
# Assume wg0 already has keys/peers; we only add routing policy.
wg set wg0 fwmark 1234
ip route add default dev wg0 table 2468
ip rule add not fwmark 1234 table 2468
ip rule add table main suppress_prefixlength 0
```

Step	Meaning
`fwmark 1234` on `wg0`	Ciphertext UDP from WireGuard carries mark 1234
`default dev wg0 table 2468`	Tunnel table sends everything into `wg0`
`not fwmark 1234 table 2468`	Plaintext (unmarked) uses the tunnel table
`suppress_prefixlength 0`	Still allow non-default routes from `main` (LAN) for unmarked packets

Without the mark exemption, tunnel ciphertext would be routed back into `wg0` (routing loop). With it, only application traffic takes the tunnel default.

```
# Tear-down sketch
ip rule del not fwmark 1234 table 2468 2>/dev/null || true
ip rule del table main suppress_prefixlength 0 2>/dev/null || true
ip route flush table 2468 2>/dev/null || true
wg set wg0 fwmark 0
```

Verification on this scout host: commands were **not** executed with root. On a lab VM, `wg show wg0` should list `fwmark: 0x4d2`, and `ip rule` should show the two policy lines.

Case 3: Multi-tunnel with distinct marks

```
  wg0 (work)   fwmark 0x10   table 100   rule: from all fwmark 0x10 lookup 100
  wg1 (lab)    fwmark 0x20   table 200   rule: from all fwmark 0x20 lookup 200

  Mark traffic deliberately:
    - systemd: IPAccounting / socket marking via nft / cgroup classifiers
    - "work browser" runs under a unit that sets mark 0x10
    - default unmarked traffic: either ISP (no kill switch) or drop (kill switch)
```

```
# Sketch - two tunnel tables (root; not run here)
wg set wg0 fwmark 16
wg set wg1 fwmark 32
ip route add default dev wg0 table 100
ip route add default dev wg1 table 200
ip rule add fwmark 16 table 100
```

```
ip rule add fwmark 32 table 200
# Exempt each interface's own ciphertext from its tunnel table using
# "not fwmark <that-iface-mark>" rules, same idea as Case 2.
```

`AllowedIPs` on each peer should match the **prefixes you actually send** into that interface. A work VPN peer might get `0.0.0.0/0` *inside table 100 only*; a lab peer gets `10.50.0.0/16` only.

Case 4: Kill switch = firewall, not WireGuard

WireGuard has no `KillSwitch=true`. Operators implement "if the tunnel is down, do not leak" with packet filters:

```
# nftables sketch - drop forwarded/output traffic not marked for a tunnel
# (illustrative; tighten to your iface list and local subnets before use)
nft add table inet killswitch
nft add chain inet killswitch output '{ type filter hook output priority 0; policy drop; }'
nft add rule  inet killswitch output meta mark 16 accept   # work tunnel mark
nft add rule  inet killswitch output meta mark 32 accept   # lab tunnel mark
nft add rule  inet killswitch output oifname "lo" accept
nft add rule  inet killswitch output ip daddr 192.168.0.0/16 accept  # example LAN
# cipher path: accept UDP to known endpoints OR accept mark that wg sets on socket
```

When `wg0` goes down, marks never get applied by your classifier → output drops → no ISP leak. Mis-write the LAN exceptions and you lock yourself out of SSH; test on console.

Case 5: Namespace alternative for "all traffic through one tunnel"

From the official netns guide: move `eth0` into a `physical` namespace, birth `wg0` there, move `wg0` to init, default-route init through `wg0`. Ciphertext UDP stays in `physical`; ordinary processes never see `eth0`. That design **avoids** half of the `main`-table fights and is often cleaner than multiple full tunnels in one namespace. Multiple *simultaneous* full tunnels still want marks, VRFs, or separate netns per tunnel.

The trap

- Two `AllowedIPs = 0.0.0.0/0` peers without separate policy domains.
- Forgetting fwmark exemption → mysterious loss of handshake (ciphertext sucked into the tunnel route).
- Calling `wg-quick` "the kill switch" — `SaveConfig` / `Table=` help routing; leak prevention is firewall policy.
- Skipping endpoint / LAN exceptions and bricking remote admin access.
- Treating X marketing ("multi-tunnel + kill switch") as WireGuard core features — they are **operator composition** on top of cryptokey routing.

The boring rule

One default owner per routing domain; mark ciphertext; steer plaintext with `ip rule`; implement kill switches in `nftables`/`iptables`; prefer netns when the whole host should only speak through one tunnel.

Try this

1. On a disposable VM, bring up one `wg-quick` full tunnel and `wg show` / `ip rule` / `ip route show table all` to see marks and tables.
2. Add a second split-tunnel interface with a narrow `AllowedIPs` and ping across both.
3. Implement a minimal nft output drop policy and prove that stopping `wg-quick` blocks IPv4 to `1.1.1.1` while LAN still works.
4. Read wireguard.com/netns end-to-end and sketch when you would choose netns over fwmark.

Sources

- WireGuard · Routing & Network Namespaces
- `wg(8)`, `wg-quick(8)`, `ip-rule(8)`, `ip-route(8)`
- Prior chapter in this book: cryptokey routing and PSK (`02b-wireguard-cryptokey-routing-and-psk.qmd`)
- Inspiration: CumulusVPN on X (AllowedIPs fights, policy routing, fwmarks, kill switch as firewall)

VXLAN Data Plane

VXLAN Data Plane

VXLAN encapsulates Ethernet frames in UDP/IP so you can stretch **L2 segments over an L3 underlay**. This chapter focuses on the **data plane**: VNI, VTEP, flood behavior, and a Linux VXLAN lab in Containerlab (mid-2026). Control planes (EVPN) come next.

Learning goals

By the end of this chapter you can:

- Define VTEP, VNI, and underlay vs overlay roles for VXLAN
- Configure Linux `ip link add type vxlan` with static flood lists or bridge
- Capture VXLAN UDP (port 4789) and relate outer/inner headers
- Predict MAC learning and flooding behavior without EVPN
- Handle MTU for VXLAN overhead

Concepts

Roles

Term	Meaning
VTEP	VXLAN Tunnel End Point—encap/decap device
VNI	VXLAN Network Identifier—segment ID (~24-bit space)
Underlay	IP network between VTEP loopbacks/addresses
Overlay	Ethernet segments identified by VNI

```
VM/host -- L2 -- [ VTEP ]==== underlay IP ====[ VTEP ] -- L2 -- VM/host
                   VNI 100                                VNI 100
```

Header (mental)

```
Outer Eth | Outer IP | UDP dport 4789 | VXLAN (VNI) | Inner Eth | Inner payload
```

UDP source port often encodes entropy for ECMP.

Data-plane only learning

Without EVPN (or a controller):

1. Unknown unicast / broadcast / multicast may **flood** to all VTEPs in the flood list (head-end replication) or underlay multicast group.
2. Source MAC learning binds MAC → remote VTEP.
3. Scale limits and wasteful flood—why EVPN exists.

Multicast vs head-end replication

Mode	Idea
Underlay multicast	BUM to group; needs multicast routing
Head-end replication (HER)	Ingress VTEP unicasts BUM to each peer VTEP

Labs often use **HER / static remote VTEP list** for simplicity.

When VXLAN is appropriate

- DC fabric tenant segments
- Lab simulation of multi-leaf L2
- Separating many logical L2s without many VLANs on underlay

Not always needed for simple L3-only campuses.

Topology — two VTEPs + underlay + hosts

```
name: vxlan-dp

topology:
  nodes:
```

```
  leaf1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 bridge iputils tcpdump
  leaf2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 bridge iputils tcpdump
  spine:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils tcpdump
      - sysctl -w net.ipv4.ip_forward=1
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils

links:
  - endpoints: ["h1:eth1", "leaf1:eth1"]
  - endpoints: ["h2:eth1", "leaf2:eth1"]
  - endpoints: ["leaf1:eth2", "spine:eth1"]
  - endpoints: ["leaf2:eth2", "spine:eth2"]
```

Addressing

Node	Underlay	Access
leaf1 lo	10.0.0.1/32	—
leaf2 lo	10.0.0.2/32	—
leaf1 eth2	10.1.1.1/30	—
spine eth1	10.1.1.2/30	—
leaf2 eth2	10.1.2.1/30	—
spine eth2	10.1.2.2/30	—
h1	—	10.10.10.10/24
h2	—	10.10.10.20/24
VNI	100	same L2 segment

Node	Underlay	Access

Underlay routes: leaf1 reaches 10.0.0.2 via spine; advertise loopbacks with statics:

```
docker exec clab-vxlan-dp-leaf1 sh -c '
  ip addr add 10.0.0.1/32 dev lo
  ip addr add 10.1.1.1/30 dev eth2; ip link set eth2 up
  ip route add 10.0.0.2/32 via 10.1.1.2
  ip route add 10.1.2.0/30 via 10.1.1.2
'
docker exec clab-vxlan-dp-spine sh -c '
  ip addr add 10.1.1.2/30 dev eth1; ip link set eth1 up
  ip addr add 10.1.2.2/30 dev eth2; ip link set eth2 up
  ip route add 10.0.0.1/32 via 10.1.1.1
  ip route add 10.0.0.2/32 via 10.1.2.1
'
# leaf2 similar with 10.0.0.2 and 10.1.2.1
```

VXLAN + bridge on leaf1

```
docker exec clab-vxlan-dp-leaf1 sh -c '
  ip link add br100 type bridge
  ip link set br100 up
  ip link add vxlan100 type vxlan id 100 \
    local 10.0.0.1 dev eth2 dstport 4789
  # static remote VTEP (HER)
  bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst 10.0.0.2
  ip link set vxlan100 up
  ip link set eth1 up
  ip link set eth1 master br100
  ip link set vxlan100 master br100
'
```

leaf2:

```
docker exec clab-vxlan-dp-leaf2 sh -c '
  ip link add br100 type bridge
  ip link set br100 up
  ip link add vxlan100 type vxlan id 100 \
    local 10.0.0.2 dev eth2 dstport 4789
  bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst 10.0.0.1
  ip link set vxlan100 up
  ip link set eth1 up
  ip link set eth1 master br100
```

```
  ip link set vxlan100 master br100
'
```

Hosts (same subnet, no default needed for local L2 test):

```
docker exec clab-vxlan-dp-h1 sh -c '
  ip addr add 10.10.10.10/24 dev eth1; ip link set eth1 up
'
docker exec clab-vxlan-dp-h2 sh -c '
  ip addr add 10.10.10.20/24 dev eth1; ip link set eth1 up
'
```

Syntax note: `bridge fdb` / `ip link add vxlan` flags vary slightly by iproute2 version; consult `man ip-link` on your image. Some use `vxlan ... remote 10.0.0.2` for point-to-point single peer.

Deploy checks

```
docker exec clab-vxlan-dp-leaf1 ping -c 2 10.0.0.2
docker exec clab-vxlan-dp-h1 ping -c 3 10.10.10.20
docker exec clab-vxlan-dp-leaf1 bridge fdb show
docker exec clab-vxlan-dp-spine tcpdump -ni eth1 -c 10 udp port 4789
```

Predict

- Underlay loopback ping works
- Host ping works over VNI 100
- Spine sees UDP/4789, not bare inner ARP as local LAN

Observe

```
docker exec clab-vxlan-dp-spine tcpdump -ni eth1 -vv -c 5 udp port 4789
docker exec clab-vxlan-dp-leaf1 ip -d link show vxlan100
```

Fix

Failure	Checks
Underlay down	static routes, spine forward
No FDB flood entry	remote VTEP missing
VNI mismatch	id 100 both sides

Failure	Checks
Host not in bridge	master br100

Drill — MTU

VXLAN overhead commonly ~50 bytes class. Set underlay MTU 1550 or overlay-aware 1450:

```
docker exec clab-vxlan-dp-leaf1 ip link set vxlan100 mtu 1450
docker exec clab-vxlan-dp-leaf2 ip link set vxlan100 mtu 1450
docker exec clab-vxlan-dp-h1 ping -c 2 -M do -s 1472 10.10.10.20
docker exec clab-vxlan-dp-h1 ping -c 2 -M do -s 1400 10.10.10.20
```

Predict

Too-large DF fails; tuned sizes pass.

Drill — wrong VNI isolation

Create VNI 200 on leaf2 only for a third host experiment—or change leaf2 to `id 200` temporarily:

Predict

h1 cannot reach h2 (different segments).
Fix

Align VNI; document allocation.

Drill — MAC move / learning

```
docker exec clab-vxlan-dp-leaf1 bridge fdb show br100
docker exec clab-vxlan-dp-h1 ping -c 1 10.10.10.20
docker exec clab-vxlan-dp-leaf1 bridge fdb show br100
```

Predict

After traffic, remote MAC points at vxlan device / dst VTEP.

Flood scope risk

Static `00:00:00:00:00:00` FDB means BUM replicates to listed VTEPs. More leaves → more replication. This is why control planes advertise **who needs which VNI**.

FRR / SR Linux awareness

NOS VTEPs bind VNI to VLAN/IRB and underlay loopbacks. Data-plane counters and `show vxlan` equivalents matter operationally. Linux lab teaches headers; production uses integrated L2/L3.

Verification checklist

Check	Intent
Underlay VTEP reachability	Outer IP works
UDP 4789 on path	Encap happening
Same VNI	Segment match
FDB remote	Mapping exists
Host ARP/ping	Inner L2
MTU tests	Size tax handled

Common mistakes

Mistake	Symptom
VTEP IP not loopback stable	Flaps when link IP changes
No underlay route to remote VTEP	Silent blackhole
Bridging mis-order	Loops or no pass
Expecting EVPN routes without config	Only static works
One huge flood list forever	BUM explosion

Summary

- VXLAN is Ethernet-in-UDP/IP with **VNI** segment IDs
- VTEPs encap/decap; underlay only sees outer IP/UDP
- Static HER labs teach data plane before EVPN
- Verify with underlay ping, host ping, and spine captures

- Plan MTU and flood scope deliberately

Next: **EVPN control-plane intro**—distributing MAC/IP and VNI membership without pure flood-and-learn.

EVPN Control-Plane Intro

EVPN Control-Plane Intro

EVPN (Ethernet VPN) is a **BGP-based control plane** for overlay reachability—most often paired with VXLAN data planes in modern fabrics. This chapter is an **introduction to models and lab reality on free stacks** (FRR EVPN/VXLAN where available, optional SR Linux community). Syntax shifts; objects do not.

Learning goals

By the end of this chapter you can:

- Explain what EVPN advertises versus what VXLAN carries
- Name core route types at a practical level (2, 3, 5—awareness)
- Relate RD/RT-style membership ideas to "who joins which VNI"
- Sketch an FRR EVPN + VXLAN lab and know how to verify BGP EVPN tables
- Predict failure when underlay OK but EVPN session down

Why EVPN exists

Data-plane-only VXLAN:

- Floods BUM to all peers
- Learns MAC reactively
- Scales poorly; slow on moves

EVPN:

- Distributes MAC/IP bindings and VTEP locations via BGP
- Reduces unknown flooding (in good designs)
- Supports L2 and L3 (IRB) services in one framework

```
BGP EVPN  (control)     VXLAN/GRE/...  (data)

          same fabric intent
```

Objects (vendor-agnostic)

Object	Role
VTEP IP	Underlay endpoint
VNI	L2 segment id in data plane
MAC-VRF / EVI	Service instance
RD	Disambiguates routes
RT (route target)	Import/export membership
Type-2 route	MAC (/IP) advertisement
Type-3 route	Inclusive multicast / flood tree membership ideas
Type-5 route	IP prefix (L3 VPN style overlay)
IRB / L3 VNI	Inter-subnet routing in fabric

You do not need full RFC memorization to operate: **session up → EVPN routes present → data plane VNI up → hosts pass**.

Control vs data verification split

Layer	Healthy signs
Underlay	VTEP loopbacks ping; ECMP present
BGP EVPN	Neighbors Established; AFI/SAFI EVPN negotiated
EVPN RIB	Type-2 for host MACs; correct next-hop VTEP
Data plane	VXLAN encap; host ping; FDB consistent

Minimal design story — 2 leaf

```
h1--leaf1----spine----leaf2--h2
      \      underlay     /
       BGP EVPN (often via spine route-reflector or eBGP unnumbered fabric)
```

Common free lab patterns:

1. **iBGP EVPN** to a route reflector (spine or dedicated).
2. **eBGP EVPN** unnumbered on fabric links (advanced; FRR supports variants).

Start with **iBGP + RR** mentally even if your first lab is two leaves peering directly.

FRR capability note (mid-2026)

FRR continues to expand EVPN/VXLAN integration. Treat the following as **structural config**—verify against your image's `vtysh` help and docs:

```
# daemons: zebra, bgpd, staticd (and platform vxlan integration)
router bgp 65000
 bgp router-id 10.0.0.1
 neighbor 10.0.0.3 remote-as 65000
 neighbor 10.0.0.3 update-source lo
 !
 address-family l2vpn evpn
  neighbor 10.0.0.3 activate
  advertise-all-vni
 exit-address-family
!
# VNI / bridge mapping is platform-specific:
# Linux FRR may integrate with kernel bridge/vxlan via zebra
```

Always confirm with:

```
vtysh -c 'show bgp l2vpn evpn summary'
vtysh -c 'show bgp l2vpn evpn'
vtysh -c 'show evpn vni'
vtysh -c 'show evpn mac vni all'
```

If a command is missing, your build differs—fall back to kernel VXLAN static (previous chapter) and read FRR release notes.

Topology sketch (EVPN lab)

```
name: evpn-intro

topology:
  nodes:
    leaf1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/leaf1:/etc/frr
    leaf2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/leaf2:/etc/frr
```

```
    rr:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/rr:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["leaf1:eth1", "rr:eth1"]
    - endpoints: ["leaf2:eth1", "rr:eth2"]
    - endpoints: ["h1:eth1", "leaf1:eth2"]
    - endpoints: ["h2:eth1", "leaf2:eth2"]
```

Underlay: give leaf1/leaf2/rr loopbacks `10.0.0.1/32`, `.2`, `.3` and point-to-point links with static or OSPF so loopbacks mesh. EVPN BGP between leaves and rr using loopbacks.

RR config idea

```
router bgp 65000
 bgp router-id 10.0.0.3
 neighbor 10.0.0.1 remote-as 65000
 neighbor 10.0.0.1 update-source lo
 neighbor 10.0.0.2 remote-as 65000
 neighbor 10.0.0.2 update-source lo
 address-family l2vpn evpn
  neighbor 10.0.0.1 activate
  neighbor 10.0.0.1 route-reflector-client
  neighbor 10.0.0.2 activate
  neighbor 10.0.0.2 route-reflector-client
 exit-address-family
```

Predict → observe → fix drills

Drill 1 — Session before service

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn summary'
```

Predict: Established to RR.
If Idle: underlay loopback ping, update-source, firewall, daemons.

Drill 2 — Host MAC appears as EVPN route

```
docker exec clab-evpn-intro-h1 ping -c 2 10.10.10.20
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn'
docker exec clab-evpn-intro-leaf2 vtysh -c 'show evpn mac vni all'
```

Predict: Type-2-like entries; next-hop remote VTEP.
Fix: VNI advertise config, bridge binding, host traffic to trigger learning if required.

Drill 3 — Kill EVPN, keep underlay

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'conf t' -c 'router bgp 65000' -c 'neighbor 10.0.
docker exec clab-evpn-intro-h1 ping -c 5 10.10.10.20
```

Predict: underlay pings between leaves still work; overlay host traffic fails or relies on stale/f flood only.
Observe: BGP down, EVPN table ages out.
Fix: `no neighbor … shutdown`.

Drill 4 — RT mismatch (concept)

If export/import RTs do not match, routes exist on one leaf but are not imported on the other.

Predict: `show bgp l2vpn evpn` has routes locally originated but remote missing or not installed.
Fix: align VRF/VNI RT policy.

L2 VNI vs L3 VNI (awareness)

Service	Host experience	EVPN idea
L2 VNI	Same subnet two leaves	MAC routes (type-2)
L3 VNI / IRB	Different subnets routed in fabric	Type-5 prefixes + symmetric/asymmetric IRB models

Do not implement every IRB mode on day one. Get **same-subnet across two leaves** solid first.

Optional SR Linux community path

Nokia SR Linux is a common Containerlab free/community kind for EVPN-VXLAN labs. Workflow:

1. Underlay eBGP or OSPF
2. Overlay BGP EVPN
3. `mac-vrf` / `vxlan-interface` style objects

Map to the same verification split. Use SR Linux documentation for exact CLI; journal the object mapping:

```
VNI   vxlan-interface
MAC-VRF   bridge table
bgp-evpn   advertisement
```

Security / isolation checklist

- VNI allocation authority
- RT import discipline (no accidental cross-tenant)
- Underlay ACL optional for UDP/4789 between VTEPs only
- Control-plane protection on BGP

What this intro deliberately skips

- Full multihoming Ethernet segments (ESI) deep dive
- Complex IRB interop matrices
- Multicast optimized BUM underlays
- DCI stretch best practices

Those are fabric/scale topics once the skeleton works.

Common mistakes

Mistake	Symptom
EVPN before underlay	Endless BGP down
Wrong AFI/SAFI	Session up but no EVPN
Data plane VNI advertised	Control OK, ping fail
Treating EVPN as L2 switch only	Miss L3 service models later
No RR scaling plan	Full mesh pain

Summary

- EVPN is BGP control for overlay membership and MAC/IP reachability
- VXLAN (or similar) remains the data plane
- Verify **underlay → BGP EVPN → EVPN RIB → host data** in order
- FRR and SR Linux are free-ish paths to practice; check your image's feature set
- Master two-leaf same-subnet before IRB gymnastics

Next: **overlay lab**—a single integrated workout combining underlay, VXLAN, and (if available) EVPN verification.

Overlay Lab

Overlay Lab

This chapter is a **portfolio lab**: build a small multi-site / multi-leaf overlay on free tools, break it on purpose, and prove each plane. Choose track A (GRE sites), track B (VXLAN static), or track C (EVPN if your FRR/SR Linux build supports it). Mid-2026 defaults: Containerlab + FRR + Alpine.

Learning goals

By the end of this chapter you can:

- Deliver one complete overlay lab folder that rebuilds from git
- Show captures proving encapsulation
- Run a failure matrix (underlay, tunnel endpoint, VNI/policy)
- Document MTU and security stance
- Meet the book's definition of done for overlay work

Pick a track

Track	Focus	Difficulty
A — Dual-site GRE/WG	Site tunnels + routing	
B — VXLAN HER two-leaf	L2 overlay data plane	
C — EVPN two-leaf + RR	Control + data	

Do **A then B** if new to overlays. Attempt C only after B host pings work.

Definition of done (all tracks)

- ☐ `README.md` with diagram + addressing
- ☐ `*.clab.yml` + configs/scripts in git

- ☐ Clean `containerlab deploy` on cold start
- ☐ `verify.sh` green
- ☐ Capture artifact (pcap or logged tcpdump) showing outer header
- ☐ 3 failure drills with notes
- ☐ MTU test recorded
- ☐ Retrospective: what was underlay vs overlay

Track A — Dual-site GRE (reference build)

Reuse topology from **Site Tunnels**; package it cleanly.

```
labs/overlay-a/
  site-tun.clab.yml
  addressing.md
  scripts/bringup.sh
  verify.sh
  drills.md
  captures/.gitkeep
```

Success metrics

- h1 h2 continuous ping
- inet tcpdump shows GRE
- Underlay link down stops overlay; restore recovers
- DF large ping behavior documented

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-site-tun-h1 ping -c2 -W1 192.168.20.10
docker exec clab-site-tun-h2 ping -c2 -W1 192.168.10.10
docker exec clab-site-tun-sitea ip link show gre1 | grep -q UP
echo OK-A
```

Failure matrix A

#	Action	Expect
A1	`inet` eth2 down	overlay down
A2	delete gre on sitea	blackhole
A3	remove LAN route	gre peer lives, hosts die
A4	lower mtu wrongly	large packet fail

Track B — VXLAN two-leaf (reference build)

Package **VXLAN data plane** chapter topology as `labs/overlay-b/`.

Success metrics

- Underlay VTEP lo ping
- h1 h2 same subnet across leaves
- Spine capture UDP/4789
- Wrong VNI experiment isolates
- FDB shows remote MAC after traffic

bringup excerpt

```
#!/usr/bin/env bash
set -euo pipefail
# called after containerlab deploy
sudo containerlab deploy -t vxlan-dp.clab.yml
# apply addresses + vxlan/bridge (idempotent scripts per node)
./scripts/addr.sh
./scripts/vxlan.sh
./verify.sh
```

verify.sh B

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo FAIL:$* >&2; exit 1; }
docker exec clab-vxlan-dp-leaf1 ping -c1 -W1 10.0.0.2 || fail underlay
```

```
docker exec clab-vxlan-dp-h1 ping -c2 -W1 10.10.10.20 || fail overlay
echo OK-B
```

Failure matrix B

#	Action	Expect
B1	spine forward off	underlay fail → overlay fail
B2	remove FDB flood entry	BUM/unknown fail
B3	VNI mismatch	isolation
B4	host unbridge eth1	local access fail

Capture helper

```
docker exec clab-vxlan-dp-spine sh -c \
  'tcpdump -ni eth1 -c 20 udp port 4789 -w /tmp/vx.pcap'
# docker cp out to captures/vx.pcap
```

Track C — EVPN intro (stretch)

In raw Containerlab, configuring BGP EVPN with VXLAN encapsulation on FRR requires intricate route distinguishers (RD), route targets (RT), VTEP interfaces, and EVPN address families.

Option 1: Netlab EVPN-VXLAN Topology (Recommended)

With Netlab (`provider: clab`), EVPN and VXLAN are first-class modules:

```
# topology.yml (Track C)
---
provider: clab
defaults.device: frr
module: [ ospf, bgp, vlan, vxlan, evpn ]

vlans:
  tenant_a:
    vni: 100100
    links: [ h1-l1, h2-l2 ]

nodes:
  spine: { role: spine }
  l1: { role: leaf }
  l2: { role: leaf }
```

```
  h1: { device: linux }
  h2: { device: linux }

links:
  - l1: spine
  - l2: spine
  - h1: l1
  - h2: l2

validate:
  evpn_sessions:
    description: "Verify BGP EVPN control plane is Established"
    nodes: [ l1, l2 ]
    plugin: bgp_neighbor(family='evpn', state='Established')
  tenant_ping:
    description: "Verify tenant host reachability across the EVPN overlay"
    nodes: [ h1 ]
    exec:
      ping_h2: ping -c 3 172.16.0.2
```

Deploy and validate with:

```
netlab up
netlab validate
```

Option 2: Manual FRR Configuration

If authoring the manual FRR configurations, package the EVPN chapter with explicit **feature probe**:

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn summary' \
  || { echo "EVPN not available-document fallback to Track B"; exit 0; }
```

Success metrics

- BGP EVPN Established
- EVPN MAC routes visible (`show bgp l2vpn evpn`)
- Host ping works across tenant VNI
- Neighbor shutdown kills overlay learning path
- Underlay remains pingable

Failure matrix C

#	Action	Expect
C1	RR down	leaves lose routes (no alternate)
C2	advertise-all-vni removed	control stops updates
C3	underlay lo route removed	BGP over loopback dies
C4	RT mismatch experiment	import empty

Integrated “mini fabric snack” (optional merge)

If ambitious, combine track B underlay style with dual spines from the fabrics part later. **Do not** expand scope until verify is green on two leaves.

```
    s1
   /  \
 l1    l2
 |      |
 h1     h2
```

Underlay ECMP + VXLAN HER to both directions is enough complexity for one week.

Observability expectations

Tool	Use
`ping` / `traceroute`	Inner vs outer endpoints
`tcpdump` / `tshark`	Prove encap
`bridge fdb` / `wg show` / `ip xfrm`	State tables
`vtysh show bgp l2vpn evpn`	Control plane
`ip route get`	Which path outer takes

Hardening lite (overlay)

- ☐ Underlay allows only needed outer ports between VTEPs
- ☐ No accidental route leaking inner prefixes to “inet”
- ☐ Keys for WireGuard/IPsec not in public git
- ☐ VNI inventory file (`vnis.md`)

Lab report template

```
# Overlay lab report
Track: B
Date: 2026-...
Underlay: static loopbacks via spine
Overlay: VXLAN VNI 100 HER
Evidence:
- captures/vx.pcap
- verify.sh OK
Drills: B1-B4 notes...
What surprised me:
Next: EVPN or dual-spine
```

Predict → observe → fix (capstone habit)

For every drill row, write three lines in `drills.md`. If observe predict, you learned something—**that is the point**.

Time-box guidance

Block	Time
Deploy + address	30–60 min
Overlay up	1–2 h
Captures + verify	30 min
Failure matrix	1–2 h
Writeup	30 min

If blocked >1 h on EVPN syntax, drop to Track B and file an issue note—do not thrash.

Common mistakes

Mistake	Symptom
Mixing all tracks half-done	Nothing verifies
No cold-start test	"Works on my dirty ns"
Capturing on wrong node	Never see UDP 4789
Skipping underlay prove	Overlay debug theater
Giant jumbo scope	No finish

Summary

- One finished overlay lab beats three partial ones
- Tracks A→B→C escalate control-plane complexity
- Always prove **encap with packets** and **failures with a matrix**
- Keep VNI/tunnel inventory and MTU notes in git
- Next part scales underlays into **fabrics and multi-area design**

When this lab is green, you are ready for Clos-style underlays and multi-implementation leaves.

Fabrics and Multi-Area

Clos Leaf-Spine

Clos Leaf-Spine

Modern data-center fabrics prefer **scale-out leaf-spine (Clos-inspired)** topologies over deep access-aggregation trees. This chapter builds the underlay model and a free Containerlab fabric snack with FRR + Linux endpoints (mid-2026).

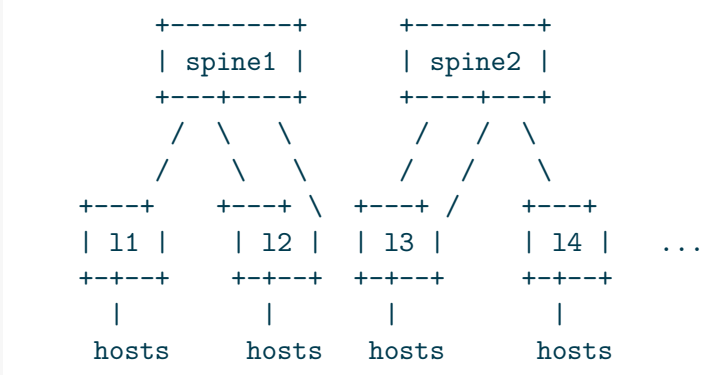
Learning goals

By the end of this chapter you can:

- Explain leaf, spine, and east-west vs north-south roles
- Design a 2-spine × 2-leaf IP underlay with ECMP
- Choose eBGP vs OSPF underlay trade-offs at a model level
- Deploy a Clos-ish lab and verify multipath
- Predict behavior when one spine or one leaf uplink fails

Why Clos-style fabrics

Classic tree pain	Leaf-spine idea
Oversubscription at aggregation	East-west via spines with planned ratio
STP blocking / L2 sprawl	L3 underlay to leaf; overlay for L2 if needed
Hard horizontal scale	Add leaves/spines with more links
Big failure domains	Limit L2; contain blasts



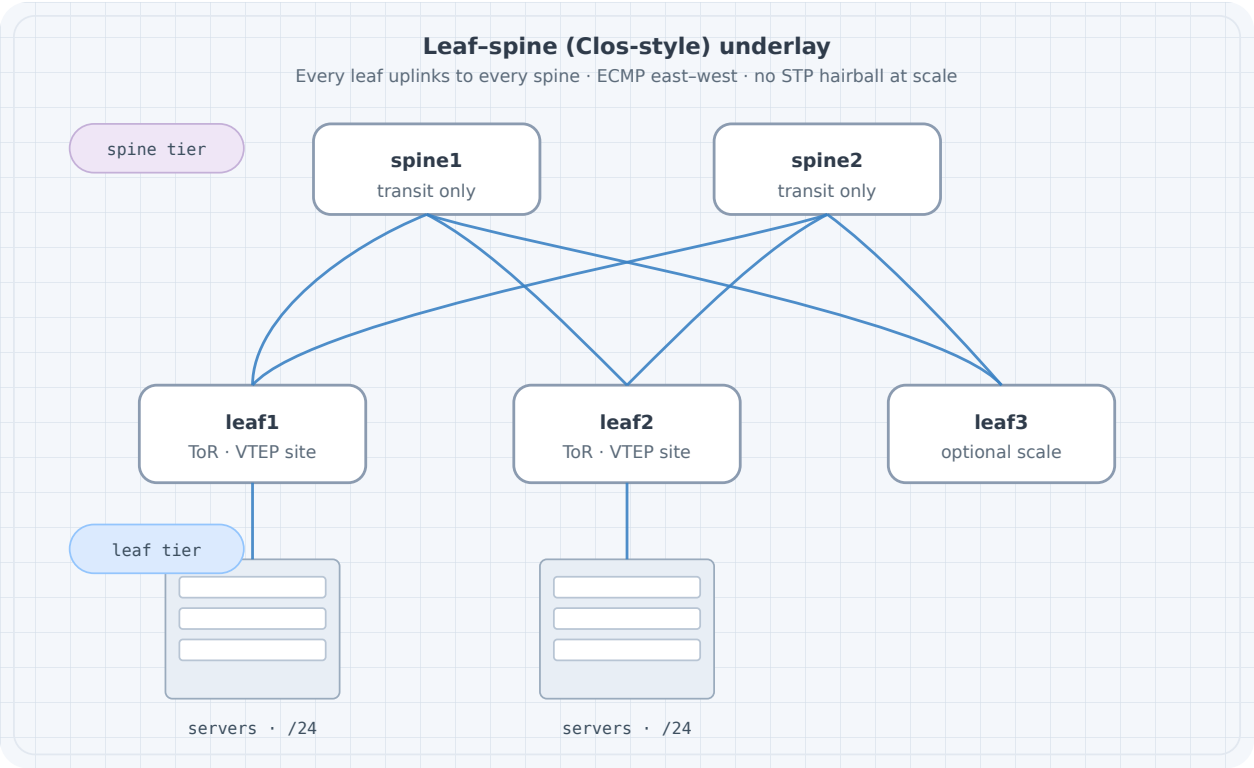


Figure 1: Leaf-spine Clos underlay

Small lab: **2 spines × 2 leaves** (diagram shows a third leaf for scale intuition).

Roles

Role	Function
Leaf	Connects servers; VTEP if overlay; default GW / anycast GW often here
Spine	Transit underlay only (usually no endpoints)
Super-spine / border	Optional higher tier or DC edge (later)

Spines should not become "core with lots of features" on day one—**move packets between leaves**.

Oversubscription (awareness)

```
oversubscription  (sum leaf downlink speeds) / (sum leaf uplink speeds)
```

Lab ignores real bandwidth; production designs document the ratio.

Underlay protocol choices

Underlay	Pros	Cons
eBGP (often unnumbered)	Scale, policy, isolation	More session design
OSPF / IS-IS	Simple adjacency model	Large single domain care
Static	Tiny labs	No scale

This book's first fabric: **OSPF or eBGP on FRR**—pick one and stay consistent. eBGP fabric is popular; OSPF is fine for 2×2 learning.

Addressing plan (2×2)

Node	Loopback	Links
s1	10.0.0.1/32	to l1, l2
s2	10.0.0.2/32	to l1, l2
l1	10.0.0.11/32	to s1, s2; host eth
l2	10.0.0.12/32	to s1, s2; host eth
h1	192.168.1.10/24	via l1
h2	192.168.2.10/24	via l2

Point-to-point `/31` or `/30` on fabric links—example `/30`:

Link	Subnet
l1–s1	10.1.1.0/30
l1–s2	10.1.2.0/30
l2–s1	10.1.3.0/30
l2–s2	10.1.4.0/30

Host subnets via leaf: advertise leaf loopbacks + host subnets into underlay **or** keep hosts only for L3 via leaf and use overlay later.

For pure underlay ECMP lab: ping between leaf loopbacks and between hosts with leaf default routing + OSPF advertised LANs.

Topology YAML

```
name: clos22

topology:
  nodes:
```

```
  s1:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/s1:/etc/frr
  s2:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/s2:/etc/frr
  l1:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l1:/etc/frr
  l2:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l2:/etc/frr
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.1.10/24 dev eth1
      - ip link set eth1 up
      - ip route add default via 192.168.1.1
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.2.10/24 dev eth1
      - ip link set eth1 up
      - ip route add default via 192.168.2.1

links:
  - endpoints: ["l1:eth1", "s1:eth1"]
  - endpoints: ["l1:eth2", "s2:eth1"]
  - endpoints: ["l2:eth1", "s1:eth2"]
  - endpoints: ["l2:eth2", "s2:eth2"]
  - endpoints: ["h1:eth1", "l1:eth3"]
  - endpoints: ["h2:eth1", "l2:eth3"]
```

FRR daemons

```
zebra=yes
ospfd=yes
# or bgpd=yes for eBGP underlay
```

OSPF underlay sketch (l1)

```
frr version 10.2.1
hostname l1
!
interface lo
 ip address 10.0.0.11/32
 ip ospf area 0
!
interface eth1
 ip address 10.1.1.1/30
 ip ospf network point-to-point
 ip ospf area 0
!
interface eth2
 ip address 10.1.2.1/30
 ip ospf network point-to-point
 ip ospf area 0
!
interface eth3
 ip address 192.168.1.1/24
 ip ospf area 0
!
router ospf
 ospf router-id 10.0.0.11
 passive-interface eth3
!
line vty
```

Spines: loopbacks + fabric links in area 0; **no host interfaces**. l2 mirrors with its addresses. Point-to-point OSPF avoids DR drama on p2p links.

eBGP underlay sketch (optional alternate)

```
router bgp 65101
 bgp router-id 10.0.0.11
 neighbor 10.1.1.2 remote-as 65100
 neighbor 10.1.2.2 remote-as 65100
 address-family ipv4 unicast
  network 10.0.0.11/32
  network 192.168.1.0/24
  neighbor 10.1.1.2 activate
  neighbor 10.1.2.2 activate
  maximum-paths 4
 exit-address-family
```

Spines use ASN 65100 (or each spine unique ASN—design choice). Leaves unique ASNs. Enable multipath.

Deploy and ECMP verify

```
sudo containerlab deploy -t clos22.clab.yml

docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'
docker exec clab-clos22-l1 vtysh -c 'show ip route 10.0.0.12'
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-clos22-h1 ping -c 3 192.168.2.10
docker exec clab-clos22-h1 traceroute -n 192.168.2.10
```

Predict

- Each leaf: 2 OSPF neighbors (both spines)
- Route to remote leaf/host shows **multiple nexthops** (ECMP) if multipath installed
- traceroute may show s1 or s2

Observe multipath

```
docker exec clab-clos22-l1 ip route show 192.168.2.0/24
# or
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0/24'
```

FRR/zebra may need `maximum-paths` under OSPF:

```
router ospf
 maximum-paths 4
```

Drill 1 — Spine loss

```
docker exec clab-clos22-s1 ip link set eth1 down
docker exec clab-clos22-s1 ip link set eth2 down
sleep 3
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-clos22-h1 ping -c 20 192.168.2.10
```

Predict → observe → fix

Predict | ECMP shrinks to remaining spine; brief loss |
Observe | single nexthop via s2 |
Fix | if blackhole, check l2 uplink to s2 and SPF |

Restore interfaces after.

Drill 2 — One leaf uplink down

```
docker exec clab-clos22-l1 ip link set eth1 down
docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'
docker exec clab-clos22-h1 traceroute -n 192.168.2.10
```

Predict

Still reachable via other spine; neighbor count 1.

Drill 3 — Polarization awareness

Send many parallel iperf flows; without entropy, paths may skew. Underlay traceroute from different sources may stick to one spine. Note in journal; overlays later add UDP entropy.

Pattern	Path

East-west vs north-south

Pattern	Path
East-west h1 h2	leaf → spine → leaf
North-south to Internet	leaf → border/spine path to edge

Do not attach WAN carelessly to all spines without a border design.

Overlay placement (preview)

VTEPs usually on **leaves**. Spines stay underlay-only. That separation keeps failure domains clean—connects to VXLAN/EVPN chapters.

Verification checklist

Check	Intent
All fabric adjacencies	Full mesh leaf-spine links up
Loopback mesh	Any lo to any lo
ECMP present	2 nexthops where topology allows
Host traffic	Data plane
Spine failure	Continuity

Common mistakes

Mistake	Symptom
Hosts on spines	Weird roles, scale pain
L2 across spines	Back to tree problems
Different OSPF areas accidentally	Missing routes
No p2p network type	Extra DR complexity
Expecting overlay without VTEP	Hosts same L2 fantasy

Summary

- Leaf-spine Clos-style fabrics scale east-west with L3 underlays
- Spines transit; leaves attach endpoints (and VTEPs)
- 2×2 FRR lab is enough to feel ECMP and spine loss
- Pick OSPF or eBGP underlay deliberately
- Keep overlays on leaves in later designs

Next: **multi-area OSPF**—when a single area is no longer the right underlay shape (campus/WAN more than fabric).

Multi-Area OSPF

Multi-Area OSPF

Single-area OSPF is the right first lab. Multi-area OSPF exists to **bound flooding, summarize, and structure large domains**. This chapter extends FRR OSPF into a small multi-area topology in Containerlab (mid-2026 free path)—useful for campus/WAN; fabrics often prefer BGP underlays instead.

Learning goals

By the end of this chapter you can:

- Explain area, ABR, ASBR, and LSA-type roles at operator level
- Build a three-router multi-area lab (backbone 0 + spoke area)
- Summarize at an ABR and predict reachability holes
- Contrast stub-ish ideas without drowning in edge-case flags
- Verify with database, route, and failure drills

Concepts

Why areas

Single area	Multi-area
Every router full topology	Topology flooding scoped
SPF on full graph	Smaller SPF domains
Simple	Needs backbone rules

Backbone rule (practical)

- Area **0** is the backbone
- Non-backbone areas attach via **ABRs** that touch area 0
- Virtual links exist for scars—not for greenfield labs

Roles

Role	Meaning
ABR	Router with interfaces in multiple areas; can summarize
ASBR	Injects external routes (redistribution)
Internal	All interfaces one area

LSA awareness (not flash cards)

Type idea	Carries
Router / network	Topology inside area
Summary (ABR)	Inter-area prefixes
External	From outside OSPF

You operate with: `show ip ospf database`, `show ip ospf border-routers`, `show ip route`.

Summarization

ABR can advertise `10.1.0.0/16` instead of many `/24`s. Mis-summary → **blackholes** if holes exist inside the aggregate.

Topology

```
     area 1              area 0              area 2
h1--r1---------------abr-r2---------------r3--h2
    (1.1.1.1)         (2.2.2.2)          (3.3.3.3)
```

Node	Areas	Notes
r1	1 only	internal
r2	0 + 1 + 2	ABR
r3	2 only	internal

Links:

Link	Subnet	Area
r1–r2	10.0.12.0/24	1
r2–r3	10.0.23.0/24	2
r1 LAN	192.168.1.0/24	1
r3 LAN	192.168.3.0/24	2

Link	Subnet	Area
r2 lo	2.2.2.2/32	0

Optional second area-0 link later; one ABR is enough to learn.

Topology YAML

```yaml
name: ospf-ma

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.3.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.3.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
```

```
    - endpoints: ["h1:eth1", "r1:eth2"]
    - endpoints: ["h2:eth1", "r3:eth2"]
```

FRR configs

r1 (area 1)

```
hostname r1
!
interface lo
 ip address 1.1.1.1/32
 ip ospf area 1
!
interface eth1
 ip address 10.0.12.1/24
 ip ospf area 1
!
interface eth2
 ip address 192.168.1.1/24
 ip ospf area 1
!
router ospf
 ospf router-id 1.1.1.1
 passive-interface eth2
!
line vty
```

r3 (area 2)

Mirror with `3.3.3.3`, `10.0.23.2/24` area 2, LAN `192.168.3.1/24` area 2.

r2 (ABR)

```
hostname r2
!
interface lo
 ip address 2.2.2.2/32
 ip ospf area 0
!
interface eth1
 ip address 10.0.12.2/24
 ip ospf area 1
```

```
!
interface eth2
 ip address 10.0.23.1/24
 ip ospf area 2
!
router ospf
 ospf router-id 2.2.2.2
 area 1 range 192.168.0.0/16
 area 2 range 192.168.0.0/16
!
line vty
```

Careful: broad `192.168.0.0/16` summary is a teaching weapon—it can hide holes. Prefer tight ranges:

```
 area 1 range 192.168.1.0/24
 area 2 range 192.168.3.0/24
```

Or summarize only when you own contiguous blocks.

Deploy and baseline

```
sudo containerlab deploy -t ospf-ma.clab.yml

docker exec clab-ospf-ma-r2 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-ma-r2 vtysh -c 'show ip ospf border-routers'
docker exec clab-ospf-ma-r1 vtysh -c 'show ip ospf database'
docker exec clab-ospf-ma-r1 vtysh -c 'show ip route'
docker exec clab-ospf-ma-h1 ping -c 3 192.168.3.10
```

Predict

- r2 has neighbors in area 1 and 2
- r1 sees inter-area path to 192.168.3.0/24 via r2
- Database on r1 does **not** contain full area 2 topology detail (summary only)

Observe

Compare `show ip ospf database` on r1 vs r2. r2 should show more topology.

Drill 1 — Summarization blackhole

On r2:

```
area 1 range 192.168.0.0/16
```

Do **not** put a real 192.168.99.0/24 anywhere, but inject a static on r1 into OSPF… or simpler: summarize too wide then withdraw a component.

Teaching sequence:

1. Advertise 192.168.1.0/24 only.
2. Add `area 1 range 192.168.0.0/16`.
3. On r3, install a static to 192.168.99.1 via null or missing next hop and redistribute—or ping 192.168.99.1 from r3 side expecting blackhole from r1 if summary exists without component.

Simpler blackhole lab:

```
# r2 summarizes 192.168.0.0/16 from area 1 while only .1.0/24 exists
# from r3 ping 192.168.1.10 works; ping 192.168.50.10 fails (good)
# If r3 thinks 192.168.50.0 is under summary toward area1, traffic blackholes at ABR
```

Predict

Summaries without covering specifics attract traffic to ABR then drop.

Fix

Tight ranges; discard routes; or don't summarize.

Drill 2 — ABR failure

```
docker stop clab-ospf-ma-r2
docker exec clab-ospf-ma-h1 ping -c 5 192.168.3.10
```

Predict

Total partition—single ABR is SPOF.
Design lesson

Redundant ABRs for real campus; dual attachment to area 0.

Drill 3 — Area mismatch

Put r1 eth1 in area 0 by mistake:

```
interface eth1
 ip ospf area 0
```

Predict

Adjacency fails or forms wrong; routes missing.
Observe

`show ip ospf neighbor` empty/wrong.
Fix

Restore area 1 both sides.

Stub area ideas (awareness)

Stub/totally stubby/NSSA reduce external LSAs into an area. FRR supports variants—use when external flood is noisy. For this chapter, know **why** (less type-5 in area) more than every flag combo.

```
router ospf
 area 1 stub
```

Retest default injection behavior when you try it.

Multi-area vs fabric BGP

Multi-area OSPF	Clos eBGP underlay
Campus/WAN hierarchy	DC scale-out
ABR summarization	Prefix + policy at leaves
Flooding domains	AS boundary policy

Do not force multi-area OSPF onto a leaf-spine if eBGP underlay is the house style.

Check	Command

Verification checklist

Check	Command
Neighbors per area	`show ip ospf neighbor`
ABR role	`show ip ospf border-routers`
DB scope	`show ip ospf database`
Inter-area routes	`show ip route` IA flags
Data plane	ping h1 h2

Common mistakes

Mistake	Symptom
Discontiguous area 0	Weird inter-area failures
Summary too broad	Blackholes
Expecting full topology remote area	Confusion reading DB
Redistribute without filters	Loops/externals everywhere
Single ABR production	Outage on maintenance

Summary

- Multi-area OSPF bounds flooding and enables ABR summarization
- Area 0 backbone rules still matter in greenfield design
- Always verify **database scope** and **inter-area routes**, not only neighbors
- Summaries can blackhole—tight ranges and drills
- Prefer BGP fabrics for Clos; multi-area OSPF for hierarchical campus/WAN

Next: **fabric with FRR**—package a tighter leaf-spine implementation path and verification kit.

Fabric with FRR

Fabric with FRR

This chapter turns the Clos model into an **FRR-operable fabric kit**: config layout, unnumbered eBGP option, verification scripts, and failure drills. Open stack only—Containerlab + FRR + Alpine (mid-2026). Overlay remains optional on leaves.

Learning goals

By the end of this chapter you can:

- Organize a fabric lab repo for N leaves × M spines
- Implement OSPF or eBGP underlay consistently on FRR
- Use `maximum-paths` and read multipath in RIB/FIB
- Smoke-test a fabric with a single `verify.sh`
- Know where to bolt VXLAN/EVPN on leaves later

Lab repo layout

```
labs/fabric-frr/
  clos22.clab.yml
  addressing.md
  config/
    s1/frr.conf daemons
    s2/...
    l1/...
    l2/...
  scripts/
    render.sh      # optional template expander
    verify.sh
    fail-spine.sh
  captures/
  README.md
```

Templating (jinja/make/python) pays off at 4×4; 2×2 can be hand-written.

Design defaults for this book

Choice	Default
Size	2 spine × 2 leaf
Underlay	OSPF area 0 p2p **or** eBGP
Loopbacks	10.0.0.x/32
Hosts	one per leaf, /24
Overlay	off (underlay first)
Images	`quay.io/frrouting/frr:10.2.1` + `alpine:3.20`

eBGP unnumbered (modern fabric style)

When link-local + IPv6 RA or RFC5549-style patterns available, unnumbered BGP reduces IP inventory. FRR support has matured—verify on your image:

```
interface eth1
 ip address 10.1.1.1/30
!
router bgp 65101
 neighbor fabric peer-group
 neighbor fabric remote-as external
 neighbor fabric capability extended-nexthop
 neighbor eth1 interface peer-group fabric
 !
 address-family ipv4 unicast
  neighbor fabric activate
  network 10.0.0.11/32
 exit-address-family
```

If interface peering is painful in your build, use explicit `/30` neighbors (Clos chapter)—**same topology math**.

Numbered eBGP fabric (reliable teaching)

Node	ASN
s1	65100
s2	65100
l1	65101
l2	65102

Spines same ASN (or not—both designs exist). Leaves unique.

Spine s1 excerpt

```
hostname s1
!
interface lo
 ip address 10.0.0.1/32
!
interface eth1
 ip address 10.1.1.2/30
!
interface eth2
 ip address 10.1.3.2/30
!
router bgp 65100
 bgp router-id 10.0.0.1
 neighbor 10.1.1.1 remote-as 65101
 neighbor 10.1.3.1 remote-as 65102
 !
 address-family ipv4 unicast
  network 10.0.0.1/32
  neighbor 10.1.1.1 activate
  neighbor 10.1.3.1 activate
  maximum-paths 4
 exit-address-family
!
line vty
```

Leaf l1 excerpt

```
hostname l1
!
interface lo
 ip address 10.0.0.11/32
!
interface eth1
 ip address 10.1.1.1/30
!
interface eth2
 ip address 10.1.2.1/30
!
interface eth3
 ip address 192.168.1.1/24
!
router bgp 65101
 bgp router-id 10.0.0.11
```

```
 neighbor 10.1.1.2 remote-as 65100
 neighbor 10.1.2.2 remote-as 65100
 !
 address-family ipv4 unicast
  network 10.0.0.11/32
  network 192.168.1.0/24
  neighbor 10.1.1.2 activate
  neighbor 10.1.2.2 activate
  maximum-paths 4
 exit-address-family
!
line vty
```

`daemons`: `zebra=yes`, `bgpd=yes`.

OSPF alternate (same wiring)

Reuse Clos chapter OSPF configs; add:

```
router ospf
 maximum-paths 4
```

Declarative Fabric with Netlab (Fast & Scalable)

For Clos fabrics, authoring individual router configs and IPAM spreadsheets for every spine and leaf switch quickly becomes error-prone. With Netlab (`provider: clab`), you declare the leaf/spine roles, ASNs, and links; Netlab generates the entire underlay, assigns loopback/transit IPs, enables ECMP multipath, and runs automated health checks:

```
# topology.yml
---
provider: clab
defaults.device: frr
module: [ bgp, bfd ]

nodes:
  s1: { role: spine, bgp.as: 65100 }
  s2: { role: spine, bgp.as: 65100 }
  l1: { role: leaf,  bgp.as: 65101 }
  l2: { role: leaf,  bgp.as: 65102 }
  h1: { device: linux }
  h2: { device: linux }

links:
```

```
  - l1:
    s1:
  - l1:
    s2:
  - l2:
    s1:
  - l2:
    s2:
  - h1:
    l1:
  - h2:
    l2:

validate:
  bgp_established:
    description: "Verify each leaf has 2 active eBGP sessions to spines"
    nodes: [ l1, l2 ]
    plugin: bgp_neighbor(count=2, state='Established')
  end_to_end_hosts:
    description: "Verify host-to-host traffic traverses the fabric"
    nodes: [ h1 ]
    exec:
      ping_h2: ping -c 3 10.0.0.6
```

Deploy and Verify with Netlab

```
# Generate configs, calculate IPAM, and deploy in Containerlab
netlab up

# Run automated validation across all BGP sessions and host pings
netlab validate

# Inject link failure drill: drop link between leaf1 and spine1
netlab tc set l1 eth1 --loss 100%
netlab validate
netlab tc clear l1 eth1
```

Manual Containerlab & FRR Implementation

If deploying without Netlab, you must manage individual configuration files and volume mounts. Below is the manual configuration pattern for reference and deep-dive troubleshooting.

Containerlab topology (`clos22.clab.yml`)

Same as Clos chapter `clos22`—keep one file, swap configs via binds.

Operational show kit

```
# sessions / neighbors
docker exec clab-clos22-l1 vtysh -c 'show bgp summary'
docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'

# multipath
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0/24'
docker exec clab-clos22-l1 ip route show 192.168.2.0/24

# loopback mesh
for d in 10.0.0.1 10.0.0.2 10.0.0.11 10.0.0.12; do
  docker exec clab-clos22-l1 ping -c1 -W1 $d || echo fail $d
done

# hosts
docker exec clab-clos22-h1 ping -c2 192.168.2.10
```

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }
P=clab-clos22

docker exec $P-l1 ping -c1 -W1 10.0.0.12 || fail "l1->l2 lo"
docker exec $P-l1 ping -c1 -W1 10.0.0.1 || fail "l1->s1 lo"
docker exec $P-h1 ping -c2 -W1 192.168.2.10 || fail "h1->h2"
docker exec $P-h2 ping -c2 -W1 192.168.1.10 || fail "h2->h1"

# multipath soft check: at least one route line with multiple nexthops or two entries
R=$(docker exec $P-l1 vtysh -c 'show ip route 192.168.2.0/24' || true)
echo "$R" | grep -Eq '192.168.2.0|via' || fail "no route to h2 subnet"

echo OK-FABRIC
```

Failure scripts

```
#!/usr/bin/env bash
# fail-spine.sh s1
set -euo pipefail
SP=${1:-s1}
docker exec clab-clos22-$SP ip link set eth1 down
docker exec clab-clos22-$SP ip link set eth2 down
echo "$SP uplinks down; run ping/traceroute; then restore"
```

```
#!/usr/bin/env bash
# restore-spine.sh s1
SP=${1:-s1}
docker exec clab-clos22-$SP ip link set eth1 up
docker exec clab-clos22-$SP ip link set eth2 up
```

Predict → observe → fix

Drill	Predict	Observe
Spine down	Paths via other spine	single nexthop
Leaf uplink down	Still works	1 BGP/OSPF neighbor
Wrong ASN	Session down	`show bgp summary` Active/Idle
Missing network stmt	Host subnet absent	RIB missing

Policy on fabric (light)

Even underlays need hygiene:

```
# do not accept default from random peers in fabric
# do not advertise lab management prefixes
ip prefix-list FABRIC-LO seq 5 permit 10.0.0.0/24 le 32
route-map EXPORT-FABRIC permit 10
 match ip address prefix-list FABRIC-LO
```

Apply out toward spines if you practice policy discipline.

Adding overlay later (hook points)

On each leaf:

1. Keep underlay BGP/OSPF as-is

2. Add VTEP on loopback

3. EVPN AF on BGP or static VXLAN to other leaf loopbacks

4. Spines remain ignorant of VNIs

```
leaf = underlay router + VTEP + (optional) IRB
spine = underlay router only
```

Scale checklist 2×2 → 4×4

Item	Action
IP plan	Spreadsheet / `addressing.md` generator
Configs	Templates
RAM	FRR light; watch host memory
verify	Loop all leaf pairs
Cabling matrix	Automate clab links generation

Observability

- Enable FRR logging to bound files
- Graph: `containerlab graph -t clos22.clab.yml`
- Optional: scrape with your ops chapter tools

Optional SONiC / SR Linux leaf

Replace one leaf image with community NOS; keep underlay protocol same. Document CLI mapping for `show bgp` / `show ip route`. Goal: **multi-implementation confidence** (next chapter deepens this).

Common mistakes

Mistake	Symptom
Spines advertising host defaults weirdly	Traffic trombones
maximum-paths left at 1	No ECMP
Overlay on spine	Operational regret
No cold deploy test	Config drift in running only
Same router-id	BGP/OSPF chaos

Mistake	Symptom

Summary

- FRR is enough to run a real teaching fabric underlay
- Standardize repo layout, ASNs, and verify scripts
- Prove ECMP and spine loss before overlays
- Leaves are the VTEP attachment point later
- Policy-light underlays still need prefix discipline

Next: **multi-implementation lab**—FRR + Linux + optional SR Linux in one topology.

Multi-Implementation Lab

Multi-Implementation Lab

Vendor-agnostic skill means the **same design** survives different CLIs. This capstone-style lab mixes **FRR, Linux endpoints, and optional Nokia SR Linux (community)** in one Containerlab topology (mid-2026 free/community images only).

Learning goals

By the end of this chapter you can:

- Run a leaf-spine underlay with heterogeneous leaf implementations
- Map show-command dialects to the same verification questions
- Keep addressing, ASN, and success metrics identical across nodes
- Document differences without abandoning models
- Produce a multi-NOS lab report

Design principle

```
Same:
  - topology roles (spine/leaf/host)
  - ASNs / areas / loopbacks
  - prefixes advertised
  - failure drills

Different:
  - config language
  - show syntax
  - feature knobs names
```

If behavior diverges, **stop and explain** (bug, knobs, or design mismatch)—do not "fix it" with silent statics on one leaf only.

Recommended topology

```
          s1 (FRR)      s2 (FRR)
          /    \        /    \
         /      \      /      \
   l1 (FRR)      l2 (SR Linux community OR FRR)
      |                 |
     h1                h2
```

If SR Linux image pull is unavailable, run **l2 as FRR** and still complete the dual-leaf discipline—or use VyOS/SONiC community kinds if present in your Containerlab kind list. The **process** matters.

Addressing / ASN (fixed)

Node	Kind	lo	ASN
s1	FRR	10.0.0.1/32	65100
s2	FRR	10.0.0.2/32	65100
l1	FRR	10.0.0.11/32	65101
l2	SR Linux or FRR	10.0.0.12/32	65102
h1	Alpine	192.168.1.10/24	—
h2	Alpine	192.168.2.10/24	—

Fabric links: same `/30` map as Clos chapter.

Topology YAML (FRR + optional srl)

```yaml
name: multi-imp

topology:
  nodes:
    s1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/s1:/etc/frr
    s2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/s2:/etc/frr
    l1:
```

```
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l1:/etc/frr
  l2:
    # Option A - all FRR:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l2:/etc/frr
    # Option B - SR Linux community (uncomment/adjust per containerlab docs):
    # kind: nokia_srlinux
    # image: ghcr.io/nokia/srlinux:latest   # pin a known free tag in real labs
    # binds: ...
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
      - ip route add default via 192.168.1.1
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
      - ip route add default via 192.168.2.1

links:
  - endpoints: ["l1:eth1", "s1:eth1"]
  - endpoints: ["l1:eth2", "s2:eth1"]
  - endpoints: ["l2:eth1", "s1:eth2"]
  - endpoints: ["l2:eth2", "s2:eth2"]
  - endpoints: ["h1:eth1", "l1:eth3"]
  - endpoints: ["h2:eth1", "l2:eth3"]
```

Pin image tags in real work; `latest` is a lab convenience only. Consult Containerlab kinds for current SR Linux free image references.

FRR side

Reuse fabric-with-FRR spine/leaf configs. Spines must peer to both leaves regardless of leaf NOS.

SR Linux mapping (conceptual)

Intent	FRR-ish	SR Linux-ish
Router id / lo	`interface lo` + BGP router-id	system / network-instance loopback
Interface IP	`ip address`	interface subinterface IPv4
eBGP neighbor	`neighbor A.B.C.D remote-as`	bgp neighbor under network-instance
Advertise lo + LAN	`network` statements	export policy + prefixes
Multipath	`maximum-paths`	ECMP / multipath knobs
Verify BGP	`show bgp summary`	`show network-instance default protocols bgp …` (version-specific)
Verify route	`show ip route`	`show route` / fib show variants

Exact paths change by SR Linux release—**write the mapping table in your lab README** with the commands that worked for you.

Verification questions (dialect-free)

Answer these on **every** leaf and spine:

1. Are fabric links up at L1/L2?
2. Is the underlay protocol adjacent to both spines?
3. Is my loopback in the fabric RIB on the far leaf?
4. Is the remote host subnet reachable with ECMP?
5. Do hosts pass bidirectional ICMP?
6. After killing s1, does traffic continue?

FRR answers

```
docker exec clab-multi-imp-l1 vtysh -c 'show bgp summary'
docker exec clab-multi-imp-l1 vtysh -c 'show ip route 192.168.2.0/24'
```

SR Linux answers (illustrative)

```
# container name pattern depends on clab naming
docker exec clab-multi-imp-l2 sr_cli 'show interface'
docker exec clab-multi-imp-l2 sr_cli 'show network-instance default route-table'
# adjust to your image's CLI entrypoint (sr_cli / srlinux)
```

predict → observe → fix matrix

Drill	Predict	FRR observe	SRL observe
Deploy cold	all sessions up	bgp summary	bgp show
h1→h2 ping	ok	ping	ping from h2 side
s1 down	ok via s2	single path	single path
shutdown l2 LAN network	h2 isolated	route gone on l1	route gone
MTU 1400 fabric	large ping fail	ping -M do	same

verify.sh (heterogeneous)

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }
P=clab-multi-imp

docker exec $P-h1 ping -c2 -W1 192.168.2.10 || fail "h1->h2"
docker exec $P-h2 ping -c2 -W1 192.168.1.10 || fail "h2->h1"
docker exec $P-l1 ping -c1 -W1 10.0.0.12 || fail "l1->l2 lo"

# FRR leaf sessions
docker exec $P-l1 vtysh -c 'show bgp summary' | grep -q '65100' \
  || fail "l1 missing spine asn in summary"

echo OK-MULTI
```

Extend with SRL-specific checks when l2 is SR Linux.

Overlay optional add-on

Once underlay multipath works on both implementations:

- Static VXLAN between l1 and l2 loopbacks (Linux vxlan on FRR node may need extra tooling; pure FRR leaf might use other integration)
- Or EVPN only if **both** leaves support it—heterogeneous EVPN is an advanced interop project, not a first lab

Declare non-goals if overlay skipped.

Hardening gates (reuse part 08)

- ☐ Only fabric prefixes advertised
- ☐ Mgmt not exposed on fabric links
- ☐ Image tags pinned
- ☐ destroy/recreate clean

Lab report template

```
# Multi-implementation fabric
Date:
Images: FRR x.y, SR Linux z (or FRR-only fallback)
Underlay: eBGP numbered
Results:
- cold verify.sh: PASS/FAIL
- spine failure: notes
- CLI mapping table: attached
Differences found:
Next improvements:
```

When interop hurts

Symptom	Approach
Session up, no routes	policy / AF / network statements
ECMP one side only	multipath knobs asymmetric
Hosts one-way	RPF, missing return prefix
SRL slow boot	wait; recheck before debug spiral

Time-box interop: 2 hours then fall back to dual FRR with a written "SRL blocked on X" note.

Common mistakes

Mistake	Symptom
Different ASNs than plan	sessions down
Comparing wrong show outputs	false failure
Fixing only FRR side	asymmetric design
Unpinned SRL image	lab broke next month
Skipping host bidirectional test	false green

Summary

- Multi-implementation labs prove **models over CLI**
- Freeze addressing and success metrics; vary only NOS
- Maintain a personal show-command dictionary
- Prefer underlay interop first; EVPN interop later
- FRR-only fallback still completes the fabric curriculum

You now have published depth from edge services through fabrics. Return to ops automation capstones and harden the labs you actually keep.